

MANUAL FUNDAMENTAL: REN'PY

EL MOTOR DE NOVELAS VISUALES

RECURSOS Y REFERENCIAS

NARRACIÓN INTERACTIVA
(DIALOGUE SYSTEM)

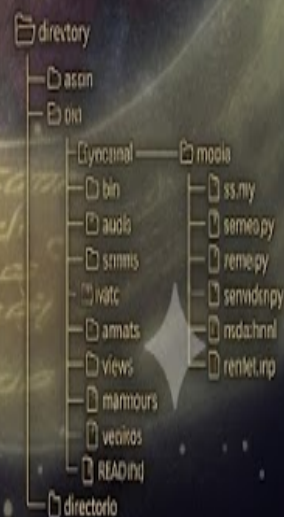
GESTOR DE ESCENAS
(SCENE MANAGEMENT)

SISTEMA DE MENU
(GUI CUSTOMIZATION)

MINIJUEGOS
(MINIGAMES)



VERSIÓN 8.0



ESTRUCTURA DEL PROYECTO

Sho Duncan 17

Manual de Ren'Py

Índice

1. El Lanzador de Ren'Py
2. Un Juego Simple
 - a)* Personajes
 - b)* Imágenes
 - c)* Transiciones
 - d)* Posiciones
 - e)* Música y Sonido
 - f)* Finalizar el Juego
 - g)* Menús, Etiquetas y Saltos
 - h)* Python y Sentencias If
 - i)* Publicar tu Juego
 - j)* Script de "The Question"
3. La Estructura de los Scripts de Ren'Py
 - a)* Líneas
 - b)* Comentarios
 - c)* Bloques
4. Bloques Init
 - a)* Definición de Imágenes
 - b)* Definición de Personajes
5. Comandos Básicos del Script
 - a)* Diálogo
 - b)* Show
 - c)* Hide
 - d)* Scene
 - e)* With
6. Audio
 - a)* Play

- b)* Stop
- c)* Queue

7. Etiquetas (Labels)

- a)* Jump
- b)* Call y Return

8. Variables y Estructuras de Control

- a)* Menu
- b)* If
- c)* Call
- d)* While
- e)* Pass

9. Tutorial de Personalización de Estilos

- a)* Personalizaciones Comunes
- b)* Personalizar Cualquier Estilo
- c)* Inspector de Estilos
- d)* Jerarquía de Estilos
- e)* Indexación y Herencia de Estilos
- f)* Establecer Propiedades de Estilo

10. Estructura de Scripts, Líneas y Bloques

11. Construcciones Sintácticas

12. Sentencias

- a)* Call
- b)* Define
- c)* Hide
- d)* If
- e)* Image
- f)* Init
- g)* Jump
- h)* Label
- i)* Menu
- j)* Pause
- k)* Play
- l)* Pass
- m)* Python

- n*) Queue
- n̂*) Return
- o*) Say
- p*) Scene
- q*) Show
- r*) Stop
- s*) Window
- t*) With
- u*) While

13. Sentencias del Script de Ren'Py

14. Sintaxis y Semántica ATL

15. Sentencias ATL

- a*) Interpolation
- b*) Time
- c*) Expression
- d*) Pass
- e*) Repeat
- f*) Block
- g*) Choice
- h*) Parallel
- i*) Event
- j*) On
- k*) Contains
- l*) Function

16. Warpers

17. Propiedades de Transformación

18. Eventos Externos

19. Equivalente en Python

20. Archivos y Directorios

21. Definición de Personajes

22. Texto

23. Displayables

24. Transiciones

25. Funciones de Interacción
26. Audio
27. Películas (Movies)
28. Funciones UI
29. Variables de Configuración
30. Variables Store
31. Overlays
32. Guardado, Carga y Rollback
33. Datos Persistentes
34. Ofuscación de tu Juego
35. Localización de Ren'Py
36. Propiedades y Estilos
37. Lista de Propiedades
38. Personalización de la Interfaz
39. Layouts
40. Definición de Nuevos Layouts
41. Temas (Themes)
42. Menú Principal y Menú de Juego
43. Modo Compatibilidad
44. Preferencias
45. Displayables Definidos por el Usuario
46. Módulos Python
47. Herramientas de Desarrollo
48. Modo NVL
49. Soporte de Voz
50. Modelo de Renderizado
51. Sentencias Definidas por el Usuario
52. Variables de Entorno
53. Índice de Funciones

El Lanzador de Ren'Py

Antes de comenzar a crear un juego, deberías tomarte un tiempo para aprender cómo funciona el lanzador de Ren'Py. El lanzador te permite crear, gestionar, editar y ejecutar proyectos de Ren'Py.

Para empezar

Para empezar, tendrás que descargar Ren'Py y descomprimirlo. Luego, iniciarás el lanzador ejecutando el programa `renpy`.

Elegir un proyecto

Primero deberías ver cómo es el juego completo de *The Question*. Para ello, inicia el lanzador de Ren'Py y elige **“Seleccionar proyecto”**. Aparecerá un menú de proyectos. Elige **“the_question”** de la lista. Volverás al menú principal y ahora puedes elegir **“Lanzar”** para iniciar *The Question*. Puedes volver a la demo de Ren'Py haciendo lo mismo, pero eligiendo **“demo”** en lugar de **“the_question”**.

Crear un nuevo proyecto

Crea un nuevo proyecto eligiendo **“Nuevo proyecto”** en el lanzador. El lanzador te pedirá que elijas una plantilla. Elige **“template”**. Luego, el lanzador te pedirá un nombre para el proyecto. Como **“the_question”** ya está ocupado, deberías escribir algo diferente, como **“my_question”**. Después, el lanzador te pedirá que elijas un tema de color para el proyecto. En este punto no importa lo que elijas, solo escoge algo que te guste. Volverás al menú principal del lanzador con tu nuevo juego seleccionado.

Un juego sencillo

```
label start:
    "Le preguntaré..."
    "Yo" "Um... ¿quieres..."
    "Yo" "¿Quieres ser mi artista para una novela visual?"
    "Silencio."
    "Ella está impactada, y entonces..."
    "Sylvie" "Claro, pero ¿qué es una \"novela visual\"?"
```

Este es quizás uno de los juegos de Ren'Py más simples. No incluye imágenes ni nada por el estilo, pero muestra una conversación entre los dos personajes.

Para probarlo, ve al lanzador, cambia al proyecto **“My Question”** y elige **“Editar guión”**. Esto abrirá los archivos del guión en un editor de texto. Elige el archivo `script.rpy` y borra todo su contenido. Como empezamos desde cero, no necesitas lo que hay allí. Copia el ejemplo de arriba en `script.rpy` y guárdalo.

Ya estás listo para ejecutar este ejemplo. Vuelve al lanzador y haz clic en **Ejecutar**. Ren'Py se iniciará. Observa cómo, sin ningún trabajo adicional, Ren'Py te ha dado menús que te permiten guardar y cargar la partida, y cambiar varias preferencias. Cuando estés listo, haz clic en **“Comenzar juego”** y juega este ejemplo.

Este ejemplo muestra algunas de las sentencias de Ren'Py más utilizadas.

La primera línea es una sentencia `label`. La sentencia `label` se usa para dar un nombre a un lugar dentro del programa. En este caso, creamos una etiqueta llamada `"start"`. La etiqueta `start` es especial, ya que es donde los guiones de Ren'Py comienzan a ejecutarse cuando el usuario hace clic en **“Comenzar juego”** en el menú principal.

Las otras líneas son sentencias `say` (decir). Hay dos formas de la sentencia `say`. La primera es una cadena (que comienza con comillas dobles, contiene caracteres y termina con comillas dobles) en una línea sola, que se usa para la narración y los pensamientos del personaje principal. La segunda forma consiste en dos cadenas. Se usa para el diálogo: la primera cadena es el nombre del personaje y la segunda es lo que ese personaje dice.

Observa que todas las sentencias `say` están sangradas con cuatro espacios. Esto es porque forman un bloque que pertenece a la sentencia `label`. En Ren'Py, los bloques deben estar sangrados con respecto a la sentencia anterior, y todas las sentencias de un bloque deben tener la misma sangría.

Cuando las cadenas contienen comillas dobles, esos caracteres deben ir precedidos por una barra invertida. Esto se hace en la última línea de nuestro ejemplo.

Aunque este sencillo juego no es gran cosa a simple vista, es un ejemplo de lo fácil que es conseguir que algo funcione en Ren'Py. Añadiremos las imágenes en un momento, pero primero veamos cómo declarar personajes.

Personajes

Un problema del primer ejemplo es que te obliga a escribir repetidamente el nombre de un personaje cada vez que habla. En un juego con mucho diálogo, esto puede suponer mucho tecleo. Además, ambos nombres de personaje se muestran de la misma manera, con un texto blanco bastante soso. Para solucionar esto, Ren'Py permite definir los personajes con antelación. Esto te permite asociar un nombre corto a un personaje y cambiar el color de su nombre.

```
define s = Character('Sylvie', color="#c8ffc8")
define m = Character('Yo', color="#c8c8ff")

label start:
    "Le preguntaré..."
    m "Um... ¿quieres..."
    m "¿Quieres ser mi artista para una novela visual?"
    "Silencio."
    "Ella está impactada, y entonces..."
    s "Claro, pero ¿qué es una \"novela visual\"?"
```

La primera y la segunda línea definen personajes. La primera línea define un personaje con el nombre corto `"s"`, el nombre largo `"Sylvie"` y un nombre que se muestra en un color verdoso (los colores son tripletas hexadecimales rojo-verde-azul, como las que se usan en las páginas web).

La segunda línea crea un personaje con el nombre corto `"m"`, el nombre largo `"Yo"` y el nombre mostrado en un color rojizo. Se pueden definir otros personajes copiando una de las líneas de personaje y cambiando el nombre corto, el nombre largo y el color.

También hemos cambiado las sentencias `say` para que usen objetos de personaje en lugar de una cadena con el nombre. Esto le indica a Ren'Py que use los personajes que definimos en el bloque `init`.

Imágenes

Una novela visual no es gran cosa sin imágenes. Añadamos algunas a nuestro juego.

```
image bg meadow = "meadow.jpg"
image bg uni = "uni.jpg"
image sylvie smile = "sylvie_smile.png"
image sylvie surprised = "sylvie_surprised.png"

define s = Character('Sylvie', color="#c8ffc8")
define m = Character('Yo', color="#c8c8ff")

label start:
    scene bg meadow
    show sylvie smile
    "Le preguntaré..."
    m "Um... ¿quieres..."
    m "¿Quieres ser mi artista para una novela visual?"
    show sylvie surprised
    "Silencio."
    "Ella está impactada, y entonces..."
    show sylvie smile
    s "Claro, pero ¿qué es una \"novela visual\"?"
```

Lo primero que necesitamos hacer fue declarar las imágenes, usando sentencias `image` en las líneas 2, 3, 5 y 6, dentro del bloque `init`. Estas sentencias `image` asignan un nombre a la imagen y el archivo en el que se encuentra.

Por ejemplo, la línea 5 declara una imagen llamada `"sylvie smile"` que se encuentra en el archivo `"sylvie_smile.png"`, con la etiqueta `"sylvie"`.

Tenemos una sentencia `scene` en la línea 12. Esta sentencia limpia la pantalla y muestra la imagen `"bg meadow"`. La siguiente línea es una sentencia `show`, que muestra la imagen `"sylvie smile"` en la pantalla.

La primera parte del nombre de una imagen es la etiqueta de imagen (*image tag*). Si una imagen está siendo mostrada y otra imagen con la misma etiqueta está en la pantalla, la imagen que está en la pantalla se reemplaza por la que se está mostrando. Esto ocurre en la línea 19, la segunda sentencia `show`. Antes de ejecutar la línea 19, la imagen `"sylvie smile"` está en la pantalla. Cuando se ejecuta la línea 19, esa imagen se reemplaza por `"sylvie surprised"`, ya que comparten la etiqueta `"sylvie"`.

Para que Ren'Py encuentre los archivos de imagen, deben estar colocados en el directorio `game` del proyecto actual. El directorio `game` se encuentra en `«NombreDelProyecto»/game/` o haciendo clic en el botón **“Directorio del juego”** en el lanzador. Probablemente tendrás que copiar los archivos de imagen del directorio `"the_question/game/"` al directorio `"my_question/game/"` para poder ejecutar este ejemplo.

Ren'Py no hace ninguna distinción entre el arte de personajes y el de fondos, ya que ambos se tratan como imágenes. En general, el arte de personajes debe ser transparente, lo que significa que debería ser un archivo PNG. El arte de fondos puede ser archivos JPEG o PNG. Por convención, las imágenes de fondo comienzan con la etiqueta `"bg"`.

La sentencia `hide`

Ren'Py también admite una sentencia `hide`, que oculta la imagen dada.

```
s "¡Me pongo con ello enseguida!"
hide sylvie
"...
m "¡No era eso lo que quería decir!"
```

En realidad es bastante raro que necesites usar `hide`. Se puede usar `show` cuando un personaje cambia de emoción, mientras que `scene` se usa cuando todos se van. Solo necesitas usar `hide` cuando un personaje se va y la escena permanece igual.

Transiciones

Que las imágenes aparezcan y desaparezcan sin más es aburrido, así que Ren'Py implementa transiciones que pueden hacer que los cambios en la pantalla sean más interesantes. Las transiciones cambian la pantalla desde cómo se veía al final de la última interacción (diálogo, menú o transición) hasta cómo queda después de las sentencias `scene`, `show` y `hide`.

```
label start:
    scene bg uni
    show sylvie smile
    s "Oh, hola, ¿volvemos juntos a casa?"
    m "Sí..."
    "Dije, y mi voz ya temblaba."
    scene bg meadow
    with fade
    "Llegamos a los prados justo a las afueras de nuestro pueblo."
    "El otoño era tan hermoso allí."
    "Cuando éramos niños, solíamos jugar aquí."
    m "Oye... ummm..."
    show sylvie smile
    with dissolve
    "Se giró hacia mí y sonrió."
    "Le preguntaré..."
    m "Ummm... ¿quieres..."
    m "¿Quieres ser mi artista para una novela visual?"
```

La sentencia `with` toma el nombre de una transición para usar. La más común es `dissolve`, que funde una pantalla en la siguiente. Otra transición útil es `fade`, que funde la pantalla a negro y luego funde la nueva pantalla.

Cuando se coloca una transición después de varias sentencias `scene`, `show` o `hide`, se aplica a todas a la vez. Por ejemplo, si escribieras:

```
scene bg meadow
show sylvie smile
with dissolve
```

Tanto el fondo "bg meadow" como "sylvie smile" aparecerían fundidos al mismo tiempo. Para fundirlos uno tras otro, necesitas escribir dos sentencias `with`:

```
scene bg meadow
with dissolve
show sylvie smile
with dissolve
```

Así, primero aparece fundido el prado y luego se funde Sylvie. Si quisieras mostrar el prado al instante y luego mostrar a Sylvie, podrías escribir:

```
scene bg meadow
with None
show sylvie smile
with dissolve
```

Aquí, `None` se usa para indicar una transición especial que actualiza la idea que Ren'Py tiene de la pantalla anterior, sin mostrar realmente nada al usuario.

Posiciones

Por defecto, las imágenes se muestran centradas horizontalmente y con su borde inferior tocando la parte inferior de la pantalla. Esto suele estar bien para fondos y personajes individuales, pero cuando se muestra más de un personaje en la pantalla, probablemente tenga sentido hacerlo en otra posición. También puede tener sentido recolocar a un personaje por motivos argumentales.

```
show sylvie smile at right
```

Para hacer este reposicionamiento, añade una cláusula `at` a la sentencia `show`. La cláusula `at` toma una posición y muestra la imagen en esa posición. Ren'Py incluye varias posiciones predefinidas: `left` para el lado izquierdo de la pantalla, `right` para el lado derecho, `center` para centrado horizontalmente (el valor por defecto) y `truecenter` para centrado horizontal y verticalmente.

El usuario puede definir sus propias posiciones, pero eso queda fuera del alcance de esta guía rápida.

Música y sonido

La mayoría de los juegos reproducen música de fondo. En Ren'Py, los archivos de música se repiten automáticamente en bucle hasta que el usuario los detiene. La música se reproduce con la sentencia `play music`.

```
play music "illurock.ogg"
```

Al cambiar de música, se puede proporcionar una cláusula `fadeout`, que se usa para desvanecer la música antigua cuando se reproduce la nueva.

```
play music "illurock.ogg" fadeout 1.0
```

La música se puede detener con la sentencia `stop music`, que también puede tomar opcionalmente una cláusula `fadeout`.

```
stop music
```

Los efectos de sonido se pueden reproducir con la sentencia `play sound`:

```
play sound "effect.ogg"
```

Ren'Py admite muchos formatos para sonido y música, pero se prefiere OGG Vorbis. Al igual que los archivos de imagen, los archivos de sonido y música deben colocarse en el directorio `game`.

Finalizar el juego

Puedes finalizar el juego ejecutando la sentencia `return` sin haber llamado a nada antes. Antes de hacerlo, es mejor poner algo en el juego que indique que está terminando, y quizás dar al usuario un número o un nombre de final.

```
".:. Final bueno."
return
```

Eso es todo lo que necesitas para hacer una novela cinética, un juego sin elecciones. Ahora veremos lo que se necesita para hacer un juego que presente menús al usuario.

Menús, etiquetas y saltos

La sentencia `menu` te permite presentar una opción al usuario.

```
s "Claro, pero ¿qué es una \"novela visual\"?"
menu:
```

```
    "Es una historia con imágenes.":
```

```
        jump vn
```

```
    "Es un juego hentai.":
```

```
        jump hentai
```

```
label vn:
```

```
    m "Es una historia con imágenes y música."
```

```
    jump marry
```

```
label hentai:
```

```
    m "Pues es un juego con mucho sexo."
```

```
    jump marry
```

```
label marry:
```

```
    scene black with dissolve
```

```
    "--- años después ---"
```

Este ejemplo muestra cómo se usan los menús en Ren'Py. La sentencia `menu` introduce un menú dentro del juego. La sentencia `menu` toma un bloque de líneas, cada una compuesta por una cadena seguida de dos puntos. Estas son las opciones del menú que se presentan al usuario. Cada opción del menú debe ir seguida de un bloque de una o más sentencias de Ren'Py. Cuando se elige una opción, se ejecutan las sentencias que la siguen.

En nuestro ejemplo, cada opción del menú ejecuta una sentencia `jump`. La sentencia `jump` transfiere el control a una etiqueta definida con la sentencia `label`. Se ejecuta el código que sigue a esa etiqueta.

En el ejemplo anterior, después de que Sylvie hace su pregunta, se le presenta al usuario un menú con dos opciones. Si el usuario elige *.Es una historia con imágenes.*, se ejecuta la primera sentencia `jump` y el control se transfiere a la etiqueta `vn`. Esto hará que el personaje del protagonista diga *.Es una historia con imágenes y música.*, y luego el control se transfiere a la etiqueta `marry`.

Las etiquetas pueden definirse en cualquier archivo que esté en el directorio `game` y que termine en `.rpy`. El nombre del archivo no importa para Ren'Py, solo las etiquetas que contiene. Una etiqueta solo puede aparecer en un único archivo.

Lanzando tu juego

Una vez que hayas hecho un juego, hay varias cosas que deberías hacer antes de lanzarlo:

- **Editar `options.rpy`.** El archivo `options.rpy`, creado cuando creas un nuevo juego, contiene una serie de ajustes que quizás quieras personalizar. Algunos de ellos, como el alto y el ancho de la pantalla, probablemente deberían establecerse antes de hacer el juego. Otros, como el título de la ventana, se pueden configurar en cualquier momento.
- **Añadir una mención a Ren'Py.** Este paso es completamente opcional, pero pedimos que si tienes créditos en tu juego, menciones a Ren'Py en ellos. Sugerimos usar algo como *“Hecho con el motor de novelas visuales Ren'Py”*, pero es solo una sugerencia, y lo que escribas queda a tu criterio. Creemos que los juegos que la gente hace son la mejor publicidad para Ren'Py, y esperamos que al incluir esto, ayudes a que más personas aprendan a hacer novelas visuales con Ren'Py.
- **Crear un archivo `README`.** Probablemente quieras incluir un archivo `README.txt` en el directorio base de tu proyecto (es decir, el directorio que está por encima del directorio `game`). Incluimos un archivo de ejemplo con Ren'Py, que puedes personalizar según sea necesario. También deberías considerar elegir una licencia para tu juego, que determine cómo la gente puede distribuirlo.
- **Verificar si hay una nueva versión de Ren'Py.** Periódicamente se lanzan nuevas versiones de Ren'Py para corregir errores y añadir nuevas funcionalidades. Deberías consultar la página de descargas para ver si ha salido una nueva versión. También puede que quieras ver si hay parches de errores disponibles en esa página.
- **Revisar el guión.** En el Lanzador, ve a la página de Herramientas y elige **“Revisar guión (Lint)”**. Esto revisará tu juego en busca de errores que podrían afectar a algunos usuarios. Estos errores pueden afectar a usuarios de las plataformas Mac y Linux, así que es importante corregirlos todos, incluso si no los ves en tu ordenador.

- **Construir distribuciones.** Desde la página de Herramientas, haz clic en **Distribuir**. El lanzador revisará tu guión de nuevo, te hará algunas preguntas y luego construirá la distribución de tu juego.
- **Probar.** Lint no es un sustituto de las pruebas exhaustivas. Es tu responsabilidad revisar tu juego antes de lanzarlo.
- **Lanzar.** Deberías subir los archivos generados (para Windows, Mac y Linux) a algún sitio web y decirle a la gente dónde descargarlos.

¡Enhorabuena, has lanzado un juego! Si quieres, puedes contárnoslo en los foros de Lemma Soft.

Guion de The Question

Puedes ver el guión completo de *The Question* aquí.

Esta página proporciona una visión general sencilla del lenguaje de Ren'Py, pensada para no programadores. También hay una versión más técnica de esta página para usuarios más avanzados. Esta versión sencilla puede no contener el mismo nivel de detalle, y puede no ser estrictamente precisa en un sentido técnico, pero pretende dar al lector una comprensión del uso común de las características del lenguaje descritas, omitiendo detalles exactos en favor de la utilidad.

La estructura de los guiones de Ren'Py

Líneas

Los guiones de Ren'Py consisten en uno o más archivos `.rpy`. Estos archivos de guión pueden leerse en cualquier orden, y todos juntos conforman un guión de Ren'Py. Cuando se inicia el juego, Ren'Py encuentra la etiqueta `start` y recorre las líneas una por una en orden, ejecutando lo que cada línea le indica, hasta que se acaba el guión.

Mientras Ren'Py ejecuta el guión, generalmente trata cada línea del archivo como una instrucción separada; sin embargo, hay algunas excepciones:

- Si has abierto un paréntesis y no lo has cerrado, Ren'Py seguirá leyendo en la siguiente línea hasta encontrar el paréntesis de cierre. Del mismo modo, Ren'Py emparejará las comillas que encierran texto a través de líneas. Por ejemplo, lo siguiente se considera una sola instrucción:

```
$ a = Character(
    'Abigail',
    color="#454580"
)
"Estaba desorientada,
sintiendo que mis pensamientos
estaban divididos y
dispersos de alguna manera."
```

- Si quieres dividir algo en varias líneas, puedes terminar una línea con una barra invertida (`\`) y continuar en la siguiente línea, por ejemplo:


```
$ e = m * \
      c * c
```

es equivalente a:

```
$ e = m * c * c
```

Comentarios

Si quieres hacer anotaciones dentro de tu guión, puedes usar comentarios. Generalmente se considera una buena práctica escribir comentarios que describan cualquier cosa que creas que necesitarás recordar, quizás por qué se comprueba una condición en particular o para qué se usa una variable. Para escribir un comentario, simplemente escribe una almohadilla (#). Ren'Py ignorará todo desde ese punto hasta el final de la línea, así que puedes escribir lo que quieras.

```
# Esta línea es un comentario y será ignorada.
# Esta línea también será ignorada.
"Esta línea será tomada por Ren'Py y usada como diálogo"

"Esto es diálogo" # pero esta parte al final es un comentario.
```

Ten en cuenta que, como Ren'Py ignora los comentarios, no verifica paréntesis, comillas ni barras invertidas al final de la línea; por lo tanto, los comentarios nunca pueden extenderse a lo largo de más de una línea en tu archivo de guión sin poner otra almohadilla.

Bloques

A veces, es necesario agrupar un conjunto de líneas. Esto ocurre con mayor frecuencia para las etiquetas, pero también para sentencias condicionales, bucles, etc. Las líneas agrupadas se denominan *bloques* y se escriben con una sangría adicional en el archivo de guión:

```
line 1
    line a
    line b
line 2
    line c
    line d
```

En este ejemplo, hay tres bloques. Un bloque contiene todo, el segundo bloque comprende las líneas a y b, y el tercero contiene las líneas c y d.

Observa que se dice que las líneas a y b están asociadas con la línea 1; típicamente, la línea 1 describirá por qué las líneas a y b están en un bloque, por ejemplo:

```
if (x < 10):
    # Las líneas de este bloque solo se ejecutan si x es menor que diez
    "Sam" "¡Oh no!"
    "Sam" "¡Ecks es menor que diez!"
# Pero estas líneas (fuera del bloque del if) se ejecutan independientemente del valor de x
"Caminamos un poco más por el camino, buscando algo tan interesante como ese ecks."
```

Observa que las líneas que inician un nuevo bloque terminan con dos puntos (:). Si una línea termina con dos puntos, Ren'Py esperará encontrar un nuevo bloque debajo; si una línea no termina con dos puntos, no se debe iniciar un nuevo bloque después de esa línea. Todo el guión se dividirá en bloques de etiquetas y bloques `init`; estos controlan el flujo general del juego; Ren'Py siempre comenzará en la etiqueta `start`.

Bloques Init

Los bloques `init` se utilizan para definir elementos que se van a usar una y otra vez a lo largo del guión. Normalmente, esto incluye imágenes y personajes. Generalmente es una mala idea definir algo que tenga un estado cambiante en un bloque `init`; es mejor ceñirse a cosas que permanecen igual durante todo el juego, como imágenes de fondo, sprites de personajes, etc.

Cuando el juego se inicia por primera vez, se ejecutan todos los bloques `init` de todos los guiones; si más tarde se encuentran bloques `init` en medio del guión, se ignoran y se saltan por completo.

Un bloque `init` se inicia con la palabra `init`, seguida de dos puntos, por ejemplo:

```
init:
    # aquí va el código del bloque init
    # más código init
```

Lo más común que se encuentra dentro de un bloque `init` son definiciones de Imágenes y Personajes:

Definir imágenes

Las imágenes deben definirse en un bloque `init`.

Cuando definimos una imagen, le damos un nombre a un archivo de imagen para poder referirnos a él más tarde en el guión. Así, cuando queramos mostrar un sprite de personaje o comenzar una nueva escena con un fondo diferente, podremos usar las imágenes que definimos anteriormente.

Una imagen se define con el comando `image`, seguido de un nombre para esa imagen, luego un signo igual y luego el nombre del archivo de imagen que queremos usar. Coloca el archivo de imagen en el directorio `game`, junto al guión, y Ren'Py podrá encontrarlo.

```
init:
    image eileen = "eileen.png"
```

Ten en cuenta que puedes dar nombres a las imágenes con espacios; si haces esto, Ren'Py presta especial atención a la primera palabra del nombre (ver los comandos `show` y `hide` para más detalles), por lo que es aconsejable agrupar tus imágenes por esa primera parte, por ejemplo:

```
init:
    image eileen happy = "eileen-happy.png"
    image eileen sad = "eileen-sad.png"
```

Definir personajes

También es común definir en bloques `init` los Personajes. Un Personaje permite asociar de forma rápida y sencilla un nombre a los diálogos, formatear el texto, etc., en lugar de tener que escribir el nombre completo del personaje y asignar colores cada vez que ese personaje habla.

En su forma más simple, se define un personaje así:

```
init:
    $ 1 = Character("Lucy")
```

El signo de dólar al principio de la línea le dice a Ren'Py que esta es una línea de Python. Esto es necesario para definir personajes porque no hay un comando de Ren'Py que lo haga por nosotros: ¡los Personajes tienen demasiadas opciones! La siguiente parte (1 en el ejemplo anterior) es el nombre con el que nos referimos a ese personaje en el resto del guión. La última parte crea el Personaje en sí; aquí simplemente proporcionamos el nombre que queremos que se muestre cuando se use ese personaje.

Crear un Personaje para cada uno de los personajes de tu novela visual no es en absoluto esencial, pero al menos puede ayudarte a ahorrarte tener que escribir el nombre del personaje cada vez que habla.

Para más detalles sobre los Personajes y las muchas otras opciones que tienen, consulta el artículo sobre *Definir personajes*.

Comandos básicos del guión

La siguiente lista contiene comandos comunes que Ren'Py entiende, junto con una explicación de uso y un ejemplo para cada uno. Aunque se ha procurado ordenar la lista de forma sensata, es inevitable que a veces sea necesario hacer referencia a elementos posteriores.

Diálogo y narración

El discurso muestra texto en la pantalla y espera a que el usuario haga clic con el ratón o pulse espacio o Intro antes de continuar con la siguiente instrucción. El discurso se divide en dos categorías: narración o diálogo.

La narración es lo más simple: consiste en el texto a narrar, encerrado entre comillas, por ejemplo:

```
"Era una noche oscura y tormentosa."
```

Observa que puedes usar comillas simples o dobles, pero debes comenzar y terminar el texto con el mismo tipo de comilla. Por supuesto, esto significa que puedes incluir comillas simples en texto entre comillas dobles y viceversa:

```
"'Era la noche antes de Navidad'"
'Podía decir que iba a ser uno de esos días de "esos".'
```

El diálogo se usa para que hablen personajes concretos. Dado que el lector no puede distinguir la voz de un personaje de la de otro cuando todo el diálogo es solo texto, el nombre del personaje que habla se muestra junto al diálogo. Por supuesto, esto significa

que debes especificar el nombre del personaje en el guión, lo cual se hace escribiendo primero el nombre del personaje (entre comillas) y luego el diálogo de ese personaje inmediatamente después, por ejemplo:

```
"Hamlet" "¡Ay, pobre Yorick! Lo conocí, Horacio, un hombre de infinito ingenio, de ex
```

Si has definido un Personaje (en un bloque `init`), puedes usar ese Personaje en lugar del nombre al escribir el diálogo, por ejemplo:

```
init:
    $ e = Character("Eileen")

label start:
    e "¡Hola!"
```

Show

El comando `show` se usa para añadir un sprite a una escena. El sprite debe haber sido definido previamente como una imagen en un bloque `init`, ya que el comando `show` usa el nombre de la imagen.

En su forma más básica, el comando `show` consiste en la palabra `show` seguida del nombre de la imagen a mostrar, por ejemplo:

```
init:
    image eileen = "eileen.png"

label start:
    show eileen
```

Esto mostrará la imagen llamada `eileen` (la del archivo `eileen.png`) en el centro inferior de la pantalla.

Cuando muestras una imagen, Ren'Py buscará una imagen que ya esté en la escena y que comparta la misma primera parte del nombre; si existe tal imagen, Ren'Py la eliminará antes de mostrar la nueva. Esto significa que, por ejemplo, no tienes que ocultar los sprites antiguos del mismo personaje cuando cambias una expresión:

```
init:
    image eileen happy = "eileen-happy.png"
    image eileen sad = "eileen-sad.png"

label start:
    show eileen happy
    show eileen sad
```

Cuando se ejecuta el segundo comando `show`, Ren'Py mira la primera parte del nombre de la imagen (en este caso `eileen`) y examina todas las imágenes ya presentes en la escena en busca de una imagen con esa etiqueta. La anterior `eileen happy` también tenía `eileen` como primera parte del nombre, por lo que se elimina de la escena antes de que se realice la segunda aparición. Esto asegura que solo haya una Eileen en pantalla al mismo tiempo. Por supuesto, no es muy útil poder mostrar cosas solo en el centro de la pantalla, así que el comando `show` también puede ir seguido de un `at` que indique una posición en la que colocar la nueva imagen en la escena:

```
show eileen at left
```

Esto mostrará la imagen `eileen` en el lado izquierdo de la pantalla. Las posiciones que Ren'Py entiende de serie son las siguientes:

- `left`: en el lado izquierdo, contra la parte inferior de la pantalla.
- `right`: en el lado derecho, contra la parte inferior de la pantalla.
- `center`: en el centro, contra la parte inferior de la pantalla.
- `truecenter`: centrado tanto horizontal como verticalmente.
- `offscreenleft`: justo fuera del lado izquierdo de la pantalla, contra la parte inferior.
- `offscreenright`: justo fuera del lado derecho de la pantalla, contra la parte inferior.

Alternativamente, puedes definir tus propias posiciones con la función `Position`.

Canales

La salida de audio se divide en “canales”. Solo puedes reproducir una pieza de audio (sonido o música) en un solo canal a la vez. Si quieres reproducir dos sonidos simultáneamente, deberás especificar que el segundo se reproduzca en un canal diferente. Esto se puede hacer añadiendo `channel` y luego el número del canal que desees usar al final del comando:

```
play sound "explosion.ogg" channel 0  
play sound "yeehah.ogg" channel 1
```

Por defecto, los sonidos se reproducen en el canal 0 y la música en el canal 7. Por supuesto, esto significa que si solo reproduces un sonido a la vez, no necesitas preocuparte por los canales: los sonidos se pueden reproducir sobre la música sin problemas.

Fade

Por defecto, Ren'Py detendrá cualquier audio que ya estés reproduciendo en un canal si reproduces algo nuevo. Si quieres especificar un desvanecimiento de salida (*fadeout*) para una música o efecto de sonido que ya se está reproduciendo, añade `fadeout` seguido de un tiempo en segundos al final del comando:

```
play music "waltz.ogg" fadeout 1
```

Del mismo modo, si quieres desvanecer la nueva pieza musical al entrar (*fadein*), usa la palabra clave `fadein`. Se pueden combinar cualquiera o todas estas palabras clave en el mismo comando:

```
play music "waltz.ogg" fadeout 1 fadein 2
```

El ejemplo anterior desvanecerá la música que se esté reproduciendo actualmente durante un segundo, y desvanecerá `"waltz.ogg"` durante dos segundos.

Listas de reproducción

Por último, si prefieres proporcionar varias piezas musicales o sonidos para que se reproduzcan uno tras otro, puedes dar al comando `play` una lista de archivos en lugar de un solo nombre de archivo. Para usar una lista, utiliza corchetes (`[, ]`) alrededor de una lista de nombres de archivo separados por comas:

```
play music ["waltz.ogg", "tango.ogg", "rumba.ogg"]
```

El ejemplo anterior reproducirá `"waltz.ogg"`, luego, cuando termine, reproducirá `"tango.ogg"`, luego `"rumba.ogg"`, y luego comenzará de nuevo desde el principio con `"waltz.ogg"`.

Formatos

Ren'Py reproduce archivos en los siguientes formatos:

- Ogg Vorbis (.ogg)
- PCM Wave (.wav)
- FLAC (.flac)
- Shorten (.shn)
- MP3 (.mp3)
- Varios formatos mod

Los archivos de audio deben guardarse a 44100 Hz para evitar crujidos.

Stop

`stop` es lo opuesto al comando `play`; detiene el audio que se esté reproduciendo actualmente. De manera similar a `play`, puedes elegir entre `stop sound` para detener un efecto de sonido, o `stop music` para detener la reproducción de música:

```
stop sound
```

Al igual que con `play`, puedes especificar un desvanecimiento de salida para el audio que estás deteniendo añadiendo la palabra `fadeout` seguida de un tiempo en segundos:

```
stop music fadeout 1.5
```

El ejemplo anterior desvanece la música que se está reproduciendo actualmente durante un segundo y medio.

También puedes detener el audio que se reproduce en un canal concreto. De nuevo, esto se hace de manera similar al comando `play`: simplemente añade `channel` y el número de canal que desees detener después del comando `stop`, por ejemplo:

```
stop sound channel 2
```

Dado que solo estás deteniendo audio que ya se ha reproducido, los canales predeterminados son los mismos: canal 0 para efectos de sonido y canal 7 para música.

Queue

El comando **queue** encola archivos de música o efectos de sonido para que se reproduzcan después de que el audio actual haya terminado. Ren'Py encolará el archivo que especifiques para que se reproduzca inmediatamente después de que termine la música que se está reproduciendo actualmente.

El uso simple del comando es similar al comando **play**: el comando **queue** va seguido de **sound** o **music**, dependiendo de si quieres encolar un efecto de sonido o una pieza musical, seguido del nombre del archivo:

```
queue music "next_song.ogg"
queue sound "effect2.ogg"
```

Al igual que con **play**, también puedes usar listas de reproducción con **queue**, especificar canales, añadir **fadein** para el nuevo audio, etc.

Queue (continuación)

Si llamas al comando **queue** por segunda vez antes de que termine la primera pieza musical, tu nuevo archivo encolado reemplazará al elemento encolado existente y se reproducirá tan pronto como el audio actual haya terminado.

```
play music "waltz.ogg"
queue music "tango.ogg"
"Hay un vals sonando de fondo..."
queue music "rumba.ogg"
```

En el ejemplo anterior, primero comienza a sonar **"waltz.ogg"** y luego se encola **"tango.ogg"**. Después, el juego muestra algo de diálogo y espera a que el usuario haga clic para continuar.

- Si el usuario hace clic antes de que termine el vals, entonces se encola **rumba.ogg**; esto reemplaza al tango ya encolado y, después de que termine el vals, sonará la rumba.
- Si el usuario espera a que termine el vals antes de continuar, entonces el tango previamente encolado sonará inmediatamente después del vals, y cuando se encola la rumba, esta se reproducirá inmediatamente después de que termine el tango.

En cualquiera de los dos escenarios, una vez que termina la rumba, continuará en bucle solo la rumba.

Si deseas encolar más de un archivo a la vez, puedes proporcionar una lista de nombres de archivo, separados por comas, entre corchetes (**[,]**) en lugar de un solo nombre de archivo:

```
play music "rumba.ogg"
queue music ["waltz.ogg", "tango.ogg"]
```

En este ejemplo, suena la rumba. Se añaden dos archivos a la cola, por lo que cuando termine la rumba, sonará el vals a continuación; cuando termine el vals, sonará el tango. Cuando termine el tango, toda la cola se repetirá en bucle, por lo que sonará el vals, luego el tango, luego el vals de nuevo, luego el tango de nuevo, y así sucesivamente.

De manera similar al comando `play`, puedes especificar un canal en el que encolar el audio añadiendo `channel` seguido del número de canal al final del comando:

```
queue sound "explosion.ogg" channel 2
```

Ten en cuenta que no puedes encolar música en un canal que esté reproduciendo efectos de sonido, ni efectos de sonido en un canal que esté reproduciendo música. Cada canal tiene su propia cola, por lo que puedes encolar, por ejemplo, música en el canal predeterminado y efectos de sonido en el canal predeterminado, y ambos elementos encolados se reproducirán de forma independiente.

Etiquetas (Labels)

Las etiquetas se usan para dividir un guión de Ren'Py en secciones y también para permitir una progresión no lineal a través de esas secciones.

Generalmente, Ren'Py ejecuta cada línea del guión en el orden en que las encuentra, por ejemplo:

```
"Esta línea se ejecuta primero"
"Esta línea se ejecuta segunda"
"Esta línea se ejecuta tercera"
# y así sucesivamente...
```

Sin embargo, para escribir una historia ramificada, o quizás una historia en la que algunas partes se repiten, usamos etiquetas. Las etiquetas contienen bloques de guión de Ren'Py y consisten en la palabra `label` seguida de un nombre, seguida de dos puntos (`:`) que indican que contiene un bloque:

```
label start:
    "Esta línea está bajo la etiqueta 'start'."
    "Y esta también."
```

```
label next:
    "Pero esta está bajo la etiqueta 'next'."
```

```
label last:
    "Y esta está bajo 'last'."
```

Los nombres de las etiquetas deben ser únicos en todo el guión, porque Ren'Py los usa para identificar ese lugar particular en el guión. No puedes tener dos etiquetas con el mismo nombre, porque Ren'Py no podría distinguir esos dos bloques de etiquetas. Los nombres de etiquetas no pueden contener espacios, por lo que es común usar guiones bajos para separar palabras en un nombre de etiqueta:


```
label not valid:
    # La etiqueta anterior no funcionará porque tiene espacios en el nombre.

label perfectly_valid:
    # La etiqueta 'perfectly_valid', en cambio, es correcta.
```

Cuando Ren'Py llega al final de un bloque de etiqueta (por ejemplo, la línea "Y esta también."^{en} el ejemplo anterior), simplemente continúa con el siguiente bloque de etiqueta, por lo que en el ejemplo anterior las cuatro líneas se entregan en el orden en que están escritas, una tras otra, como si las declaraciones de etiqueta no estuvieran allí en absoluto. Sin embargo, usando los comandos `jump` y `call`, podemos movernos entre etiquetas de forma no lineal.

Salto (Jump)

El comando `jump` le indica a Ren'Py que salte inmediatamente a la etiqueta que se indica, ignorando todo el guión que hay entre la posición actual y esa etiqueta. Los saltos pueden ser hacia adelante o hacia atrás en el archivo, o incluso a un archivo de guión diferente en el mismo directorio del juego.

El comando `jump` consiste en la palabra `jump` seguida del nombre de la etiqueta a la que saltar:

```
jump some_label
```

Como la etiqueta puede estar en cualquier lugar, el comando `jump` nos permite escribir guiones en un orden no lineal, por ejemplo:

```
label start:
    "Esta línea de diálogo se dirá primero."
    jump stage_two

label stage_three:
    "Esta línea se dirá en tercer lugar."
    jump end
    "Esta línea nunca se ejecutará."

label stage_two:
    "Esta línea se dirá en segundo lugar."
    jump stage_three

label end:
    "Y esta línea será la última."
```

Ren'Py no puede ejecutar la línea del medio (^{Esta línea nunca se ejecutará}) porque es imposible avanzar línea por línea desde una etiqueta hasta esa línea de diálogo. El comando `jump end` se interpondrá y redirigirá a Ren'Py a la etiqueta `end` antes de que llegue a esa línea de diálogo.

Queue (continuación)

Para encolar una pista de música o un efecto de sonido, escribe el comando **queue** seguido de **music** o **sound** (según lo que quieras encolar) y luego el nombre del archivo de audio:

```
queue music "waltz.ogg"
```

Si llamas al comando **queue** por segunda vez antes de que termine la primera pieza musical, tu nuevo archivo encolado reemplazará al elemento encolado existente y se reproducirá tan pronto como el audio actual haya terminado.

```
play music "waltz.ogg"
queue music "tango.ogg"
"Hay un vals sonando de fondo..."
queue music "rumba.ogg"
```

En el ejemplo anterior, primero comienza a sonar **"waltz.ogg"** y luego se encola **"tango.ogg"**. Después, el juego muestra algo de diálogo y espera a que el usuario haga clic para continuar.

- Si el usuario hace clic antes de que termine el vals, entonces se encola **rumba.ogg**; esto reemplaza al tango ya encolado y, después de que termine el vals, sonará la rumba.
- Si el usuario espera a que termine el vals antes de continuar, entonces el tango previamente encolado sonará inmediatamente después del vals, y cuando se encola la rumba, esta se reproducirá inmediatamente después de que termine el tango.

En cualquiera de los dos escenarios, una vez que termina la rumba, continuará en bucle solo la rumba.

Si deseas encolar más de un archivo a la vez, puedes proporcionar una lista de nombres de archivo, separados por comas, entre corchetes ([,]) en lugar de un solo nombre de archivo:

```
play music "rumba.ogg"
queue music ["waltz.ogg", "tango.ogg"]
```

En este ejemplo, suena la rumba. Se añaden dos archivos a la cola, por lo que cuando termine la rumba, sonará el vals a continuación; cuando termine el vals, sonará el tango. Cuando termine el tango, toda la cola se repetirá en bucle, por lo que sonará el vals, luego el tango, luego el vals de nuevo, luego el tango de nuevo, y así sucesivamente.

De manera similar al comando **play**, puedes especificar un canal en el que encolar el audio añadiendo **channel** seguido del número de canal al final del comando:

```
queue sound "explosion.ogg" channel 2
```

Ten en cuenta que no puedes encolar música en un canal que esté reproduciendo efectos de sonido, ni efectos de sonido en un canal que esté reproduciendo música. Cada canal tiene su propia cola, por lo que puedes encolar, por ejemplo, música en el canal predeterminado y efectos de sonido en el canal predeterminado, y ambos elementos encolados se reproducirán de forma independiente.

Etiquetas (Labels)

Las etiquetas se usan para dividir un guión de Ren'Py en secciones y también para permitir una progresión no lineal a través de esas secciones.

Generalmente, Ren'Py ejecuta cada línea del guión en el orden en que las encuentra, por ejemplo:

```
"Esta línea se ejecuta primero"  
"Esta línea se ejecuta segunda"  
"Esta línea se ejecuta tercera"  
# y así sucesivamente...
```

Sin embargo, para escribir una historia ramificada, o quizás una historia en la que algunas partes se repiten, usamos etiquetas. Las etiquetas contienen bloques de guión de Ren'Py y consisten en la palabra `label` seguida de un nombre, seguida de dos puntos (`:`) que indican que contiene un bloque:

```
label start:  
    "Esta línea está bajo la etiqueta 'start'."  
    "Y esta también."
```

```
label next:  
    "Pero esta está bajo la etiqueta 'next'."
```

```
label last:  
    "Y esta está bajo 'last'."
```

Los nombres de las etiquetas deben ser únicos en todo el guión, porque Ren'Py los usa para identificar ese lugar particular en el guión. No puedes tener dos etiquetas con el mismo nombre, porque Ren'Py no podría distinguir esos dos bloques de etiquetas. Los nombres de etiquetas no pueden contener espacios, por lo que es común usar guiones bajos para separar palabras en un nombre de etiqueta:

```
label not valid:  
    # La etiqueta anterior no funcionará porque tiene espacios en el nombre.
```

```
label perfectly_valid:  
    # La etiqueta 'perfectly_valid', en cambio, es correcta.
```

Cuando Ren'Py llega al final de un bloque de etiqueta (por ejemplo, la línea "Y esta también."^{en} el ejemplo anterior), simplemente continúa con el siguiente bloque de etiqueta, por lo que en el ejemplo anterior las cuatro líneas se entregan en el orden en que están escritas, una tras otra, como si las declaraciones de etiqueta no estuvieran allí en absoluto. Sin embargo, usando los comandos `jump` y `call`, podemos movernos entre etiquetas de forma no lineal.

Salto (Jump)

El comando `jump` le indica a Ren'Py que salte inmediatamente a la etiqueta que se indica, ignorando todo el guión que hay entre la posición actual y esa etiqueta. Los saltos pueden ser hacia adelante o hacia atrás en el archivo, o incluso a un archivo de guión diferente en el mismo directorio del juego.

El comando `jump` consiste en la palabra `jump` seguida del nombre de la etiqueta a la que saltar:

```
jump some_label
```

Como la etiqueta puede estar en cualquier lugar, el comando `jump` nos permite escribir guiones en un orden no lineal, por ejemplo:

```
label start:
    "Esta línea de diálogo se dirá primero."
    jump stage_two
```

```
label stage_three:
    "Esta línea se dirá en tercer lugar."
    jump end
    "Esta línea nunca se ejecutará."
```

```
label stage_two:
    "Esta línea se dirá en segundo lugar."
    jump stage_three
```

```
label end:
    "Y esta línea será la última."
```

Ren'Py no puede ejecutar la línea del medio (^{Esta} "Esta línea nunca se ejecutará") porque es imposible avanzar línea por línea desde una etiqueta hasta esa línea de diálogo. El comando `jump end` se interpondrá y redirigirá a Ren'Py a la etiqueta `end` antes de que llegue a esa línea de diálogo.

Llamada y retorno (Call and Return)

El comando `call` instruye a Ren'Py para que vaya a ejecutar el código en otra etiqueta y luego regrese para continuar desde este punto.

El comando `call` consiste en la palabra `call`, seguida del nombre de la etiqueta a la que llamar:

```
call subroutine
```

En el otro extremo, el comando `return` le dice a Ren'Py que vuelva al punto desde donde se hizo la llamada:

```
label subroutine:
    "Hacer algunas cosas."
    return
```

Esto significa que si hay una parte de tu guión que quieres reutilizar en múltiples puntos, puedes usar `call` para visitar esa parte desde cualquier otro lugar de tu guión y regresar a esos diferentes lugares sin tener que conocer el nombre de cada una de sus etiquetas.

```
label start:
```

```
    "El profesor empieza a hablar monótonamente sobre Hamlet..."
    call boring
    "Creo que me perdí algo. Ahora está hablando de Rosencrantz y... alguien..."
    "Tom Stoppard? ¿Estaba en Hamlet?"
    call boring
    jump end
```

```
label boring:
```

```
    "No puedo mantener los ojos abiertos... qué aburrido..."
    "¡Ah! ¿Me he quedado dormido?"
    return
```

```
label end:
```

```
    "Vaya. Parece que la clase ha terminado. ¡Qué rápido!"
```

Esto producirá el siguiente diálogo:

- *.^{El}* profesor empieza a hablar monótonamente sobre Hamlet..."
- "No puedo mantener los ojos abiertos... qué aburrido..."
- "¡Ah! ¿Me he quedado dormido?"
- Creo que me perdí algo. Ahora está hablando de Rosencrantz y... alguien..."
- "Tom Stoppard? ¿Estaba en Hamlet?"
- "No puedo mantener los ojos abiertos... qué aburrido..."
- "¡Ah! ¿Me he quedado dormido?"
- "Vaya. Parece que la clase ha terminado. ¡Qué rápido!"

Observa que las líneas en cursiva (o resaltadas en el original) son las que se incluyen en la etiqueta `boring` a la que `call` salta, y después de las cuales regresa al punto del guión desde el que fue llamada.

Variables y estructuras de control

Menú (Menu) Código de ejemplo:

```
menu:
```

```
    "yes":
        "yes"
        jump yes
    "no":
        "no"
```

```

        jump no

label yes:
    "I agree."

label no:
    "I don't agree."

```

Las opciones `yes` y `no` son tus opciones; puedes añadir tantas como quieras. Los dos puntos (`:`) después de cada opción te permiten indicarle a Ren'Py qué sucede a continuación, como ir a otra página del guión o añadir texto.

Otros comandos de control comunes incluyen:

- `if` (si) – para ejecutar código condicionalmente.
- `call` (llamar) – para llamar a subrutinas (como se explicó anteriormente).
- `while` (mientras) – para crear bucles.
- `pass` (pasar) – para no hacer nada (útil como marcador de posición).

Estos comandos se utilizan normalmente dentro de bloques de Python (con el prefijo `$` o dentro de un bloque `python:`) para controlar la lógica del juego de forma más avanzada.

Tutorial de personalización de estilos

Por defecto, los juegos de Ren'Py se crean con un estilo utilizable pero relativamente genérico. El propósito de este tutorial es enseñarte cómo personalizar el sistema de estilos de Ren'Py para hacer tu juego más atractivo.

Antes de empezar, debemos señalar que el sistema de estilos es solo una forma de personalizar tu juego, y puede que no sea capaz de lograr todas las personalizaciones.

- Los **diseños (layouts)** definen el aspecto básico de las diversas pantallas del menú principal y del menú del juego, como qué botones y otros elementos mostrables están presentes. Hay varios diseños integrados, incluidos los basados en mapas de imagen, y es posible escribir los tuyos propios.
- Los **temas (themes)** manejan los cambios más grandes en el sistema de estilos, definiendo apariencias coherentes en muchos estilos.
- Una vez que hayas elegido tu diseño y tema, puedes seguir ajustando el aspecto de tu juego usando el sistema de estilos.

Este tutorial se divide en dos partes. La primera parte ofrece recetas preparadas para las personalizaciones de estilos más comunes. La segunda parte explica cómo personalizar cualquier estilo y propiedad de estilo en el sistema, utilizando las herramientas del inspector de estilos y el árbol de estilos.

Personalizaciones de estilos comunes

Personalizar cualquier estilo

Lo primero que hay que hacer al personalizar estilos es determinar qué estilo es el que realmente quieres personalizar. La forma más fácil de hacerlo es usando el inspector de estilos. También es posible usar la herramienta de jerarquía de estilos para determinar los estilos.

Inspector de estilos Para usar el inspector de estilos, coloca el mouse sobre cualquier elemento del juego y presiona **Mayús + I**. Si no sucede nada, establece `config.developer` en `True` y reinicia tu juego. De lo contrario, aparecerá una pantalla similar a la siguiente, que muestra los estilos y los elementos mostrables (*Displayables*) que hay debajo del mouse.

Inspector de estilos

```
- Window : mm_root (800x600)
- Window : mm_menu_frame (196x132)
- MultiBox : mm_menu_frame_box (184x120)
  - Button : mm_button[u'Iniciar juego'] (184x30)
  - Text : mm_button_text[u'Iniciar juego'] (125x26)
```

(haz clic para continuar)

(imagen)

Esta herramienta te permite identificar rápidamente el estilo de cualquier elemento visible en tu juego, para que puedas modificarlo con precisión. En la segunda parte del tutorial se explica cómo usar esta información para cambiar las propiedades de los estilos. Esta captura de pantalla se creó colocando el cursor sobre el botón **“Iniciar juego”** en el menú principal del juego de demostración y presionando **Mayús + I**.

Cada línea contiene los siguientes tres campos:

1. El tipo de elemento mostrable (*displayable*) que se está mostrando.
2. El nombre del estilo que está utilizando ese elemento mostrable.
3. El tamaño del elemento mostrable.

Las líneas están ordenadas según el orden en que se dibujan las cosas en la pantalla, de modo que la primera línea es la más alejada del usuario y la última línea es la más cercana. La sangría se usa para representar el anidamiento de los elementos mostrables. Dicho anidamiento puede afectar a la posición de los diversos elementos mostrables.

Jerarquía de estilos

Los estilos pueden heredar unos de otros. La naturaleza precisa de la herencia de estilos varía según los diseños y temas que se estén utilizando. Para obtener un registro completo de los estilos que conoce el sistema, presiona **Mayús + D** y elige **“Style Hierarchy”** (Jerarquía de estilos). Esto mostrará una lista de los estilos que conoce el sistema.

Los estilos que se puede esperar que existan en todos los juegos son:

- `style.default` (Todo) - La raíz de la jerarquía de estilos. Debe tener un valor para
- o `style.bar` (Barra) - Barras horizontales destinadas a transmitir información sin pe
- o `style.button` (Botón) - Botones de tamaño normal.
- o `style.button_text` (Texto) - Texto dentro de los botones.
- o `style.small_button` (Botón) - Botones más pequeños de lo normal.
- o `style.small_button_text` (Texto) - Texto dentro de botones más pequeños de lo nor
- o `style.centered_text` (Texto) - Texto mostrado por el personaje centrado predefinido
- o `style.centered_window` (Ventana) - Ventana que contiene el texto mostrado por el pe
- o `style.frame` (Marco) - Los marcos son ventanas destinadas a contener botones, etiqu
- o `style.menu_frame` (Marco) - Marcos que forman parte de los menús principales o de
- o `style.gm_root` (Ventana) - Una ventana vacía colocada en la raíz del menú del juego
- o `style.hbox` (Caja) - Una caja que distribuye sus hijos de izquierda a derecha.
- o `style.hyperlink_text` (Texto) - Texto con hipervínculo, usando la etiqueta de texto
- o `style.image_button` (Botón) - `ui.imagebuttons`.
- o `style.input_prompt` (Texto) - Texto de solicitud usado por `renpy.input`.
- o `style.input_text` (Entrada) - Campos de entrada producidos por `renpy.input` y `ui.inp`
- o `style.label` (Ventana) - Ventana que rodea el texto de una etiqueta. Las etiquetas
- o `style.label_text` (Texto) - El texto de una etiqueta.
- o `style.large_button` (Botón) - Un botón grande, como los usados para los espacios en
- o `style.large_button_text` (Texto) - Texto usado por botones grandes.
- o `style.menu` (Caja) - Caja que contiene un menú dentro del juego, como los que se m
- o `style.menu_caption` (Texto) - Texto de un título usado en un menú dentro del juego
- o `style.menu_choice` (Texto) - Texto de una opción de menú.
- o `style.menu_choice_chosen` (Texto) - Texto de una opción de menú que ha sido elegi
- o `style.menu_choice_button` (Botón) - Botón que contiene una opción de menú.
- o `style.menu_choice_chosen_button` (Botón) - Botón que contiene una opción de menú c
- o `style.menu_window` (Ventana) - Ventana que contiene un menú dentro del juego.
- o `style.mm_root` (Ventana) - Ventana vacía mostrada en la raíz del menú principal.
- o `style.prompt` (Ventana) - Una ventana que contiene texto de solicitud. El texto de
- o `style.prompt_text` (Texto) - Texto de solicitud.
- o `style.say_dialogue` (Texto) - Diálogo en la sentencia `say`.
- o `style.say_label` (Texto) - El nombre del personaje que habla en la sentencia `say`.
- o `style.say_thought` (Texto) - Diálogo hablado por el narrador predeterminado.
- o `style.scrollbar` (Barra) - Barras de desplazamiento horizontales que pueden o no es
- o `style.slider` (Barra) - Barras horizontales destinadas a ser ajustadas por el usuar
- o `style.vbar` (Barra) - Barras verticales destinadas a transmitir información sin pe
- o `style.vbox` (Caja) - Una caja que distribuye sus hijos de arriba a abajo.
- o `style.vscrollbar` (Barra) - Barras de desplazamiento verticales que pueden o no es
- o `style.vslider` (Barra) - Barras verticales destinadas a ser ajustadas por el usuar
- o `style.window` (Ventana) - Ventana que se usa para mostrar diálogos y otro texto de

layout.button_menu Invocar `layout.button_menu`, como hace la plantilla de juego predeterminada, realiza un par de cambios en la jerarquía de estilos predeterminada.

- `menu_choice` pasa a depender de `button_text`.
- `menu_choice_button` pasa a depender de `button`. Esto hace que las opciones de menú parezcan botones.

Indexación y herencia de estilos

Algunos estilos utilizados por el juego pueden estar indexados. Por ejemplo, el estilo `mm_button[u"Start Game"]` es el estilo `mm_button` indexado con la cadena `u"Start Game"`. La indexación se usa para personalizar específicamente un solo botón o etiqueta.

Según la herramienta de jerarquía de estilos, la jerarquía de herencia predeterminada para el estilo `mm_button` es:

1. `mm_button`
2. `button`
3. `default`

Cuando se indexa con `u"Start Game"`, se convierte en:

1. `mm_button[u"Start Game"]`
2. `mm_button`
3. `button[u"Start Game"]`
4. `button`
5. `default[u"Start Game"]`
6. `default`

Ren'Py buscará en los estilos en este orden hasta que encuentre el primer estilo en el que esté definido un valor para la propiedad.

Establecer propiedades de estilo

Cada elemento mostrable se encuentra en uno de dos roles: seleccionado o no seleccionado. El rol seleccionado se usa, por ejemplo, para indicar la página que se está mostrando o el valor actual de una preferencia. El rol no seleccionado se usa para todos los demás elementos mostrables.

Cada elemento mostrable se encuentra en uno de cuatro estados:

- `insensitive` (insensible), incapaz de responder a la entrada del usuario.
- `idle` (inactivo), capaz de responder a la entrada del usuario, pero sin foco.
- `hover` (sobre), capaz de responder a la entrada del usuario y con foco.
- `activate` (activado), elegido por el usuario (por ejemplo, un botón pulsado).

Los roles y estados corresponden a prefijos que se aplican a las propiedades de estilo. Los prefijos de rol son:

- (sin prefijo) - se establece tanto para roles seleccionados como no seleccionados.
- `"selected_"` - se establece solo para el rol seleccionado.

Los prefijos de estado son:

- (sin prefijo) - se establece para todos los estados.
- `insensitive_` - se establece para el estado insensible.
- `idle_` - se establece para el estado inactivo.
- `hover_` - se establece para los estados de hover y active.
- `.activate_` - se establece para el estado activado.

Para establecer una propiedad, se asigna un valor a la expresión Python:

```
style.<nombre del estilo>.<prefijo de rol><prefijo de estado><propiedad>
```

Por ejemplo:

```
init python:
    style.mm_button.background = "#f00"
    style.mm_button.hover_background = "#ff0"
    style.mm_button.selected_hover_background = "#00f"
    style.mm_button[u"Start Game"].background = "#0f0"
```

La primera línea establece el fondo de todos los botones del menú principal en rojo. La segunda cambia el fondo de los botones del menú principal al pasar el cursor por encima a amarillo. La tercera cambia el fondo de los botones del menú principal seleccionados y con el cursor encima a azul; esto realmente no hace nada, ya que los botones del menú principal nunca están seleccionados. Finalmente, la última cambia el estilo indexado para que tenga un fondo verde. Dado que el estilo indexado está más alto en el orden de herencia, tendrá prioridad sobre todos los demás estilos.

Observa que el orden de las declaraciones importa. El código:

```
init python:
    style.mm_button.hover_background = "#ff0"
    style.mm_button.background = "#f00"
```

Hará que los botones del menú principal al pasar el cursor tengan un fondo rojo. Esto se debe a que la segunda declaración, que establece todos los roles y estados, tiene prioridad sobre la primera. Generalmente, tiene sentido establecer primero las propiedades sin prefijo y luego las propiedades con prefijo.

Un script simple de Ren'Py

El siguiente es un script de Ren'Py simple pero completo. Los colores se añaden para facilitar la lectura y no forman parte del script en sí.

```
image bg whitehouse = "whitehouse.jpg"
image eileen happy = "eileen_happy.png"
image eileen upset = "eileen_upset.png"

define e = Character('Eileen')
```

```
label start:
    scene bg whitehouse
    show eileen happy
    e "Estoy frente a la Casa Blanca."
    show eileen upset
    e "Una vez quise hacer un recorrido por el Ala Oeste, pero hay que conocer a algu
    "Por alguna razón, parece realmente molesta por esto."
    e "Pensé en colarme, pero probablemente no sea una buena idea."
```

El comienzo de este script consiste en declaraciones de los recursos que se utilizarán a lo largo del juego. Primero, se definen tres imágenes:

```
image bg whitehouse = "whitehouse.jpg"
image eileen happy = "eileen_happy.png"
image eileen upset = "eileen_upset.png"
```

Estas definiciones crean lo que en terminología de Ren'Py se llaman *Displayables* (elementos mostrables), a los que luego se puede hacer referencia en otras declaraciones del script, como `show`, por su nombre. Vale la pena señalar que los nombres deben ser nombres de variable de Python válidos, que no pueden comenzar con números.

En la siguiente línea del bloque `init`, se define una variable de personaje (*Character*):

```
define e = Character('Eileen')
```

En este caso, la declaración está creando una variable, `e`, y asignándole el resultado de una llamada a la función `Character` con el argumento `'Eileen'`. En términos más simples, crea un acceso directo para usar al escribir líneas de diálogo en el script, de modo que no sea necesario escribir el nombre completo *Eileen* cada vez que ella dice algo.

La siguiente línea introduce la declaración `label`:

```
label start:
```

Una etiqueta proporciona un lugar al que otras declaraciones del script de Ren'Py pueden llamar o saltar. Por defecto, el menú principal de Ren'Py salta a la etiqueta `start` cuando comienzas un nuevo juego.

Después de la etiqueta `start` viene la parte del script donde se escribe el juego interactivo propiamente dicho. Primero, una declaración `scene`:

```
scene bg whitehouse
```

Esta declaración `scene` simplemente añade a la pantalla el *Displayable* previamente definido `bg whitehouse`.

A continuación, una declaración `show`:

```
show eileen happy
```

Esta declaración `show` añade a la pantalla el *Displayable* `eileen happy` previamente definido, encima de la imagen mostrada por la declaración `scene` anterior.

```
e "Estoy frente a la Casa Blanca."
```

Esta línea es un ejemplo de una declaración `say`, aunque notarás que no comienza con la palabra clave `say`. Dado que las declaraciones `say` constituyen la mayor parte de un guión de juego, utilizan la sintaxis más básica posible.

Aquí, la declaración comienza con la variable de Python `e` definida anteriormente, que se usa para indicar al intérprete de Ren'Py que la línea de diálogo debe mostrarse en pantalla con el nombre definido en la llamada a su función `Character` (es decir, 'Eileen'). A continuación, otra declaración `show`:

```
show eileen upset
```

Esta línea demuestra una de las ventajas sutiles de usar una declaración `show` en combinación con una etiqueta de *Displayable* de varias palabras. Recuerda que en el bloque `init` había definiciones para `eileen happy` y `eileen upset`. Cuando se ejecuta la declaración `show`, primero verifica si ya hay imágenes en la pantalla donde la primera palabra del nombre coincida con la primera palabra de la nueva imagen a mostrar. Si encuentra alguna, la imagen anterior se oculta y se reemplaza por la nueva.

En pocas palabras, la imagen `eileen happy` que ya está en la pantalla se oculta y se reemplaza por `eileen upset`, ya que ambas comienzan con `eileen`.

La siguiente es otra línea de diálogo de Eileen:

```
e "Una vez quise hacer un recorrido por el Ala Oeste, pero hay que conocer a alguien p
```

Esta línea está dividida en dos líneas físicas del script, pero se unirán al mostrarse en pantalla. Ten en cuenta que los saltos de línea del script no se trasladarán a la línea mostrada; si quieres forzar un salto de línea en pantalla, deberás usar un salto de línea explícito (`\n`).

A continuación, otra declaración `say`, en su forma más simple:

```
"Por alguna razón, parece realmente molesta por esto."
```

La declaración `say` más simple es solo una cadena entre comillas. Se mostrará en pantalla sin una etiqueta con el nombre del personaje, por lo que generalmente se usa como narración de los pensamientos del personaje protagonista (*POV*).

Finalmente, la última línea es más diálogo:

```
e "Pensé en colarme, pero probablemente no sea una buena idea."
```

Después de hacer clic en la última línea de diálogo en el juego, volverás al menú principal.

Declaraciones

Declaración Call

La declaración `call` se utiliza para transferir el control a la declaración con el nombre dado. También coloca el nombre de la declaración siguiente a esta en la pila de llamadas, lo que permite que la declaración `return` devuelva el control a la declaración siguiente a esta.

```
call_statement -> "call" name ( "from" name )?
call_statement -> "call" "expression" simple_expression ( "from" name )?
call_statement -> "call" name "(" arguments ")" ( "from" name )?
call_statement -> "call" "expression" simple_expression "pass" "(" arguments ")" ( "f
```

Si la palabra clave **expression** está presente, la expresión se evalúa y la cadena así calculada se usa como el nombre de la declaración a llamar. Si la palabra clave **expression** no está presente, el nombre de la declaración a llamar debe darse explícitamente.

Si la cláusula **from** opcional está presente, tiene el efecto de incluir una declaración **label** con el nombre dado como la declaración inmediatamente siguiente a la declaración **call**. Se requiere una etiqueta explícita aquí para asegurar que las partidas guardadas con pilas de retorno puedan volver al lugar adecuado cuando se carguen en un guión modificado. Las cláusulas **from** deben incluirse para todas las llamadas en los juegos lanzados.

Dado que las cláusulas **from** pueden distraer cuando un juego aún está en desarrollo, proporcionamos con Ren'Py un programa, llamado **add_from**, que añade cláusulas **from** a todas las llamadas simples en cualquier directorio de juego. Se puede encontrar en **tools/add_from**, aunque necesita ejecutarse desde el directorio base. La forma más fácil de hacerlo en Windows es ejecutando **tools/game_add_from.bat**. Debe ejecutarse antes del lanzamiento final de tu juego. Asegúrate de hacer una copia de seguridad de tus directorios de juego antes de ejecutar **add_from**. Ten en cuenta también que **add_from** produce archivos **.bak** para todos los archivos que puede modificar, así que elimínalos cuando estés satisfecho de que todo funcionó.

```
e "Primero, llamaremos a una subrutina."
call subroutine from _call_site_1
# ...
label subroutine:
    e "A continuación, volveremos de la subrutina."
    return
```

La declaración **call** puede tomar argumentos, que se procesan como se describe en PEP 3102. Si la declaración **return** devuelve un valor, ese valor se almacena en la variable **_return**, que tiene un alcance dinámico para cada contexto.

Cuando se usa una expresión **call** con una lista de argumentos, se debe insertar la palabra clave **pass** entre la expresión y la lista de argumentos. De lo contrario, la lista de argumentos se analizará como parte de la expresión, no como parte de la llamada.

Declaración Define

La declaración **define** hace que se evalúe su expresión y se asigne al nombre proporcionado. Si no está dentro de un bloque **init**, la declaración **define** se ejecutará automáticamente con prioridad **init 0**.

```
define_statement -> "define" name "=" python_expression
define e = Character("Eileen")
```

Declaración Hide

La declaración **hide** se utiliza para ocultar una imagen de la pantalla.

```
hide_statement -> "hide" image_name ( "onlayer" name )?
```

Una declaración **hide** opera en la capa proporcionada en la cláusula **onlayer** de la especificación de la imagen, por defecto **"master"** si no se ha proporcionado dicha cláusula.

Busca una imagen que comience con la etiqueta de imagen del nombre de la imagen y la elimina de esa capa.

Ten en cuenta que la declaración `hide` rara vez se usa en la práctica. `show` se puede usar por sí sola cuando un personaje cambia de emoción, mientras que `scene` se usa para eliminar todas las imágenes al final de una escena. `hide` solo es necesaria cuando un personaje se va antes del final de una escena.

Declaración If

La declaración `if` se utiliza para ejecutar condicionalmente un bloque de declaraciones.

```
if_statement -> "if" python_expression ":"
elif_clause  -> "elif" python_expression ":"
else_clause  -> "else" ":"
```

La declaración `if` es la única declaración que consta de más de una línea lógica en el mismo bloque. La declaración `if` inicial puede ir seguida de cero o más cláusulas `elif`, concluyendo con una cláusula `else` opcional. La expresión se evalúa para cada cláusula por turno, y si se evalúa como un valor verdadero, se ejecuta el bloque asociado a esa cláusula. Si ninguna expresión se evalúa como verdadera, se ejecuta el bloque asociado a la cláusula `else`. (Si existe una cláusula `else`, la ejecución continúa inmediatamente con la siguiente declaración). En cualquier caso, al final del bloque, el control se transfiere a la declaración siguiente a la declaración `if`.

```
if points >= 10:
    e "¡Enhorabuena! ¡Estás obteniendo el mejor final!"
elif points >= 5:
    e "Es el final bueno para ti."
else:
    e "Lo siento, estás a punto de obtener el final malo."
```

Declaración Image

La declaración `image` se utiliza para declarar imágenes a Ren'Py. Las declaraciones `image` pueden aparecer en bloques `init`, o se colocan implícitamente en un bloque `init` con prioridad 990 (cambiado en la versión 6.9.0).

```
image_statement -> "image" image_name "=" python_expression
```

Una declaración `image` vincula un nombre de imagen a un elemento mostrable (*displayable*). El elemento mostrable se calcula mediante la expresión Python proporcionada, pasando el resultado de la expresión a la función `im.Image` en modo flexible. Esto significa que si la asignación es una sola cadena, se interpreta como un nombre de archivo de imagen. Los elementos mostrables se pasan sin modificaciones. Una vez que se ha definido una imagen usando una declaración `image`, se puede usar con las declaraciones `scene`, `show` y `hide`.

Para obtener una lista completa de funciones que definen elementos mostrables, consulta la página de elementos mostrables (*displayables*).

```
image eileen happy = "eileen/happy.png"
image eileen upset = "eileen/upset.png"
```

Declaración Init

La declaración `init` se utiliza para ejecutar bloques de declaraciones de Ren'Py antes de que se ejecute el guión. Los bloques `init` se utilizan para definir imágenes y personajes, configurar estructuras de datos de juego inmutables y personalizar Ren'Py. El código dentro de los bloques `init` no debe interactuar con el usuario ni cambiar ninguna de las capas, por lo que no debe contener declaraciones `say`, `menu`, `scene`, `show` o `hide`, ni llamadas a funciones que puedan hacer estas cosas.

```
init_statement -> "init" (number)? ":"
```

Una declaración `init` se introduce con la palabra clave `init`, seguida de un número de prioridad opcional y dos puntos obligatorios. Si no se da la prioridad, su valor predeterminado es 0. Los números de prioridad deben estar en el rango de -999 a 999. Los números fuera de este rango están reservados para el código de Ren'Py.

El número de prioridad se utiliza para determinar cuándo se ejecuta el código dentro del bloque `init`. Los bloques `init` se ejecutan en orden de prioridad de menor a mayor. Dentro de un archivo, los bloques `init` con la misma prioridad se ejecutan en orden de arriba a abajo del archivo. El orden de evaluación de los bloques de prioridad con la misma prioridad entre archivos no está definido.

Todos los bloques `init` se ejecutan una vez, durante una fase `init` especial. Cuando el control llega al final de un bloque `init` durante la ejecución normal, la ejecución de ese bloque termina. Si se encuentra una declaración `init` durante la ejecución normal, el bloque `init` no se ejecuta. En su lugar, el control pasa a la siguiente declaración.

Declaración Jump

La declaración `jump` se utiliza para transferir el control a la declaración con el nombre dado.

```
jump_statement -> "jump" name
jump_statement -> "jump" "expression" simple_expression
```

Si la palabra clave `expression` está presente, la expresión se evalúa y la cadena así calculada se usa como el nombre de la declaración a la que saltar. Si la palabra clave `expression` no está presente, el nombre de la declaración a la que saltar debe darse explícitamente.

A diferencia de `call`, `jump` no coloca el destino en ninguna pila. Como resultado, no hay forma de volver al lugar desde donde se saltó.

```
label loop_start:
    e "¡Oh no! Parece que estamos atrapados en un bucle infinito."
    jump loop_start
```

Declaración Label

Las declaraciones `label` permiten asignar un nombre a un punto del programa. Existen únicamente para ser llamadas o saltadas, ya sea por el código del guión o por la configuración de Ren'Py.

```
label_statement -> "label" name ":"
label_statement -> "label" name "(" parameters ")" ":"
```

Una declaración `label` puede tener un bloque asociado. En ese caso, el control entra en el bloque cuando se alcanza la declaración `label` y continúa con la declaración siguiente a `label` cuando se llega al final del bloque.

La declaración `label` puede tomar una lista opcional de parámetros. Estos parámetros se procesan como se describe en PEP 3102, con dos excepciones:

- Los valores de los parámetros predeterminados se evalúan en el momento de la llamada.
- Las variables tienen un alcance dinámico, en lugar de léxico.

Dentro de una etiqueta llamada, las variables pueden declararse dinámicas usando la función `renpy.dynamic`:

```
Function: renpy.dynamic (*vars):
```

Esto declara una variable como de alcance dinámico para la llamada actual de Ren'Py. La primera vez que se llama a `renpy.dynamic` en la llamada actual, se almacenan los valores de las variables nombradas en las cadenas proporcionadas. Cuando se regresa de la llamada actual, las variables recuperan los valores que tenían en el momento en que se llamó a `renpy.dynamic`. Si las variables no están definidas cuando se llama a `renpy.dynamic`, no estarán definidas después de que la llamada actual retorne. Si se llama a `renpy.dynamic` dos veces para la misma variable en una misma llamada, no tiene efecto la segunda vez.

Declaración Menu

Los menús se utilizan para presentar al usuario una lista de opciones entre las que puede elegir. En una novela visual, los menús son el principal medio por el cual el usuario puede influir en la historia.

```
menu_statement -> "menu" ( name )? ":"
```

Una declaración `menu` se introduce con la palabra clave `menu`, un nombre opcional y dos puntos. Si se proporciona el nombre, se trata como una etiqueta para esta declaración `menu`, como si la declaración `menu` estuviera precedida por una declaración `label`.

Una declaración `menu` debe tener un bloque asociado. Este es un bloque de elementos de menú que debe contener uno o más elementos de menú. Hay varios tipos de elementos de menú que se pueden incluir en un bloque de elementos de menú.

```
caption_menuitem -> string
```

El primer tipo de elemento de menú es una cadena. Esta cadena se coloca en un menú como un título que no se puede seleccionar. En general, los títulos se utilizan para indicar para qué sirve el menú, especialmente cuando no queda claro por las opciones.

```
choice_menuitem -> string ( "if" python_expression )? ":"
```


El segundo tipo de elemento de menú ofrece una opción que el usuario puede elegir. Cada opción debe tener un bloque de declaraciones de Ren'Py asociado. Si el usuario selecciona la opción, se ejecuta el bloque de declaraciones asociado a esa opción. Una opción también puede tener una cláusula `if` opcional que incluye una expresión de Python. Esta cláusula proporciona una condición que debe cumplirse para que la opción se presente al usuario. Los dos puntos finales se utilizan para indicar que este elemento de menú es una opción.

```
set_menuitem -> "set" variable_name
```

El tercer tipo de elemento de menú proporciona una variable en la que almacenar la lista de opciones que el usuario ha elegido, y evita que el usuario elija la misma opción si el menú se visita varias veces. Esta variable debe definirse antes de la declaración `menu` y debe ser una lista vacía, `[]`. Cuando el usuario elige una opción del menú, esa opción se almacena en la lista. Cuando el juego llega a otra declaración `menu` que usa el mismo nombre de variable en una cláusula `set` (o llega al mismo menú de nuevo), las opciones que coinciden con los elementos de la lista no se mostrarán.

```
with_menuitem -> "with" simple_expression
```

El último tipo de elemento de menú es una cláusula `with`. Consulta la sección sobre Transiciones para obtener más información sobre las cláusulas `with`.

```
menu what_to_do:
    "¿Qué deberíamos hacer hoy?"
    "Ir al cine.":
        "Fuimos al cine."
    "Ir de compras.":
        "Fuimos de compras y las chicas compraron trajes de baño."
        $ have_swimsuits = True
    "Ir a la playa." if have_swimsuits:
        "Fuimos a la playa juntos. Pude ver a las chicas con sus nuevos trajes de baño."
```

Detalles Cuando se va a mostrar un menú al usuario, lo primero que ocurre es que se construye una lista de títulos y opciones a partir de los elementos de menú asociados al menú. Cada una de las opciones que tiene una expresión asociada evalúa esa expresión, y si no es verdadera, esa opción se elimina de la lista. Si no sobrevive ninguna opción a este proceso, el menú no se muestra y la ejecución continúa con la siguiente declaración. De lo contrario, se llama a la función `menu` con la lista de opciones, se muestra el menú al usuario y devuelve una opción elegida. La ejecución continúa con el bloque correspondiente a la opción elegida. Si la ejecución llega al final de ese bloque, continúa con la declaración siguiente al menú.

Declaración `Pause`

La declaración `pause` hace que Ren'Py se detenga hasta que se haga clic con el ratón. Si se proporciona la expresión opcional, se evaluará como un número y la pausa terminará automáticamente una vez transcurrido ese número de segundos.

```
pause_statement -> "pause" ( simple_expression )?
```

Declaración Play

La declaración `play` se utiliza para reproducir sonido y música. Si un archivo se está reproduciendo actualmente, se interrumpe y se reemplaza con el nuevo archivo.

```
play_statement -> "play" name simple_expression
  ( "fadeout" simple_expression )?
  ( "fadein" simple_expression )?
  ( "loop" | "noloop" )?
  ( "if_changed" )?
```

Se espera que la primera `simple_expression` en la declaración `play` se evalúe como una cadena que contiene un nombre de archivo, o como una lista de nombres de archivo para reproducir. Se espera que el nombre sea el nombre de un canal (normalmente, "sound", "music" o "movie"). El archivo o lista de archivos se reproduce usando `renpy.music.play`.

Las otras cláusulas son todas opcionales. `fadeout` proporciona el tiempo de desvanecimiento para la música que se está reproduciendo actualmente, en segundos, mientras que `fadein` proporciona el tiempo que tarda en desvanecerse la nueva música. `loop` hace que la música se repita en bucle, mientras que `noloop` fuerza que no se repita. `if_changed` hace que la música solo cambie si no es la música que se está reproduciendo actualmente.

```
play music "mozart.ogg"
play sound "woof.ogg"
"Probemos algo más complicado."
play music [ "a.ogg", "b.ogg" ] fadeout 1.0 fadein 1.0
```

Declaración Pass

La declaración `pass` no realiza ninguna acción. Existe porque los bloques de declaraciones de Ren'Py requieren al menos una declaración en ellos, y no siempre es sensato realizar una acción en esos bloques.

```
pass_statement -> "pass"
menu:
    "¿Debería ir al cine?"
    "Sí":
        call go_see_movie
    "No":
        pass
"Ahora se acerca la hora de cenar y estoy muerto de hambre."
```

Declaración Python

La declaración `python` permite ejecutar código Python en un guión de Ren'Py. Esto permite usar código Python para declarar cosas a Ren'Py, invocar gran parte de la funcionalidad de Ren'Py y almacenar datos en variables a las que puede acceder el código del usuario. Hay dos formas de la declaración `python`:

```
python_statement -> "$" python_code
python_statement -> "python" ( "hide" )? ":"
```

La primera forma de una declaración **python** consiste en un signo de dólar (\$) seguido de código Python que se extiende hasta el final de la línea. Esta forma se utiliza para ejecutar una sola declaración de Python.

Una segunda forma consiste en la palabra clave **python**, opcionalmente la palabra clave **hide**, y dos puntos. Esto se utiliza para ejecutar un bloque de código Python, proporcionado después de la declaración. Normalmente, el código Python se ejecuta en un espacio de nombres global del guión, pero si se proporciona la palabra clave **hide**, se crea un nuevo espacio de nombres para este bloque (se puede acceder al espacio de nombres global del guión desde el bloque, pero no se puede asignar a él).

```
$ score += 1
python:
    ui.text("Este es un texto en la pantalla.")
    ui.saybehavior()
    ui.interact()
```

Declaración Init Python. Por conveniencia, hemos creado la declaración **init python**. Esta declaración combina una declaración **init** y una declaración **python** en una sola declaración, para reducir la indentación requerida en archivos con mucho Python.

```
init python_statement -> "init" ( number )? "python" ( "hide" )? ":"
```

Declaración Queue

La declaración **queue** se utiliza para encolar archivos de audio. Se reproducirán cuando el canal termine de reproducir el archivo actual.

```
queue_statement -> "queue" name simple_expression ( "loop" | "nolop" )?
```

Se espera que el nombre sea el nombre de un canal, mientras que se espera que **simple_expression** se evalúe como una cadena que contiene un nombre de archivo, o como una lista de nombres de archivo para encolar. Los archivos se encolan usando **renpy.music.queue**. **loop** hace que la música encolada se repita en bucle, mientras que **nolop** hace que se reproduzca solo una vez.

```
queue sound "woof.ogg"
queue music [ "a.ogg", "b.ogg" ]
```

Declaración Return

La declaración **return** extrae la ubicación superior de la pila de llamadas y transfiere el control a ella. Si la pila de llamadas está vacía, la declaración **return** realiza un reinicio completo de Ren'Py.

```
return_statement -> "return"
return_statement -> "return" expression
```

Si se proporciona la expresión opcional a **return**, se evalúa y su resultado se almacena en la variable **_return**. Esta variable tiene un alcance dinámico para cada contexto.

Declaración Say

La declaración **say** se utiliza para presentar texto al usuario, en forma de diálogo o pensamientos. Dado que la mayor parte del contenido de un guión será diálogo o pensamientos, es importante que la declaración **say** sea lo más conveniente posible. Por ello, la declaración **say** es la única declaración que no se delimita con una palabra clave u otro tipo de delimitador. En su lugar, consiste en una cadena, con una **simple_expression** opcional antes para designar quién habla, y una cláusula **with** opcional después para especificar una transición.

```
say_statement -> ( simple_expression )? string ( "with" simple_expression )?
```

Hay dos formas de la declaración **say**, dependiendo de si se proporciona la expresión simple. La forma de un solo argumento consiste en una sola cadena (con o sin la cláusula **with** opcional). Esta forma hace que la cadena se muestre al usuario sin ninguna etiqueta que indique quién la dice. Convencionalmente, se usa para indicar los pensamientos del personaje protagonista o la narración.

```
"Me moví a mi izquierda, y ella se movió a su derecha."
```

```
"Así que seguíamos bloqueándonos el camino el uno al otro."
```

```
"Entonces me moví a mi derecha, y al mismo tiempo ella se movió a su izquierda."
```

```
"Podríamos estar así todo el día."
```

La forma de dos argumentos de la declaración **say** consiste en una **simple_expression**, una cadena y, opcionalmente, una cláusula **with**. Esta forma de la declaración se utiliza para indicar diálogo. Se espera que el primer argumento sea un objeto (normalmente un objeto **Character** o **DynamicCharacter**) que sepa cómo mostrar el diálogo al usuario. La cadena se pasa entonces a ese objeto, que es responsable de mostrarla al usuario.

La **simple_expression** también puede ser una cadena, en lugar de un objeto. Las cadenas se utilizan directamente como el nombre del personaje.

```
"Girl" "Hola, me llamo Eileen."
```

```
e "A partir de hoy, viviré aquí."
```

Detalles La declaración **say** de dos argumentos primero evalúa la expresión simple proporcionada. Luego intenta llamar a ese valor (el valor **who**) con la cadena que da la línea de diálogo (la cadena **what**). Si puede hacerlo, ha terminado, ya que el objeto que se llama es responsable de interactuar con el usuario.

Si no puede llamar al valor de la expresión, entonces copia el objeto de personaje **name_only**, proporcionando la cadena dada como un nuevo nombre de personaje, y luego usa ese objeto para decir el diálogo. (Esto lo hacen las funciones **say** y **predict_say** que se encuentran en el **store**. Cambiar estas funciones puede cambiar este comportamiento).

La forma de un solo argumento de la declaración simplemente llama a la función especial (u objeto) **narrator** con la cadena que se va a mostrar. Esta función es responsable de mostrar la cadena al usuario. Los objetos **Character** y **DynamicCharacter** son adecuados para usar como **narrator**.

La cláusula **with** se utiliza para especificar una transición; consulta la sección sobre la declaración **With** y sus cláusulas para obtener más detalles.

Declaración Scene

La declaración **scene** limpia una capa eliminando todas las imágenes de ella. Luego puede mostrar una imagen proporcionada al usuario. Esto la hace apropiada para cambiar el fondo de una escena.

```
scene_statement -> "scene" ("onlayer" name)? (...)?
```

La declaración **scene** primero elimina todas las imágenes de una capa, usando por defecto la capa **"master"** si no se especifica otra. Si hay argumentos adicionales, se manejan como si se hubiera proporcionado una declaración **show**. Por defecto, no se añade ningún fondo a la pantalla, por lo que recomendamos que cada guión comience con una declaración **scene** que muestre un fondo a pantalla completa al usuario.

Declaración Show

La declaración **show** se utiliza para añadir una imagen a una capa. La imagen debe haber sido definida previamente con la declaración **image**.

```
show_statement -> "show" image_name
  ( "at" transform_list )?
  ( "as" name )?
  ( "behind" name_list )?
  ( "onlayer" name )?
  ( "with" simple_expression )?
```

Al añadir una imagen, la declaración **show** primero comprueba si ya existe en la capa una imagen con la misma etiqueta (*tag*) (por defecto, la primera parte del nombre de la imagen). Si es así, esa imagen se reemplaza sin cambiar el orden. Esto significa que rara vez es necesario ocultar imágenes.

La declaración **show** admite varias cláusulas opcionales:

- La cláusula **at** toma una lista separada por comas de posiciones y movimientos. Estos se crean usando las funciones de posición y movimiento.
- La cláusula **as** especifica la etiqueta de imagen directamente. Esto permite que la misma imagen se muestre dos veces en la pantalla.
- La cláusula **behind** toma una lista separada por comas de etiquetas de imagen. La imagen se mostrará detrás de todas las imágenes con esa etiqueta.
- La cláusula **onlayer** especifica la capa en la que se mostrará la imagen.
- La cláusula **with** especifica una transición que ocurre cuando se muestra esta imagen. Consulta la declaración **with** para más detalles.

Cuando se muestra una imagen, Ren'Py comprueba si había una imagen anterior con esa etiqueta y si esa imagen usaba una transformación. Si es así, Ren'Py hace dos cosas:

1. Si la nueva imagen no es una transformación, la envuelve en una transformación.
2. La transformación se inicializa con las propiedades de la transformación anterior.

Esto generalmente tiene el efecto de recordar la posición de las imágenes mostradas en la pantalla. En algunos casos, este efecto de memoria puede anular una posición codificada en una imagen. En ese caso, la imagen debe ocultarse y mostrarse de nuevo.

```
scene living_room
show eileen happy at left
show golden glow as halo at left behind eileen
e "Ahora mismo me siento feliz."
show eileen upset at left
show darkness as halo at left behind eileen
e "Pero a veces, puedo enfadarme sin una buena razón."
```

La declaración `show` también se puede usar para mostrar texto, utilizando el elemento mostrable `ParameterizedText`. El elemento mostrable de texto está definido por defecto y usa el estilo `centered_text`:

```
show text "Texto centrado"
```

Existe una segunda forma de la declaración `show` que toma una expresión que devuelve un elemento mostrable. Esto se puede usar para mostrar un elemento mostrable directamente. La etiqueta del elemento mostrable no está definida, a menos que se proporcione con la cláusula `as`.

```
show_statement -> "show" "expression" simple_expression
    ( "as" name )?
    ( "onlayer" name )?
    ( "at" transform_list )?
    ( "behind" name_list )?
    ( "with" simple_expression )?

show expression "myimage.png"
```

Declaración Stop

La declaración `stop` se utiliza para detener la reproducción de sonido y música.

```
stop_statement -> "stop" name
    ( "fadeout" simple_expression )?
```

Se espera que el nombre sea el nombre de un canal de audio. El sonido o la música se detienen usando `renpy.music.stop`. La cláusula opcional `fadeout` espera un tiempo en segundos y hará que la música tarde ese tiempo en desvanecerse.

```
stop sound
stop music fadeout 1.0
```

Declaración Window

La declaración `window` se utiliza para controlar si se muestra una ventana cuando un personaje no está hablando (por ejemplo, durante transiciones y pausas).

```
window_statement -> "window" ( "show" | "hide" ) ( simple_expression )?
```

La instrucción `window show` hace que se muestre la ventana, mientras que `window hide` la oculta. Si se proporciona la `simple_expression` opcional, es una transición que se usa para mostrar y ocultar la ventana. Si no se proporciona, se usan por defecto `config.window_show_transition` y `config.window_hide_transition`. Proporcionar `None` como transición evita que ocurra.

La ventana en sí se muestra llamando a `config.empty_window`. Por defecto, el narrador dice una cadena vacía.

```
show bg washington
show eileen happy
with dissolve
window show dissolve
"Puedo decir cosas..."
show eileen happy at right
with move
"... y moverme, mientras la ventana permanece mostrada."
window hide dissolve
```

Declaración With y cláusulas With

La declaración `with` y las cláusulas `with` se utilizan para mostrar transiciones al usuario. Estas transiciones siempre van desde la última pantalla mostrada al usuario hasta la pantalla actual. Al final de una declaración `with`, la última pantalla mostrada al usuario se establece en la pantalla actual. La última pantalla mostrada al usuario también puede actualizarse mediante declaraciones `say` o `menu`, así como mediante código Python.

La declaración `with` tiene la forma:

```
with_statement -> "with" simple_expression
```

Se espera que la `simple_expression` se evalúe como una función de transición. Si se evalúa como el valor `None`, la última pantalla mostrada al usuario se actualiza a la pantalla actual, sin realizar una transición. Esto es útil para evitar que los elementos de interfaz transitorios (como una declaración `say` anterior) participen en una transición.

Por conveniencia, varias declaraciones admiten cláusulas `with`. En el caso de las declaraciones `scene`, `show` y `hide`, la cláusula `with` equivale a colocar una declaración `with None` antes de la declaración `scene`, `show` o `hide`, y una declaración `with transition` después. Por ejemplo, la declaración:

```
show eileen happy with dissolve
```

es equivalente a:

```
with None
show eileen happy
with dissolve
```

Este comportamiento puede provocar efectos secundarios no deseados. El código:

```
show bg whitehouse with dissolve
show eileen happy with dissolve
```

causará dos transiciones. Para asegurar que solo ocurra una transición, se debe escribir:

```
with None
show bg whitehouse
show eileen happy
with dissolve
```

Las cláusulas **with** también se pueden aplicar a declaraciones **say** y **menu**. En este caso, la transición ocurre cuando el diálogo o el menú se muestra por primera vez al usuario. Para funciones de transición predefinidas que se pueden usar en cualquier guión, consulta *Transiciones predefinidas*. Para funciones que devuelven funciones de transición, consulta *Constructores de transiciones*.

Declaración While

La declaración **while** se utiliza para ejecutar un bloque de declaraciones de Ren'Py mientras se mantenga una condición verdadera.

```
while_statement -> "while" python_expression ":"
```

Cuando se ejecuta una declaración **while**, se evalúa la **python_expression**. Si se evalúa como verdadera, el control se transfiere a la primera declaración del bloque asociado a esta declaración **while**. Si es falsa, el control se envía a la declaración siguiente al **while**. Cuando el control llega al final del bloque asociado a la declaración **while**, vuelve a la declaración **while**. Esto permite que **while** verifique la condición de nuevo y evalúe el bloque si la condición sigue siendo verdadera.

```
while not endgame:
    "Es de mañana. Hora de levantarse y aprovechar el día."
    call morning
    call afternoon
    call evening
    "Bueno, hora de terminar la noche."
"Ahora es hora de despertar y enfrentar el final del juego."
```

El Lenguaje de Animación y Transformación (ATL)

El Lenguaje de Animación y Transformación (ATL) proporciona una forma de alto nivel de elegir un elemento mostrable para mostrar, posicionarlo en la pantalla y aplicar transformaciones como rotación, zoom y modificación de alfa. Estos se pueden cambiar con el tiempo y en respuesta a eventos.

Declaraciones de guión de Ren'Py

Los bloques de código ATL se pueden introducir de tres formas.

Declaración Transform

La primera forma es usando la declaración `transform`. La declaración `transform` crea una función de posición y movimiento que se puede proporcionar a la cláusula `at` de una declaración `show` o `scene`. La sintaxis de la declaración `transform` es:

```
transform_statement -> "transform" name "(" parameters ")" ":"
    atl_block
```

La declaración `transform` debe ejecutarse en tiempo de `init`. Si se encuentra fuera de un bloque `init`, se coloca automáticamente dentro de un bloque `init` con prioridad 0.

La transformación puede tener una lista de parámetros, que deben proporcionarse cuando se llama. La sintaxis es la misma que para las etiquetas, excepto que no permitimos un número variable de argumentos.

Se debe proporcionar un nombre, que debe ser un identificador de Python. La transformación creada a partir del bloque ATL se vincula a este nombre.

Declaración Image con bloque ATL

La segunda forma de usar ATL es como parte de una declaración `image` con bloque ATL. Esto vincula un nombre de imagen a la transformación dada. Como no hay forma de proporcionar parámetros a esta transformación, solo es útil si la transformación define una animación. La sintaxis para una declaración `image` con bloque ATL es:

```
image_statement -> "image" image_name ":"
    atl_block
```

Declaraciones Scene y Show con bloque ATL

La forma final de usar ATL es como parte de una declaración `scene` o `show`. Esto envuelve la imagen que se está mostrando dentro de una transformación ATL.

```
atl_scene_statement -> scene_statement ":"
    atl_block
atl_show_statement -> show_statement ":"
    atl_block
```

Sintaxis y semántica de ATL

Un bloque ATL consiste en una o más líneas lógicas, todas con la misma sangría, y con sangría relativa a la declaración que contiene el bloque. Cada línea lógica en un bloque ATL debe contener una o más declaraciones ATL.

Hay dos tipos de declaraciones ATL: simples y complejas. Las declaraciones simples no toman un bloque ATL. Una sola línea lógica puede contener una o más declaraciones ATL, separadas por comas. Una declaración compleja contiene un bloque y debe estar en su propia línea. La primera línea de una declaración compleja siempre termina con dos puntos (":").

Por defecto, las declaraciones en un bloque se ejecutan en el orden en que aparecen, comenzando con la primera declaración del bloque. La ejecución termina cuando se llega

al final del bloque. Las declaraciones de tiempo cambian esto, como se describe en la sección correspondiente a continuación.

La ejecución de un bloque termina cuando todas las declaraciones del bloque han terminado.

Si una declaración ATL requiere la evaluación de una expresión, dicha evaluación ocurre cuando la transformación se añade por primera vez a la lista de escenas (por ejemplo, al usar una declaración `show` o una función de interfaz de usuario).

Declaraciones ATL

Las siguientes son las declaraciones ATL.

Declaración de interpolación

```
interpolation_statement ->
    ( warper simple_expression | "warp" simple_expression simple_expression )?
    ( property simple_expression ( "knot" simple_expression )*
      | "clockwise"
      | "counterclockwise"
      | "circles" simple_expression
      | simple_expression )
```

La declaración de interpolación es la forma principal en que ATL controla las transformaciones.

La primera parte de la declaración de interpolación se usa para seleccionar una función que deforma el tiempo de la interpolación (es decir, una función de tiempo lineal a tiempo no lineal). Esto se puede hacer dando el nombre de un *warper* registrado en ATL, o dando la palabra clave `warp` seguida de una expresión que proporciona una función. En cualquier caso, va seguido de un número que indica el número de segundos que debe durar la interpolación.

Si no se proporciona ninguna función de deformación, la interpolación se ejecuta durante 0 segundos, utilizando la función `pause`.

El *warper* y la duración se utilizan para calcular una fracción de finalización. Esto se hace dividiendo el tiempo transcurrido de la interpolación por la duración de la interpolación. Esto se limita a la duración y luego se pasa al *warper*. El resultado devuelto por el *warper* es la fracción de finalización.

La declaración de interpolación puede contener varias otras cláusulas. Cuando hay una propiedad y un valor presentes, el valor es el valor que la propiedad obtendrá al final de la declaración. El valor se puede obtener de varias formas:

- Si el valor va seguido de uno o dos *knots* (nudos), se utiliza un movimiento spline. El punto de inicio es el valor de la propiedad al inicio de la interpolación, el punto final es el valor de la propiedad, y los nudos se utilizan para controlar la spline.
- Si la declaración de interpolación contiene una cláusula `clockwise` (en el sentido de las agujas del reloj) o `counterclockwise` (en sentido contrario), se utiliza un movimiento circular, como se describe a continuación.
- De lo contrario, el valor se interpola linealmente entre las posiciones inicial y final, utilizando la fracción de finalización.

Las cláusulas `clockwise`, `counterclockwise` y `circles` se utilizan en el movimiento circular, descrito a continuación.

Si hay una expresión simple presente, debe evaluarse como una transformación con una sola declaración de interpolación, sin *warper*, splines ni movimiento circular. Las propiedades de la transformación se procesan como si estuvieran incluidas en esta declaración.

Algunas interpolaciones de ejemplo:

`show logo base:`

```
# Mostrar el logotipo en la parte superior derecha de la pantalla.
xalign 0.0 yalign 1.0
# Tarda 1.0 segundos en mover las cosas de vuelta a la izquierda.
linear 1.0 xalign 0.0
# Tarda 1.0 segundos en mover las cosas a la ubicación especificada
# en la transformación truecenter. Usa el warper ease para hacerlo.
ease 1.0 truecenter
# Solo hacer una pausa de un segundo.
pause 1.0
# Establecer la ubicación alrededor de la cual girar.
alignaround (.5, .5)
# Usar movimiento circular para llevarnos en espiral hacia la parte superior de
# la pantalla. Tarda 2 segundos en hacerlo.
linear 2.0 yalign 0.0 clockwise circles 3
# Usar un movimiento spline para movernos por la pantalla.
linear 2.0 align (0.5, 1.0) knot (0.0, .33) knot (1.0, .66)
```

Declaración de tiempo

```
time_statement -> "time" simple_expression
```

La declaración `time` es una declaración de control simple. Contiene una sola `simple_expression`, que se evalúa para dar un tiempo, expresado en segundos desde el inicio de la ejecución del bloque contenedor.

Cuando se alcanza el tiempo dado en la declaración, comienza a ejecutarse la siguiente declaración. Esta transferencia de control ocurre incluso si una declaración anterior aún se está ejecutando, y hace que cualquier declaración anterior termine inmediatamente.

Las declaraciones `time` están implícitamente precedidas por una declaración `pause` con un tiempo infinito. Esto significa que si el control llegara a la declaración `time`, espera hasta que la declaración `time` tome el control.

Cuando hay múltiples declaraciones `time` en un bloque, deben aumentar estrictamente en orden.

`image backgrounds:`

```
"bg band"
time 2.0
"bg whitehouse"
time 4.0
"bg washington"
```

Declaración de expresión

`expression_statement -> simple_expression ("with" simple_expression)?`

Una declaración de expresión es una declaración simple que comienza con una expresión simple. Luego contiene una cláusula `with` opcional, con una segunda expresión simple.

La primera expresión simple puede evaluarse como una transformación (definida con la declaración `transform`) o como un elemento mostrable. Si es una transformación, se ejecuta esa transformación. Las cláusulas `with` se ignoran cuando se proporciona una transformación.

La primera expresión simple puede evaluarse como un número entero o de punto flotante. En ese caso, se toma como un número de segundos para pausar la ejecución.

De lo contrario, la expresión se interpreta como un elemento mostrable. Este elemento mostrable reemplaza al hijo de la transformación cuando se ejecuta esta cláusula, lo que la hace útil para la animación. Si hay una cláusula `with` presente, la segunda expresión se evalúa como una transición, y la transición se aplica a los elementos mostrables antiguos y nuevos.

```
image atl example:
    # Mostrar logo_base.png
    "logo_base.png"
    # Pausa de 1.0 segundos.
    1.0
    # Mostrar logo_bw.png, con una disolución.
    "logo_bw.png" with Dissolve(0.5, alpha=True)
    # Ejecutar la transformación move_right.
    move_right
```

Declaración Pass

`pass_statement -> "pass"`

La declaración `pass` es una declaración simple que no hace que ocurra nada. Esto se puede usar cuando se desea separar declaraciones, como cuando hay dos conjuntos de declaraciones de elección que de otro modo estarían consecutivas.

Declaración Repeat

`repeat_statement -> "repeat" (simple_expression)?`

La declaración `repeat` es una declaración simple que hace que el bloque que la contiene reanude la ejecución desde el principio. Si la expresión está presente, se evalúa para dar un número entero de veces que se ejecutará el bloque (por lo que un bloque que termina con `repeat 2` se ejecutará como máximo dos veces).

La declaración `repeat` debe ser la última declaración en un bloque.

```
show logo base:
    xalign 0.0
    linear 1.0 xalign 1.0
    linear 1.0 xalign 0.0
    repeat
```

Declaración Block

```
block_statement -> "block" ":"
    atl_block
```

La declaración `block` es una declaración compleja que contiene un bloque de código ATL. Esto se puede usar para agrupar declaraciones que se repetirán.

```
label start:
    show logo base:
        alpha 0.0 xalign 0.0 yalign 0.0
        linear 1.0 alpha 1.0
        block:
            linear 1.0 xalign 1.0
            linear 1.0 xalign 0.0
        repeat
```

Declaración Choice

```
choice_statement -> "choice" (simple_expression)? ":"
    atl_block
```

La declaración `choice` es una declaración compleja que define una de un conjunto de opciones potenciales. Ren'Py elegirá una de las opciones del conjunto y ejecutará el bloque ATL asociado, y luego continuará la ejecución después de la última opción del conjunto de opciones.

Las declaraciones `choice` se agrupan de forma codiciosa en un conjunto de opciones cuando aparecen más de una declaración `choice` consecutivamente en un bloque. Si se proporciona la `simple_expression`, es un peso de punto flotante dado a ese bloque; de lo contrario, se asume 1.0.

```
image eileen random:
    choice:
        "eileen happy"
    choice:
        "eileen vhappy"
    choice:
        "eileen concerned"
    pause 1.0
    repeat
```

Declaración Parallel

La declaración `parallel` se utiliza para definir un conjunto de bloques ATL para ejecutar en paralelo.

```
parallel_statement -> "parallel" ":"
    atl_block
```

Las declaraciones `parallel` se agrupan de forma codiciosa en un conjunto paralelo cuando aparecen más de una declaración `parallel` consecutivamente en un bloque. Los bloques de todas las declaraciones `parallel` se ejecutan simultáneamente. La declaración `parallel` termina cuando el último bloque termina.

Los bloques dentro de un conjunto deben ser independientes entre sí y manipular propiedades diferentes. Cuando dos bloques cambian la misma propiedad, el resultado no está definido.

```
show logo base:
```

```
  parallel:
    xalign 0.0
    linear 1.3 xalign 1.0
    linear 1.3 xalign 0.0
    repeat
  parallel:
    yalign 0.0
    linear 1.6 yalign 1.0
    linear 1.6 yalign 0.0
    repeat
```

Declaración Event

```
event_prod_statement -> "event" name
```

La declaración `event` es una declaración simple que produce un evento con el nombre dado.

Cuando se produce un evento dentro de un bloque, se comprueba si existe un manejador de eventos para el nombre dado. Si existe, el control se transfiere al manejador de eventos. De lo contrario, el evento se propaga a cualquier manejador de eventos contenedor.

Declaración On

La declaración `On` es una declaración compleja que define un manejador de eventos. Las declaraciones `On` se agrupan de forma codiciosa en una sola declaración.

```
on_statement -> "on" name ":"
  atl_block
```

La declaración `on` se utiliza para manejar eventos. Cuando se maneja un evento, el manejo de cualquier otro evento termina y el manejo del nuevo evento comienza inmediatamente. Cuando un manejador de eventos termina sin que ocurra otro evento, se produce el evento `"default"` (a menos que ya estuviéramos manejando el evento `"default"`).

La ejecución de la declaración `on` nunca terminará de forma natural (pero puede terminar mediante la declaración `time` o un manejador de eventos contenedor).

```
show logo base:
```

```
  on show:
    alpha 0.0
    linear .5 alpha 1.0
  on hide:
    linear .5 alpha 0.0
```

Declaración Contains

La declaración `contains` establece el elemento mostrable contenido por esta transformación ATL (el hijo de la transformación). Hay dos variantes de la declaración `contains`.

Contains Expression. La variante `contains expression` toma una expresión y la establece como el hijo de la transformación. Esto es útil cuando una transformación ATL desea contener, en lugar de incluir, una segunda transformación ATL.

```
contains_statement -> "contains" expression
```

```
transform an_animation:
```

```
    "1.png"
```

```
    pause 2
```

```
    "2.png"
```

```
    pause 2
```

```
    repeat
```

```
image move_an_animation:
```

```
    contains an_animation
```

```
    # Si no usáramos contains, seguiríamos en bucle y nunca llegaríamos aquí.
```

```
    xalign 0.0
```

```
    linear 1.0 yalign 1.0
```

Contains Block. El bloque `contains` permite definir un bloque ATL que se utiliza para el hijo de esta transformación ATL. Una o más declaraciones de bloque `contains` se agruparán de forma codiciosa, se envolverán dentro de un `Fixed` y se establecerán como el hijo de esta transformación.

```
contains_statement -> "contains" ":"
```

```
    atl_block
```

Cada bloque debe definir un elemento mostrable para usar, o se producirá un error. La declaración `contains` se ejecuta instantáneamente, sin esperar a que los hijos se completen. Esta declaración es principalmente azúcar sintáctico, ya que permite que los argumentos se pasen fácilmente a los hijos.

```
image test double:
```

```
    contains:
```

```
        "logo.png"
```

```
        xalign 0.0
```

```
        linear 1.0 xalign 1.0
```

```
        repeat
```

```
    contains:
```

```
        "logo.png"
```

```
        xalign 1.0
```

```
        linear 1.0 xalign 0.0
```

```
        repeat
```

Declaración Function

La declaración `function` permite a ATL usar funciones de Python para controlar las propiedades ATL.

`function_statement -> "function" expression`

Las funciones tienen la misma firma que las utilizadas con **Transform**:

- El primer argumento es un objeto de transformación. Se pueden establecer propiedades de transformación en este objeto.
- El segundo argumento es la base de tiempo mostrada, el número de segundos desde que la función comenzó a ejecutarse.
- El tercer argumento es la base de tiempo de animación, que es el número de segundos que algo con la misma etiqueta ha estado en la pantalla.
- Si la función devuelve un número, se llamará de nuevo después de que hayan transcurrido ese número de segundos (0 segundos significa llamar a la función lo antes posible). Si la función devuelve `None`, el control pasará a la siguiente declaración ATL.

```
init python:
    def slide_function(trans, st, at):
        if st > 1.0:
            trans.xalign = 1.0
            return None
        else:
            trans.xalign = st
            return 0
```

```
label start:
    show logo base:
        function slide_function
        pause 1.0
        repeat
```

Warpers

Un *warper* es una función que puede cambiar la cantidad de tiempo que una declaración de interpolación considera que ha transcurrido. Los siguientes warpers están definidos por defecto. Se definen como funciones de t a t' , donde t y t' son números de punto flotante entre 0.0 y 1.0. (Si la declaración tiene duración 0, entonces t es 1.0 cuando se ejecuta).

- **pause**: Pausa, luego salta al nuevo valor. Si $t == 1,0$, $t' = 1,0$. De lo contrario, $t' = 0,0$.
- **linear**: Interpolación lineal. $t' = t$.
- **ease**: Comienza lento, acelera, luego desacelera. $t' = 0,5 - \cos(\pi \cdot t)/2,0$.

- **easein**: Comienza rápido, luego desacelera. $t' = \cos((1,0 - t) \cdot \pi/2, 0)$.
- **easeout**: Comienza lento, luego acelera. $t' = 1,0 - \cos(t \cdot \pi/2, 0)$.

Se pueden definir nuevos warpers usando el decorador `renpy.atl_warper` en un bloque `python early`. Debe colocarse en un archivo que se analice antes que cualquier archivo que use el warper. El código se ve así:

```
python early hide:
    @renpy.atl_warper
    def linear(t):
        return t
```

Propiedades de transformación

Existen las siguientes propiedades de transformación.

Cuando el tipo se indica como **position** (posición), puede ser un entero, absoluto o flotante. Si es un flotante, se interpreta como una fracción del tamaño del área contenedora (para **pos**) o del elemento mostrable (para **anchor**).

Ten en cuenta que no todas las propiedades son independientes. Por ejemplo, **xalign** y **xpos** actualizan parte de los mismos datos subyacentes. En una declaración **parallel**, solo un bloque debe ajustar la posición horizontal, y uno debe ajustar la posición vertical (pueden ser el mismo bloque). Las propiedades **angle** y **radius** establecen tanto la posición horizontal como la vertical.

pos

- Tipo: (posición, posición)
- Valor predeterminado: (0, 0)
- La posición relativa a la esquina superior izquierda del área contenedora.

xpos

- Tipo: posición
- Valor predeterminado: 0
- La posición horizontal relativa al lado izquierdo del área contenedora.

ypos

- Tipo: posición
- Valor predeterminado: 0
- La posición vertical relativa a la parte superior del área contenedora.

anchor

- Tipo: (posición, posición)
- Valor predeterminado: (0, 0)
- La posición del ancla relativa a la esquina superior izquierda del elemento mostrable.

xanchor

- Tipo: posición
- Valor predeterminado: 0
- La posición horizontal del ancla relativa al lado izquierdo del elemento mostrable.

yanchor

- Tipo: posición
- Valor predeterminado: 0
- La posición vertical del ancla relativa a la parte superior del elemento mostrable.

align

- Tipo: (float, float)
- Valor predeterminado: (0.0, 0.0)
- Equivalente a establecer **pos** y **anchor** al mismo valor.

xalign

- Tipo: float
- Valor predeterminado: 0.0
- Equivalente a establecer **xpos** y **xanchor** a este valor.

yalign

- Tipo: float
- Valor predeterminado: 0.0
- Equivalente a establecer **ypos** y **yanchor** a este valor.

rotate

- Tipo: float o None
- Valor predeterminado: None
- Si es **None**, no se produce rotación. De lo contrario, la imagen se rotará esta cantidad de grados en el sentido de las agujas del reloj. Rotar el elemento mostrable hace que se redimensione, según la configuración de **rotate_pad**, más abajo. Esto puede cambiar el posicionamiento si **xanchor** y **yanchor** no son 0.5.

rotate_pad

- Tipo: booleano
- Valor predeterminado: True
- Si es **True**, un elemento mostrable rotado se rellena de modo que el ancho y el alto sean iguales a la hipotenusa del ancho y alto originales. Esto asegura que la transformación no cambie de tamaño a medida que su contenido rota. Si es **False**, la transformación tendrá el tamaño mínimo que contenga el elemento mostrable transformado. Esto es más adecuado para rotaciones fijas.

zoom

- Tipo: float
- Valor predeterminado: 1.0
- Hace que el elemento mostrable se escale por el factor proporcionado. Siempre debe ser mayor que 0.5.

xzoom

- Tipo: float
- Valor predeterminado: 1.0
- Hace que el elemento mostrable se escale horizontalmente por el factor proporcionado. Siempre debe ser mayor que 0.5.

yzoom

- Tipo: float
- Valor predeterminado: 1.0
- Hace que el elemento mostrable se escale verticalmente por el factor proporcionado. Siempre debe ser mayor que 0.5.

alpha

- Tipo: float
- Valor predeterminado: 1.0
- Controla la opacidad del elemento mostrable.

around

- Tipo: (posición, posición)
- Valor predeterminado: (0.0, 0.0)
- Si no es **None**, especifica el centro de coordenadas polares, relativo a la esquina superior izquierda del área contenedora. Establecer el centro usando esto permite el movimiento circular en modo de posición.

alignaround

- Tipo: (float, float)
- Valor predeterminado: (0.0, 0.0)
- Si no es **None**, especifica el centro de coordenadas polares, relativo a la esquina superior izquierda del área contenedora. Establecer el centro usando esto permite el movimiento circular en modo de alineación.

angle

- Tipo: float
- Obtiene el componente de ángulo de la posición en coordenadas polares. Esto no está definido cuando el centro de coordenadas polares no está establecido.

radius

- Tipo: posición
- Obtiene el componente de radio de la posición en coordenadas polares. Esto no está definido cuando el centro de coordenadas polares no está establecido.

crop

- Tipo: **None** o (int, int, int, int)
- Valor predeterminado: **None**
- Si no es **None**, hace que el elemento mostrable se recorte en el rectángulo dado. El rectángulo se especifica como una tupla de (x, y, ancho, alto).

corner1

- Tipo: **None** o (int, int)
- Valor predeterminado: **None**
- Si no es **None**, da la esquina superior izquierda del rectángulo de recorte. Tiene prioridad sobre **crop**.

corner2

- Tipo: **None** o (int, int)
- Valor predeterminado: **None**
- Si no es **None**, da la esquina inferior derecha del rectángulo de recorte. Tiene prioridad sobre **crop**.

size

- Tipo: `None` o `(int, int)`
- Valor predeterminado: `None`
- Si no es `None`, hace que el elemento mostrable se escale al tamaño dado.

subpixel

- Tipo: booleano
- Valor predeterminado: `False`
- Si es `True`, hace que las cosas se dibujen en la pantalla usando posicionamiento subpíxel.

delay

- Tipo: `float`
- Valor predeterminado: `0.0`
- Si esta transformación se está usando como transición, esta es la duración de la transición.

Estas propiedades se aplican en el siguiente orden:

1. `crop`, `corner1`, `corner2`
2. `size`
3. `rotate`
4. `zoom`, `xzoom`, `yzoom`
5. Propiedades de posición

Movimiento circular

Cuando una declaración de interpolación contiene las palabras clave `clockwise` (en el sentido de las agujas del reloj) o `counterclockwise` (en sentido contrario), la interpolación producirá un movimiento circular. Ren'Py comparará las ubicaciones inicial y final y calculará el centro de coordenadas polares. Luego, Ren'Py calculará el número de grados que tomará ir desde el ángulo inicial al ángulo final, en la dirección de rotación especificada. Si se proporciona la cláusula `circles`, Ren'Py se asegurará de que se realice el número apropiado de círculos.

Ren'Py interpolará entonces las propiedades `angle` y `radius`, según corresponda, para que se produzca el movimiento circular. Si la transformación está en modo de alineación, establecer `angle` y `radius` establecerá la propiedad `align`. De lo contrario, se establecerá la propiedad `pos`.

Eventos externos

Los siguientes eventos se pueden activar automáticamente:

- **start**: Un pseudo-evento, activado al entrar en una declaración **on**, si no ha ocurrido ningún evento de mayor prioridad.
- **show**: Activado cuando la transformación se muestra usando la declaración **show** o **scene**, y no existe ninguna imagen con la etiqueta dada.
- **replace**: Activado cuando la transformación se muestra usando la declaración **show**, reemplazando una imagen con la etiqueta dada.
- **hide**: Activado cuando la transformación se oculta usando la declaración **hide**. La imagen no se ocultará realmente hasta que el bloque ATL termine.
- **hover**, **idle**, **selected_hover**, **selected_idle**: Activados cuando el botón que contiene esta transformación, o un botón contenido por esta transformación, entra en el estado nombrado.

Equivalente en Python

El equivalente en Python de una transformación ATL es un objeto **Transform**. No hay forma de crear código ATL mediante programación.

Archivos y directorios

Hay dos directorios principales utilizados por los juegos de Ren'Py.

Directorio base

El directorio base es el directorio que contiene todos los archivos que se distribuyen con el juego (también puede contener algunos archivos que no se distribuyen con el juego). Archivos como README deben colocarse en el directorio base, desde donde se distribuirán. El directorio base se crea dentro del directorio de Ren'Py y tiene el nombre de tu juego. Por ejemplo, si tu directorio de Ren'Py se llama **renpy-6.11.0** y tu juego se llama "HelloWorld", tu directorio base será **renpy-6.11.0/HelloWorld**.

Directorio del juego

El directorio del juego es casi siempre un directorio llamado **game** dentro del directorio base. Por ejemplo, si tu directorio base es **renpy-6.11.0/HelloWorld**, tu directorio del juego será **renpy-6.11.0/HelloWorld/game**.

Sin embargo, Ren'Py busca directorios en el siguiente orden:

1. El nombre del ejecutable, sin el sufijo. Por ejemplo, si el ejecutable se llama **moonlight.exe**, buscará un directorio llamado **moonlight** dentro del directorio base.
2. El nombre del ejecutable, sin el sufijo, y con un prefijo que termina en **_** eliminado. Por ejemplo, si el ejecutable es **moonlight_en.exe**, Ren'Py buscará un directorio llamado **en**.

3. Los directorios **game**, **data** y **launcher**, en ese orden.

Sin embargo, el lanzador solo reconocerá correctamente los directorios **game** y **data**. El directorio del juego contiene todos los archivos utilizados por el juego. Se escanea en busca de archivos **.rpy** y **.rpyc**, y estos se combinan para formar el guión del juego. También se escanea en busca de archivos de archivo **.rpa**, que son utilizados automáticamente por el juego. Finalmente, cuando el juego proporciona una ruta a un archivo para cargar, se carga en relación con el directorio del juego (pero ten en cuenta que **config.searchpath** puede cambiar esto).

Archivos ignorados

Los siguientes archivos y directorios no se distribuyen con los juegos de Ren'Py, incluso si están en el directorio base.

- **archived** - Este directorio contiene archivos que se han añadido a los archivos por el archivador incluido con el lanzador de Ren'Py. El lanzador escanea este directorio, junto con el directorio del juego, al construir archivos.
- **icon.ico** - Este archivo se utiliza para establecer el icono de un ejecutable de Windows.
- **icon.icns** - Este archivo se utiliza para establecer el icono de una aplicación de Mac.

Definición de personajes

Al mostrar diálogo al usuario usando la forma de dos argumentos de la declaración **say**, el primer argumento de la declaración **say** casi siempre debe ser una variable vinculada a un objeto **Character**. Estos objetos implementan (o llaman a funciones que implementan) gran parte de la lógica necesaria para mostrar el diálogo al usuario.

Los personajes se declaran vinculando variables a los objetos **Character**. Dichas declaraciones deben tener lugar dentro de bloques **init**, para asegurar que los enlaces no se guarden con el estado del juego, ya que esto impediría que las versiones revisadas de un juego recojan los cambios en el bloque **init**. Quizás la definición más simple es:

```
init:
    $ e = Character("Eileen")
```

Esto crea un nuevo objeto de personaje y lo vincula a la variable **e**. Cuando se ejecuta la declaración **say**:

```
e "¡Hola, mundo!"
```

La línea de texto "¡Hola, mundo!" irá precedida por la etiqueta **.Eileen**, indicando quién está hablando. Es común personalizar el color de esta etiqueta. Esto se puede hacer proporcionando el argumento de palabra clave **color** a **Character**, como en:

```
init:
    $ e = Character("Eileen", color=(200, 255, 200, 255))
```

El argumento `color` está estableciendo la propiedad `color` en la etiqueta. La propiedad `color` toma una tupla RGBA que consta de cuatro números, encerrados entre paréntesis y separados por comas, con cada número en el rango de 0 a 255.

La función `Character` se define de la siguiente manera:

Function: `Character (name, kind=adv, **kwargs):`

Crea un nuevo objeto de personaje. Este objeto es adecuado para usarlo como el primer argumento de una declaración `say` de dos argumentos. Cuando se usa así, muestra una ventana que contiene el nombre del personaje y el segundo argumento de la declaración `say`. Este comportamiento se personaliza pasando parámetros a esta función.

`name` es el nombre del personaje que está hablando. Puede ser una cadena que contenga el nombre del personaje, o `None` para indicar que la etiqueta no debe mostrarse. Un valor de `name` que consiste en un solo espacio () indica que la etiqueta debe ser una línea en blanco, lo que permite que un personaje narrador alinee la narración con el diálogo de los personajes. `name` es el único requisito de `Character`.

`kind`, si se proporciona, da el nombre de otro objeto `Character`. Los valores predeterminados para todas las configuraciones del objeto de personaje recién creado se toman de ese objeto de personaje. Si no se da, el valor predeterminado es `adv`. Otro valor útil es `nvl`, que por defecto causa interacciones en modo NVL.

Argumentos de palabra clave

Además de `name`, `Character` toma argumentos de palabra clave que controlan su comportamiento.

- `dynamic` - Si es verdadero, entonces `name` se interpreta como una cadena que contiene una expresión de Python que se evalúa para producir el nombre del personaje.
- `image` - Si es verdadero, entonces `name` se interpreta como una imagen. Esta imagen se añade a la ventana de diálogo en lugar de la etiqueta.
- `condition` - Si está presente, debe ser una cadena que contenga una expresión de Python. Esta expresión se evalúa cada vez que se dice una línea a través de este objeto de personaje. Si se evalúa como falsa, la línea no se muestra al usuario.
- `interact` - Si es verdadero (el valor predeterminado), provoca que ocurra una interacción cuando se muestra una línea al usuario. Si es falso, la interacción no tiene lugar, y se debe llamar a `ui.interact` (u otro medio) para que ocurra una interacción.
- `with_none` - Si es `True`, provoca que se ejecute una declaración `with None` después de cada interacción. Si es `None` (el valor predeterminado), comprueba `config.implicit_with_none` para determinar si se debe ejecutar un `with None`.
- `type` - El tipo de interacción. Consulta `renpy.last_interact_type` para saber cómo se usa esto.

Prefijos y sufijos

Los siguientes argumentos de palabra clave se pueden usar para añadir un prefijo o sufijo a todo lo que se dice a través de un objeto de personaje. Estos se pueden usar cuando las

líneas de diálogo deben ir entre comillas, como alternativa preferible a añadir esas comillas a cada línea de diálogo del guión.

- **who_prefix** - Texto que se antepone al nombre del personaje al formar la etiqueta del diálogo.
- **who_suffix** - Texto que se añade al nombre del personaje al formar la etiqueta del diálogo.
- **what_prefix** - Texto que se antepone a la línea de diálogo antes de mostrarla al usuario.
- **what_suffix** - Texto que se añade a la línea de diálogo antes de mostrarla al usuario.

Indicador de clic para continuar

Estos argumentos de palabra clave se utilizan para controlar el indicador de clic para continuar:

- **ctc** - Si está presente, este argumento toma un elemento mostrable que se usa como indicador de clic para continuar. Si no está presente o es **None**, no se muestra ningún indicador de clic para continuar.
- **ctc_pause** - Si se proporciona, un indicador de clic para continuar que se muestra cuando la visualización del texto se ha pausado con las etiquetas de texto **{p}** o **{w}**.
- **ctc_timedpause** - Si se proporciona y no es **None**, un indicador de clic para continuar que se muestra cuando una etiqueta **{p}** o **{w}** tiene un tiempo asociado. Se puede usar **ctc_pause=None()** para deshabilitar **ctc_pause** solo en el caso de una pausa temporizada.
- **ctc_position** - Si es **"nestled"**, el indicador de clic para continuar se muestra anidado con el final del texto. Si es **"fixed"**, el indicador de clic para continuar se muestra directamente en la pantalla, con sus diversas propiedades de posición determinando dónde se muestra realmente.

Funciones

Los siguientes son argumentos de palabra clave que permiten personalizar masivamente el comportamiento de un objeto de personaje:

- **show_function** - La función que se llama para mostrar cada paso de un diálogo al usuario (un diálogo puede dividirse en pasos mediante etiquetas de texto de pausa). Debe tener la misma firma que **renpy.show_display_say**.
- **predict_function** - La función que se llama para predecir las imágenes de este diálogo. Debe tener la firma de **renpy.predict_display_say**.
- **callback** - Una función de llamada de retorno o lista de funciones de llamada de retorno que se llaman en varios puntos durante el proceso de mostrar el diálogo. La función de llamada de retorno se llama con un solo argumento posicional, que indica el evento que ocurrió, y los siguientes argumentos de palabra clave:

- `interact` - Verdadero si este diálogo se está mostrando de forma interactiva.
- `type` - El valor de `type` proporcionado a `Character`.

Los argumentos de palabra clave adicionales y los eventos desconocidos deben ignorarse para futuras expansiones. Los eventos que provocan que se llame a la función de llamada de retorno son:

- `"begin"` - llamado al inicio de mostrar el diálogo.
- `"show"` - llamado antes de que se muestre cada segmento de diálogo.
- `"show_done"` - llamado después de que se muestre cada segmento de diálogo (pero antes de que ocurra la interacción).
- `"slow_done"` - llamado cuando el texto lento termina de mostrarse.
- `".end"` - llamado al final de mostrar el diálogo.

Ten en cuenta que cuando `interact=False`, `slow_done` puede ocurrir después de `end`. Si no se proporciona `callback` o es `None`, se usa `config.character_callback`, si no es `None`. Las funciones de llamada de retorno de personajes en `config.all_character_callbacks` se llaman para todos los personajes, antes que cualquier función de llamada de retorno más específica.

Función de visualización predeterminada

Los siguientes son argumentos de palabra clave que son procesados por la función de visualización predeterminada:

- `show_two_window` - Coloca el nombre del personaje que dice el diálogo en su propia ventana.
- `show_side_image` - Especifica un elemento mostrable que se muestra con este diálogo. El elemento mostrable debe establecer sus propiedades de posición de una manera que lo coloque donde el usuario quiera. Del mismo modo, las propiedades de la ventana deben establecerse de manera que proporcionen espacio para la imagen lateral. La función `ShowingSwitch` puede ser útil para hacer que este elemento mostrable varíe según qué otras imágenes son visibles.
- `show_two_window_vbox_properties` - Si se proporciona, un diccionario que contiene propiedades que se añaden al `vbox` que contiene ambas ventanas, cuando `show_two_window` es verdadero.
- `show_who_window_properties` - Si se proporciona, un diccionario que contiene propiedades que se añaden a la ventana que contiene el nombre de quién está hablando, cuando `show_two_window` es verdadero.
- `show_say_vbox_properties` - Si se proporciona, un diccionario que contiene propiedades que se añaden al `vbox` dentro de la ventana principal.
- `show_transform` - Si se proporciona, una transformación que se aplica a la(s) ventana(s) que se están mostrando.

Estilos

Los siguientes argumentos de palabra clave controlan los estilos utilizados por las partes del diálogo:

- `who_style` - Predeterminado a `'say_label'`, el estilo de la etiqueta.
- `what_style` - Predeterminado a `'say_dialogue'`, el estilo del texto que se dice.
- `window_style` - Predeterminado a `'say_window'`, el estilo de la ventana que contiene el diálogo.

Argumentos de palabra clave adicionales

Los argumentos de palabra clave adicionales se interpretan de la siguiente manera:

- Los argumentos de palabra clave que comienzan con `"window_"` se interpretan como propiedades de la ventana que contiene el diálogo, con el prefijo `"window_"` eliminado.
- Los argumentos de palabra clave que comienzan con `"what_"` se interpretan como propiedades del texto que se dice, con el prefijo `"what_"` eliminado.
- Los argumentos de palabra clave que comienzan con `"show_"` se proporcionan como argumentos de palabra clave a `show_function` y `predict_function`, con el prefijo `"show_"` eliminado.
- Los argumentos de palabra clave que comienzan con `çb_"` se proporcionan como argumentos de palabra clave a la(s) función(es) de llamada de retorno, con el prefijo `çb_"` eliminado.
- Todos los demás argumentos de palabra clave se interpretan como propiedades de la etiqueta (`color` es la propiedad más común que se pasa como palabra clave en esta función).

También existe la función `DynamicCharacter`:

Function: `DynamicCharacter (name, **kwargs):`

Equivalente a llamar a `Character` con los mismos argumentos y con el argumento `dynamic` establecido en verdadero.

Método: `Character.copy (...)`

El método `copy` en `Character` toma los mismos argumentos que el constructor `Character`, con la excepción de que el argumento `name` es opcional. Este método devuelve un nuevo objeto `Character`, creado combinando los argumentos del constructor original y este método, con los argumentos proporcionados por este método teniendo prioridad.

Llamar a objetos Character

Los objetos `Character` se pueden llamar directamente, como si fueran funciones. Toman un parámetro posicional, la línea de diálogo que se mostrará al usuario. También toman argumentos de palabra clave:

- `interact`, que determina si debe tener lugar una interacción.
- `force_ctc`, fuerza a que se muestre el indicador de clic para continuar incluso si `interact` es `False`.

Este uso puede reemplazar programáticamente las declaraciones `say`. Por ejemplo, la declaración `say`:

```
e "¡Hola, mundo!"
```

es equivalente a:

```
$ e("¡Hola, mundo!", interact=True)
```

Personajes predefinidos

Proporcionamos algunos personajes predefinidos para trabajar:

- `centered` es un personaje que hará que lo que dice se muestre centrado, en el medio de la pantalla, fuera de cualquier ventana.
- `extend` hará que el último personaje que habló diga una línea de diálogo que consiste en la última línea de diálogo hablada, `"{fast}"` y el diálogo dado a `extend`. Esto se puede usar para que la pantalla cambie durante el curso del diálogo. `extend` es consciente del modo NVL y lo trata correctamente.

```
# Mostrar la primera línea de diálogo, esperar un clic, cambiar la expresión y mostrar
# el resto.
```

```
show eileen concerned
```

```
e "A veces, me siento triste."
```

```
show eileen happy
```

```
extend " ¡Pero normalmente me recupero rápido!"
```

```
# Similar, pero cambia automáticamente la expresión cuando la primera línea termina
# de mostrarse. Esto solo tiene sentido cuando el usuario no tiene la velocidad de te
# ajustada al máximo.
```

```
show eileen concerned
```

```
e "A veces, me siento triste.{nw}"
```

```
show eileen happy
```

```
extend " ¡Pero normalmente me recupero rápido!"
```

Texto

Esta sección cubre aspectos del texto en Ren'Py. Primero cubre la interpolación, compatible con las declaraciones `say` y `menu`, que permite sustituir valores en el texto. Luego analiza las etiquetas de texto, que permiten personalizar el estilo de partes de las cadenas de texto. Finalmente, cubre cómo Ren'Py maneja las fuentes.

Interpolación

La interpolación es compatible con las declaraciones `say` y `menu`. Estas declaraciones admiten la interpolación de cadenas de Python sobre el contenido del `store`. Las cadenas utilizadas por las declaraciones admiten especificadores de conversión de la forma `%(variable)s`, donde `variable` es el nombre de una variable y `s` es una conversión. Las conversiones útiles incluyen `'s'`, que interpola una cadena; `'d'`, que interpola un entero; y `'f'`, que interpola un número de punto flotante. Las conversiones también pueden incluir caracteres que modifican cómo se convierte la cadena. Se puede encontrar más información sobre las conversiones en la referencia de operaciones de formato de cadenas de Python. En las cadenas donde se admite la interpolación, los caracteres de porcentaje (`%`) deben duplicarse (a `%%`) para evitar que se interpreten como introducción de interpolación.

```
$ name = 'Samantha'
$ age = 19
$ withyou = 110
girl "Mi nombre es %(name)s, y tengo %(age)d años. Estoy contigo %(withyou)d%%"
```

Cuando no se admite la interpolación, el efecto a menudo se puede simular colocando los marcadores de posición `%s`, `%d` y `%f` con `%` (lista de variables) fuera de la cadena (ver el ejemplo a continuación).

```
ui.text("%s's Vital Statistics: %d" % (name, blood))
```

El procesamiento de etiquetas de texto se realiza después de la interpolación, por lo que es importante asegurarse de que la interpolación no introduzca etiquetas de texto.

Etiquetas de texto

El texto mostrado por Ren'Py admite etiquetas de texto. Mientras que los estilos solo se pueden aplicar a un elemento mostrable `Text` completo, las etiquetas de texto permiten personalizar solo una parte del texto en el elemento mostrable. Como las etiquetas de texto son parte del elemento mostrable `Text`, se pueden usar en cualquier cadena que se muestre en la pantalla. Sin embargo, algunas de las etiquetas de texto solo tendrán efecto si se usan en el contexto apropiado.

Las etiquetas de texto deben usarse con bastante moderación. Si te encuentras usando etiquetas de texto en cada línea del juego, es muy posible que las etiquetas de texto no sean la solución adecuada para el problema en cuestión. Investiga los estilos de texto como posible alternativa a las etiquetas de texto.

Las etiquetas de texto comienzan con una llave izquierda (`{`) y continúan hasta la llave derecha correspondiente (`}`). Inmediatamente después de la llave izquierda está el nombre de la etiqueta. El nombre puede ir seguido de un argumento, que se separa del nombre de la etiqueta por un signo igual (`=`). Algunas etiquetas requieren un argumento, mientras que otras requieren que se omita. Las etiquetas de texto distinguen entre mayúsculas y minúsculas y son sensibles a los espacios en blanco.

Algunas etiquetas requieren una etiqueta de cierre correspondiente. Una etiqueta de cierre es cualquier etiqueta de texto donde el nombre comienza con una barra diagonal (`/`). Las etiquetas de cierre deben estar correctamente anidadas: `"{b}{i}esto está bien{/i}{/b}"`, mientras que `"{b}{i}esto está mal{/b}{/i}"`. Aunque el anidamiento incorrecto de las etiquetas de texto generalmente no causará un error, este problema

puede causar resultados de renderizado no deseados. El texto entre una etiqueta y la etiqueta de cierre correspondiente se llama texto encerrado. Todas las etiquetas deben estar cerradas al final de la cadena de texto.

Para incluir una sola llave izquierda en el texto, se deben incluir dos llaves izquierdas (`{{`) en su lugar.

Ren'Py admite las siguientes etiquetas de texto:

- `{a=argumento}` y `{/a}` hacen que su texto encerrado se renderice como un hipervínculo. El texto encerrado puede contener etiquetas de texto, pero no puede contener otro hipervínculo. Un hipervínculo se renderiza como un botón que contiene el texto encerrado. El texto en el botón no participa en los saltos de línea. Más bien, si el texto crece demasiado para la línea en la que está, todo el botón se mueve al inicio de la siguiente línea. Cuando se hace clic en un hipervínculo, se llama a la función de llamada de retorno definida en `config.hyperlink_callback` con el texto del argumento del hipervínculo. El hipervínculo predeterminado comprueba si el hipervínculo comienza con `"http:"`. Si es así, el enlace se abre en el navegador web del usuario. Si no, el hipervínculo se interpreta como una etiqueta, que se llama en un nuevo contexto. Este es un comportamiento apropiado cuando se usan hipervínculos para definiciones de términos desconocidos. Los hipervínculos no deben usarse como una herramienta de control de flujo general, ya que transfieren el control a un nuevo contexto cuando se hace clic. Si el usuario guarda mientras está dentro de ese contexto, cuando cargue el juego volverá a la pantalla que contiene el hipervínculo.

```
init:
    $ definition = Character(None, window_yfill=True,
                             window_xmargin=20, window_ymargin=20,
                             window_background=Solid((0, 0, 0, 192)))

label start:
    "¿Un juego que enseña cómo hacer un juego? ¿No es eso una especie de {a=d
    # ...

label define_quine:
    definition "Quine:\n\nUn programa que se imprime a sí mismo en su salida.
    return
```

- `{b}texto{/b}` renderiza el texto encerrado en negrita.
- `{color=espec}texto{/color}` cambia el color del texto encerrado. El color puede ser un triplete hexadecimal o un sexteto hexadecimal, opcionalmente precedido por un signo de almohadilla (el mismo formato que acepta `color()`). `f00`, `ff0000`, `#f00` y `#ff0000` son todas representaciones válidas de rojo.
- `{fast}` provoca la visualización inmediata del texto anterior. Se puede usar para especificar dónde debe comenzar a mostrarse el texto lento. Solo tiene efecto si la preferencia de velocidad de texto no es infinita. Úsalo si tienes dos líneas de diálogo donde la segunda es una extensión de la primera, como una variante del efecto

de pausa que permite cambios mientras está en pausa. Esta etiqueta no toma una etiqueta de cierre.

- `{font=nombreadarchivo.ttf}texto{/font}` renderiza el texto encerrado en la fuente proporcionada.
- `{i}texto{/i}` hace que el texto encerrado esté en cursiva.
- `{image=nombreadarchivo}` hace que la imagen proporcionada se cargue y se incluya en el texto. Las imágenes se tratan como si fueran texto y no deben ser más altas que una sola línea de texto. `{image}` no toma una etiqueta de cierre.
- `{nw}`, al final de un bloque de texto, hace que la interacción actual termine inmediatamente una vez que el texto esté completamente mostrado. Solo tiene sentido usar esto en texto de diálogo, donde el efecto es hacer que el texto lento descarte la interacción una vez que se haya mostrado completamente.
- `{p}` hace que la visualización del texto se detenga hasta que el usuario haga clic para continuar, o hasta que ocurra un avance automático. Esto permite que el texto se muestre de forma incremental. `{p}` no toma etiquetas de cierre y hace que se inserte un salto de línea después del texto dado. La lógica para manejar la pausa se implementa en el objeto `Character` y puede no funcionar para otros widgets de texto.
- `{plain}texto{/plain}` hace que el texto encerrado sea simple, eliminando los estilos de negrita, cursiva y subrayado.
- `{s}texto{/s}` hace que el texto encerrado aparezca tachado (nuevo en 6.6.2).
- `{size=espec}texto{/size}` cambia el tamaño del texto. La `espec` proporcionada puede ser un número, en cuyo caso es el número de píxeles de alto que tendrá el texto. Si es un número precedido por un signo más (como `-10"`), significa aumentar el tamaño en ese número de píxeles, mientras que si va precedido por un signo menos, significa disminuir.
- `{u}texto{/u}` renderiza el texto encerrado con subrayado.
- `{w}` es similar a `{p}`, excepto que no provoca un salto de línea. También admitimos la forma `{w=número}`, que pausa durante `número` segundos (por ejemplo, `{w=.5}` pausa durante 0.5 segundos antes de continuar).
- `{=estilo}texto{/=}` aplica el estilo nombrado al texto encerrado. Piensa en esto como una forma de diseñar tus propias etiquetas de texto (nuevo en 6.6.2).

La última etiqueta `{fast}` es la que se usa. Las etiquetas `{nw}`, `{p}` y `{w}` solo tienen efecto si están después de la última etiqueta `{fast}` en el texto.

El siguiente es un ejemplo de una declaración `say` que usa muchas etiquetas de texto. No se recomienda el uso de tantas etiquetas de texto en un juego de alta calidad.

"Usando etiquetas de texto, podemos hacer que el texto sea `{size=+12}más grande{/size}`.

Fuentes

El elemento mostrable **Text** intenta encontrar una fuente apropiada usando información sobre el nombre de la fuente, negrita, cursiva, subrayado y tamaño. Esta información se proporciona al elemento mostrable **Text** mediante propiedades de estilo, pero luego se puede modificar usando etiquetas de texto. Ren'Py traduce esto en una fuente usando el siguiente algoritmo.

1. Se busca un triplete (nombre, negrita, cursiva) en `config.font_replacement_map`. Si está presente, se espera que se asigne a un triplete que dé valores de reemplazo para nombre, negrita y cursiva. Esta búsqueda no es recursiva, por lo que solo se realiza una búsqueda. Esta búsqueda permite utilizar formas de fuente específicas en negrita o cursiva.
2. Si se ha registrado un **SFont** con el mismo nombre, tamaño, negrita, cursiva y subrayado, se utiliza ese **SFont**.
3. De lo contrario, Ren'Py interpreta el nombre de la fuente como el nombre de archivo de una fuente TrueType. Se busca este nombre de archivo en el `searchpath` y en los archivos. Si se encuentra, se usa como el archivo de fuente.
4. De lo contrario, Ren'Py busca una fuente con el nombre de archivo dado en el directorio de fuentes del sistema. Si se encuentra, se usa ese archivo.
5. De lo contrario, Ren'Py interpreta el nombre de archivo como el nombre de una fuente TrueType, que se carga si se encuentra.

El código de carga de fuentes TrueType escalará y subrayará automáticamente la fuente según sea necesario. Si no has proporcionado asignaciones de fuentes en negrita y cursiva, también engrosará e inclinará artificialmente la fuente cuando sea necesario.

Para obtener los mejores resultados, las fuentes deben ser archivos TrueType que se distribuyan con el juego. Esto asegura que las fuentes requeridas estén siempre presentes en el sistema del usuario.

Admitimos colecciones TrueType: "0@font.ttc" es la primera fuente de la colección, "1@font.ttc" la segunda, y así sucesivamente (nuevo en 6.10.0).

Fuentes basadas en imágenes

Además de las fuentes TrueType habituales, Ren'Py admite SFonts basados en imágenes, MudgeFonts y BMFonts.

Las fuentes basadas en imágenes tienen varias ventajas y varias desventajas. Una de las ventajas es que estas fuentes están basadas en mapas de bits, lo que significa que es fácil crear fuentes personalizadas y encontrar ejemplos gratuitos (en contraste, las licencias de las fuentes TrueType de alta calidad rara vez permiten la redistribución). Además, dado que las fuentes son imágenes, es posible aplicarles efectos que Ren'Py no admitiría de otra manera. Los inconvenientes de las fuentes basadas en imágenes provienen del hecho de que Ren'Py no las renderiza, sino que simplemente copia caracteres de ellas. Debido a esto, es necesario proporcionar otra imagen a Ren'Py si se quiere que la fuente se escale, se ponga en negrita, cursiva o subrayado. Ren'Py recolorará las fuentes de imagen, mapeando el blanco al color proporcionado y otros colores se interpolarán canal por canal.

Ten en cuenta que debes registrar una fuente basada en imágenes para cada combinación de fuente, tamaño, negrita, cursiva y subrayado que use tu juego. Luego se pueden usar estableciendo la propiedad `font` de un estilo al nombre de la fuente.

Para usar SFonts, primero deben registrarse con Ren'Py usando la función `renpy.register_sfont`.

Para más información sobre SFonts, consulta [1].

Para usar MudgeFonts, deben registrarse con la función `renpy.register_mudgefont`.

Para más información sobre MudgeFonts, consulta [2].

Para usar BMFonts, deben registrarse con la función `renpy.register_bmfont`. Para más información sobre BMFonts, consulta [3]. Recomendamos BMFont para fuentes de mapa de bits, ya que BMFonts admiten unicode y kerning adecuado.

Los problemas legales relacionados con el uso de fuentes de contorno con derechos de autor en la creación de fuentes basadas en imágenes se discuten en *Fuentes de mapa de bits y derechos de autor*.

Elementos mostrables (Displayables)

Un elemento mostrable es un objeto de Python que implementa una interfaz que permite mostrarlo en la pantalla. Los elementos mostrables se pueden asignar a un nombre de imagen usando una declaración `image`, y luego se pueden mostrar al usuario usando las declaraciones `show` y `hide`. Se pueden proporcionar como argumento a la función `ui.add`. También se utilizan como argumento para ciertas propiedades de estilo.

En esta sección, describiremos funciones que crean elementos mostrables.

Cuando Ren'Py requiere que se especifiquen colores para un elemento mostrable, permite especificarlos en varias formas. En los siguientes ejemplos, las letras minúsculas representan dígitos hexadecimales, mientras que las letras mayúsculas son abreviaturas de números entre 0 y 255. `r`, `g`, `b` y `a` son abreviaturas de rojo, verde, azul y alfa.

- Una cadena de la forma `"#rgb"`. (amarillo = `"#ff0"`, azul = `"#00f"`)
- Una cadena de la forma `"#rgba"`. (amarillo = `"#ff0f"`, azul = `"#00ff"`)
- Una cadena de la forma `"#rrggbb"`. (amarillo = `"#ffff00"`, azul = `"#0000ff"`)
- Una cadena de la forma `"#rrggbbaa"`. (amarillo = `"#ffff00ff"`, azul = `"#0000ffff"`)
- Una tupla de 4 elementos de la forma `(R, G, B, A)`. (amarillo = `(255, 255, 0, 255)`, azul = `(0, 0, 255, 255)`)

Cuando Ren'Py espera que se especifique un elemento mostrable, permite que estén en las siguientes formas:

- Se puede proporcionar un elemento mostrable directamente, en cuyo caso se usa.
- Una cadena que comienza con `"#"` se usa para crear un objeto `Solid`, con el color interpretado como arriba.
- Las cadenas con al menos un punto se interpretan como nombres de archivo de imágenes y se usan para crear objetos de imagen.
- Otras cadenas se interpretan como el nombre de una imagen definida con la declaración `image`.

Manipuladores de imágenes

Una `Image` es un tipo de elemento mostrable que contiene datos de mapa de bits que se pueden mostrar en la pantalla. Todas las imágenes son elementos mostrables, pero no todos los elementos mostrables son imágenes. Las imágenes se diferencian de los elementos mostrables en que se pueden calcular estáticamente y almacenar en la caché de imágenes. Esta carga ocurre en algún momento antes de que la imagen se vaya a usar (pero no en el momento en que se crea la `Image`). Esto generalmente hace que las `Image` sean más rápidas que los elementos mostrables arbitrarios. Un manipulador de imágenes es una función que devuelve una `Image`.

Cuando un manipulador de imágenes requiere otro como entrada, el segundo manipulador de imágenes se puede especificar de varias formas.

- Se puede proporcionar como una cadena, que se interpreta como un nombre de archivo y se carga con `im.Image`.
- Puede ser un manipulador de imágenes especificado directamente, en cuyo caso se pasa sin cambios.

Codificación de longitud de ejecución (Run-length Encoding)

Ren'Py admite el uso de codificación de longitud de ejecución para dibujar eficientemente imágenes con grandes cantidades de transparencia. Todos los manipuladores de imágenes admiten un argumento `rle`. Cuando es `True`, se usa la codificación de longitud de ejecución. Cuando es `False`, no se usa. Cuando es `None` (el valor predeterminado), Ren'Py muestrea aleatoriamente 10 puntos en la imagen para intentar encontrar transparencia, y usa RLE solo si se encuentra transparencia. A cambio de un mayor tiempo de carga y memoria adicional proporcional al número de píxeles no transparentes en la imagen, la codificación de longitud de ejecución puede dibujar imágenes en la pantalla en un tiempo proporcional al número de píxeles no transparentes en la imagen. Cuando una imagen es mayormente transparente, esto puede llevar a una aceleración significativa en el tiempo de dibujo.

```
init:
    image eileen happy = Image("eileen_happy.png", rle=True)
```

Caché

Por defecto, los manipuladores de imágenes se almacenan en caché en memoria antes de cargarse. Proporcionar el argumento `cache=False` a un manipulador de imágenes evitará este almacenamiento en caché. Esto solo debe usarse cuando un manipulador de imágenes nunca se usa directamente (como por una declaración `image`).

```
init:
    image eileen happy = im.AlphaMask(Image("eileen_happy.base.jpg", cache=False),
                                       Image("eileen_happy.mask.jpg", cache=False))
```

Lista de manipuladores de imágenes

Los manipuladores de imágenes son:

Function: `im.Image (filename, **properties):`

Este manipulador de imágenes carga una imagen desde un archivo.

- `filename` - El nombre de archivo del que se cargará la imagen.

Nota: `Image` es un alias de `im.Image`.

Function: `Image (...):`

Un alias para `im.Image`.

Function: `im.Alpha (image, alpha, **properties):`

Devuelve una versión con máscara alfa de la imagen. `alpha` es el alfa máximo que puede tener esta imagen, un número entre 0.0 (completamente transparente) y 1.0 (opaco). Si una imagen ya tiene un canal alfa, los valores en ese canal alfa se reducen según corresponda.

Function: `im.AlphaMask (base, mask, **properties):`

`base` y `mask` deben ser manipuladores de imágenes. Esta función toma el canal rojo de `mask` y lo aplica al canal alfa de `base` para crear una nueva imagen.

Function: `im.Composite (size, *args, **properties):`

Este manipulador de imágenes compone una o más imágenes juntas.

Toma un número variable de argumentos. El primer argumento es `size`, que es el tamaño deseado de la imagen (en píxeles), o `None` para indicar que el tamaño debe ser el tamaño de la primera imagen.

Luego toma un número par de argumentos adicionales (para un número impar de argumentos totales). El segundo y otros argumentos en posiciones pares contienen tuplas de posición, mientras que el tercero y otros argumentos en posiciones impares dan imágenes (o manipuladores de imágenes). Un argumento de posición da la posición de la imagen que le sigue inmediatamente, con la posición expresada como una tupla que da un desplazamiento desde la esquina superior izquierda de la imagen. Las imágenes se componen en orden de abajo hacia arriba, siendo la última imagen la más cercana al usuario.

Function: `im.Crop (im, x, y, w, h, **properties):`

Recorta la imagen que es su hijo.

Function: `im.FactorScale (im, width, height=None, bilinear=True, **properties):`

Escala el manipulador de imágenes `im` por los factores de ancho y alto dados. Si no se da `height`, se usa el valor de `width`.

- `bilinear` - Si es `True`, se usa el escalado bilineal.

Function: `im.Flip (im, horizontal=False, vertical=False, **properties):`

Este es un manipulador de imágenes que puede voltear la imagen horizontal o verticalmente.

- `im` - La imagen a voltear.
- `horizontal` - `True` para voltear la imagen horizontalmente.
- `vertical` - `True` para voltear la imagen verticalmente.

Function: `im.Map (im, rmap=im.ramp(0, 255), gmap=im.ramp(0, 255), bmap=im.ramp(0, 255), amap=im.ramp(0, 255), force_alpha=False, **properties):`

Ajusta los colores de la imagen que es su hijo. Toma como argumentos 4 cadenas de 256 caracteres. Si un canal de píxel tiene un valor de 192, entonces el valor del carácter 192 en la cadena se usa para el componente de píxel mapeado.

La función `im.Map` se puede usar de forma simple, como `im.Recolor`, para escalar o eliminar componentes de color de la imagen fuente, o se puede usar de una manera más compleja para reasignar completamente el color de la imagen fuente.

Function: `im.ramp (start, end):`

Devuelve una rampa lineal de 256 caracteres, donde el primer carácter tiene el valor `start` y el último carácter tiene el valor `end`. Tal rampa se puede usar como argumento de mapa de `im.Map`.

Function: `im.Scale (im, width, height, bilinear=True, **properties):`

Este es un manipulador de imágenes que escala otro manipulador de imágenes al ancho y alto especificados.

- `bilinear` - Si es `True`, se usa la interpolación bilineal. Si es `False`, se usa el filtrado de vecino más cercano.

Function: `im.Recolor (im, rmul=255, gmul=255, bmul=255, amul=255, force_alpha=False, **properties):`

Ajusta los colores de la imagen proporcionada, `im`. Toma cuatro argumentos, `rmul`, `gmul`, `bmul` y `amul`, correspondientes a los canales rojo, verde, azul y alfa, siendo cada uno un entero entre 0 y 255. Cada canal tiene su valor mapeado de modo que 0 es 0, 255 es el argumento proporcionado para ese canal, y otros valores se mapean linealmente entre ellos.

Function: `im.Twocolor (im, white, black, force_alpha=False, **properties):`

Toma como argumentos dos colores, `white` y `black`. La imagen se mapea de modo que los píxeles en blanco tengan el color blanco, los píxeles en negro tengan el color negro, y los tonos de gris se interpolan linealmente entre ellos. El canal alfa se mapea linealmente entre 0 y el alfa encontrado en el color blanco; el alfa del color negro se ignora.

Function: `im.Rotozoom (im, angle, zoom, **properties):`

Este es un manipulador de imágenes que es una rotación y zoom suaves de otro manipulador de imágenes.

- `im` - La imagen a rotar y escalar.
- `angle` - El número de grados en sentido antihorario que se va a rotar la imagen.
- `zoom` - El factor de zoom. Números mayores que 1.0 hacen que la imagen se agrande.

Function: `im.Tile (im, size=None, **properties):`

Mosaico de la imagen, repitiéndola vertical y horizontalmente hasta que sea tan grande como el tamaño especificado. Si no se da ningún tamaño, el tamaño predeterminado es el tamaño de la pantalla.

`ImageReference` se usa para acceder a imágenes declaradas con la declaración `image`.

Function: `ImageReference (name):`

Este elemento mostrable accede a imágenes declaradas usando la declaración `image`. `name` puede ser una cadena, o una tupla de cadenas que dan los componentes del nombre de la imagen.

Esto siempre es un elemento mostrable, y también puede ser un manipulador de imágenes si `name` está declarado para referirse a un manipulador de imágenes.

im.MatrixColor

El manipulador de imágenes `im.MatrixColor` toma una matriz de 20 o 25 elementos y la utiliza para transformar linealmente los colores de una imagen.

Function: `im.MatrixColor (im, matrix):`

Este es un operador de imágenes que crea una imagen usando una matriz para transformar linealmente los colores de la imagen `im`.

`matrix` debe ser una lista, tupla o `im.matrix` de 20 o 25 elementos de longitud. Si el objeto tiene 25 elementos, los elementos más allá del 20 se ignoran. Si los elementos de la matriz se nombran de la siguiente manera:

```
[ a, b, c, d, e,
  f, g, h, i, j,
  k, l, m, n, o,
  p, q, r, s, t ]
```

y `R`, `G`, `B` y `A` son los componentes rojo, verde, azul y alfa del color, respectivamente, entonces el color transformado `R'`, `G'`, `B'`, `A'` se calcula de la siguiente manera:

$$R' = (a * R) + (b * G) + (c * B) + (d * A) + (e * 255)$$

$$G' = (f * R) + (g * G) + (h * B) + (i * A) + (j * 255)$$

$$B' = (k * R) + (l * G) + (m * B) + (n * A) + (o * 255)$$

$$A' = (p * R) + (q * G) + (r * B) + (s * A) + (t * 255)$$

`R'`, `G'`, `B'` y `A'` se limitan al rango $[0, 255]$.

A menudo es conveniente especificar la matriz usando un objeto `im.matrix`. Estos objetos admiten varias operaciones matemáticas, como multiplicación de matrices, multiplicación escalar y adición escalar. Multiplicar matrices entre sí permite realizar múltiples manipulaciones de color a la vez, al costo de una sola operación `MatrixColor`.

Función: im.matrix (matrix):

Construye un objeto `im.matrix` a partir de la matriz dada. Los objetos `im.matrix` representan matrices de 5x5 y admiten varias operaciones matemáticas. Las operaciones admitidas son multiplicación de matrices, multiplicación escalar, adición elemento por elemento y sustracción elemento por elemento. Estas operaciones se invocan usando los operadores matemáticos estándar (`*`, `*`, `+` y `-`), respectivamente. Si se multiplican dos objetos `im.matrix`, se realiza una multiplicación de matrices; de lo contrario, se usa la multiplicación escalar.

`matrix` es una lista de 20 o 25 elementos. Si la lista tiene 20 elementos, se rellena con $[0, 0, 0, 0, 1]$ para formar una matriz de 5x5, adecuada para la multiplicación.

Las siguientes funciones producen objetos `im.matrix`:

Function: `im.matrix.identity ():`

Devuelve una matriz identidad `im.matrix` (una que no cambia el color ni el alfa).

Function: `im.matrix.saturation (level, desat=(0.2126, 0.7152, 0.0722)):`

Construye un `im.matrix` que altera la saturación de una imagen. El canal alfa no se modifica.

- `level` - La cantidad de saturación en la imagen resultante. 1.0 es la imagen sin alterar, mientras que 0.0 es escala de grises.

- **desat** - Esta es una tupla de 3 elementos que controla cuánto de los canales rojo, verde y azul se colocará en los tres canales de una imagen completamente desaturada. El valor predeterminado se basa en las constantes utilizadas para el canal de luminancia de una señal de televisión NTSC. Dado que el ojo humano es más sensible al verde, se conserva más del canal verde que de los otros dos canales.

Function: `im.matrix.desaturate ()`:

Devuelve una matriz que desatura la imagen (la convierte en escala de grises). Esto es equivalente a llamar a `im.matrix.saturation(0)`.

Function: `im.matrix.tint (r, g, b)`:

Construye un `im.matrix` que tiñe una imagen, dejando el canal alfa intacto. `r`, `g` y `b` deben ser números entre 0 y 1, y controlan qué fracción del canal dado se coloca en la imagen final. (Por ejemplo, si `r` es 0.5 y un píxel tiene un componente rojo de 100, el píxel correspondiente tendrá un componente rojo de 50).

Function: `im.matrix.invert ()`:

Construye un `im.matrix` que invierte los canales rojo, verde y azul de la imagen fuente, dejando el canal alfa sin cambios.

Function: `im.matrix.brightness (b)`:

Construye un `im.matrix` que altera el brillo de una imagen, dejando el canal alfa intacto. `b` debe estar entre -1 y 1, siendo -1 la imagen más oscura posible y 1 la más brillante.

Function: `im.matrix.contrast (c)`:

Construye un `im.matrix` que altera el contraste de una imagen. `c` debe ser mayor que 0.0, con valores mayores que 1.0 que aumentan el contraste y menores que 1.0 que lo disminuyen.

Function: `im.matrix.opacity (o)`:

Construye un `im.matrix` que altera la opacidad de una imagen, dejando los canales rojo, verde y azul sin cambios. Un `o` de 0.0 es completamente transparente, mientras que 1.0 es completamente opaco.

Function: `im.matrix.hue (h)`:

Devuelve una matriz que rota el tono en `h` grados, preservando la luminosidad.

Las matrices construidas con estas funciones se pueden componer mediante multiplicación de matrices. Por ejemplo, se puede desaturar una imagen y luego teñirla de azul claro:

`init:`

```
image city_blue = im.MatrixColor("city.jpg",
                                im.matrix.desaturate() * im.matrix.tint(0.9, 0.9
```

Es mucho más eficiente multiplicar matrices que componer operaciones `im.MatrixColor`, ya que la multiplicación de matrices requiere aproximadamente 125 operaciones de multiplicación, mientras que `im.MatrixColor` usa 16 por píxel multiplicado por el número de píxeles en la imagen.

Existen dos manipuladores de imágenes que envuelven usos comunes de `im.MatrixColor`:

Function: `im.Grayscale (im, desat=(0.2126, 0.7152, 0.0722))`:

Este operador de imágenes convierte la imagen dada a escala de grises. `desat` es como en `im.matrix.saturation`.

Function: `im.Sepia (im, tint=(1.0, .94, .76), desat=(0.2126, 0.7152, 0.0722))`:

Este operador de imágenes aplica un tono sepia a una imagen. `desat` es como en `im.matrix.saturation`. `tint` se descompone y se usa como los parámetros para `im.matrix.tint`.

Fondos (Backgrounds)

Hay dos elementos mostrables que son eminentemente adecuados para usar como fondos:

Function: Frame (image, xborder, yborder, tile=False, bilinear=False):

Devuelve un elemento mostrable que es un marco, basado en el nombre de archivo de imagen proporcionado. Un marco es una imagen que se redimensiona automáticamente al tamaño que se le asigna. La imagen tiene bordes que solo se escalan en un eje. La región dentro de **xborder** píxeles de los bordes izquierdo y derecho solo se escala en la dirección y, mientras que la región dentro de **yborder** píxeles de los ejes superior e inferior se escala solo en la dirección x. Las esquinas no se escalan en absoluto, mientras que el centro de la imagen se escala tanto en la dirección x como en la y.

- **image** - La imagen (que puede ser un nombre de archivo u objeto de imagen) que se escalará.
- **xborder** - El número de píxeles en la dirección x para usar como borde.
- **yborder** - El número de píxeles en la dirección y para usar como borde.
- **tile** - Si es verdadero, se realiza un mosaico en lugar de escalado.
- **bilinear** - Si es verdadero (y **tile** es falso), se realiza un escalado bilineal en lugar de un escalado de vecino más cercano.

Para un mejor rendimiento, haz que la imagen comparta una dimensión de longitud común con el tamaño en el que se renderizará el marco. Detectamos esto y evitamos el escalado si es posible.

Function: Solid (color):

Devuelve un elemento mostrable que es sólido y está relleno con un solo color. Un **Solid** se expande para llenar todo el espacio que se le asigna, lo que lo hace adecuado para usar como fondo.

- **color** - El color con el que se rellenará la pantalla, dado como una tupla RGBA o una cadena de estilo HTML.

Texto (Text)

Text se puede usar para mostrar texto al usuario, como un elemento mostrable.

Function: Text (text, slow=False, slow_done=None, slow_speed=None, slow_start=0, slow_abortable=False, style='default', **properties):

Un elemento mostrable que puede formatear y mostrar texto en la pantalla.

- **text** - El texto que se mostrará en la pantalla.
- **slow** - Si es **True**, el texto se escribirá en la pantalla a una velocidad determinada por la propiedad **slow_cps**, si está establecida, o por la preferencia "Velocidad de texto". Si es **None** (el valor predeterminado), se escribirá a una velocidad determinada por la propiedad **slow_cps**. Si es **False**, aparecerá instantáneamente.
- **style** - Un estilo que se aplicará al texto.
- **properties** - Propiedades adicionales que se aplican al texto.

- `slow_done` - Una llamada de retorno que ocurre cuando el texto lento termina.
- `slow_speed` - La velocidad del texto lento. Si no se proporciona, se toma de las preferencias.
- `slow_offset` - El desplazamiento en el texto para comenzar el texto lento.
- `slow_abortable` - Si es `True`, al hacer clic se aborta el texto lento.

Ren'Py también admite un objeto de texto parametrizado, que muestra texto como si fuera una imagen en la pantalla. Pero se debe tener cuidado, ya que casi siempre es mejor usar un objeto `Character` para mostrar texto. Por defecto, hay una imagen `ParameterizedText` llamada `text` declarada, pero el usuario puede declarar más de una para mostrar múltiples bloques de texto en la pantalla a la vez.

Function: `ParameterizedText (style='default', **properties):`

Esto se puede usar como una imagen. Cuando se usa, se espera que esta imagen tenga un solo parámetro, una cadena que se renderiza como la imagen.

Dinámico (Dynamic)

`DynamicDisplayable` se puede usar en estilos para cambiar el elemento mostrable que se muestra durante el transcurso del juego.

Function: `DynamicDisplayable (function, *args, **kwargs):`

Este elemento mostrable evalúa una función y usa el resultado de esa función para determinar qué mostrar.

`function` debe ser una función que acepte al menos dos argumentos. El primer argumento es el tiempo (en segundos) que el `DynamicDisplayable` se ha mostrado. El segundo argumento es el número de segundos que un elemento mostrable con la misma etiqueta de imagen se ha mostrado. Los argumentos posicionales y de palabra clave adicionales pasados a `DynamicDisplayable` también se proporcionan a la función. Se espera que la función devuelva una tupla de 2 elementos. El primer elemento de esta tupla debe ser un elemento mostrable. El segundo debe ser el tiempo (en segundos) que el valor de retorno es válido, o `None` para indicar que el valor de retorno es válido indefinidamente.

La función se evalúa al menos una vez por cada interacción. También se evalúa de nuevo cuando ha transcurrido el tiempo especificado.

Ten en cuenta que `DynamicDisplayable` no acepta propiedades. En su lugar, utiliza las propiedades del elemento mostrable devuelto por `function`.

Ten en cuenta que `ConditionSwitch` es un envoltorio alrededor de `DynamicDisplayable` y puede ser más simple dependiendo de tus necesidades.

Para compatibilidad con versiones anteriores a la 5.6.3 de Ren'Py, `DynamicDisplayable` también acepta una cadena como argumento de función. En este caso, la cadena se evalúa una vez por interacción, y el resultado se usa como el elemento mostrable para esa interacción.

Function: `ConditionSwitch (*args, **properties):`

Este es un envoltorio alrededor de `DynamicDisplayable` que muestra el primer elemento mostrable que coincida con una condición. Toma un número par de argumentos posicionales, donde los argumentos en posiciones impares son cadenas que contienen condiciones de Python, y los argumentos en posiciones pares son elementos mostrables. En cada interacción, evalúa las condiciones en orden para encontrar la primera que sea verdadera, y luego muestra ese elemento mostrable. Es un error que ninguna condición sea verdadera.

Si se proporcionan, los argumentos de palabra clave se usan para posicionar el elemento mostrable elegido.

Function: `ShowingSwitch (*args, **kwargs):`

Esto elige un elemento mostrable para mostrar basándose en qué imágenes se están mostrando en la pantalla. Espera un número par de argumentos posicionales. Los argumentos posicionales en posiciones impares deben ser nombres de imágenes, mientras que los argumentos posicionales en posiciones pares deben ser elementos mostrables. Una imagen coincide si es el prefijo de una imagen mostrada. Si el nombre de la imagen es `None`, siempre coincide. Es un error si no se produce ninguna coincidencia.

Toma el argumento de palabra clave `layer`, que especifica la capa que se comprobará para ver si aparecen imágenes en ella; el valor predeterminado es `"master"`. Otros argumentos de palabra clave se usan para posicionar el elemento mostrable elegido.

Los nombres de imágenes mostradas son rastreados por el mecanismo de carga predictiva de imágenes, por lo que `ShowingSwitch` cargará correctamente las imágenes de forma predictiva.

Dado que `ShowingSwitch` coincide con el prefijo de una imagen mostrada, puedes `engañarlo` para que piense que se muestra una imagen particular usando versiones nulas de imágenes:

```
init:
    image eileen happy hidden = Null()
    image eileen concerned hidden = Null()
```

Luego, si Eileen no es visible pero quieres que su imagen lateral muestre una expresión particular cuando habla, puedes usar un código como este:

```
show eileen concerned hidden
e "¿Estás seguro de que esa es una buena idea?"
";Oh! No me di cuenta de que Eileen estaba allí."
show eileen concerned
```

offset - A time offset, in seconds. Use this to have a blink that does not start at the start of the on phase. `anim_timebase`—If `True`, use the animation timebase, if `false`, the displayable timebase. `Function: anim.SMAAnimation(initial, *args, **properties) : This creates a state-machine animation. Such an animation has a name (a string) of the initial state we start in. show old—If the keyword parameter show old is True, then if True, we use the animation timebase. If False, we use the displayable timebase. This accepts as an additional argument anim.State(name, image, *atlist, **properties) : This creates a state that can be used in an anim.SMAAnimation. A string giving the name of this state. image—The displayable that is shown to the user while we are in (entering) the state. atlist—A list of functions to call on the image. (In general, if something can be used in an at clause, it can be used here.)`

Animaciones

Esta sección ha quedado algo obsoleta con la introducción de ATL en Ren'Py 6.10.

Ren'Py proporciona varios tipos de elementos mostrables de animación.

Estas funciones de animación toman un parámetro `anim_timebase` que determina qué base de tiempo utilizar. La base de tiempo de animación, utilizada cuando `anim_timebase` es `True`, comienza en el instante del primer fotograma desde el cual se ha mostrado en la pantalla la etiqueta de la imagen que contiene esta animación. Esto se puede usar para cambiar entre dos animaciones, de forma que se asegure que estén sincronizadas con la misma base de tiempo. La base de tiempo del elemento mostrable, utilizada cuando

`anim_timebase=False`, comienza en el primer fotograma después de que se muestre el elemento mostrable, y se puede usar para asegurar que se vea toda la animación, incluso si ya había una imagen con la misma etiqueta en la pantalla.

La base de tiempo del elemento mostrable se establece a cero para los hijos de un `Button`, cada vez que el botón recibe o pierde el foco. Esto significa que las animaciones que son hijas del botón (incluyendo los fondos del botón) que tienen `anim_timebase=False` se reiniciarán cuando el botón cambie de foco.

Las funciones de animación son:

Función: Animation (*args, **properties)

Un elemento mostrable que dibuja una animación, que es una serie de imágenes que se muestran con retardos de tiempo entre ellas.

Los argumentos en posiciones impares (primero, tercero, quinto, etc.) de `Animation` se interpretan como nombres de archivo de imagen, mientras que los argumentos en posiciones pares son el tiempo de retardo entre cada imagen. Si el número de argumentos es impar, la animación se detendrá con la última imagen (en realidad, se retrasa un año antes de repetirse). De lo contrario, la animación se reiniciará después del tiempo de retardo final.

- `anim_timebase` - Si es `True` (el valor predeterminado), usa la base de tiempo de animación. De lo contrario, usa la base de tiempo del elemento mostrable.

Función: anim.TransitionAnimation (*args, **kwargs)

Un elemento mostrable que dibuja una animación con cada fotograma separado por una transición.

Toma argumentos de modo que el 1º, 4º, 7º, ... argumentos son elementos mostrables, el 2º, 5º, 8º, ... son tiempos, y el 3º, 6º, 9º, ... son transiciones.

Esto muestra el primer elemento mostrable durante el tiempo dado, luego realiza la transición al segundo elemento mostrable usando la transición dada, y lo muestra durante el tiempo dado (el tiempo de la transición se resta del tiempo que se muestra el fotograma), y así sucesivamente.

Una transición puede ser `None`, para especificar que no se debe usar ninguna transición.

El último argumento puede ser una transición (en cuyo caso se usa esa transición para volver al primer fotograma), o un elemento mostrable (que se muestra para siempre).

No todas las transiciones se pueden usar con esto (en particular, las diversas formas de `MoveTransition` no pueden).

Hay un argumento de palabra clave, aparte de las propiedades de estilo habituales:

- `anim_timebase` - Si es `True` (el valor predeterminado), usa la base de tiempo de animación. De lo contrario, usa la base de tiempo del elemento mostrable.

Función: anim.Blink (image, on=0.5, off=0.5, rise=0.5, set=0.5, high=1.0, low=0.0, offset=0.0, anim_timebase=False, **properties)

Toma como argumento una imagen o widget, y parpadea esa imagen variando su alfa. La secuencia de fases es: encendido - fijar - apagado - subir - encendido - ... Todos los tiempos se dan en segundos, todos los alfas son fracciones entre 0 y 1.

- `image` - La imagen o widget que parpadeará.

- **on** - El tiempo que el widget pasa encendido, con alfa alto.
- **off** - El tiempo que el widget pasa apagado, con alfa bajo.
- **rise** - El tiempo que tarda el widget en subir de alfa bajo a alfa alto.
- **set** - El tiempo que tarda el widget en bajar de alfa alto a alfa bajo.
- **high** - El alfa alto.
- **low** - El alfa bajo.
- **offset** - Un desplazamiento de tiempo, en segundos. Úsalo para tener un parpadeo que no comience al inicio de la fase de encendido.
- **anim_timebase** - Si es **True**, usa la base de tiempo de animación; si es **False**, usa la base de tiempo del elemento mostrable.

Función: `anim.SMAnimation (initial, *args, **properties)`

Crea una animación de máquina de estados. Tal animación se crea recorriendo aleatoriamente los bordes entre estados en una máquina de estados definida. Cada estado corresponde a una imagen mostrada al usuario, con los bordes correspondientes al tiempo que se muestra una imagen y la transición con la que se muestra.

Las imágenes se muestran, quizás con una transición, cuando se transita a un estado que contiene esa imagen.

- **initial** - El nombre (una cadena) del estado inicial en el que comenzamos.
- **showold** - Si el parámetro de palabra clave **showold** es **True**, entonces se muestra la imagen antigua en lugar de la nueva imagen cuando se está en un borde.
- **anim_timebase** - Si es **True**, usamos la base de tiempo de animación. Si es **False**, usamos la base de tiempo del elemento mostrable.

Acepta como argumentos adicionales los objetos **anim.State** y **anim.Edge** que se utilizan para componer esta máquina de estados.

Función: `anim.State (name, image, *atlist, **properties)`

Crea un estado que se puede usar en un **anim.SMAnimation**.

- **name** - Una cadena que da el nombre de este estado.
- **image** - El elemento mostrable que se muestra al usuario mientras estamos en (entrando en) este estado. Por conveniencia, también puede ser una cadena o tupla, que se interpreta con **Image**.
- **image** debe ser **None** cuando este **State** se usa con movimiento, para indicar que la imagen será reemplazada por el hijo del movimiento.
- **atlist** - Una lista de funciones a llamar en la imagen (en general, si algo se puede usar en una cláusula **at**, también se puede usar aquí).

Si se proporcionan argumentos de palabra clave, se usan para construir un objeto **Position**, que modifica la posición de la imagen.

Función: `anim.Edge` (`old`, `delay`, `new`, `trans=None`, `prob=1`)

Crea un borde que se puede usar con un `anim.SMAAnimation`.

- `old` - El nombre (una cadena) del estado del que proviene esta transición.
- `delay` - El número de segundos que dura esta transición.
- `new` - El nombre (una cadena) del estado al que va esta transición.
- `trans` - La transición que se usará para mostrar la imagen que se encuentra en el nuevo estado. Si es `None`, la imagen se muestra inmediatamente.
- `prob` - El número de veces que se añade este borde. Esto se puede usar para hacer que una transición sea más probable que otras. Por ejemplo, si una transición desde un estado tiene `prob=5` y la otra tiene `prob=1`, entonces la que tiene `prob=5` se ejecutará 5/6 de las veces, mientras que la que tiene `prob=1` solo ocurrirá 1/6 de las veces (no hagas esto demasiado grande, ya que el uso de memoria es proporcional a este valor).

Presentamos dos ejemplos de esto en acción. El primero muestra cómo se puede crear un personaje que ocasionalmente, de forma aleatoria, realiza un parpadeo de 3 fotogramas aproximadamente una vez por minuto.

```
init:
    image blinking = anim.SMAAnimation("a",
        anim.State("a", "eyes_open.png"),
        # Este borde nos mantiene mostrando los ojos abiertos durante un segundo.
        anim.Edge("a", 1.0, "a", prob=60),
        # Este borde hace que los ojos comiencen a cerrarse...
        anim.Edge("a", 0.25, "b"),
        # ...porque nos trae aquí.
        anim.State("b", "eyes_half.png"),
        # Y así sucesivamente...
        anim.Edge("b", 0.25, "c"),
        anim.State("c", "eyes_closed.png"),
        anim.Edge("c", 0.25, "d"),
        anim.State("d", "eyes_half.png"),
        # Y de vuelta a a.
        anim.Edge("d", 0.5, "a")
    )
```

Recuerda, `State` puede tomar una `Position`, y `Edge` puede tomar una transición. Esto te permite mover cosas por la pantalla, disolver imágenes en otras, y hacer todo tipo de cosas complicadas e inesperadas (pero ten cuidado... no todas las transiciones hacen lo que esperarías cuando se usan con `SMAAnimation`).

Función: `anim.Filmstrip (image, framesize, gridsize, delay, frames=None, loop=True, **properties)`

Esta función no está obsoleta.

Crea una animación a partir de una sola imagen. Esta imagen debe consistir en una cuadrícula de fotogramas, donde el número de columnas y filas en la cuadrícula se toma de `gridsize`, y el tamaño de cada fotograma en la cuadrícula se toma de `framesize`. Toma los fotogramas y los coloca en una `Animation`, con el retardo dado entre cada fotograma. Los fotogramas se toman recorriendo de izquierda a derecha a lo largo de la primera fila, de izquierda a derecha a lo largo de la segunda fila, y así sucesivamente hasta que se consuman todos los fotogramas o se tome un número específico de fotogramas.

- `image` - La imagen de la que se deben tomar los fotogramas.
- `framesize` - Una tupla (ancho, alto) que da el tamaño de cada uno de los fotogramas en la animación.
- `gridsize` - Una tupla (columnas, filas) que da el número de columnas y filas en la cuadrícula.
- `delay` - El retardo, en segundos, entre fotogramas.
- `frames` - El número de fotogramas en esta animación. Si es `None`, entonces el valor predeterminado es columnas * filas fotogramas, es decir, tomando todos los fotogramas de la cuadrícula.
- `loop` - Si es `True`, se repite al final de la animación. Si es `False`, la animación se realiza una vez y luego se detiene.

Otros argumentos de palabra clave son como para `anim.SMAAnimation`.

Layouts

Estos elementos mostrables se utilizan para distribuir múltiples elementos mostrables a la vez.

Función: `LiveComposite (size, *args, **properties)`

Es similar a `im.Composite`, pero se puede usar con elementos mostrables en lugar de imágenes. Esto permite usarlo para componer, por ejemplo, una animación encima de la imagen.

Es menos eficiente que `im.Composite`, ya que necesita dibujar todos los elementos mostrables en la pantalla. Por otro lado, permite que los elementos mostrables cambien mientras están en la pantalla, lo cual es necesario para la animación.

Toma un número variable de argumentos. El primer argumento es `size`, que debe ser una tupla que da el ancho y alto de los widgets compuestos, para fines de diseño.

Luego toma un número par de argumentos adicionales (para un número impar de argumentos totales). El segundo y otros argumentos en posiciones pares contienen tuplas de posición, mientras que el tercero y otros argumentos en posiciones impares dan elementos mostrables. Un argumento de posición da la posición del elemento mostrable que le sigue inmediatamente, con la posición expresada como una tupla que da un desplazamiento desde la esquina superior izquierda del `LiveComposite`. Los elementos mostrables se dibujan en orden de abajo hacia arriba, siendo el último el más cercano al usuario.

Función: Fixed (*args, **properties)

Un diseño que se expande para ocupar el tamaño que se le asigna. A cada elemento mostrable se le asigna el tamaño completo del diseño, estando el primer elemento mostrable más alejado del usuario que el segundo, y así sucesivamente.

Esta función toma tanto argumentos posicionales como argumentos de palabra clave. Los argumentos posicionales deben ser elementos mostrables o imágenes para diseñar. Los argumentos de palabra clave se interpretan como propiedades de estilo, excepto el argumento de palabra clave `style`, que es el nombre del estilo padre de este diseño.

Función: HBox (*args, **properties)

Un diseño que distribuye los elementos mostrables de izquierda a derecha.

Esta función toma tanto argumentos posicionales como argumentos de palabra clave. Los argumentos posicionales deben ser elementos mostrables o imágenes para diseñar. Los argumentos de palabra clave se interpretan como propiedades de estilo, excepto el argumento de palabra clave `style`, que es el nombre del estilo padre de este diseño.

Función: VBox (*args, **properties)

Un diseño que distribuye los elementos mostrables de arriba a abajo.

Esta función toma tanto argumentos posicionales como argumentos de palabra clave. Los argumentos posicionales deben ser elementos mostrables o imágenes para diseñar. Los argumentos de palabra clave se interpretan como propiedades de estilo, excepto el argumento de palabra clave `style`, que es el nombre del estilo padre de este diseño.

Widgets

Estos se usan generalmente con las funciones de interfaz de usuario (ui), pero a veces también se pueden usar como elementos mostrables.

Función: Bar (range, value, clicked=None, **properties)

Crea un elemento mostrable `Bar`, el equivalente mostrable de `ui.bar`. Los parámetros son los mismos que para `ui.bar`.

Función: Button (child, clicked=None, hovered=None, unhovered=None, role=None, **properties)

Crea un elemento mostrable de botón que el usuario puede pulsar. Cuando se hace clic en este botón o se selecciona de otro modo, se llama a la función proporcionada como argumento `clicked`. Si devuelve un valor, ese valor se devuelve desde `ui.interact`.

- `child` - El elemento mostrable que se contiene dentro de este botón. Es obligatorio.
- `clicked` - Una función que se llama cuando se hace clic en este botón.
- `hovered` - Una función que se llama cuando este botón gana el foco.
- `unhovered` - Una función que se llama cuando este botón pierde el foco.
- `role` - El rol que desempeña este botón. Puede ser una cadena vacía, o `"selected_"`.

Función: Window (child, **properties)

Crea un elemento mostrable Window con el hijo y las propiedades dadas.

Función: Viewport (child_size=(None, None), xadjustment=None, yadjustment=None, set_adjustments=True, mousewheel=False, draggable=False, style='viewport', **properties)

Crea un elemento mostrable de viewport. Un viewport restringe el tamaño de su hijo y permite que el hijo se muestre con un desplazamiento. Esto se puede usar para recortar y desplazar las imágenes.

Consulta `ui.viewport` para conocer el significado de los parámetros.

Función: Null (width=0, height=0, **properties)

Un elemento mostrable que no muestra nada. Al establecer las propiedades de ancho y alto de este elemento mostrable, se puede usar para ocupar espacio.

Movimiento de partículas

Ren'Py admite movimiento de partículas. El movimiento de partículas es el movimiento de muchas partículas en la pantalla a la vez, con una vida útil más corta que una sola interacción. El movimiento de partículas se puede usar para tener múltiples elementos moviéndose en la pantalla a la vez, como nieve, pétalos de cerezo, burbujas, luciérnagas y más. Hemos proporcionado dos interfaces para el motor de partículas. La función `SnowBlossom` es un constructor conveniente para los casos más comunes de partículas que caen o suben linealmente, mientras que la función `Particles` da control total sobre el motor de partículas.

Función: SnowBlossom (image, count=10, border=50, xspeed=(20, 50), yspeed=(100, 200), start=0, horizontal=False)

Implementa el efecto `snowblossom`, que es un movimiento lineal simple hacia arriba, abajo, izquierda o derecha. Este efecto se puede usar para pétalos de cerezo cayendo, nieve cayendo, burbujas subiendo, dientes de león flotando y otras cosas.

- **image** - La imagen que se usará para las partículas. Puede ser cualquier elemento mostrable, por lo que está bien usar una `Animation` como argumento.
- **count** - El número de partículas que se mantienen en un momento dado.
- **border** - Cuántos píxeles fuera de la pantalla se mantiene una partícula. Se usa para asegurar que una partícula no se muestre en la pantalla cuando se crea y que esté completamente fuera de la pantalla cuando se destruye.
- **xspeed** - La velocidad horizontal de las partículas, en píxeles por segundo. Puede ser un solo entero o una tupla de dos enteros. En este último caso, los dos números se usan como un rango del cual elegir la velocidad horizontal para cada partícula. Los números pueden ser positivos o negativos, siempre que el segundo sea mayor que el primero.

- **yspeed** - La velocidad vertical de las partículas, en píxeles por segundo. Puede ser un solo entero, pero casi seguro que debe ser un par de enteros que se usan como un rango para elegir la velocidad vertical de cada partícula. El segundo número de la tupla debe ser mayor que el primero.
- **start** - El número de segundos que tomará poner en movimiento todas las partículas. Establecer esto a un número distinto de cero evita que una oleada inicial de partículas abrume la pantalla. Cada partícula comenzará en un tiempo aleatorio menor que este número de segundos.
- **fast** - Si es **true**, todas las partículas se inician a la vez y se inician en lugares aleatorios de la pantalla, en lugar de en la parte superior o inferior.
- **horizontal** - Si es **true**, las partículas comienzan en el borde izquierdo o derecho de la pantalla. Si es **false**, comienzan a lo largo del borde superior o inferior.

El resultado de **SnowBlossom** se usa mejor para definir una imagen, que luego se puede mostrar al usuario.

```
init:
    image blossoms = SnowBlossom(Animation("sakura1.png", 0.15, "sakura2.png", 0.15))
```

Puede tener sentido mostrar múltiples **SnowBlossom** a la vez. Por ejemplo, en una escena con pétalos de cerezo cayendo, se pueden tener pequeños pétalos de cerezo cayendo lentamente detrás de un personaje, mientras que otros más grandes caen más rápido delante de ella.

Si **SnowBlossom** no hace lo que quieres, puede tener sentido definir tu propio movimiento de partículas. Esto se hace llamando a la función **Particles**.

Función: `Particles (factory, style='default', **properties)`

Admite movimiento de partículas.

- **factory** - Un objeto de fábrica.

La función **particles** espera tomar como argumento un objeto de fábrica. Este objeto (que debe ser serializable con **pickle**) debe admitir dos métodos.

El método **create** del objeto de fábrica se llama una vez por fotograma con dos argumentos. El primero es una lista de partículas existentes, o **None** si es la primera vez que se muestra este **Particles** (y por lo tanto no hay partículas en la pantalla). El segundo argumento es el tiempo en segundos desde algún punto arbitrario, que aumenta cada vez que se llama a **create**. Se espera que el método devuelva una lista de nuevas partículas creadas para este fotograma, o una lista vacía si no se van a crear partículas en este fotograma.

El método **predict** del objeto de fábrica se llama cuando se solicita la predicción de imágenes para el **Particles**. Se espera que devuelva una lista de elementos mostrables y/o nombres de archivo de imagen que se usarán.

Las partículas se representan mediante los objetos devueltos por cada función de fábrica. Cada objeto de partícula debe tener un método **update**. Este método se llama una vez por fotograma por partícula, generalmente con el tiempo desde el mismo punto arbitrario que se usó para crear el objeto (el punto arbitrario puede cambiar cuando se oculta

y se muestra, por lo que el código de partículas debe estar preparado para manejar esto). El método `update` puede devolver `None` para indicar que la partícula está muerta. No se muestra nada para una partícula muerta y nunca se llama a `update` en ella. El método `update` también puede devolver una tupla (`xpos`, `ypos`, `time`, `displayable`). Los parámetros `xpos` e `ypos` son una posición en la pantalla para mostrar la partícula, interpretada de la misma manera que las propiedades de estilo `xpos` e `ypos`. `time` es el tiempo de visualización. Debe comenzar con el parámetro `time`, pero puede tener sentido desplazarlo para que múltiples animaciones de partículas estén desfasadas. Finalmente, `displayable` es un elemento mostrable o nombre de archivo de imagen que se muestra como la partícula.

Funciones de posición y movimiento

Esta sección ha quedado algo obsoleta con la introducción de ATL en Ren'Py 6.10. El resultado de estas funciones es adecuado para usar como argumento de la cláusula `at` de las declaraciones `scene` y `show`. El resultado también se puede llamar (usando una segunda llamada) para devolver un elemento mostrable.

Función: `Position (**properties)`

`Position`, cuando se le dan propiedades de posición como argumentos, devuelve un objeto invocable que se puede pasar a la cláusula `at` de una declaración `show` o `scene` para mostrar la imagen en la ubicación dada. Consulta la sección de Propiedades de posición para obtener una explicación completa de cómo se usan para distribuir los elementos.

Función: `Motion (function, period, repeat=False, bounce=False, time_warp=None, add_sizes=False, anim_timebase=False, **properties)`

`Motion`, cuando se le dan los argumentos apropiados, devuelve un objeto que, cuando se proporciona como la cláusula `at` de una imagen, hace que la imagen se mueva en la pantalla.

- `function` es una función que toma uno o dos argumentos. El primer argumento es una fracción del período, un número entre 0 y 1. Si `add_sizes` es verdadero, `function` debe tomar un segundo argumento, una tupla de 4 elementos que da el ancho y alto del área en la que se mostrará el hijo, y el ancho y alto del hijo mismo.
- `function` debe devolver una tupla que contenga dos o cuatro valores. Los dos primeros valores se interpretan como `xpos` e `ypos` del movimiento. (Ten en cuenta que si estos valores son números de punto flotante, se interpretan como una fracción de la pantalla. Si son enteros, se interpretan como la posición absoluta del ancla del movimiento.) Si se devuelven cuatro valores, el tercero y cuarto se interpretan como `xanchor` e `yanchor`.

Ten en cuenta que la función puede ser serializada con `pickle`, lo que significa que no puede ser una función interna o una lambda, sino que debe ser una función definida en un bloque `init` de tu guión. En general, es mejor usar un `Pan` o un `Move` en lugar de definir tu propio movimiento.

- `period` es el tiempo, en segundos, que tarda en completarse un ciclo del movimiento.

- **repeat** - Si es **True**, el ciclo se repite cuando termina; si es **False**, el movimiento se detiene después de un período.
- **bounce** - Si es **True**, el argumento de la función va de 0 a 1 a 0 en un solo período; si es **False**, va de 0 a 1.
- **time_warp** - Si se proporciona, es una función que toma un período de tiempo fraccionario (un número entre 0.0 y 1.0) y devuelve un período de tiempo fraccionario. Esto permite movimientos no lineales. Esta función también puede ser serializada con **pickle**.
- **anim_timebase** - **true** para usar la base de tiempo de animación, **false** para usar la base de tiempo del elemento mostrable.
- **add_sizes** - Añadido en la versión 5.6.6.

Función: Pan (startpos, endpos, time, repeat=False, bounce=False, time_warp=None, **properties)

Pan, cuando se le dan los argumentos apropiados, devuelve un objeto que se puede pasar a la cláusula **at** de una imagen para hacer que la imagen se desplace en la pantalla.

Los parámetros **startpos** y **endpos** son tuplas que contienen las coordenadas **x** e **y** de la esquina superior izquierda de la pantalla en relación con la imagen. **time** es el tiempo que tardará esta posición en moverse de **startpos** a **endpos**. **repeat**, **bounce** y **time_warp** son como en **Motion**.

Dado que la implementación actual de Ren'Py es bastante limitada, hay muchas restricciones que ponemos en **Pan**. La principal es que siempre debe haber una cantidad de píxeles equivalente a una pantalla a la derecha y debajo de las posiciones inicial y final. No asegurar esto puede llevar a un renderizado inconsistente.

Ten en cuenta que el **Pan** se mostrará inmediatamente y que Ren'Py no esperará a que se complete antes de pasar a la siguiente declaración. Esto puede llevar a que el **Pan** se superponga con texto o diálogo. Es posible que quieras usar una llamada a **renpy.pause** para retrasar el tiempo que tardará en completarse el **Pan**. Finalmente, también ten en cuenta que cuando se completa un **Pan**, la imagen se fija en la posición final.

Función: Move (startpos, endpos, time, repeat=False, bounce=False, time_warp=None, **properties)

Move es similar a **Pan** en cuanto a que implica mover cosas. Pero mientras que **Pan** mueve la pantalla a través de una imagen, **Move** mueve una imagen en la pantalla. Específicamente, **Move** cambia la propiedad de estilo de posición de una imagen con el tiempo.

Move toma como parámetros una posición inicial, una posición final, el tiempo que tarda en moverse de la posición inicial a la final, y propiedades de posición adicionales. Las posiciones pueden ser pares que dan las propiedades **xpos** e **ypos**, o tuplas de 4 elementos que dan **xpos**, **ypos**, **xanchor** e **yanchor**. Estas propiedades se pueden dar como enteros o números de punto flotante, pero para cualquier propiedad no es permisible mezclar ambos. **repeat**, **bounce** y **time_warp** son como en **Motion**.

En general, se quiere usar **Pan** cuando una imagen es más grande que la pantalla, y **Move** cuando es más pequeña. Tanto **Pan** como **Move** son casos especiales de **Motion**.

Función: SplineMotion (points, time, anchors=(0.5, 0.5), repeat=False, bounce=False, anim_timebase=False, time_warp=None, **properties)

Crea un movimiento basado en splines, donde una spline puede consistir en segmentos lineales, bezier cuadráticos y bezier cúbicos.

La ruta es una lista de segmentos, donde cada segmento es una tupla que contiene entre 1 y 3 puntos, y un tiempo opcional. Un punto se representa por un par (x, y) o una tupla (x, y, xanchor, yanchor). Cuando no se especifican en un punto, los anclajes se toman del parámetro `anchors`. El tiempo se representa como un número de punto flotante entre 0.0 y 1.0, y da el momento en que el movimiento a lo largo del segmento debe completarse.

- Un segmento lineal consiste en un solo punto, el punto al que moverse.
- Una curva cuadrática contiene dos puntos: el punto al que moverse y el punto de control único.
- Una curva cúbica contiene tres puntos: el punto al que moverse y los dos puntos de control.

Cualquier tiempo que no especifiques manualmente se interpola linealmente entre los tiempos anteriores y siguientes que se especifican. Esto permite especificar que en un momento concreto, el movimiento estará en un punto concreto. Por defecto, el tiempo inicial es 0.0 y el tiempo final es 1.0, a menos que especifiques algo diferente.

Hay un editor de splines que ayuda con la edición de splines.

`repeat`, `bounce`, `anim_timebase` y `time_warp` son como en `Motion`.

`anim.SMAAnimation` también se puede usar para declarar movimientos complicados. Usa `None` en lugar de una imagen en los `State`, y proporciona una transición de movimiento al moverse entre estados. Un `SMAAnimation` así creado se puede pasar a la cláusula `at` de una imagen, permitiendo mover cosas por la pantalla.

Estas cláusulas de movimiento también se pueden usar como transiciones, en cuyo caso afectan a la posición de una sola capa o de toda la pantalla, según corresponda.

Función: Zoom (size, start, end, time, after_child=None, time_warp=None, bilinear=True, opaque=True, anim_timebase=False, repeat=False, **properties)

Provoca un zoom utilizando el escalado de imágenes. Cuando se usa como una cláusula `at`, crea un elemento mostrable. El renderizado de este elemento mostrable siempre tiene el tamaño proporcionado. Se renderiza el hijo y se recorta un rectángulo de él. Este rectángulo se interpola entre los rectángulos inicial y final. Luego, el rectángulo se escala al tamaño proporcionado. El zoom tardará `time` segundos, después de los cuales mostrará el rectángulo final, a menos que se proporcione un `after_child`.

Los rectángulos inicial y final deben caber completamente dentro del tamaño del hijo, de lo contrario se producirá un error.

- `size` - El tamaño al que se escala el rectángulo, una tupla (ancho, alto).
- `start` - El rectángulo inicial, una tupla (desplazamiento x, desplazamiento y, ancho, alto).

- **end** - El rectángulo final, una tupla (desplazamiento x, desplazamiento y, ancho, alto).
- **time** - El tiempo que tomará interpolar del rectángulo inicial al final.
- **after_child** - Si está presente, un segundo widget hijo. Este elemento mostrable se renderizará después de que se complete el zoom. Úsalo para cambiar a un elemento mostrable nítido después del zoom.
- **time_warp** - Si no es `None`, una función que toma un período fraccionario (entre 0.0 y 1.0) y devuelve un período fraccionario. Úsalo para implementar zooms no lineales. Esta función puede ser serializada con `pickle`, por lo que no puede ser una lambda u otra función no de nivel superior.
- **bilinear** - Si es `True` (el valor predeterminado), se usa el filtrado bilineal. Si es `false`, se usa un filtrado de vecino más cercano, feo pero rápido.
- **opaque** - Si es `True` y **bilinear** es `True`, se usa un método muy eficiente que no admite transparencia. Si es `False`, admite transparencia, pero es menos eficiente.
- **anim_timebase** - `true` para usar la base de tiempo de animación, `false` para usar la base de tiempo del elemento mostrable.
- **repeat** - Hace que el zoom se repita cada **time** segundos.

Función: FactorZoom (start, end, time, after_child=None, time_warp=None, bilinear=True, opaque=True, anim_timebase=False, repeat=False, **properties)

Provoca un zoom utilizando el escalado de imágenes. Cuando se usa como una cláusula **at**, crea un elemento mostrable. El renderizado de este elemento mostrable siempre tiene el tamaño proporcionado. Se renderiza el hijo y luego se escala por un factor apropiado, interpolado entre los factores inicial y final. El rectángulo se escala al tamaño proporcionado. El zoom tardará **time** segundos, después de los cuales mostrará el factor final, a menos que se proporcione un **after_child**.

El algoritmo utilizado para escalar no realiza ninguna interpolación u otro suavizado.

- **start** - El factor de zoom inicial, un número de punto flotante.
- **end** - El factor de zoom final, un número de punto flotante.
- **time** - El tiempo que tomará interpolar de los factores inicial a final.
- **after_child** - Si está presente, un segundo widget hijo. Este elemento mostrable se renderizará después de que se complete el zoom. Úsalo para cambiar a un elemento mostrable nítido después del zoom.
- **time_warp** - Si no es `None`, una función que toma un período fraccionario (entre 0.0 y 1.0) y devuelve un período fraccionario. Úsalo para implementar zooms no lineales. Esta función puede ser serializada con `pickle`, por lo que no puede ser una lambda u otra función no de nivel superior.

- **bilinear** - Si es **True** (el valor predeterminado), se usa el filtrado bilineal. Si es **false**, se usa un filtrado de vecino más cercano, feo pero rápido.
- **opaque** - Si es **True** y **bilinear** es **True**, se usa un método muy eficiente que no admite transparencia. Si es **False**, admite transparencia, pero es menos eficiente.
- **anim_timebase** - **true** para usar la base de tiempo de animación, **false** para usar la base de tiempo del elemento mostrable.
- **repeat** - Hace que el zoom se repita cada **time** segundos.

Función: RotoZoom (rot_start, rot_end, rot_delay, zoom_start, zoom_end, zoom_delay, rot_repeat=False, zoom_repeat=False, rot_bounce=False, zoom_bounce=False, rot_anim_timebase=False, zoom_anim_timebase=False, rot_time_warp=None, zoom_time_warp=None, opaque=False, **properties)

Esta función devuelve un objeto, adecuado para usar en una cláusula **at**, que rota y/o escala su hijo.

- **rot_start** - El número de grados de rotación en el sentido de las agujas del reloj en el tiempo inicial.
- **rot_end** - El número de grados de rotación en el sentido de las agujas del reloj en el tiempo final.
- **rot_delay** - El tiempo que tarda en completarse la rotación.
- **zoom_start** - El factor de zoom en el tiempo inicial.
- **zoom_end** - El factor de zoom en el tiempo final.
- **zoom_delay** - El tiempo que tarda en completarse un zoom.
- **rot_repeat** - Si es **true**, el ciclo de rotación se repite una vez que termina.
- **zoom_repeat** - Si es **true**, el ciclo de zoom se repite una vez que termina.
- **rot_bounce** - Si es **true**, rota de **rot_start** a **rot_end** a **rot_start** cada ciclo. Si es **false**, rota de **rot_start** a **rot_end** una vez.
- **zoom_bounce** - Si es **true**, escala de **zoom_start** a **zoom_end** a **zoom_start** cada ciclo. Si es **false**, escala de **zoom_start** a **zoom_end** una vez.
- **rot_anim_timebase** - Si es **true**, los tiempos de rotación se juzgan desde que se mostró por primera vez un elemento mostrable con la misma etiqueta. Si es **false**, los tiempos se juzgan desde que se mostró por primera vez este elemento mostrable.
- **zoom_anim_timebase** - Si es **true**, los tiempos de zoom se juzgan desde que se mostró por primera vez un elemento mostrable con la misma etiqueta. Si es **false**, los tiempos se juzgan desde que se mostró por primera vez este elemento mostrable.
- **opaque** - Debe ser **True** si el hijo es completamente opaco, y **False** en caso contrario.

RotoZoom utiliza un método de interpolación bilineal que es preciso cuando el factor de zoom es ≥ 0.5 .

La rotación es alrededor del centro del elemento mostrable hijo. Ten en cuenta que esto reduce en 1 píxel horizontal y verticalmente lo que está rotando. Las imágenes deben ser 1 píxel más grandes de lo que serían de otro modo.

El elemento mostrable producido tiene un alto y ancho igual a la hipotenusa de los lados del hijo. Esto puede alterar un poco el diseño. Para minimizar esto, posiciona el centro del **RotoZoom**.

El tiempo que tarda **RotoZoom** es proporcional al área de la imagen rotada y escalada que se muestra en la pantalla.

Función: Alpha (start, end, time, repeat=False, bounce=False, time_warp=None, add_sizes=False, anim_timebase=False, **properties)

Alpha devuelve una función que, cuando se llama con un elemento mostrable, permite que el alfa de ese elemento mostrable cambie con el tiempo. Es una versión más cara de **im.Alpha** que funciona con cualquier elemento mostrable y puede cambiar con el tiempo.

- **start** - El alfa al inicio, un número entre 0 y 1.
- **end** - El alfa al final, un número entre 0 y 1.
- **time** - El tiempo, en segundos, que tarda en completarse un ciclo.
- **repeat** - Si es **True**, el ciclo se repite cuando termina; si es **False**, el movimiento se detiene después de un período.
- **bounce** - Si es **True**, el alfa va de **start** a **end** a **start** en un solo período; si es **False**, va de **start** a **end** solo.
- **time_warp** - Si se proporciona, es una función que toma un período de tiempo fraccionario (un número entre 0.0 y 1.0) y devuelve un período de tiempo fraccionario. Esta función debe ser serializable con **pickle**.
- **anim_timebase** - **true** para usar la base de tiempo de animación, **false** para usar la base de tiempo del elemento mostrable.

Función: Revolve (start, end, time, around=(0.5, 0.5), cor=(0.5, 0.5), **kwargs)

Se usa para hacer girar a su hijo alrededor de un punto en su padre. **around** es el punto en el padre alrededor del cual giramos, mientras que **cor** es el centro de revolución para el hijo. **start** es la revolución inicial y **end** es la revolución final, ambas en grados en el sentido de las agujas del reloj.

Otros argumentos de palabra clave son como en **Motion**.

Para aplicar los resultados de estas funciones a un elemento mostrable, usa la función **At**.

Función: At (displayable, *positions_and_motions)

La función **At** se usa para aplicar los resultados de funciones de posición y movimiento a elementos mostrables, produciendo un elemento mostrable.

Definiciones de posición

Estas posiciones se pueden usar como argumento de la cláusula **at** de una declaración **scene** o **show**.

Definición: left Una posición en la que el lado izquierdo de la imagen está alineado con el lado izquierdo de la pantalla, con la parte inferior pegada a la parte inferior de la pantalla.

Definición: right Una posición en la que el lado derecho de la imagen está alineado con el lado derecho de la pantalla, con la parte inferior de la imagen pegada a la parte inferior de la pantalla.

Definición: center Una posición en la que la imagen está centrada horizontalmente, con la parte inferior de la imagen pegada a la parte inferior de la pantalla.

Definición: offscreenleft Una posición en la que la imagen se coloca justo fuera del lado izquierdo de la pantalla. Ten en cuenta que una imagen colocada en esta posición, aunque no sea visible para el usuario, sigue consumiendo recursos, por lo que las imágenes deben ocultarse cuando no sean visibles. Esta posición está destinada a ser usada con la transición **move**.

Definición: offscreenright Una posición en la que la imagen se coloca justo fuera del lado derecho de la pantalla. Ten en cuenta que una imagen colocada en esta posición, aunque no sea visible para el usuario, sigue consumiendo recursos, por lo que las imágenes deben ocultarse cuando no sean visibles. Esta posición está destinada a ser usada con la transición **move**.

Transform

Un **Transform** permite usar una función de Python para controlar muchos aspectos de su hijo. Aunque es más complicado, **Transform** es lo suficientemente potente como para implementar todas las funciones de posición y movimiento. Un **Transform** se puede usar como un elemento mostrable (proporcionando un hijo) o como una función de posición y movimiento (omitiendo el hijo).

Function: Transform (child=None, function=None, **properties):

Una transformación aplica rotación, zoom y mezcla alfa a su hijo, en ese orden. Estas operaciones, junto con el posicionamiento del objeto transformado, están controladas por campos en el objeto **Transform**. Los objetos **Transform** se pueden componer con una sobrecarga mínima.

Parámetros. Un **Transform** toma estos parámetros:

- **child** es el hijo del **Transform**. Se puede dejar como **None**, en cuyo caso el **Transform** actuará como una función de Posición y Movimiento.
- **function** es una función de llamada de retorno que se llama antes de que se renderice el **Transform**, con el objeto **Transform**, la base de tiempo mostrada y la

base de tiempo de animación. Puede establecer cualquiera de los campos descritos a continuación para hacer que el **Transform** altere cómo se muestra. Se espera que devuelva un número de punto flotante que indique la cantidad de tiempo antes de que el **Transform** deba volver a renderizarse, en segundos. Si se devuelve 0, el **Transform** se volverá a renderizar en el siguiente fotograma.

Campos. Las propiedades de transformación ATL están disponibles como campos en un objeto **Transform**.

Los campos adicionales son:

- **hide_request** - Se establece en **true** cuando el **Transform** se oculta.
- **hide_response** - Cuando **hide_request** es **True**, se puede establecer en **False** para evitar que el **Transform** se oculte inmediatamente.

Métodos. Un **Transform** tiene el siguiente método:

- **Method: update ()**: El método **update** debe llamarse en una transformación después de que uno o más campos se hayan cambiado fuera de la función de llamada de retorno de esa transformación.
- **Method: set_child (d)**: Establece el hijo de este **Transform** en **d**.

Transiciones

Por defecto, Ren'Py muestra cada escena reemplazando la escena antigua por una nueva. Esto es apropiado en general (como para cambios de emoción), pero puede ser aburrido para cambios grandes, como un cambio de ubicación o la entrada o salida de un personaje de la escena. Ren'Py admite transiciones que controlan cómo se exponen al usuario los cambios en las listas de escenas.

Las transiciones ocurren entre la última escena que se mostró al usuario y la escena actual que se ha actualizado mediante las declaraciones **scene**, **show** o **hide**. Una transición se ejecuta durante un tiempo determinado, pero el usuario puede descartarla antes de tiempo. Una vez que se muestra una transición, la escena se considera mostrada a efectos de futuras transiciones.

Las transiciones se introducen con la declaración **with**. La declaración **with** toma una expresión que es adecuada para usar con la declaración **with** (es decir, un objeto invocable que toma como entrada dos listas de escenas) y ejecuta esa transición. Alternativamente, si la expresión es **None**, la declaración **with** tiene el efecto de mostrar la escena al usuario y regresar instantáneamente. Esto es útil en combinación con una declaración **with** futura, de modo que solo algunos cambios en la lista de escenas se transicionen.

Un ejemplo es oportuno. Primero, definamos algunos objetos que se pueden pasar como argumento a una declaración **with**:

```
init:
    # Fude a negro y luego a la nueva escena.
    fade = Fade(0.5, 0, 0.5)
    # Disuelve entre escenas antiguas y nuevas.
    dissolve = Dissolve(0.5)
```


Un uso simple de `with` sería colocarlo después de una serie de declaraciones `show` y `hide` del programa. Por ejemplo:

```
scene bg whitehouse
show eileen happy
with fade
```

Esta serie de declaraciones hará que la escena antigua (mostrada antes de que comenzaran estas declaraciones) se funda a negro y sea reemplazada por la nueva escena de una sola vez. Este es un comportamiento útil, por ejemplo, cuando reemplazamos una escena por una nueva, como cuando la historia cambia de ubicación.

```
scene bg whitehouse
with None
show eileen happy with dissolve
```

La declaración `with None` es útil para dividir los cambios en la lista de escenas en partes, como en el ejemplo anterior. Cuando se ejecuta, el fondo se muestra instantáneamente y luego la imagen del personaje se disuelve sobre el fondo.

Otro uso de la declaración `with None` es eliminar elementos transitorios antes de que comience una transición. Por defecto, la lista de escenas incluye elementos transitorios como diálogos, pensamientos y menús. `with None` siempre se ejecuta sin estos elementos y, por lo tanto, los elimina.

Las declaraciones `show`, `hide` y `scene` admiten una cláusula `with`. Una de estas declaraciones con una cláusula `with` asociada se convierte en tres declaraciones: una declaración `with None`, la declaración original sin la cláusula `with`, y la cláusula `with` como una declaración `with`. Por ejemplo:

```
scene bg whitehouse with fade
show eileen happy at left with dissolve
show lucy happy at right with dissolve
```

se convierte en:

```
with None
scene bg whitehouse
with fade
with None
show eileen happy at left
with dissolve
with None
show lucy happy at right
with dissolve
```

Esto tiene el efecto de fundir la escena antigua y fundir el nuevo fondo, luego disolver a los personajes uno tras otro.

```
show albert at far_left
show bernardette at left
show charles at center
show derek at right
show eleanor at far_right
with dissolve
```

Además, también puedes mostrar múltiples imágenes a la vez ordenándolas consecutivamente y poniendo la declaración **with** al final, seguida de tu transición.

También permitimos que se proporcionen cláusulas **with** para declaraciones **say** y **menu**. Cuando se proporciona una cláusula **with** en una de estas declaraciones, la transición se usa para introducir el elemento **say** o **menu**. Por ejemplo:

```
e "¿Cómo estás?" with dissolve
```

Disolverá una línea de diálogo. La línea de diálogo se descartará inmediatamente, a menos que vaya seguida de una declaración o cláusula **with** que haga que transicione a otra cosa. Hay dos variables que controlan las transiciones:

Variable: default_transition = None Si no es **None**, especifica una transición predeterminada que se aplica a todas las declaraciones **say** y **menu** que no tienen una cláusula **with**. Esto solo se considera si la preferencia de transiciones está establecida en `^11`.

Variable: _window_during_transitions = False Si se establece en **true**, mostrará una línea de narración vacía durante las transiciones, siempre que estés en el contexto más externo y no haya nada transitorio en la pantalla.

Esto solo funciona para el `config.with_callback` predeterminado.

Esta variable se puede cambiar fuera de un bloque **init**, pero se debe ejecutar una declaración **with None** después de hacerlo para que el cambio surta efecto.

Esto está más o menos obsoleto por la declaración **window**.

Transiciones predefinidas

Definición: fade Una instancia de la transición **Fade** que tarda 0.5 segundos en fundir a negro y luego 0.5 segundos en fundir a la nueva pantalla.

Definición: dissolve Una instancia de la transición **Dissolve** que tarda 0.5 segundos en completarse.

Definición: pixellate Una instancia de la transición **Pixellate**, que tarda 1 segundo en completarse y crea píxeles de hasta 32x32 en el transcurso de 5 pasos en cualquier dirección.

Definición: move Una instancia de la transición **MoveTransition**, que tarda 0.5 segundos en mover las imágenes que cambiaron de posición a sus nuevas ubicaciones.

Definición: moveinright, moveinleft, moveintop, moveinbottom Estas mueven las imágenes que entran a la pantalla desde el lado correspondiente, tardando 0.5 segundos en hacerlo.

Definición: moveoutright, moveoutleft, moveouttop, moveoutbottom Estas mueven las imágenes que salen de la pantalla por el lado correspondiente, tardando 0.5 segundos en hacerlo.

Definición: `ease`, `easeinright`, `easeinleft`, `easeintop`, `easeinbottom`, `easeoutright`, `easeoutleft`, `easeouttop`, `easeoutbottom` Estas son similares a la familia de transiciones `move-`, excepto que usan una curva basada en coseno para ralentizar el inicio y el final de la transición.

Definición: `zoomin` Esta acerca las imágenes que entran, tardando 0.5 segundos en hacerlo.

Definición: `zoomout` Esta aleja las imágenes que salen, tardando 0.5 segundos en hacerlo.

Definición: `zoominout` Esta acerca las imágenes que entran y aleja las imágenes que salen, tardando 0.5 segundos en hacerlo.

Definición: `vpunch` Cuando se invoca, esta transición agita la pantalla verticalmente durante un cuarto de segundo.

Definición: `hpunch` Cuando se invoca, esta transición agita la pantalla horizontalmente durante un cuarto de segundo.

Definición: `blinds` Transiciona la pantalla con un efecto de persianas verticales que dura 1 segundo.

Definición: `squares` Transiciona la pantalla con un efecto de cuadrados que dura 1 segundo.

Definición: `wiperight` Una instancia de `CropMove` que tarda 1 segundo en barrer la pantalla hacia la derecha.

Definición: `wipeleft` Una instancia de `CropMove` que tarda 1 segundo en barrer la pantalla hacia la izquierda.

Definición: `wipeup` Una instancia de `CropMove` que tarda 1 segundo en barrer la pantalla hacia arriba.

Definición: `wipedown` Una instancia de `CropMove` que tarda 1 segundo en barrer la pantalla hacia abajo.

Definición: `slideright` Una instancia de `CropMove` que tarda 1 segundo en deslizar la pantalla hacia la derecha.

Definición: `slideleft` Una instancia de `CropMove` que tarda 1 segundo en deslizar la pantalla hacia la izquierda.

Definición: `slideup` Una instancia de `CropMove` que tarda 1 segundo en deslizar la pantalla hacia arriba.

Definición: `slidedown` Una instancia de `CropMove` que tarda 1 segundo en deslizar la pantalla hacia abajo.

Definición: `slideawayright` Una instancia de `CropMove` que tarda 1 segundo en deslizar la pantalla hacia la derecha y fuera de ella.

Definición: `slideawayleft` Una instancia de `CropMove` que tarda 1 segundo en deslizar la pantalla hacia la izquierda y fuera de ella.

Definición: `slideawayup` Una instancia de `CropMove` que tarda 1 segundo en deslizar la pantalla hacia arriba y fuera de ella.

Definición: `slideawaydown` Una instancia de `CropMove` que tarda 1 segundo en deslizar la pantalla hacia abajo y fuera de ella.

Definición: `irisout` Una instancia de `CropMove` que abre el iris de la pantalla durante 1 segundo.

Definición: `irisin` Una instancia de `CropMove` que cierra el iris de la pantalla durante 1 segundo.

Constructores de transiciones

Las siguientes son funciones que devuelven objetos útiles como transiciones. El usuario no debe proporcionar los parámetros `new_widget` o `old_widget`, ya que Ren'Py los proporciona cuando comienza una transición.

Función: `CropMove` (`time`, `mode='fromleft'`, `startcrop=(0.0, 0.0, 0.0, 0.0)`, `startpos=(0.0, 0.0)`, `endcrop=(0.0, 0.0, 0.0, 0.0)`, `endpos=(0.0, 0.0)`, `topnew=True`, `old_widget=None`, `new_widget=None`)

La transición `CropMove` funciona colocando la imagen antigua y la nueva en dos capas, llamadas superior e inferior (normalmente la nueva imagen está en la parte superior, pero eso se puede cambiar en algunos modos). La capa inferior siempre se dibuja por completo. La imagen superior se recorta primero a un rectángulo, y luego ese rectángulo se dibuja en la pantalla en una posición especificada. Los rectángulos de recorte y las posiciones iniciales y finales se pueden seleccionar mediante el modo proporcionado, o especificar manualmente.

El resultado es una transición sorprendentemente flexible. Esta transición tiene muchos modos que simplifican su uso. Podemos agrupar estos modos en tres grupos: barridos, deslizamientos y otros.

- En un barrido (*wipe*), la imagen permanece fija y se revela más a medida que avanza la transición. Por ejemplo, en `"wiperight"`, un barrido de izquierda a derecha, primero se revela el borde izquierdo de la imagen en el borde izquierdo de la pantalla, luego el centro de la imagen y finalmente el lado derecho de la imagen en el lado derecho de la pantalla. Otros barridos admitidos son `"wipeleft"`, `"wipedown"` y `"wipeup"`.

- En un deslizamiento (*slide*), la imagen se mueve. Así que en un "slideright", el borde derecho de la imagen comienza en el borde izquierdo de la pantalla y se mueve hacia la derecha a medida que avanza la transición. Otros deslizamientos son "slideleft", "slidedown" y "slideup".
- También hay deslizamientos de salida (*slideaways*), en los que la imagen antigua se mueve sobre la nueva imagen. Los deslizamientos de salida incluyen "slideawayright", "slideawayleft", "slideawayup" y "slideawaydown".

También admitimos un iris rectangular con `irisin` y un iris rectangular de salida con `irisout`. Finalmente, `custom` permite al usuario definir nuevas transiciones si estas no son suficientes.

- `time` - El tiempo que durará esta transición, en segundos.
- `mode` - Uno de los modos dados anteriormente.

Los siguientes parámetros solo se respetan si el modo es `custom`.

- `startcrop` - El rectángulo inicial que se recorta de la imagen superior. Una tupla de 4 elementos que contiene x, y, ancho y alto.
- `startpos` - El lugar inicial donde se dibuja la imagen superior en la pantalla, una tupla de 2 elementos que contiene x e y.
- `endcrop` - El rectángulo final que se recorta de la imagen superior. Una tupla de 4 elementos que contiene x, y, ancho y alto.
- `endpos` - El lugar final donde se dibuja la imagen superior en la pantalla, una tupla de 2 elementos que contiene x e y.
- `topnew` - Si es `True`, la capa superior contiene la nueva imagen. De lo contrario, la capa superior contiene la imagen antigua.

Función: Dissolve (time, old_widget=None, new_widget=None, alpha=False)

Disuelve la escena antigua en la nueva, superponiendo la nueva escena sobre la antigua y variando su alfa de 0 a 255. `Dissolve` solo funciona correctamente cuando ambas escenas tienen el mismo tamaño.

- `time` - El tiempo que tardará la disolución.
- `alpha` - Si es `True`, el elemento mostrable resultante tendrá un canal alfa, a costa de cierta velocidad. Si es `False`, se tratará como opaco, pero será más rápido.

Función: Fade (`out_time`, `hold_time`, `in_time`, `old_widget=None`, `new_widget=None`, `color=None`, `widget=None`)

Devuelve un objeto que se puede usar como argumento de una declaración `with` para fundir la escena antigua en un color sólido, esperar durante un tiempo determinado y luego fundir el color sólido en la nueva escena.

- `out_time` - El tiempo que se dedicará a fundir de la escena antigua al color sólido. Un flotante, en segundos.
- `hold_time` - El tiempo que se dedicará a mostrar el color sólido. Un flotante, en segundos.
- `in_time` - El tiempo que se dedicará a fundir del color sólido a la nueva escena. Un flotante, en segundos.
- `color` - El color sólido al que se fundirá. Una tupla que contiene tres componentes, cada uno entre 0 y 255. También puede ser `None`.
- `widget` - Un widget al que se fundirá, si `color` es `None`. Esto permite un fundido a una imagen en lugar de solo a un color sólido.

Si tanto `color` como `widget` son `None`, el fundido es a negro.

Función: ImageDissolve (`image`, `time`, `reverse=False`, `old_widget=None`, `new_widget=None`)

Disuelve la escena antigua en la nueva, usando una imagen para controlar el proceso de disolución. Básicamente, esto significa que los píxeles blancos entran primero y los negros al final.

Los dos hijos deben tener el mismo tamaño, o el comportamiento de `ImageDissolve` no está definido.

- `image` - La imagen que se usará para controlar esta transición. La imagen debe tener el mismo tamaño que la escena que se está disolviendo.
- `time` - El tiempo que tardará la disolución.
- `reverse` - Invierte la rampa y la dirección del deslizamiento de la ventana. Cuando es `True`, los píxeles negros se disuelven primero y los blancos al final.
- `alpha` - Si es `True`, el elemento mostrable resultante tendrá un canal alfa, a costa de cierta velocidad. Si es `False`, se tratará como opaco, pero será más rápido.

Función: MoveTransition (`delay`, `factory=None`, `enter_factory=None`, `leave_factory=None`, `old=False`, `layers=['master']`)

Esta transición intenta encontrar imágenes que hayan cambiado de posición y las mueve de la posición antigua a la nueva transición, tardando `delay` segundos en completar el movimiento.

Si se proporciona `factory`, se espera que sea una función que tome como argumentos: una posición antigua, una posición nueva, el retardo y un elemento mostrable, y devuelva

un elemento mostrable como argumento. Si no se proporciona, el comportamiento predeterminado es mover el elemento mostrable desde las posiciones iniciales a las finales. Las posiciones siempre se dan como tuplas (`xpos`, `ypos`, `xanchor`, `yanchor`).

Si se proporcionan `enter_factory` o `leave_factory`, se espera que sean funciones que tomen como argumentos una posición, un retardo y un elemento mostrable, y devuelvan un elemento mostrable. Se aplican a los elementos mostrables que entran o salen de la escena, respectivamente. El valor predeterminado es mostrar en el lugar los elementos mostrables que entran y no mostrar los que salen.

Si `old` es `True`, entonces `factory` mueve el elemento mostrable antiguo con la etiqueta dada. De lo contrario, mueve el nuevo elemento mostrable con esa etiqueta (nuevo en 6.6.1).

`layers` es una lista de capas a las que se aplicará la transición (nuevo en 6.9.0).

Las imágenes se consideran iguales si tienen la misma etiqueta, de la misma manera que la etiqueta se usa para determinar qué imagen reemplazar u ocultar. También se consideran iguales si son el mismo elemento mostrable.

Si usas esta transición para deslizar una imagen fuera del lado de la pantalla, recuerda ocultarla cuando hayas terminado (o simplemente úsala con un `leave_factory`).

Hay varios constructores que crean funciones para usar con `enter_factory` y `leave_factory`:

Función: `MoveFactory (**kwargs)` Se puede usar con `factory` para proporcionar argumentos a `Move`.

Función: `MoveIn (pos, **kwargs)` Se puede usar con `enter_factory` para mover los elementos que entran.

- `pos` - Una tupla (`xpos`, `ypos`, `xanchor`, `yanchor`) que da la posición desde la que moverse. Cualquier componente puede ser `None` para tomar el componente correspondiente de la posición final.

Los argumentos de palabra clave se pasan a `Move`.

Función: `MoveOut (pos, **kwargs)` Se puede usar con `leave_factory` para mover los elementos que salen.

- `pos` - Una tupla (`xpos`, `ypos`, `xanchor`, `yanchor`) que da la posición a la que moverse. Cualquier componente puede ser `None` para tomar el componente correspondiente de la posición original.

Los argumentos de palabra clave se pasan a `Move`.

Función: `ZoomInOut (start, end, **kwargs)` Se puede usar con `enter_factory` o `leave_factory`. Se usa para acercar o alejar los elementos mostrables que entran o salen.

- `start` - El factor de zoom al inicio de la transición.
- `end` - El factor de zoom al final de la transición.

Los argumentos de palabra clave se pasan a `FactorZoom`.

Función: RevolveInOut (start, end, **kwargs) Se puede usar con `enter_factory` o `leave_factory`. Se utiliza para hacer girar hacia adentro o hacia afuera los elementos mostrables que entran o salen.

- **start** - La revolución en el sentido de las agujas del reloj al inicio de la transición.
- **end** - La revolución en el sentido de las agujas del reloj al final de la transición.

Los argumentos de palabra clave adicionales se pasan a `Revolve`.

Función: Pause (delay) Devuelve una transición que muestra la nueva pantalla durante `delay` segundos.

Esto es útil para implementar una pausa que se comporta como una transición, una que se omite cuando las transiciones están desactivadas.

Función: Pixellate (time, steps, old_widget=None, new_widget=None) Esta pixela la escena antigua y luego pixela la nueva escena, tomando el tiempo dado y el número dado de pasos de pixelación en cada dirección.

- **time** - El tiempo que durará la transición, en segundos.
- **steps** - El número de pasos de pixelación en cada dirección.

Función: ComposeTransition (trans, before=None, after=None) Esto permite componer transiciones antes de que se apliquen.

- **trans** - La transición de nivel superior.
- **before** - Si no es `None`, una transición que se aplica a las pantallas antigua y nueva, y se usa como la pantalla antigua para **trans**.
- **after** - Si no es `None`, una transición que se aplica a las pantallas antigua y nueva, y se usa como la pantalla nueva para **trans**.

Esto puede no funcionar con todas las transiciones.

Función: MultipleTransition (args) Crea una transición que consiste en una o más transiciones. **args** debe ser una lista de longitud impar, que contenga al menos 3 elementos. Los elementos en posiciones impares de esta lista se consideran elementos mostrables, mientras que los elementos en posiciones pares son transiciones. Cuando se usa, esto hace que el primer elemento (un elemento mostrable) transicione al tercer elemento (un elemento mostrable), usando el segundo elemento como transición. Si el cuarto y quinto elementos están presentes, ocurrirá una transición del tercer elemento (un elemento mostrable) al quinto elemento (un elemento mostrable) usando el cuarto elemento (una transición).

Hay dos valores especiales que se reconocerán como elementos mostrables. `False` se reemplazará con un elemento mostrable que contiene la escena antes de la transición, mientras que `True` se reemplazará con un elemento mostrable que contiene la escena después de la transición.

Algunas transiciones también se pueden aplicar a capas específicas, utilizando la función `renpy.transition`. Solo las transiciones que no son completamente opacas se pueden usar de esta manera.

Familias de transiciones

Esta función define una familia de transiciones similares.

Función: `define.move_transitions(prefix, delay, time_warp=None, in_time_warp=None, out_time_warp=None, layers=['master'], **kwargs)` Define una familia de transiciones de movimiento, similares a las transiciones `move` y `ease`. Para un `prefix` dado, define las transiciones:

- `prefix` - Una transición que tarda `delay` segundos en mover las imágenes que cambiaron de posición a sus nuevas ubicaciones.
- `prefixinleft`, `prefixinright`, `prefixintop`, `prefixinbottom` - Una transición que tarda `delay` segundos en mover las imágenes que cambiaron de posición a sus nuevas ubicaciones, con las imágenes recién mostradas entrando desde el lado correspondiente.
- `prefixoutleft`, `prefixoutright`, `prefixouttop`, `prefixoutbottom` - Una transición que tarda `delay` segundos en mover las imágenes que cambiaron de posición a sus nuevas ubicaciones, con las imágenes recién ocultas saliendo por el lado correspondiente.
- `time_warp`, `in_time_warp`, `out_time_warp` - El argumento `time_warp` proporcionado a `Move` para imágenes que permanecen en la pantalla, imágenes recién mostradas e imágenes recién ocultas (respectivamente).
- `old` - Hace que las transiciones muevan los elementos mostrables antiguos, en lugar de los nuevos.
- `layers` - Las capas a las que se aplicará la transición.

Los argumentos de palabra clave adicionales se pasan (indirectamente) a los movimientos. El argumento de palabra clave adicional más útil es probablemente `subpixel=True`, que hace que se use un movimiento subpíxel.

Funciones de interacción

Las funciones de interacción de Ren'Py están disponibles para que puedas personalizar las interacciones con el usuario aprovechando el mismo código que utilizan las interacciones normales.

Funciones auxiliares de declaraciones

Las siguientes funciones implementan nuevos comportamientos de juego que no merecían su propia declaración, o complementan el comportamiento de las declaraciones.

Function: `renpy.block_rollback()`:

Previene que el juego retroceda (rollback) hasta antes de la declaración actual.

Function: `renpy.checkpoint(data=None)`:

Crea un punto de control al que el usuario puede retroceder (rollback). El punto de control se coloca en la declaración siguiente a la última declaración que interactuó con el usuario. Una vez que se ha llamado a esta función, no debe haber más interacción con el usuario en el bloque Python actual.

- **data** - Si no es `None`, contiene datos a los que se puede acceder mediante `renpy.roll_forward_info` si y cuando la declaración actual se vuelve a ejecutar debido a que el usuario retrocede y luego avanza de nuevo.

`renpy.checkpoint` debe llamarse después de `ui.interact`.

Function: `renpy.choice_for_skipping ()`:

Se llama para indicar al código de omisión que hemos llegado a una opción. Si estamos omitiendo y la preferencia de omitir después de las opciones no es `True`, esto deshabilita la omisión. Solo se necesita en una función de interacción personalizada.

Function: `renpy.clear_game_runtime ()`:

Reinicia el temporizador de tiempo de juego a 0.

El contador de tiempo de juego cuenta el número de segundos que han transcurrido mientras se esperaba la entrada del usuario en el contexto actual (por lo que no cuenta el tiempo pasado en el menú del juego).

Function: `renpy.current_interact_type ()`:

Devuelve el valor del parámetro `type` proporcionado a `ui.interact()` durante la interacción actual. Consulta `renpy.last_interact_type` para conocer los tipos de interacción posibles.

Function: `renpy.display_menu (items, window_style='menu_window', interact=True, with_none=None)`:

Muestra un menú que contiene los elementos dados, devolviendo el valor del elemento que el usuario selecciona.

- **items** - Una lista de tuplas que son los elementos que se añadirán a este menú. El primer elemento de una tupla es una cadena que se usa para este elemento de menú. El segundo elemento es el valor que se devuelve si se selecciona este elemento, o `None` si este elemento es un título no seleccionable.
- **interact** - Si es `True`, se produce una interacción. Si es `False`, no se produce tal interacción y el usuario debe llamar a `ui.interact` manualmente.
- **with_none** - Si es `True`, hace que se ejecute una declaración `with None` después de cada interacción. Si es `None` (el valor predeterminado), comprueba `config.implicit_with_none` para determinar si se debe ejecutar un `with None`.

Function: `renpy.display_say (who, what, ...)`:

Llamado por `Character` para implementar la declaración `say`.

Function: `renpy.full_restart (transition=config.end_game_transition, label="_invoke_main_menu", target="_main_menu")`:

Provoca un reinicio completo de Ren'Py. Restablece el estado de Ren'Py a lo que era cuando los bloques `init` terminaron de ejecutarse, y luego reinicia el juego. Después de que se ejecute algo de código `init`, se establecerá `transition` para que se ejecute en la siguiente interacción, y el control se transferirá a `label`. La etiqueta predeterminada inicializa el contexto del menú principal y luego invoca `target`. `target` puede ser una pantalla, como `"preferences_screen"` o `"load_screen"`. Generalmente no tiene sentido especificar tanto `label` como `target` (cambiado en 6.9.0).

Function: `renpy.game_menu (screen=None)`:

Invoca el menú del juego.

- **screen** - La pantalla a la que acceder, normalmente una de "load_screen", "save_screen" o "preferences_screen", pero el usuario puede definir otras pantallas del menú del juego. Si es None, se usa el valor de `_game_menu_screen`.

Function: `renpy.get_game_runtime ()`:

Devuelve el número de segundos que han transcurrido en el juego desde la última llamada a `clear_game_timer`, como un flotante.

El contador de tiempo de juego cuenta el número de segundos que han transcurrido mientras se esperaba la entrada del usuario en el contexto actual (por lo que no cuenta el tiempo pasado en el menú del juego).

Function: `renpy.get_reshow_say ()`:

Obtiene una función que, cuando se llama, volverá a mostrar la declaración `say` anterior. No provoca que ocurra una interacción.

Function: `renpy.get_roll_forward ()`:

Obtiene la información de avance para la interacción actual. Esta función está pensada para ser usada por funciones de superposición (*overlay*). La función de superposición debe devolver el resultado de esta función para que se produzca un avance.

Ten en cuenta que solo tiene sentido usar esto dentro de una interacción. Consulta `renpy.roll_forward_info` para obtener información sobre cómo obtener la información de avance fuera de una interacción.

Function: `renpy.get_transition (layer=None)`:

Obtiene la transición que se ha programado para ocurrir mediante `renpy.transition`.

- **layer** - Si no es None, encuentra la transición que ocurre para esa capa específica.

Function: `renpy.imagemap (ground, selected, hotspots, unselected=None, overlays=False, style='imagemap', with_none=None, **properties)`:

Muestra un mapa de imagen (*imagemap*). Un mapa de imagen consiste en dos imágenes y una lista de zonas interactivas (*hotspots*) definidas en esa imagen. Cuando el usuario hace clic en una zona interactiva, se devuelve el valor asociado a esa zona. A pesar del nombre, esto toma elementos mostrables arbitrarios, no solo manipuladores de imágenes.

- **ground** - El nombre del archivo que contiene la imagen de fondo. La imagen de fondo se muestra para las áreas que no son parte de ninguna zona interactiva.
- **selected** - El nombre del archivo que contiene la imagen seleccionada. Esta imagen se muestra en las zonas interactivas cuando el ratón está sobre ellas.
- **hotspots** - Una lista de tuplas que definen las zonas interactivas en este mapa de imagen. Cada tupla tiene el formato `(x0, y0, x1, y1, resultado)`. `(x0, y0)` da las coordenadas de la esquina superior izquierda de la zona, `(x1, y1)` da la esquina inferior derecha, y `resultado` da el valor devuelto por esta función si se hace clic en la zona.
- **unselected** - Si se proporciona, es el nombre de un archivo que contiene la imagen que se usa para rellenar las zonas interactivas que no están seleccionadas como parte de ninguna imagen. Si no se proporciona, se usa la imagen de fondo.
- **overlays** - Si es `True`, las superposiciones se muestran cuando este mapa de imagen está activo. Si es `False`, las superposiciones se suprimen.

- `with_none` - Si es `True`, hace que se ejecute una declaración `with None` después de cada interacción. Si es `None` (el valor predeterminado), comprueba `config.implicit_with_none` para determinar si se debe ejecutar un `with None`.

Function: `renpy.input (prompt, default='', allow=None, exclude='{}', length=None, with_none=None):`

Abre una ventana solicitando que el usuario ingrese algo de texto. Devuelve el texto ingresado.

- `prompt` - Un mensaje que se usa para pedir al usuario el texto.
- `default` - Un valor predeterminado para el texto que esta entrada puede devolver.
- `length` - Si se proporciona, un límite a la cantidad de texto que devolverá esta función.
- `allow` - Si no es `None`, si un carácter de entrada no está en esta cadena, se ignora.
- `exclude` - Si no es `None`, si un carácter de entrada está en este conjunto, se ignora.
- `with_none` - Si es `True`, hace que se ejecute una declaración `with None` después de cada interacción. Si es `None` (el valor predeterminado), comprueba `config.implicit_with_none` para determinar si se debe ejecutar un `with None`.

Function: `renpy.last_interact_type ():`

Devuelve el valor del parámetro `type` proporcionado a `ui.interact()` durante la última interacción. Los tipos predeterminados son:

- `"say"` - Declaraciones `say` normales.
- `"nvl"` - Declaraciones `say` en modo NVL.
- `imagemap` - `renpy.imagemap`
- `"pause"` - `renpy.pause`
- `input` - `renpy.input`
- `"menu"` - Menús.
- `"misc"` - Otras interacciones.

Además de estos, se pueden crear otros valores dando el argumento `say` a `Character` o a `ui.interact`.

Function: `renpy.pause (delay=None, music=None):`

Cuando se llama sin argumentos, pausa y espera a que el usuario haga clic antes de avanzar en el guión. Si se proporciona un parámetro `delay`, Ren'Py esperará ese tiempo antes de continuar, a menos que el usuario haga clic para interrumpir la pausa. Esto es útil, por ejemplo, para mostrar una imagen que permanecerá en la pantalla durante un cierto tiempo y luego proceder a la siguiente declaración del guión sin esperar un clic indefinidamente.

- `delay` - El número de segundos a esperar.

- **music** - Si se proporciona y la música se está reproduciendo, tiene prioridad sobre el parámetro **delay**. Da un tiempo, en segundos, dentro de la pista de música que se está reproduciendo actualmente. Ren'Py pausará hasta que la música haya llegado a ese punto.

Devuelve **True** si el usuario interrumpió la pausa pulsando una tecla o haciendo clic con el ratón, o **False** si la pausa terminó al alcanzarse el tiempo establecido.

Function: `renpy.predict (img):`

Fuerza la predicción de la imagen o elemento mostrable proporcionado. Esto hará que se cargue en la caché de imágenes durante la siguiente interacción (y solo la siguiente), si hay tiempo libre.

Function: `renpy.predict_display_say (who, what, ...):`

Esta es la función predeterminada utilizada por **Character** para predecir las imágenes que usará `renpy.display_say`.

Function: `renpy.quit ():`

Hace que Ren'Py salga por completo.

Function: `renpy.show_display_say (who, what, who_args={}, what_args={}, window_args={}, image=False, no_ctc_what=..., **kwargs):`

Esta función es llamada (por defecto) por `renpy.display_say` para añadir los widgets correspondientes a una pantalla de diálogo al usuario. No se espera que sea llamada por el usuario, sino por `renpy.display_say`, o por una función pasada como el argumento `show_function` de **Character** o `renpy.display_say`.

- **who** - El nombre del personaje que está hablando, o **None** para no mostrar este nombre al usuario.
- **what** - Lo que dice ese personaje. Ten en cuenta que esto puede no ser una cadena, ya que también puede ser una lista que contenga texto y elementos mostrables, adecuada para usar como el primer argumento de `ui.text`.
- **who_args** - Argumentos de palabra clave adicionales destinados a ser proporcionados a `ui.text` que crea el widget **who** de este diálogo.
- **what_args** - Argumentos de palabra clave adicionales destinados a ser proporcionados a `ui.text` que crea el widget **what** de este diálogo.
- **window_args** - Argumentos de palabra clave adicionales destinados a ser proporcionados a `ui.window` que crea el widget **who** de este diálogo.
- **image** - Si es **True**, entonces **who** debe interpretarse como una imagen o elemento mostrable en lugar de una cadena de texto.
- **callback** - Si no es **None**, una función que debe llamarse sin argumentos antes de cada vez que se muestra algo.
- **no_ctc_what** - El contenido de **what** antes de que se añadiera el indicador de clic para continuar.
- **kwargs** - Los argumentos de palabra clave adicionales deben ignorarse.

Se requiere que esta función devuelva el widget `ui.text` que muestra el texto `what`.

Function: `renpy.shown_window ()`:

Debe llamarse cuando se muestra una ventana en la pantalla. Llamar a esto evita que `config.empty_window` se llame cuando `_window` es verdadero (y por lo tanto, evita que se muestre la ventana vacía).

Function: `renpy.transition (trans, layer=None, always=False)`:

Establece la transición que se usará para la siguiente interacción. Esto es útil cuando la siguiente interacción no toma una cláusula `with`, como es el caso de las interacciones en las funciones `renpy.pause`, `renpy.input` y `renpy.imagemap`.

- **trans** - La transición deseada.
- **layer** - Si la capa no es `None`, la transición se aplicará solo a la capa nombrada. Ten en cuenta que solo algunas transiciones se pueden aplicar a capas específicas.
- **always** - Si es `True`, fuerza la transición, pudiendo anular la preferencia del juego de mostrar todas o ninguna transición.

Function: `renpy.restart_interaction ()`:

Reinicia la interacción actual, manteniendo cualquier transición en curso.

Debe llamarse siempre que se añadan o eliminen elementos mostrables durante el transcurso de una interacción, o cuando la información utilizada para construir la superposición cambie.

Function: `renpy.reshow_say ()`:

Vuelve a mostrar la última declaración `say`. No provoca que ocurra una interacción.

Function: `renpy.roll_forward_info ()`:

Cuando se vuelve a ejecutar una declaración después de un retroceso, devuelve los datos almacenados mediante `renpy.checkpoint` la última vez que se ejecutó esta declaración. Si esta declaración no se está volviendo a ejecutar debido a un retroceso, devuelve `None`. A menudo, estos datos se pasan como el parámetro `roll_forward` de `ui.interact`. Es importante comprobar que el valor es válido antes de usarlo de esta manera.

Funciones de contexto

Los contextos almacenan las listas de escenas actuales y la ubicación de ejecución. Ren'Py admite una pila de contextos, pero solo el contexto de nivel superior se guarda en el archivo de guardado.

Function: `renpy.context ()`:

Devuelve un objeto que es único para el contexto actual y que participa en el retroceso y similares.

Function: `renpy.call_in_new_context (label)`:

Crea un nuevo contexto y comienza a ejecutar código desde esa etiqueta en el nuevo contexto. El retroceso está deshabilitado en el nuevo contexto (en realidad, solo te llevará de vuelta al contexto real).

Úsalo para comenzar una segunda interacción con el usuario mientras estás dentro de una interacción.

Function: `renpy.invoke_in_new_context (callable, *args, **kwargs)`:

Empuja el contexto actual e invoca la función Python dada en un nuevo contexto. Cuando esa función retorna o lanza una excepción, elimina el nuevo contexto y restaura el contexto actual.

Los argumentos adicionales y los argumentos de palabra clave se pasan al objeto invocable. Ten en cuenta que el contexto así creado no puede ejecutar código Ren'Py. Por lo tanto, las excepciones que cambian el flujo del código Ren'Py (como las creadas por `renpy.jump`) hacen que este contexto termine y son manejadas por el siguiente contexto superior.

Si quieres ejecutar código Ren'Py desde la función, puedes llamarlo con `renpy.call_in_new_context`. Úsalo para comenzar una segunda interacción con el usuario mientras estás dentro de una interacción.

Function: `renpy.jump_out_of_context (label):`

Hace que el control salga del contexto actual y luego se transfiera en el contexto padre a la etiqueta dada.

Funciones de depuración

Function: `renpy.log (msg):`

Si `config.log` no está configurado, no hace nada. De lo contrario, abre el archivo de registro (si no está ya abierto), formatea el mensaje a 70 columnas y lo imprime en el archivo de registro.

Function: `renpy.watch (expression, style='default', **properties):`

Observa la expresión Python dada, mostrándola en la esquina superior izquierda de la pantalla (aunque las propiedades de posición pueden cambiar eso). La expresión siempre debe estar definida, sin lanzar excepciones.

Una observación (*watch*) no persiste después de guardar o reiniciar.

Function: `renpy.get_filename_line ():`

Devuelve un par que da el nombre de archivo y el número de línea de la declaración actual.

Funciones de declaraciones de Ren'Py

Estas funciones corresponden a las declaraciones equivalentes del lenguaje central de Ren'Py.

Funciones de declaraciones de Ren'Py

Estas funciones corresponden a las declaraciones equivalentes del lenguaje central de Ren'Py.

Function: `renpy.jump (label):`

Transfiere el control a la etiqueta `label`. Ten en cuenta que esto terminará el bloque Python en el que se encuentra.

Function: `renpy.image (name, img):`

Se utiliza para ejecutar la declaración `image`. Toma como argumentos un nombre de imagen y un objeto de imagen, y asocia el nombre de imagen con el objeto de imagen.

Al igual que la declaración `image`, esta función solo debe ejecutarse en bloques `init`.

- `name` - El nombre de la imagen, una tupla de cadenas.
- `img` - El elemento mostrable que se asocia con ese nombre. Si es una cadena o tupla, se interpreta como un argumento para `Image`.

Function: `renpy.scene (layer='master'):`

Limpia la capa especificada, o `'master'` por defecto. Se utiliza en la ejecución de la declaración `scene`, pero solo para limpiar la capa. Si quieres añadir algo nuevo, como

haría normalmente la declaración `scene` cuando se proporciona un parámetro de imagen, llama a `renpy.show` después de esto.

Function: `renpy.show (name, at_list=[], layer='master', what=None, zorder=0, tag=None, behind=None):`

Se utiliza para ejecutar la declaración `show`, añadiendo la imagen nombrada a la pantalla.

- **name** - El nombre de la imagen a añadir a la pantalla. Puede ser una tupla de cadenas o una sola cadena. En este último caso, se divide por espacios en blanco para formar una tupla.
- **at_list** - La lista `at`, una lista de funciones que se aplican a la imagen cuando se muestra. Los miembros de la lista `at` deben ser serializables con `pickle` si `sticky_positions` es `True`.
- **layer** - La capa en la que se mostrará la imagen.
- **what** - Si no es `None`, se espera que sea un elemento mostrable. Ese elemento mostrable se muestra en lugar de pasar por el proceso de búsqueda normal.
- **zorder** - El orden `z` de esta imagen.
- **tag** - La etiqueta de esta imagen. Si es `None`, la etiqueta se toma de `name`.
- **behind** - Una lista de etiquetas detrás de las cuales se mostrará esta imagen, si están presentes en la misma capa con el mismo orden `z`, y no hay ya una imagen con la misma etiqueta en la capa.

Function: `renpy.hide (name, layer='master'):`

Busca elementos en la capa dada (o `'master'` si no se da ninguna capa) que tengan el mismo nombre que el primer componente del nombre dado, y los elimina de la capa. Se utiliza para ejecutar la declaración `hide`.

- **name** - El nombre de la imagen a ocultar de la pantalla. Puede ser una tupla de cadenas o una sola cadena. En este último caso, se divide por espacios en blanco para formar una tupla. Solo se usa el primer elemento de la tupla.
- **layer** - La capa en la que opera.

Function: `renpy.with_statement (trans, paired=None, always=False, clear=True):`

El equivalente programático de una declaración `with`.

- **trans** - La transición causada por la declaración `with`.
- **paired** - La transición emparejada que se pasa a `config.with_callback`.
- **always** - Hace que esta transición ocurra siempre, incluso si el usuario ha desactivado las transiciones.
- **clear** - Si es `True`, la capa transitoria se limpiará al final de la transición.

Acceso a la lista de escenas

Estas funciones permiten consultar las listas de escenas.

Function: `renpy.showing (name, layer='master')`:

Devuelve `True` si `name` es un prefijo de una imagen que se está mostrando en la capa. `name` puede ser una tupla de cadenas o una sola cadena. En este último caso, se divide por espacios en blanco para formar una tupla.

Cuando se llama durante la predicción de imágenes, utiliza las imágenes que se predicen que se mostrarán, en lugar de las imágenes que se están mostrando actualmente.

Function: `renpy.get_at_list (name, layer='master')`:

Obtiene la lista `at` que se utilizó para mostrar una etiqueta de imagen dada. `name` se analiza para obtener la etiqueta de imagen, y se recupera la lista `at` correspondiente a esa etiqueta. Si no se ha mostrado ninguna imagen con esa etiqueta en la capa dada, se devuelve `None`.

Function: `renpy.layer_at_list (at_list, layer='master')`:

Establece la lista `at` de capa, una lista `at` que se aplica a la capa dada en su conjunto. Esto se puede usar para hacer cosas como mover la capa o hacer zoom.

Ten en cuenta que esto interactúa mal con `MoveTransition`.

Funciones de utilidad varias

Function: `color (s)`:

Esta función convierte una cadena de código hexadecimal `s` en una tupla de color/alfa. Los signos `#` iniciales se ignoran. Los colores pueden ser `rgb` o `rgba`, con cada elemento teniendo uno o dos dígitos (por lo que las cadenas pueden tener 3, 4, 6 u 8 dígitos de longitud, sin incluir el `#` opcional). Un alfa faltante se interpreta como 255, completamente opaco.

Por ejemplo, `color('#123a')` devuelve (17, 34, 51, 170), mientras que `color('c0c0c0')` devuelve (192, 192, 192, 255).

Ten en cuenta que a partir de Ren'Py 5.6.0, las funciones que requieren un color aceptan directamente la cadena hexadecimal, sin necesidad de usar esta función `color`.

Function: `renpy.cache_pin (*image_manipulators)`:

Cuando se llama con uno o más manipuladores de imágenes, hace que esos manipuladores de imágenes se “fijen” en la caché de imágenes. Esto significa que se cargarán poco después de que comience el juego y nunca se eliminarán de la caché.

El uso habitual de esta función es precargar imágenes utilizadas en el menú del juego. Ten cuidado con ella, ya que puede aumentar el uso de memoria de Ren'Py.

Function: `renpy.cache_unpin (*image_manipulators)`:

Se puede llamar con uno o más manipuladores de imágenes. Hace que esos manipuladores dejen de estar fijados y se eliminen de la caché de imágenes.

Function: `renpy.curry (func)`:

La currificación es una operación que permite llamar a una función mediante una cadena de llamadas, cada una proporcionando algunos de los argumentos. En Ren'Py, llamar a `renpy.curry(func)` devuelve un objeto invocable. Cuando este objeto se llama con argumentos, se devuelve un segundo objeto invocable que almacena `func` y los argumentos proporcionados. Cuando se llama a este segundo objeto invocable, la función original se llama con los argumentos almacenados y los argumentos proporcionados a la segunda llamada.

Los argumentos posicionales de la primera llamada se colocan antes de los argumentos posicionales de la segunda llamada. Si se da un argumento de palabra clave en ambas llamadas, el valor de la segunda llamada tiene prioridad.

Los objetos `Curry` pueden ser serializados con `pickle` siempre que la función original permanezca disponible con su nombre original.

Function: `renpy.exists (filename):`

Devuelve `True` si el nombre de archivo dado se puede encontrar en el `searchpath`. Esto solo funciona si existe un archivo físico en el disco. No encontrará el archivo si está dentro de un archivo (*archive*).

Function: `renpy.list_files (common=False):`

Devuelve una lista de todos los archivos que se pueden encontrar en el `searchpath`. Si `common` es `True`, también se incluyen los archivos del directorio común.

Function: `renpy.file (filename):`

Devuelve un objeto similar a un archivo de solo lectura que accede a `filename`. El archivo se accede mediante el método de búsqueda estándar de Ren'Py y puede residir en un archivo. El objeto admite un amplio subconjunto de los campos y métodos que se encuentran en el objeto de archivo estándar de Python (básicamente, todos los métodos que tienen sentido para un objeto de solo lectura).

Function: `renpy.free_memory ():`

Intenta liberar algo de memoria. Útil antes de ejecutar un minijuego basado en `renpygame`.

Function: `renpy.get_all_labels ():`

Devuelve el conjunto de todas las etiquetas definidas en el programa, incluyendo las etiquetas definidas para uso interno en las bibliotecas.

Function: `renpy.get_placement (d):`

Obtiene la colocación del elemento mostrable `d`. Hay muy poca garantía sobre esta información, ya que podría cambiar cuando se renderice el elemento mostrable y podría no existir hasta que se renderice por primera vez.

Devuelve un objeto con los siguientes campos, cada uno correspondiente a una propiedad de estilo:

- `xpos`
- `xanchor`
- `xoffset`
- `ypos`
- `yanchor`
- `yoffset`
- `subpixel`

Function: `renpy.loadable (filename):`

Devuelve `True` si el nombre de archivo dado se puede cargar, lo que significa que se puede cargar desde el disco o desde dentro de un archivo. Devuelve `False` si este no es el caso.

Function: `renpy.load_module (name):`

Carga el módulo de Ren'Py llamado `name`. Un módulo de Ren'Py consiste en código Ren'Py que se carga en el espacio de nombres habitual (`store`), contenido en un archivo

llamado `name.rpym` o `name.rpymc`. Si existe un archivo `.rpym` y es más nuevo que el archivo `.rpymc` correspondiente, se carga y se crea un nuevo archivo `.rpymc`.

Todo el código `init` del módulo se ejecuta antes de que esta función retorne. Se lanza un error si no se puede encontrar el nombre del módulo o es ambiguo.

La carga de módulos solo puede ocurrir desde dentro de un bloque `init`.

Function: `renpy.seen_audio (filename):`

Devuelve `True` si el nombre de archivo dado se ha reproducido al menos una vez en el sistema del usuario actual.

Function: `renpy.seen_image (name):`

Devuelve `True` si la imagen nombrada se ha visto al menos una vez en el sistema del usuario. Una imagen se ha visto si se ha mostrado mediante la declaración `show`, la declaración `scene` o la función `renpy.show`. (Ten en cuenta que hay casos en los que el usuario no verá realmente la imagen, como un `show` seguido inmediatamente de un `hide`).

Function: `renpy.seen_label (label):`

Devuelve `True` si la etiqueta nombrada se ha ejecutado al menos una vez en el sistema del usuario actual, y `False` en caso contrario. Esto se puede usar para desbloquear galerías de escenas, por ejemplo.

Variable: `renpy.random = ...` Este objeto es un generador de números aleatorios que implementa la interfaz de generación de números aleatorios de Python. La aleatoriedad se puede generar llamando a los diversos métodos que este objeto expone. Consulta la documentación de Python para la lista completa, pero los más útiles son:

Method: `renpy.random.random ():`

Devuelve el siguiente número de punto flotante aleatorio en el rango (0.0, 1.0).

Method: `renpy.random.randint (a, b):`

Devuelve un entero aleatorio tal que $a \leq N \leq b$.

Method: `renpy.random.choice (seq):`

Devuelve un elemento aleatorio de la secuencia no vacía `seq`.

A diferencia del generador de números aleatorios estándar de Python, este objeto coopera con el retroceso, generando los mismos números independientemente de cuántas veces retrocedamos. Debe usarse en lugar del módulo `random` estándar de Python.

```
# devuelve un flotante aleatorio entre 0 y 1
$ randfloat = renpy.random.random()
# devuelve un entero aleatorio entre 1 y 20
$ d20roll = renpy.random.randint(1, 20)
# devuelve un elemento aleatorio de una lista
$ randfruit = renpy.random.choice(['manzana', 'naranja', 'ciruela'])
```

Audio

Ren'Py admite la reproducción de música y efectos de sonido en segundo plano, utilizando los siguientes formatos de archivo de audio:

- OGG Vorbis
- MP3
- WAV (solo PCM sin comprimir)

Ren'Py admite un número arbitrario de canales de audio. Tres están definidos por defecto:

- **music** - Un canal para la reproducción de música.
- **sound** - Un canal para efectos de sonido.
- **voice** - Un canal para voz.

Las configuraciones de *Volumen de música*, *Volumen de sonido* y *Volumen de voz* del menú de preferencias dentro del juego se utilizan para establecer volúmenes individuales para estos canales.

Los sonidos también se pueden configurar para que se reproduzcan cuando los botones, las opciones de menú o los mapas de imagen entren en sus estados **hovered** o **activated**. Consulta las propiedades de sonido.

Declaraciones de audio

La forma habitual de reproducir música y sonido en Ren'Py es utilizando las tres declaraciones de música/sonido:

■ La declaración **Play**.

```
play channelname audiofile(s) [fadein] [fadeout]
```

Se utiliza para reproducir sonido y música. Si un archivo se está reproduciendo actualmente, se interrumpe y se reemplaza con el nuevo archivo.

- **channelname** debe ser el nombre de un canal (normalmente "sound", "music" o "voice").
- **audiofile(s)** puede ser un archivo o una lista de archivos.
- Las cláusulas **fadein** y **fadeout** son opcionales. **fadeout** da el tiempo de desvanecimiento para la música que se está reproduciendo actualmente, en segundos, mientras que **fadein** da el tiempo que tarda en desvanecerse la nueva música.

```
play music "mozart.ogg"
play sound "woof.mp3"
play myChannel "punch.wav" # 'myChannel' necesita ser definido con renpy.music
"También podemos reproducir una lista de sonidos o música."
play music [ "a.ogg", "b.ogg" ] fadeout 1.0 fadein 1.0
```

■ La declaración **Stop**.

```
stop sound
stop music fadeout 1.0
```

■ La declaración **Queue**.

```
queue sound "woof.ogg"  
queue music [ "a.ogg", "b.ogg" ]
```

La ventaja de usar estas declaraciones es que tu programa será revisado en busca de archivos de sonido y música faltantes cuando se ejecute `lint`. Las funciones que se indican a continuación existen para permitir el control de la música y el sonido desde Python y exponer funciones avanzadas (de uso poco frecuente).

Dos variables de configuración, `config.main_menu_music` y `config.game_menu_music`, permiten que los archivos de música dados se reproduzcan como música del menú principal y del menú del juego, respectivamente.

Definir canales

Es posible definir tu propio canal llamando a `renpy.music.register_channel` en el código `init`.

Function: `renpy.music.register_channel (channel, mixer, loop, tight=False, file_prefix=, file_suffix=)`:

Registra un nuevo canal de audio llamado `channel`.

- **mixer** - El nombre del mezclador que usa el canal. Los tres mezcladores que Ren'Py conoce por defecto son "music", "sfx" y "voice".
- **loop** - Determina si los sonidos en este canal se repiten por defecto.
- **tight** - Determina si los sonidos se repiten incluso durante el desvanecimiento.
- **file_prefix** - Un prefijo que se antepone a los nombres de archivo proporcionados a este canal antes de reproducirlos.
- **file_suffix** - Un sufijo que se añade a los nombres de archivo proporcionados a este canal antes de reproducirlos.

Funciones de música

Las funciones de música proporcionan una interfaz programática para la reproducción de música.

Function: `renpy.music.play (filenames, channel="music", loop=None, fadeout=None, synchro_start=False, fadein=0, tight=False, if_changed=False)`:

Detiene la música que se reproduce actualmente en el canal nombrado, elimina de la cola cualquier música encolada y comienza a reproducir el archivo o archivos especificados.

- **filenames** - Puede ser un solo archivo o una lista de archivos.
- **loop** - Si es `True`, las pistas se repetirán cuando terminen de reproducirse. Si es `False`, no se repetirán. Si es `None`, toma el valor predeterminado del canal.
- **fadeout** - Si es `None`, el tiempo de desvanecimiento se toma de `config.fade_music`; de lo contrario, es un tiempo en segundos para desvanecerse.

- **synchro_start** - Si es **True**, todos los canales que han tenido una llamada **play** con **synchro_start** establecido en **True** se iniciarán al mismo tiempo, de forma precisa a nivel de muestra. Esto se puede usar, por ejemplo, para tener una pieza musical separada en archivos de audio de percusión, melodía y acordes de fondo, y reproducirlos simultáneamente.
- **fadein** - El número de segundos para desvanecer la música, solo en el primer bucle.
- **tight** - Si es **True**, los desvanecimientos se extenderán al siguiente sonido encolado. Si es **False**, no lo hará, y si es **None**, toma el valor predeterminado del canal.
- **if_changed** - Si es **True** y el archivo de música se está reproduciendo actualmente, no se detendrá/desvanecerá y se desvanecerá de nuevo, sino que se mantendrá reproduciendo (esto siempre encolará un bucle adicional de la música).

Function: `renpy.music.queue (filenames, channel="music", loop=None, clear_queue=True, fadein=0, tight=None):`

Encola los nombres de archivo dados en el canal nombrado.

- **filenames** - Puede ser un solo nombre de archivo o una lista de nombres de archivo.
- **loop** - Si es **True**, esta música se repetirá. Si es **False**, no lo hará. Si es **None**, toma el valor predeterminado del canal.
- **clear_queue** - Si es **True**, la cola se borra, haciendo que estos archivos sean los que se reproducen cuando el archivo actual termina. Si es **False**, estos archivos se colocan al final de la cola. En cualquier caso, si no se reproduce música, estos archivos comienzan a reproducirse inmediatamente.
- **fadein** - El número de segundos para desvanecer la música, solo en el primer bucle.
- **tight** - Si es **True**, los desvanecimientos se extenderán al siguiente sonido encolado. Si es **False**, no lo hará, y si es **None**, toma el valor predeterminado del canal.

Function: `renpy.music.stop (channel="music", fadeout=None):`

Detiene la música que se reproduce actualmente en el canal nombrado, elimina toda la música encolada y establece el último archivo encolado en **None**.

- **fadeout** - Si es **None**, la música se desvanece durante el tiempo dado en `config.fade_music`; de lo contrario, se desvanece durante el número de segundos dado.

Function: `renpy.music.set_volume (volume, delay=0, channel="music"):`

Establece el volumen del canal nombrado. El volumen es un número entre 0.0 y 1.0, y se interpreta como una fracción del volumen del mezclador para el canal.

Tarda **delay** segundos en cambiar/desvanecer el volumen del valor antiguo al nuevo. Este valor se conserva en los guardados y participa en el retroceso.

Function: `renpy.music.set_pan (pan, delay, channel="music"):`

Permite panoramizar el sonido y la música entre los dos canales estéreo.

- **pan** - Un número entre -1 y 1 que controla la ubicación del audio. Si es -1, todo el audio se envía al canal izquierdo. Si es 0, los dos canales están equilibrados. Si es 1, todo el audio se envía al oído derecho.

- **delay** - El tiempo que tarda en producirse la panoramización.
- **channel** - El canal en el que se realiza la panoramización. Puede ser un canal de sonido o de música. A menudo, es el canal 7, el canal de música predeterminado.

Function: `renpy.music.get_playing (channel="music"):`

Devuelve el nombre de archivo de la música que se reproduce en el canal dado, o `None` si no se reproduce música en ese canal. Ten en cuenta que se puede devolver `None` cuando el usuario establece el volumen de música a cero, incluso si el guión del juego solicitó que se reprodujera música en ese canal.

Function: `renpy.music.set_queue_empty_callback (callback, channel="music"):`

Establece una función de llamada de retorno que se llama cuando la cola está vacía. Esta función se llama cuando la cola se vacía por primera vez, y al menos una vez por interacción mientras la cola está vacía.

La función de llamada de retorno se llama sin parámetros. Puede encolar sonidos llamando a `renpy.music.queue` con los argumentos apropiados. Ten en cuenta que la función de llamada de retorno puede llamarse mientras se reproduce un sonido, siempre que haya un espacio en la cola vacío.

Funciones de sonido

La mayoría de las funciones de `renpy.music` tienen alias en `renpy.sound`. Estas funciones son similares, excepto que usan por defecto el canal de sonido en lugar del canal de música y no se repiten por defecto.

Películas

Ren'Py es capaz de utilizar `ffmpeg` (incluido) para reproducir películas utilizando los siguientes códecs de video:

- Theora
- MPEG 4 part 2 (incluyendo Xvid y DivX)
- MPEG 2
- MPEG 1

y los siguientes códecs de audio:

- Vorbis
- MP3
- MP2
- PCM

dentro de los siguientes formatos de contenedor:

- Matroska
- Ogg

- Avi
- Varios tipos de flujo MPEG.

(Ten en cuenta que el uso de algunos de estos formatos puede requerir licencias de patentes. En caso de duda, y especialmente para juegos comerciales, recomendamos usar Theora, Vorbis y Matroska u Ogg.)

Ren'Py espera que cada película tenga una pista de audio asociada, incluso si esa pista de audio consiste solo en silencio. Esto se debe a que la pista de audio se utiliza para fines de sincronización.

Las películas se pueden mostrar a pantalla completa o dentro de un elemento mostrable. Las películas a pantalla completa son más eficientes.

Películas a pantalla completa

La forma más sencilla de mostrar una película a pantalla completa es usando la función `renpy.movie_cutscene`. Esta función muestra una película durante un tiempo específico. Cuando ese tiempo ha transcurrido, o cuando el usuario hace clic para descartar la película, la película termina y la función retorna.

Function: `renpy.movie_cutscene (filename, delay=None, loops=0, stop_music=True):`
Muestra una escena de película a pantalla completa.

- **filename** - El nombre de un archivo que contiene una película.
- **delay** - El número de segundos a esperar antes de terminar la escena. Normalmente es la duración de la película, en segundos. Si es `None`, la duración de la película se determinará automáticamente. Si es `-1`, la escena no terminará automáticamente y continuará hasta que el usuario haga clic.
- **loops** - El número de repeticiones adicionales a mostrar; `-1` para repetir indefinidamente.
- **stop_music** - Si es `True`, detiene el canal de música mientras se reproduce la escena y lo reinicia cuando termina.

Devuelve `True` si la película fue terminada por el usuario, o `False` si el retraso dado transcurrió sin interrupción.

```
$ renpy.movie_cutscene("On_Your_Mark.mpg")
```

Películas dentro de elementos mostrables

Una película también se puede mostrar dentro de un elemento mostrable, lo que permite combinarla con otras cosas en la pantalla. Para hacer esto, primero se debe mostrar un elemento mostrable `Movie` y luego reproducir la película en un canal de audio (recomendamos usar el canal `movie` para este propósito).

```
init:
    image movie = Movie(size=(400, 300), xalign=0.5, yalign=0.5)

label movie_sign:
```



```

scene black
show movie
play movie "incubus.mkv"
"Vaya, esta película es realmente terrible."
"Quiero decir, está protagonizada por William Shatner..."
"... hablando en esperanto."
"¡HAGAN QUE SE DETENGA!"
stop movie
hide movie
"Eso es... mejor."

```

Function: Movie (fps=24, size=None, **properties):

Este es un elemento mostrable que muestra la película actual.

- **fps** - La velocidad de fotogramas a la que se debe mostrar la película (actualmente se ignora, pero el parámetro se mantiene por compatibilidad con versiones anteriores; la velocidad de fotogramas se detecta automáticamente).
- **size** - Siempre debe especificarse. Una tupla que da el ancho y alto de la película.

El contenido de este elemento mostrable cuando no se reproduce una película no está definido (y puede cambiar cuando ocurre un retroceso).

Funciones de interfaz de usuario (UI)

Si bien las declaraciones `say` y `menu` suelen ser suficientes para muchos juegos, hay momentos en los que se desean interfaces de usuario más complejas. Por ejemplo, los juegos de simulación de citas pueden requerir pantallas de programación más complejas. Estas pantallas se pueden construir utilizando las funciones de UI.

La mayoría de las funciones de UI crean widgets y los añaden a la pantalla. Las funciones de UI gestionan la complejidad del anidamiento de widgets. La pantalla se puede mostrar al usuario con `ui.interact` y se puede devolver un valor y usarlo en el guión del juego. Para empezar, veamos un fragmento de guión que usa las funciones de UI. El siguiente ejemplo muestra una ventana que contiene tres botones, dispuestos horizontalmente. La cadena asignada a `choice` varía según cuál de los tres se seleccione.

```

$ ui.window()
$ ui.hbox()
$ ui.textbutton("A", clicked=ui.returns("A"))
$ ui.textbutton("B", clicked=ui.returns("B"))
$ ui.textbutton("C", clicked=ui.returns("C"))
$ ui.close()
$ choice = ui.interact(suppress_overlay=True)

```

Hay tres tipos de widgets que se pueden crear con las funciones de UI:

1. El primer tipo de widget (del cual `ui.window` es uno) toma un solo widget hijo. Cuando este tipo de widget está abierto, el siguiente widget que se cree se añadirá a él y luego se cerrará automáticamente.

2. El segundo tipo de widget puede tomar un número arbitrario de widgets hijos. Este tipo de widget es ejemplificado por los widgets de diseño: `vbox`, `hbox` y `fixed`. Los nuevos widgets se añaden a este tipo de widget hasta que se produzca una llamada a `ui.close` coincidente, a menos que se añadan a un hijo abierto de este widget.
3. El tercer tipo de widget no puede tomar ningún hijo. Este tipo de widget es ejemplificado por `ui.textbutton`.

También hay un conjunto de funciones que se utilizan para gestionar la interacción y un conjunto de funciones que se pueden usar para realizar varias acciones cuando se hace clic en los botones.

Widgets con un solo hijo

Function: `ui.button (clicked=None, **properties):`

Crea un botón que el usuario puede pulsar. Cuando se hace clic en este botón o se selecciona de otro modo, se llama a la función proporcionada como argumento `clicked`. Si devuelve un valor, ese valor se devuelve desde `ui.interact`.

Los botones creados con esta función contienen otro widget, específicamente el siguiente widget que se añada. Como conveniencia, se puede usar `ui.textbutton` para crear un botón con una etiqueta de texto. Consulta también `ui.imagebutton`, que crea un botón en forma de imagen.

- **clicked** - Una función que se llama sin argumentos cuando se hace clic en este botón. Si devuelve un valor distinto de `None`, ese valor se devuelve desde `ui.interact`. Si `clicked` es `None`, el botón se muestra en el estilo `insensitive` (deshabilitado).
- **hovered** - Una función que se llama sin argumentos cuando este botón recibe el foco. Si devuelve un valor distinto de `None`, ese valor se devuelve desde `ui.interact`.
- **unhovered** - Una función que se llama sin argumentos cuando el botón pierde el foco. También se llama al menos una vez por interacción, cuando un botón no tiene el foco al inicio de la interacción. El valor de retorno se ignora actualmente, pero debe ser `None`.
- **role** - El rol que desempeña este botón. Puede ser una cadena vacía o `"selected_"`.
- **keymap** - Un mapa de teclas que se usa cuando este botón tiene el foco.

Ten en cuenta que el código en los métodos `clicked`, `hovered` y `unhovered` se ejecuta dentro de la interacción actual. Esto significa que la pantalla no se borra mientras se ejecuta este código. Se pueden añadir o eliminar elementos mostrables de la interacción actual, siempre que se llame a `renpy.restart_interaction` para que Ren'Py sepa que la interacción ha cambiado. No debes ejecutar código que cause una nueva interacción desde dentro de estas funciones, excepto dentro de un nuevo contexto usando `renpy.call_in_new_context`, `renpy.invoke_in_new_context` o `ui.callsinnewcontext`.

Function: `ui.frame (**properties):`

Un marco contiene un solo widget. Es similar a una ventana, pero se puede estilizar de manera diferente.

Function: `ui.sizer (maxwidth=None, maxheight=None, **properties):`

Este es un widget que puede reducir el tamaño asignado al siguiente widget añadido. Si `maxwidth` o `maxheight` no son `None`, el espacio asignado al hijo en la dirección apropiada se limita a la cantidad dada.

Ten en cuenta que esto solo funciona con widgets hijos que pueden tener un área limitada asignada (como texto), y no con los que usan un área fija (como imágenes).

- `maxwidth` - El ancho máximo del widget hijo, o `None` para no afectar al ancho.
- `maxheight` - El alto máximo del widget hijo, o `None` para no afectar al alto.

Function: `ui.transform (function=None, alpha=1, rotate=None, zoom=1, xzoom=1, yzoom=1, style='transform', **properties):`

Envuelve a su hijo en un `Transform`, permitiendo que el hijo sea rotado, escalado y modificado en alfa. Los parámetros son como en `Transform`.

Function: `ui.viewport (child_size=(None, None), xadjustment=None, yadjustment=None, set_adjustments=True, mousewheel=False, draggable=False, style='viewport', **properties):`

Muestra un viewport en la pantalla. Un viewport restringe el tamaño de su hijo y permite que el hijo se muestre con un desplazamiento.

- `child_size` - El tamaño x e y del área que se le pide al hijo que renderice. Si alguno es `None`, se usa el tamaño de este viewport.
- `xadjustment` - Un `ui.adjustment` que se usa para el eje x del viewport.
- `yadjustment` - Un `ui.adjustment` que se usa para el eje y del viewport.
- `set_adjustments` - Si es `true`, el rango y el tamaño de página de los ajustes serán establecidos por este viewport.
- `mousewheel` - Si es `true`, la rueda del ratón se puede usar para desplazar el viewport.
- `draggable` - Si es `true`, el ratón se puede usar para arrastrar el viewport.

Si `xadjustment` o `yadjustment` son `None`, se crean ajustes automáticamente. Estos ajustes están disponibles a través de las propiedades `xadjustment` y `yadjustment` de un viewport. En general, los viewports solo son útiles cuando se especifican las propiedades `xmaximum` e `ymaximum`. También querrás establecer `clipping=True` en el estilo, aunque esto es parte del estilo de viewport predeterminado.

Function: `ui.window (**properties):`

Una ventana contiene un solo widget. Dibuja esa ventana sobre un fondo y con cantidades apropiadas de margen y relleno, tomadas de las propiedades de ventana proporcionadas en esta llamada. El siguiente widget creado se añade a esta ventana.

Widgets con múltiples hijos

Function: `ui.fixed (**properties):`

Crea un diseño que coloca widgets en ubicaciones fijas relativas al origen del widget contenedor. El diseño ocupa toda el área que se le asigna. Los nuevos widgets se añaden a este widget hasta la siguiente llamada a `ui.close`.

Function: `ui.grid (cols, rows, padding=0, transpose=False, **properties):`

Crea un diseño que coloca widgets en una cuadrícula espaciada uniformemente. Los nuevos widgets se añaden a esta cuadrícula hasta que se llama a `ui.close`. Los widgets se añaden de izquierda a derecha dentro de una fila, y hacia abajo al inicio de la siguiente fila cuando una fila está llena. Todas las celdas deben estar llenas (es decir, se deben añadir exactamente `cols * rows` widgets a la cuadrícula).

Los hijos de este widget deben tener un tamaño fijo que no varíe según el espacio asignado. El incumplimiento de esta restricción podría dar lugar a diseños realmente extraños o a que las cosas se rendericen fuera de la pantalla. Esta condición se relaja en la dimensión apropiada si se establece `xfill` o `yfill`.

Cada celda de la cuadrícula tiene exactamente el mismo tamaño. Por defecto, la cuadrícula es el tamaño más pequeño que puede acomodar a todos sus hijos, pero se puede expandir para consumir todo el espacio disponible en una dimensión dada estableciendo `xfill` o `yfill` en `True`, según corresponda (de lo contrario, `xfill` e `yfill` se heredan del estilo).

- `cols` - El número de columnas en esta cuadrícula.
- `rows` - El número de filas en esta cuadrícula.
- `padding` - La cantidad de espacio que dejar entre filas y columnas.
- `xfill` - `True` si la cuadrícula debe consumir todo el ancho disponible.
- `yfill` - `True` si la cuadrícula debe consumir todo el alto disponible.
- `transpose` - Si es `True`, la cuadrícula llenará columnas antes que filas.

Function: `ui.hbox (spacing=None, style='hbox', **properties):`

Crea un diseño que coloca widgets uno al lado del otro, de izquierda a derecha. Los nuevos widgets se añaden a este `hbox` hasta que se llama a `ui.close`.

- `spacing` - El número de píxeles que dejar entre widgets. Si es `None`, toma la cantidad de espacio del estilo.

Function: `ui.vbox (spacing=None, style='vbox', **properties):`

Crea un diseño que coloca widgets uno debajo del otro, de arriba a abajo. Los nuevos widgets se añaden a este `vbox` hasta que se llama a `ui.close`.

- `spacing` - El número de píxeles que dejar entre widgets. Si es `None`, toma la cantidad de espacio del estilo.

Function: `ui.side (places, style='default', **properties):`

Posiciona widgets hijos para que rodeen a un widget padre. Toma un solo argumento, `places`, que controla dónde se coloca cada widget hijo. Se espera que `places` sea iterable, y cada elemento debe estar en la lista:

`'c', 't', 'b', 'l', 'r', 'tl', 'tr', 'bl', 'br'`

`'c'` significa centro, `'t'` parte superior, `'tl'` superior izquierda, `'br'` inferior derecha, y así sucesivamente. Se debe dar al lateral el mismo número de hijos que el número de entradas en la lista `places`.

Los hijos superior e inferior se renderizan con una altura solicitada de 0, mientras que los hijos izquierdo y derecho se renderizan con un ancho solicitado de 0. Por lo tanto, se sugiere que impongas un ancho mínimo en los hijos que se colocan en estas ranuras. El widget central se renderiza antes que los demás. Aparte de eso, el orden en que se renderizan los widgets no está definido.

Widgets sin hijos

Function: `ui.autobar (range, start, end, time, **properties):`

Crea una barra (con un rango de `range`) que se mueve automáticamente desde `start` hasta `end` en `time` segundos.

Function: `ui.bar (range=None, value=None, changed=None, step=None, page=None, adjustment=None, style='bar', **properties):`

Crea un widget de barra. El widget de barra se puede usar para mostrar datos en formato de gráfico de barras y, opcionalmente, para informar cuando el usuario hace clic en una ubicación de esa barra.

- **adjustment** - Siempre debe darse como un argumento de palabra clave. Si no es `None`, debe ser un `ui.adjustment` que usará esta barra, y los siguientes argumentos se ignoran. Si es `None`, se crea un nuevo `ui.adjustment` a partir de los argumentos `range`, `value`, `changed`, `step` y `page`.
- **style** - El estilo de esta barra. A partir de 6.2.0, hay cuatro estilos que puedes usar:
 - **bar** - Una barra horizontal, con el valor más grande a la derecha.
 - **scrollbar** - Una barra de desplazamiento horizontal, con el valor más grande a la derecha.
 - **vbar** - Una barra vertical, con el valor más grande en la parte superior.
 - **vscrollbar** - Una barra de desplazamiento vertical, con el valor más grande en la parte inferior.

El ancho y el alto deben establecerse con las propiedades `xmaximum` e `ymaximum`. Para obtener mejores resultados, si se establece `clicked`, el ancho debe ser al menos el doble del `range`.

Function: `ui.image (im, **properties):`

Carga una imagen y la muestra como un widget. La imagen puede ser el nombre de un archivo que la contenga o un elemento mostrable arbitrario.

Function: `ui.imagebutton (idle_image, hover_image, clicked=None, image_style='image_button', **properties):`

Crea un botón que contiene dos imágenes. La primera es la imagen inactiva, que se usa cuando el ratón no está sobre la imagen, mientras que la segunda es la imagen de hover, que se usa cuando el ratón está sobre la imagen. Si se hace clic en el botón o se selecciona de otro modo, se llama al argumento `clicked`. Si devuelve un valor, ese valor se devuelve desde `ui.interact`.

- **idle_image** - El nombre del archivo de la imagen usada cuando este botón está inactivo (no tiene el puntero del ratón sobre él, ni foco del teclado o joystick). Debe especificarse.
- **hover_image** - El nombre del archivo de la imagen usada cuando este botón está en hover (el puntero del ratón está sobre él, o el usuario lo ha enfocado con el teclado o joystick). Debe especificarse.
- **clicked** - La función que se llama cuando se hace clic en este botón.
- **hovered** - Una función que se llama sin argumentos cuando este botón está en hover. Si devuelve un valor distinto de `None`, ese valor se devuelve desde `ui.interact`.

- `unhovered` - Una función que se llama sin argumentos cuando este botón pierde el foco.
- `image_style` - El estilo que se aplica a las imágenes que se usan como parte del `imagebutton`.
- `role` - El rol que desempeña este botón. Puede ser una cadena vacía o `"selected_"`.
- `insensitive_image` - Si se especifica, el nombre del archivo de la imagen usada cuando este botón está deshabilitado (`clicked` es `None`).
- `activate_image` - Si se especifica, el nombre del archivo de la imagen usada cuando este botón está activado (el botón del ratón se mantiene presionado sobre él, o equivalentes para teclado y joystick).

También existen `selected_idle_image`, `selected_hover_image`, `selected_insensitive_image` y `selected_activate_image`. Son los equivalentes de `idle_image`, `hover_image`, `insensitive_image` y `activate_image` que se usan cuando este botón está seleccionado (representa el estado actual, como el botón *Guardar juego* en la pantalla de guardado).

Function: `ui.imagemap (ground, selected, hotspots, unselected=None, style='imagemap', button_style='imagemap_button', **properties):`

Se llama para crear mapas de imagen. Los parámetros son más o menos los mismos que en `renpy.imagemap`. El valor de la zona interactiva se devuelve cuando `ui.interact` retorna.

Function: `ui.input (default, length=None, allow=None, exclude='{}', button=None, changed=None, prefix=, suffix=, **properties):`

Muestra un área de texto que acepta entrada del usuario. Solo se introduce ASCII de forma fiable, aunque algunos idiomas no ASCII también pueden funcionar en algunas plataformas.

- `default` - El valor predeterminado para el área de texto.
- `length` - Si no es `None`, un límite en el número de caracteres que se pueden introducir.
- `allow` - Si no es `None`, una cadena que contiene los caracteres permitidos.
- `exclude` - Una cadena que contiene caracteres no permitidos. Los caracteres de llave siempre deben estar deshabilitados, ya que podrían hacer que se muestre accidentalmente una etiqueta de texto.
- `button` - Si no es `None`, debe ser un botón. La entrada solo está activa cuando este botón está enfocado.
- `changed` - Si no es `None`, esta función se llama cuando se cambia el texto.
- `prefix` - Un prefijo incluido antes del texto a editar.
- `suffix` - Un sufijo incluido después del texto a editar.

Si `changed` es `None`, el texto se devuelve como resultado de `ui.interact()` cuando se pulsa enter. De lo contrario, esto no puede hacer que `ui.interact()` termine.

Function: `ui.keymap (**keymap):`

Este es un pseudowidget que añade un mapa de teclas a la pantalla. Esta función toma como argumentos de palabra clave los nombres de enlaces o teclas. A estas palabras clave

se les debe dar como argumento una función, que se llama sin argumentos cuando se pulsa la tecla apropiada.

Function: `ui.menu (menuitems, style='menu', caption_style='menu_caption', choice_style='menu_choice', choice_chosen_style='menu_choice_chosen', choice_button_style='menu_choice_button', choice_chosen_button_style='menu_choice_chosen_button', location=None, focus=None, default=False, **properties):`

Crea un nuevo widget de menú. A diferencia de la declaración `menu` o la función `renpy.display_menu`, este widget de menú no está encerrado en ningún tipo de ventana. Tendrías que hacerlo tú mismo, si se desea.

- **menuitems** - Una lista de tuplas que son los elementos que se añadirán a este menú. El primer elemento de una tupla es una cadena que se usa para este elemento de menú. El segundo elemento es el valor que se devuelve desde `ui.interact` si se selecciona este elemento, o `None` si este elemento es un título no seleccionable.
- **location** - Algún objeto serializable y con hash que representa la ubicación de este menú en el juego (normalmente, es el nombre de la declaración que ejecuta el menú). Si se proporciona, registra qué elementos del menú se han elegido y cambia el estilo de esos elementos a `choice_seen_style`.

Function: `ui.null (width=0, height=0, **properties):`

Este widget no muestra nada en la pantalla. ¿Por qué alguien querría hacer esto? Si un widget requiere contenido, pero no tienes contenido que proporcionarle. También se puede usar para crear espacio vacío, dando argumentos de ancho y alto distintos de cero.

Function: `ui.pausebehavior (delay, result=False):`

Este es un pseudowidget que añade el comportamiento de pausa a la pantalla. El comportamiento de pausa es devolver el **result** proporcionado cuando transcurre el número de segundos dado. Este widget no debe añadirse a ningún otro widget, sino solo a la propia pantalla.

Ten en cuenta que este widget siempre pausará durante el tiempo dado. Si quieres una pausa que pueda ser interrumpida por el usuario, añade un `saybehavior`.

- **delay** - La cantidad de tiempo a pausar, en segundos.
- **result** - El resultado que se devolverá después de que transcurra el tiempo de retraso.

Function: `ui.saybehavior (afm=None, dismiss=[ 'dismiss' ], allow_dismiss=None):`

Este es un pseudowidget que añade el comportamiento de diálogo a la pantalla. El comportamiento de diálogo es devolver `True` si se hace clic con el botón izquierdo del ratón o se pulsa enter. También devuelve `True` en varios otros casos, como si la declaración actual ya se ha visto. Este widget no debe añadirse a ningún otro widget, sino solo a la propia pantalla.

Si **afm** está presente, es un bloque de texto que se da al algoritmo del modo de avance automático para determinar el tiempo de espera del avance automático.

Si **dismiss** está presente, es una lista de nombres de enlaces de teclas que se usan para descartar este `saybehavior`.

Si **allow_dismiss** está presente, debe ser una función. Esta función se llama sin argumentos cada vez que el usuario inicia una solicitud de descarte. Si la función devuelve `True`, la solicitud de descarte se permite; de lo contrario, se ignora (añadido en 5.6.6).

Solo debe haber un `saybehavior` mostrado a la vez; de lo contrario, competirán por el foco.

Cambios: El parámetro `dismiss` se cambió en 5.6.1 de un solo enlace de teclas a una lista de enlaces de teclas.

Function: `ui.soundstopbehavior (channel, result=False):`

Añadir este comportamiento a la pantalla hace que `ui.interact` devuelva `result` cuando no hay sonido reproduciéndose en el canal de música nombrado.

Function: `ui.text (label, **properties):`

Crea un widget que muestra una etiqueta de texto.

- **label** - El texto que se mostrará en la pantalla. Usa propiedades de fuente. La etiqueta también puede ser una lista que contenga cadenas de texto y widgets; en el caso de un widget, se muestra como si fuera texto de la fuente contenedora. La altura del área asignada es la de la fuente, y el ancho se toma del renderizado del widget.
- **slow** - Si es `True`, el texto se escribirá en la pantalla a una velocidad determinada por la propiedad `slow_cps`, si está establecida, o por la preferencia *Velocidad de texto*. Si es `None` (el valor predeterminado), se escribirá a una velocidad determinada por `slow_cps`. Si es `False`, aparecerá instantáneamente.
- **slow_speed** - Si `slow` es `True`, este es el número de cps al que se muestra el texto, anulando la preferencia.
- **slow_done** - Si no es `None` y `slow` es `True`, esta es una función de llamada de retorno que se llama cuando se ha terminado de mostrar el texto en la pantalla.

Function: `ui.timer (seconds, function, repeat=False, args=(), kwargs={}):`

Configura un temporizador que llamará a `function` después de que hayan transcurrido `seconds` segundos. Si `repeat` es `true`, la función se llama cada `seconds` segundos a partir de entonces. `args` y `kwargs` son los argumentos posicionales y de palabra clave que se proporcionan a la función, respectivamente.

Funciones de gestión

Function: `ui.add (w, make_current=False, once=False):`

Se utiliza para añadir un elemento mostrable al widget actual. Si `make_current` es `True`, el widget se convierte en el widget actual. Si `once` es `True`, el elemento mostrable se convertirá en el widget actual, pero solo para el siguiente widget que se añada.

Function: `ui.at (position):`

La función `ui.at` toma una posición o movimiento y lo aplica al siguiente widget creado mediante una función de UI.

Function: `ui.clear ():`

Limpia la capa actual de widgets. No es particularmente útil en la práctica y es básicamente duplicativo de `renpy.scene`.

Function: `ui.close ():`

Cierra el widget o capa actualmente abiertos. Si se cierra un widget, comenzamos a añadir a su padre, o a la capa si no hay ningún padre abierto. Si se cierra una capa, volvemos a la capa previamente abierta. Se lanza un error si cerramos la última capa abierta.

Function: `ui.interact (**kwargs):`

Muestra la escena actual al usuario, espera a que un widget indique un valor de retorno y devuelve ese valor al usuario. Como efecto secundario, deshabilita el modo de omisión rápida cuando se ejecuta.

Algunos argumentos de palabra clave útiles son:

- **type** - El tipo de esta interacción. Consulta `renpy.last_interact_type` para más detalles.
- **show_mouse** - ¿Debe mostrarse el ratón durante esta interacción? Solo es orientativo, ya que no funciona de forma fiable.
- **suppress_overlay** - Suprime la visualización de la superposición durante esta interacción.
- **suppress_underlay** - Suprime la subcapa durante esta interacción. La subcapa contiene los enlaces de teclado y ratón que permiten al usuario acceder al menú del juego (así como otras funciones útiles dentro del juego).
- **suppress_window** - Suprime la visualización de una ventana vacía cuando de otro modo se mostraría.
- **roll_forward** - El valor devuelto si el usuario intenta avanzar. Normalmente se usa junto con `renpy.roll_forward_info` y `renpy.checkpoint`.
- **clear** - Si es `True`, la capa transitoria se limpiará al final de la interacción.

Function: `ui.layer (name):`

Hace que los widgets se añadan a la capa nombrada, hasta que se produzca una llamada a `ui.close`.

Function: `ui.remove (d):`

Elimina el elemento mostrable `d` de la capa actual. Solo puede eliminar cosas directamente de la capa, no de un elemento mostrable.

Function: `ui.tag (name):`

Especifica la etiqueta de imagen para el siguiente widget que se añada a una capa. Si el siguiente widget que se crea no se añade a una capa, se ignora. Cuando se especifica una etiqueta, los widgets anteriores con la misma etiqueta se eliminan de la capa.

Funciones de acción de clic

El resultado de estas funciones es adecuado para usar como el argumento `clicked` utilizado por `ui.button`, `ui.imagebutton` y `ui.textbutton`.

Function: `ui.callsinnewcontext (label, *args, **kwargs):`

Esta función devuelve una función que, cuando se llama, llama a la etiqueta proporcionada en un nuevo contexto. Los argumentos posicionales y de palabra clave adicionales se pasan a la etiqueta llamada.

Function: `ui.jumps (label, transition=None):`

Esta función devuelve una función que, cuando se llama, salta el juego a la etiqueta dada, terminando la interacción actual en el proceso. Es mejor usarla para proporcionar el argumento `clicked` a los diversos widgets de botón.

- **transition** - Si no es `None`, especifica una transición que se usará cuando ocurra la siguiente interacción. Puede ser una transición o una cadena. Si es una cadena, se interpreta como un campo en el objeto `config`, al que se accede para determinar la transición que debe ocurrir (nuevo en 6.9.0).

Function: `ui.jumpsoutofcontext (label):`

Esta función devuelve una función que, cuando se llama, sale del contexto actual y, en el contexto padre, salta a la etiqueta nombrada. Está pensada para ser usada como el argumento `clicked` de un botón.

Function: `ui.returns (value):`

Esta función devuelve una función que, cuando se llama, devuelve el valor proporcionado. Es mejor usarla para proporcionar el argumento `clicked` a los diversos widgets de botón.

Ajuste (Adjustment)

Function: `ui.adjustment (range=1, value=0, step=None, page=0, changed=None, adjustable=True, ranged=None):`

Los objetos de ajuste se utilizan para representar un valor numérico que puede variar entre 0 y un número definido. También contienen información sobre los pasos en los que se puede ajustar el valor.

- **range** - Es el límite del rango del valor. Puede ser un entero o un flotante.
- **value** - Es el valor inicial del ajuste.
- **step** - Es el tamaño de los pasos en los que se puede ajustar el valor. El valor predeterminado para esto depende de **page** y **range**. Si se establece **page**, el valor predeterminado es 1/10 de página. Si **range** es un flotante, el valor predeterminado es 1/20 del rango; de lo contrario, el valor predeterminado es 1 píxel.
- **page** - Es el tamaño de una página, que se usa cuando la barra está en modo de paginación. Si no se establece, el valor predeterminado es 1/10 del rango.

Estos cuatro argumentos corresponden a campos en el objeto de ajuste. Los campos se pueden establecer de forma segura antes de que se renderice cualquier elemento que use el ajuste. Después de eso, deben tratarse como de solo lectura.

- **changed** - Es una función que se llama cuando se cambia el ajuste. La función se llama con un argumento, el nuevo valor del ajuste. Debe estar presente para que la barra sea ajustable.
- **adjustable** - Controla si las barras sin funciones **changed** son ajustables. Si **changed** no es `None`, la barra siempre es ajustable y este argumento se ignora.
- **ranged** - Es una función que se llama con este ajuste como argumento cuando se establece el rango del ajuste (nuevo en 6.6.0).

Los objetos de ajuste tienen un método:

Method: `Adjustment.change (value):`

Llama a esto para establecer el valor del ajuste en **value**. Esto volverá a dibujar todos los elementos mostrables afectados por este cambio.

Variables de configuración

Gran parte de la configuración de Ren'Py se realiza mediante variables de configuración. Estas variables, cuando se asignan en un bloque Python, cambian el comportamiento del intérprete. Como las variables de configuración no se guardan, y muchas deben establecerse antes de que la GUI se inicialice, tiene sentido establecer todas las variables de configuración dentro de bloques `init`. Un ejemplo de configuración de variables es:

```
init -1 python:
    config.screen_width = 640
    config.screen_height = 480
```

De uso común

Variable: `config.archives = [ ]`

Una lista de archivos que se buscarán para encontrar imágenes.

Variable: `config.developer = False`

Si se establece en `True`, se activa el modo desarrollador (ten en cuenta que la plantilla de script predeterminada lo establece en `True`).

Variable: `config.help = README.html`

Controla la funcionalidad del sistema de ayuda invocado por el botón de ayuda en los menús principal y del juego, o pulsando `f1` o `command-?`.

- Si es `None`, el sistema de ayuda está deshabilitado y no aparece en los menús.
- Si es una cadena que corresponde a una etiqueta en el guión, esa etiqueta se invoca en un nuevo contexto. Esto permite definir una pantalla de ayuda dentro del juego.
- De lo contrario, se interpreta como un nombre de archivo relativo al directorio base, que se abre en un navegador web.

(nuevo en 6.8.0)

Variable: `config.screen_height = 600`

Establece la altura de la pantalla.

Variable: `config.screen_width = 800`

Establece el ancho de la pantalla.

Variable: `config.translations = dict(...)`

Un mapa que se utiliza para traducir el texto del menú del juego a tu idioma. Consulta *Localizar Ren'Py* para saber cómo usarlo, y aquí para una lista de traducciones disponibles.

Variable: `config.window_icon = None`

Si no es `None`, se espera que sea el nombre de archivo de una imagen que proporciona un icono utilizado para la ventana en Linux y Mac OS X. Debe ser una imagen grande con alfa de 8 bits.

Variable: `config.windows_icon = None`

Si no es `None`, se espera que sea el nombre de archivo de una imagen que proporciona un icono utilizado para la ventana en Windows. Debe ser una imagen de 32x32 con alfa de 1 bit (las imágenes opacas funcionan mejor).

Variable: `config.window_title = ".^ Ren'Py Game"`

Se antepone a `_window_subtitle` para producir el título de la ventana.

Variable: `config.save_directory = "..."`

Se utiliza para generar el directorio predeterminado en el que se guardan las partidas y la información persistente. El nombre del directorio generado depende de la plataforma:

- Windows: %APPDATA%/RenPy/save_directory
- Mac OS X: /Library/RenPy/save_directory
- Linux/Otros: /.renpy/save_directory

Esto debe establecerse en un bloque `python early`, ya que el nombre del directorio de guardado debe conocerse antes de que se ejecute el código `init`, ya que la información persistente está disponible para el código `init`.

Este directorio puede ser anulado por la invocación del usuario. El código que necesita conocer el directorio de guardado debe leer `config.savedir` en su lugar (nuevo en 6.7.0).

De uso ocasional

Variable: `config.adv_nvl_transition = None`

Una transición que se usa al mostrar texto en modo NVL inmediatamente después de texto en modo ADV.

Variable: `config.after_load_transition = None`

Una transición que se usa después de cargar, al entrar en la partida cargada.

Variable: `config.auto_load = None`

Si no es `None`, el nombre de un archivo de guardado para cargar automáticamente cuando Ren'Py se inicia. Está pensado para uso del desarrollador, no para usuarios finales. Establecer esto en "1" cargará automáticamente la partida en la ranura de guardado 1 (nuevo en 6.10.0).

Variable: `config.automatic_images = None`

Si no es `None`, hace que Ren'Py defina imágenes automáticamente. Cuando no se establece en `None`, debe ser una lista de separadores (por ejemplo, [' ', ' ', ' ', ' ']). Ren'Py escaneará la lista de archivos en disco y en archivos. Cuando encuentre un archivo que termine en `.png` o `.jpg`, eliminará la extensión y lo dividirá en los separadores para crear un nombre de imagen. Si el nombre consta de al menos dos componentes y no hay una imagen con ese nombre ya definida, Ren'Py definirá esa imagen para que haga referencia a un nombre de archivo.

Por ejemplo, si tu directorio del juego contiene:

- `eileen_happy.png`, Ren'Py definirá la imagen `.eileen happy`.
- `lucy/mad.png`, Ren'Py definirá la imagen `"lucy mad"`.
- `mary.png`, Ren'Py no hará nada (ya que la imagen no tiene dos componentes).

(nuevo en 6.10.0)

Variable: `config.automatic_images_strip = []`

Una lista de cadenas que dan prefijos que se eliminan al definir imágenes automáticas. Esto se puede usar para eliminar nombres de directorios, cuando los directorios contienen imágenes (nuevo en 6.10.2).

Variable: `config.debug = False`

Activa algunas funcionalidades menores de depuración (principalmente convirtiendo algunos archivos faltantes en errores). Siempre debe estar desactivado en un lanzamiento.

Variable: `config.debug_image_cache = False`

Si es `True`, Ren'Py imprimirá el contenido de la caché de imágenes en la salida estándar (donde sea que vaya) cada vez que cambie el contenido de la caché de imágenes.

Variable: `config.debug_sound = False`

Activa la depuración de la funcionalidad de sonido. Esto deshabilita la supresión de errores al generar sonido. Sin embargo, si falta una tarjeta de sonido o es defectuosa, estos errores son normales y activar esto puede evitar que Ren'Py funcione con normalidad. Siempre debe ser `False` en un juego lanzado.

Variable: `config.default_afm_time = None`

Si no es `None`, establece el tiempo de espera predeterminado del modo de avance automático. Si no es `None`, este es el tiempo en segundos que debemos esperar al mostrar 250 caracteres. 0 se trata de forma especial como tiempo infinito, deshabilitando el modo de avance automático (nuevo en 6.11.0).

Variable: `config.default_afm_enable = None`

Si no es `None`, debe ser un booleano que controla si el modo de avance automático está habilitado por defecto. Cuando es `False`, no se producirá el avance automático. Establece esto en `False` con precaución, ya que la interfaz de usuario predeterminada de Ren'Py no proporciona una forma de cambiar su configuración (pero se puede usar una acción `Preference` en una pantalla para crear dicha interfaz) (nuevo en 6.11.0).

Variable: `config.default_fullscreen = None`

Establece el valor predeterminado de la preferencia de pantalla completa. Debe ser `True` o `False`. Si es `None`, se ignora, permitiendo que otro código establezca el valor predeterminado (normalmente se establece en `False` en `options.rpy`).

El archivo `saves/persistent` en el directorio del juego debe eliminarse para que esto surta efecto.

Variable: `config.default_text_cps = None`

Si no es `None`, establece el número predeterminado de caracteres por segundo a mostrar. 0 se trata de forma especial como un número infinito de caracteres por segundo (normalmente se establece en 0 en `options.rpy`). El archivo `saves/persistent` en el directorio del juego debe eliminarse para que esto surta efecto.

Variable: `config.empty_window = ...`

Se llama cuando `_window` es `True` y no se ha mostrado ninguna ventana en la pantalla (es decir, no ha ocurrido ninguna llamada a `renpy.shown_window`). Se espera que muestre una ventana vacía en la pantalla y retorne sin causar una interacción.

La implementación predeterminada usa el personaje `narrator` para mostrar una línea en blanco sin interactuar.

Variable: `config.end_game_transition = None`

La transición que se usa para mostrar el menú principal después de que el juego termine normalmente, ya sea invocando `return` sin lugar al que regresar o llamando a `renpy.full_restart`.

Variable: `config.end_splash_transition = None`

La transición que se usa para mostrar el menú principal después del final de la pantalla de presentación (`splashscreen`).

Variable: `config.enter_sound = None`

Si no es `None`, es un archivo de sonido que se reproduce al entrar en el menú del juego sin hacer clic en un botón (por ejemplo, al hacer clic con el botón derecho durante el juego).

Variable: `config.enter_transition = None`

Si no es `None`, esta variable debe proporcionar una transición que se usará al entrar en el menú del juego.

Variable: `config.exit_sound = None`

Si no es `None`, es un archivo de sonido que se reproduce al salir del menú del juego sin hacer clic en un botón (por ejemplo, al hacer clic con el botón derecho dentro del menú del juego).

Variable: `config.exit_transition = None`

Si no es `None`, esta variable debe proporcionar una transición que se realizará al salir del menú del juego.

Variable: `config.font_replacement_map = dict()`

Este es un mapa de (fuente, negrita, cursiva) a (fuente, negrita, cursiva), utilizado para reemplazar una fuente con otra especializada en negrita y/o cursiva. Por ejemplo, si quisieras que todo lo que usa una versión cursiva de "Vera.ttf" use "VeraIt.ttf" en su lugar, podrías escribir `config.font_replacement_map["Vera.ttf", False, True] = ("VeraIt.ttf", False, False)`. Ten en cuenta que estas asignaciones solo se aplican a variantes específicas de una fuente. En este caso, las solicitudes de una versión en negrita cursiva de Vera obtendrán una versión en negrita cursiva de Vera, en lugar de una versión en negrita de la Vera cursiva.

Variable: `config.framerate = 100`

Si no es `None`, este es el límite superior en el número de fotogramas que Ren'Py intentará mostrar por segundo.

Variable: `config.game_main_transition = None`

La transición que se usa para mostrar el menú principal después de salir del menú del juego. Se usa cuando las pantallas de carga y preferencias se invocan desde el menú principal, y también se usa cuando el usuario elige *Menú principal* desde el menú del juego.

Variable: `config.game_menu = [ ... ]`

Se usa para personalizar las opciones del menú del juego. Consulta *Menús principal y del juego* para obtener más detalles sobre el contenido de esta variable.

Variable: `config.game_menu_music = None`

Si no es `None`, un archivo de música para reproducir en el menú del juego.

Variable: `config.has_autosave = True`

Si es `true`, el juego guardará automáticamente. Si es `false`, no se realizará ningún autoguardado.

Variable: `config.hyperlink_callback = ...`

Una función que se llama con el argumento de un hipervínculo cuando se hace clic en él. El valor predeterminado es una función que interpreta el argumento como una etiqueta, que se llama en un nuevo contexto cuando se hace clic en el hipervínculo. La función predeterminada también envía argumentos que comienzan con "http:" como URL, abriendo un navegador web para ellos.

Variable: `config.hyperlink_focus = None`

Si no es `None`, una función que se llama cuando un hipervínculo gana o pierde el foco. Cuando se gana el foco, esta función se llama con un solo argumento, el argumento del hipervínculo. Cuando se pierde el foco, la función se llama con `None` como argumento.

Variable: `config.hyperlink_styler = ...`

Una función que se llama para determinar el estilo de los hipervínculos. La función se llama con un solo argumento, el argumento del hipervínculo. Debe devolver un objeto de estilo, no una cadena (nuevo en 6.6.2).

Variable: `config.image_cache_size = 8`

Se usa para establecer el tamaño de la caché de imágenes, como un múltiplo del tamaño de la pantalla. Esto es principalmente una sugerencia, ya que si Ren'Py necesita mostrar más de esta cantidad de imágenes a la vez, las almacenará en caché todas. Pero dejará de cargar imágenes de forma predictiva cuando la caché se llene más allá de esto, e intentará eliminar imágenes de la caché hasta que se reduzca por debajo de este tamaño. Si es demasiado pequeño, habrá más acceso a disco a medida que las imágenes se cargan y recargan. Si es demasiado grande, el uso de memoria aumentará a medida que las imágenes se mantengan en memoria.

Variable: `config.main_game_transition = None`

La transición que se usa al entrar en el menú del juego desde el menú principal, como se hace al hacer clic en *Cargar partida* o *Preferencias*.

Variable: `config.main_menu = [ ( "Start Game", "start", "True"), (Continue Game", ... ), ("Preferences", ... ), ("Quit Game", ... ) ]`

Se usa para proporcionar el menú principal que se muestra al usuario cuando el juego se inicia por primera vez. Es una lista de tuplas, donde el primer elemento de cada tupla es el título del botón del menú y el segundo elemento es una etiqueta a la que saltamos cuando se selecciona ese botón (este salto sale del contexto en el que se ejecuta el menú de inicio). El segundo elemento también puede ser una función, en cuyo caso se llama cuando se selecciona el elemento, en el contexto del menú. El tercer elemento en cada tupla es una expresión, que se evalúa cada vez que se muestra el menú principal. Si la expresión se evalúa como `false`, la opción del menú se deshabilita.

Para más información, consulta *Menús principal y del juego*.

Variable: `config.main_menu_music = None`

Si no es `None`, un archivo de música para reproducir en el menú principal.

Variable: `config.menu_clear_layers = []`

Una lista de nombres de capas (como cadenas) que se limpian al entrar en el menú del juego (nuevo en 6.10.2).

Variable: `config.menu_window_subtitle =`

La variable `_window_subtitle` se establece en este valor al entrar en los menús principal o del juego (nuevo en 6.7.1).

Variable: `config.missing_background = "black"`

Este es el fondo que se usa cuando `config.developer` es `True` y se usa una imagen no definida en una declaración `scene`. Es un nombre de imagen, no un elemento mostrable. La imagen debe estar definida (nuevo en 6.8.0).

Variable: `config.mouse = None`

Esta variable controla el uso de cursores de ratón definidos por el usuario. Si es `None`, se usa el cursor del sistema, que suele ser un cursor de ratón blanco y negro. De lo contrario, debe ser un diccionario que proporcione las animaciones del ratón para varios tipos de ratón. Las claves utilizadas por la biblioteca predeterminada incluyen `"default"`, `"say"`, `"with"`, `"menu"`, `"prompt"`, `imagemap`, `"pause"`, `"mainmenu"` y `"gamemenu"`. La clave `"default"` siempre debe estar presente. Los valores en el diccionario deben ser listas de fotogramas que componen el cursor del ratón. Cada fotograma es una tupla (`imagen`, `desplazamiento-x`, `desplazamiento-y`). `Imagen` es el nombre de archivo de una imagen, mientras que los desplazamientos `x` e `y` son los desplazamientos del punto caliente dentro de la imagen. Los fotogramas se reproducen a 20 Hz y la animación se repite después de que se hayan mostrado todos los fotogramas. Ten en cuenta que para mostrar un fotograma más de una vez, debe repetirse en la lista de fotogramas (esto se

implementa de forma eficiente).

Variable: `config.nvl_adv_transition = None`

Una transición que se usa al mostrar texto en modo ADV directamente después de texto en modo NVL.

Variable: `config.overlay_functions = [ ]`

Una lista de funciones. Cuando se llaman, se espera que cada función devuelva una lista de elementos mostrables, que se añaden a la lista de superposiciones. Consulta la sección sobre superposiciones para más información.

Variable: `config.thumbnail_height = 75`

La altura de las miniaturas que se toman cuando se guarda la partida. Estas miniaturas se muestran cuando se carga la partida. Ten en cuenta que la miniatura se muestra en el tamaño en que se tomó, en lugar del valor de esta configuración cuando se muestra al usuario.

El valor predeterminado lo establece el diseño de carga/guardado, por lo que puede diferir de estos valores.

Variable: `config.thumbnail_width = 100`

El ancho de las miniaturas que se toman cuando se guarda la partida. Estas miniaturas se muestran cuando se carga la partida. Ten en cuenta que la miniatura se muestra en el tamaño en que se tomó, en lugar del valor de esta configuración cuando se muestra al usuario.

El valor predeterminado lo establece el diseño de carga/guardado, por lo que puede diferir de estos valores.

Variable: `config.window_hide_transition = None`

La transición utilizada por la declaración `window hide` cuando no se ha especificado explícitamente una transición.

Variable: `config.window_overlay_functions = [ ]`

Una lista de funciones de superposición que solo se llaman cuando la ventana está visible (nuevo en 6.10.0).

Variable: `config.window_show_transition = None`

La transición utilizada por la declaración `window show` cuando no se ha especificado explícitamente una transición.

De uso poco frecuente o interno

Variable: `config.afm_bonus = 25`

El número de caracteres de bonificación añadidos a cada cadena cuando el modo de avance automático está activo.

Variable: `config.afm_callback = None`

Si no es `None`, una función Python que se llama para determinar si es seguro avanzar automáticamente. La intención es que esto pueda ser utilizado por un sistema de voz para deshabilitar el avance automático cuando se está reproduciendo una voz.

Variable: `config.afm_characters = 250`

El número de caracteres en una cadena que se necesita para que se retrase la cantidad de tiempo especificada en la preferencia del modo de avance automático antes de que el modo de avance automático surta efecto.

Variable: `config.all_character_callbacks = [ ]`

Una lista de funciones de llamada de retorno que son llamadas por todos los personajes. Esta lista se antepone a la lista de funciones de llamada de retorno específicas de cada personaje.

Variable: `config.allow_skipping = True`

Si se establece en `False`, el usuario no puede saltar el texto del juego.

Variable: `config.args = [ ]`

Una lista de argumentos de línea de comandos proporcionados al juego. Estos tienen el prefijo `-arg` en la línea de comandos que lanza Ren'Py.

Variable: `config.auto_choice_delay = None`

Si no es `None`, esta variable proporciona un número de segundos que Ren'Py pausará en un menú dentro del juego antes de elegir una opción aleatoria de ese menú. Esperamos que esta variable siempre se establezca en `None` en juegos lanzados, pero establecerla en un número permitirá demostraciones automatizadas de juegos sin mucha interacción humana.

Variable: `config.autosave_frequency = 200`

Aproximadamente, el número de interacciones que ocurrirán antes de que se realice un autoguardado. Para deshabilitar el autoguardado, establece `config.has_autosave` en `False`, no cambies esta variable.

Variable: `config.character_callback = None`

El valor predeterminado del parámetro `callback` de `Character`.

Variable: `config.clear_layers = [ ]`

Una lista de nombres de capas que se limpian al entrar en los menús principal y del juego (nuevo en 6.7.0).

Variable: `config.editor = None`

Si no es `None`, se espera que sea una línea de comandos para un editor que se invoca cuando se presiona la tecla `launch_editor` (normalmente Mayús-E). Las siguientes sustituciones tienen sentido aquí:

- `%(filename)s` - El nombre de archivo del archivo más interesante para editar. Este es el archivo que debe mostrarse al usuario.
- `%(line)d` - El número de línea del archivo más interesante para mostrar al usuario.
- `%(otherfiles)s` - Otros archivos menos interesantes para mostrar al usuario.
- `%(allfiles)s` - Todos los archivos.

`filename`, `otherfiles` y `allfiles` tienen caracteres relevantes para el shell escapados con barras invertidas. `otherfiles` y `allfiles` separan archivos con `config.editor_file_separator` (por defecto `' '`, una comilla doble, un espacio y una comilla). Dado que todos los nombres de archivo deben estar entre comillas dobles, esto significa que `otherfiles` y `allfiles` crearán varios archivos entre comillas. Si aparecen dos comillas dobles seguidas en la cadena, ambas se eliminan (esto permite usar un `allfiles` vacío) (cambiado en 6.7.1).

Un ejemplo razonable es:

```
init python:
```

```
    config.editor = 'myeditor "%(filename)s" +line:%(line)d "%(otherfiles)s"'
```

Esto toma como valor predeterminado el valor de la variable de entorno `RENPY_EDITOR`. Si no está definido por esa variable o por el código del usuario, el lanzador de Ren'Py lo establece automáticamente.

Variable: `config.editor_file_separator = ' '`

El separador utilizado entre nombres de archivo cuando se proporcionan listas de archivos al editor.

Variable: `config.editor_transient = None`

Si no es `None`, se espera que sea una línea de comandos para un editor que se invoca en archivos transitorios, como resultados de `lint`, errores de análisis y trazas de pila. Las sustituciones son como para `config.editor`.

Esto toma como valor predeterminado el valor de la variable de entorno `RENPY_EDITOR_TRANSIENT`. Si no está definido por esa variable o por el código del usuario, el lanzador de Ren'Py lo establece automáticamente.

Variable: `config.fade_music = 0.0`

Esta es la cantidad de tiempo en segundos que se dedica a desvanecer la pista antigua antes de que comience una nueva pista de música. Probablemente debería ser bastante corta, para que la música incorrecta no se reproduzca durante demasiado tiempo.

Variable: `config.fast_skipping = False`

Establece esto en `True` para permitir el salto rápido fuera del modo desarrollador.

Variable: `config.file_open_callback = None`

Si no es `None`, es una función que se llama con el nombre del archivo cuando se necesita abrir un archivo. Debe devolver un objeto similar a un archivo, o `None` para cargar el archivo usando los mecanismos habituales de Ren'Py. Tu objeto similar a un archivo debe implementar al menos los métodos `read`, `seek`, `tell` y `close` (nuevo en 6.10.0).

Variable: `config.focus_crossrange_penalty = 1024`

Esta es la cantidad de penalización que se aplica a los movimientos perpendiculares a la dirección de movimiento seleccionada, cuando se mueve el foco con el teclado.

Variable: `config.hard_rollback_limit = 100`

Este es el número de pasos que Ren'Py permitirá al usuario retroceder de forma interactiva. Establece esto en 0 para deshabilitar el retroceso por completo, aunque no lo recomendamos, ya que el retroceso es útil para permitir al usuario ver el texto que omitió por error.

Variable: `config.hide = renpy.hide`

Una función que se llama cuando se ejecuta la declaración `hide`. Debe tomar los mismos argumentos que `renpy.hide`.

Variable: `config.implicit_with_none = True`

Si es `True`, por defecto se realizará el equivalente a una declaración `with None` después de las interacciones causadas por el diálogo (a través del objeto `Character`), los menús (y `renpy.display_menu`), `renpy.input` y `renpy.imagemap`. Todas las funciones anteriores toman un parámetro `with_none`, que anulará esta variable.

Variable: `config.interact_callbacks = ...`

Una lista de funciones que se llaman (sin argumentos) cuando se inicia o reinicia una interacción.

Variable: `config.joystick = True`

Gobierna si el soporte para joystick está habilitado. Si es `False`, la detección de joystick está deshabilitada.

Variable: `config.keymap = dict(...)`

Esta variable contiene un mapa de teclas que asigna las teclas y botones del ratón a cada operación posible. Consulta la sección sobre Mapas de teclas para más información.

Variable: `config.label_callback = None`

Si no es `None`, es una función que se llama cada vez que se alcanza una etiqueta. Se llama con dos parámetros. El primero es el nombre de la etiqueta. El segundo es `true` si la etiqueta se alcanzó mediante un salto, una llamada o la creación de un nuevo contexto, y `false` en caso contrario (nuevo en 6.9.0).

Variable: `config.label_overrides = { }`

Esta variable proporciona una forma de hacer que los saltos y llamadas a etiquetas en el código Ren'Py se redirijan a otras etiquetas. Por ejemplo, si añades una asignación de "start" a "mystart", todos los saltos y llamadas a "start" irán a "mystart" en su lugar.

Variable: `config.layer_clipping = { }`

Controla el recorte de capas. Es un mapa de nombres de capas a tuplas (`x`, `y`, `height`, `width`), donde `x` e `y` son las coordenadas de la esquina superior izquierda de la capa, con `height` y `width` dando el tamaño de la capa.

Si una capa no se menciona en `config.layer_clipping`, se asume que ocupa toda la pantalla.

Variable: `config.layers = [ 'master', 'transient', 'overlay' ]`

Esta variable proporciona una lista de todas las capas que Ren'Py conoce, en el orden en que se mostrarán en la pantalla (la capa más baja es la primera entrada en la lista). Ren'Py usa las capas "master", "transient" y "overlay" internamente, por lo que siempre deben estar en esta lista.

Variable: `config.lint_hooks = ...`

Una lista de funciones que se llaman, sin argumentos, cuando se ejecuta `lint`. Se espera que las funciones comprueben los datos del guión en busca de errores e impriman cualquier error que encuentren en la salida estándar (usar la declaración `print` de Python está bien en este caso).

Variable: `config.load_before_transition = True`

Si es `True`, el inicio de una interacción se retrasará hasta que se hayan cargado todas las imágenes utilizadas por esa interacción (sí, es un nombre poco apropiado).

Variable: `config.log = None`

Si no es `None`, se espera que sea un nombre de archivo. Gran parte del texto mostrado al usuario por las declaraciones `say` o `menu` se registrará en este archivo.

Variable: `config.missing_image_callback = None`

Si no es `None`, esta función se llama cuando falla un intento de cargar una imagen. Puede devolver `None` o un manipulador de imágenes. Si se devuelve un manipulador de imágenes, ese manipulador de imágenes se carga en lugar de la imagen faltante.

Variable: `config.mouse_hide_time = 30`

El ratón se oculta después de que hayan transcurrido este número de segundos sin ninguna entrada del ratón. Esto debe establecerse en un tiempo mayor que el tiempo esperado para leer una sola pantalla, para que los usuarios de ratón no experimenten que el ratón aparece y desaparece entre clics.

Variable: `config.overlay_during_with = True`

`True` si queremos que las superposiciones se muestren durante las declaraciones `with`, o `False` si preferimos que se oculten durante las declaraciones `with`.

Variable: `config.overlay_layers = [ 'overlay' ]`

Esta es una lista de todas las capas de superposición. Las capas de superposición se limpian antes de que se llamen a las funciones de superposición. "overlay" siempre debe estar en esta lista.

Variable: `config.periodic_callback = None`

Si no es `None`, debe ser una función. La función se llama, sin argumentos, a aproximadamente 20 Hz.

Variable: `config.predict_statements = 10`

Este es el número de declaraciones, incluyendo la actual, a considerar al realizar la carga predictiva de imágenes. Se realiza una búsqueda en anchura desde la declaración actual hasta que se consideran este número de declaraciones, y cualquier imagen referenciada en esas declaraciones se carga potencialmente de forma predictiva. Establecer esto en 0 deshabilitará la carga predictiva de imágenes.

Variable: `config.profile = False`

Si se establece en `True`, se mostrará cierta información de perfil en `stdout` (donde sea que vaya).

Variable: `config.rollback_enabled = True`

¿Se debe permitir al usuario retroceder en el juego? Si se establece en `False`, el usuario no puede retroceder de forma interactiva (la función todavía funciona para cargar partidas guardadas cuando el guión cambia).

Variable: `config.rollback_length = 128`

Cuando hay más de este número de declaraciones en el registro de retroceso, Ren'Py considerará recortar el registro.

Variable: `config.say_allow_dismiss = None`

Si no es `None`, debe ser una función. La función se llama sin argumentos cuando el usuario intenta descartar una declaración `say`. Si esta función devuelve `true`, se permite el descarte; de lo contrario, se ignora.

Variable: `config.say_menu_text_filter = None`

Si no es `None`, es una función a la que se le da el texto que se encuentra en las cadenas de las declaraciones `say` y `menu`. Se espera que devuelva cadenas nuevas (o las mismas) para reemplazarlas.

Variable: `config.say_sustain_callbacks = ...`

Una lista de funciones que se llaman, sin argumentos, antes de la segunda y siguientes interacciones causadas por una línea de diálogo con pausas. Se usa para mantener la voz durante las pausas.

Variable: `config.savedir = ...`

La ruta completa al directorio en el que se guarda la partida. Solo debe establecerse en un bloque `python early`. Consulta también `config.save_directory`, que genera el valor predeterminado para esto si no se establece durante un bloque `python early`.

Variable: `config.scene = renpy.scene`

Una función que se usa en lugar de `renpy.scene` por la declaración `scene`. Ten en cuenta que esto se usa para limpiar la pantalla, y `config.show` se usa para mostrar una nueva imagen. Debe tener la misma firma que `renpy.scene`.

Variable: `config.script_version = None`

Si no es `None`, se interpreta como una versión del guión. La biblioteca usará esta versión del guión para habilitar algunas características de compatibilidad, si es necesario. Si es `None`, asumimos que es un guión de la última versión.

Normalmente, esto se establece en un archivo añadido por el lanzador de Ren'Py cuando se construyen las distribuciones.

Variable: `config.searchpath = [ 'common', 'game' ]`

Una lista de directorios que se buscan para imágenes, música, archivos y otros medios, pero no para guiones. Se inicializa con una lista que contiene `common` y el nombre del directorio del juego, que cambia según el nombre del archivo ejecutable. Esta variable no se usa para cargar guiones, ya que los guiones se cargarán antes de que pueda establecerse.

Variable: `config.show = renpy.show`

Una función que se usa en lugar de `renpy.show` por las declaraciones `show` y `scene`. Debe tener la misma firma que `renpy.show`.

Variable: `config.skip_delay = 75`

La cantidad de tiempo que se mostrará el diálogo, cuando se saltan declaraciones usando `Ctrl`, en milisegundos (aunque en la práctica no es ni mucho menos tan preciso).

Variable: `config.skip_indicator = True`

Si es `True`, la biblioteca mostrará un indicador de salto cuando se esté saltando el guión.

Variable: `config.sound = True`

Si es `True`, el sonido funciona. Si es `False`, el subsistema de sonido/mezclador está completamente deshabilitado.

Variable: `config.sound_sample_rate = 44100`

La frecuencia de muestreo a la que se ejecutará la tarjeta de sonido. Si todos tus archivos WAV son de una frecuencia más baja, cambiarla a esa frecuencia puede hacer las cosas más eficientes.

Variable: `config.start_interact_callbacks = ...`

Una lista de funciones que se llaman (sin argumentos) cuando se inicia una interacción. Estas funciones de llamada de retorno no se llaman cuando se reinicia una interacción.

Variable: `config.sticky_positions = False`

Las posiciones adhesivas hacen que la cláusula `at` de una declaración `show` o `scene` se recuerde mientras ese personaje esté en la pantalla. Las futuras declaraciones `show` sin cláusulas `at` recibirán la cláusula `at` recordada. Esto permite cambiar fácilmente la expresión del personaje sin cambiar su posición en la pantalla. Las posiciones adhesivas no interactúan bien con múltiples capas cuando se usan con las declaraciones `show`, `hide` y `scene` mediante una cláusula `onlayer`.

Variable: `config.top_layers = [ ]`

Esta es una lista de nombres de capas que se muestran encima de todas las demás capas y no participan en una transición que se aplica a todas las capas. Si un nombre de capa se lista aquí, no debe listarse en `config.layers`.

Variable: `config.transient_layers = [ 'transient' ]`

Esta variable proporciona una lista de todas las capas transitorias. Las capas transitorias son capas que se limpian después de cada interacción. `"transient"` siempre debe estar en esta lista.

Variable: `config.with_callback = None`

Si no es `None`, debe ser una función que se llama cuando ocurre una declaración `with`. Esta función puede ser responsable de mostrar cosas transitorias en la pantalla durante la transición. La función se llama con un solo argumento, que es la transición que está ocurriendo. Se espera que devuelva una transición, que puede ser o no la transición proporcionada como argumento.

Variables del store

Esta es una lista de variables especiales del store que controlan aspectos del juego. Son variables que se colocan en el store (el ámbito predeterminado del código Python) y, por lo tanto, se pueden actualizar durante el transcurso del juego. Debes tener cuidado de asegurar que los valores de estas variables se puedan serializar con `pickle` (por ejemplo, se debe crear una función con un nombre diferente dentro de un bloque `init` y luego asignarla a una de estas variables).

Lista de variables

Variable: `adv = ...`

Esta es la plantilla de personaje en modo ADV y el personaje predeterminado que se usa cuando estamos en modo ADV.

Variable: `default_transition = None`

Si no es `None`, una transición predeterminada que se usa cuando no se especifica ninguna transición. No uses esto, es realmente molesto.

Variable: `menu = renpy.display_menu`

La función que se llama para mostrar el menú. Debe tomar los mismos argumentos que `renpy.display_menu`. Asignar `nvl_menu` a esto se hace a menudo en el modo NVL.

Variable: `mouse_visible = True`

Controla si el ratón es visible. Se establece automáticamente en `true` al entrar en los menús del juego estándar.

Variable: `name_only = ...`

Este es el personaje que se usa como base para el diálogo hablado por personajes con nombres dados como una simple cadena. Lo usa la función `say`.

Variable: `narrator = ...`

Este es el personaje que dirá la narración (es decir, declaraciones `say` que consisten solo en una sola cadena).

Variable: `nvl = ...`

Este es el tipo de personaje predeterminado para los personajes en modo NVL.

Variable: `predict_menu = ...`

Se llama sin argumentos y se espera que devuelva una lista de elementos mostrables utilizados por la función `menu`.

Variable: `predict_say = ...`

Se usa para predecir las imágenes utilizadas por `say`. Se llama con los mismos dos argumentos y se espera que devuelva una lista de elementos mostrables utilizados.

Variable: `_rollback = True`

Controla si se permite el retroceso.

Variable: `say = ...`

Una función que se llama cuando la declaración `say` se llama con una cadena para un nombre. El comportamiento predeterminado de esta función crea una copia del objeto `name_only` con el nombre proporcionado y luego lo usa para mostrar el diálogo. La función toma dos argumentos, el nombre del personaje y la línea de diálogo.

Variable: `save_name =`

Este es un nombre que se usa cuando se guardan los datos.

Variable: `suppress_overlay = False`

Cuando es `true`, no se llamará a las funciones de superposición y la superposición no se mostrará. El valor predeterminado es `true`, pero se establece en `false` al entrar en los menús fuera del juego.

Variable: `_window = False`

Controla si la ventana se muestra automáticamente cuando un personaje no está hablando. Normalmente, esto lo establecen las declaraciones `window show` y `window hide`.

Variable: `_window_subtitle =`

Se añade a `config.window_title` para producir el título de la ventana del juego. Se establece automáticamente en `config.menu_window_subtitle` al entrar en el menú del juego (nuevo en 6.7.1).

Las variables y funciones específicas de la personalización de menús se documentan en *Personalizar los menús principal y del juego*.

Superposiciones (Overlays)

Las superposiciones se utilizan para mostrar información por encima de la escena que se muestra actualmente. La superposición se regenera cada vez que comienza una interacción con el usuario, lo que la hace adecuada para mostrar al usuario cosas como estadísticas o fechas. La superposición se muestra generalmente cuando se muestran cosas transitorias (como diálogos, pensamientos y menús).

Las superposiciones se configuran añadiendo a la lista `config.overlay_functions` una función Python que, cuando se llama, utiliza las funciones de UI para añadir widgets a la pantalla. Por defecto, estos widgets se añaden a la capa `'overlay'`, pero una llamada a `ui.layer` puede cambiar la capa a cualquiera de las capas listadas en `config.overlay_layers`. Estas funciones se llaman para cada interacción, lo que permite que la superposición cambie para reflejar el estado de las variables del juego. Si una variable que afecta a la superposición cambia durante una interacción, se debe llamar a `renpy.restart_interaction` para regenerar la superposición.

Como ejemplo, considera el siguiente fragmento de código. Cuando se añade a un programa, muestra una imagen de fecha en la esquina superior derecha de la pantalla (como se hace en Kanon). La imagen mostrada se basa en la variable `date`. Si `date` es `None`, no se muestra ninguna fecha. De lo contrario, se muestra un archivo PNG que comienza con el valor de `date`.

```
init:
    $ date = "mar25"
    python hide:
        def date_overlay():
            if date:
                ui.image(date + ".png",
                        xpos=1.0, xanchor="right",
                        ypos=0.0, yanchor="top")
        config.overlay_functions.append(date_overlay)
```

Como todas las variables de configuración, `config.overlay_functions` solo debe cambiarse en un bloque `init`. Si necesitas activar y desactivar una superposición, la función de superposición debe estar condicionada por alguna variable normal. Esto se hace en el ejemplo anterior cuando `date` es `None`.

Guardado, carga y retroceso

Ren'Py admite guardar una partida en disco, cargarla de vuelta desde el disco y retroceder la partida a un estado anterior. Como estas acciones comparten infraestructura, las discutiremos todas en esta sección, prestando atención a lo que se requiere para admitir la carga de una partida guardada cuando el guión cambia entre el guardado y la carga.

Ten en cuenta que la compatibilidad de las partidas guardadas no está garantizada entre versiones de Ren'Py, incluso entre versiones antiguas y nuevas de Ren'Py. Puede ser necesario tener esto en cuenta al lanzar un juego con una versión actualizada de Ren'Py.

El guardado, la carga y el retroceso pueden verse todos como operaciones que afectan al estado del juego. Guardar es una operación que persiste el estado del juego en el disco; cargar consiste en restaurar el estado del juego desde el disco; y retroceder revierte el estado del juego a lo que era en un punto anterior en el tiempo. Este estado del juego consta de dos partes: el estado del usuario consiste en variables y objetos que han sido cambiados por el usuario, mientras que el estado interno consiste en información utilizada directamente por Ren'Py.

Estado interno

Específicamente, el estado interno de Ren'Py consta de lo siguiente:

- El nombre de la declaración que se está ejecutando actualmente.
- El nombre de cada declaración en la pila de retorno.
- Las capas que contienen imágenes que se muestran en la pantalla.
- La música que se ha solicitado a Ren'Py que reproduzca.
- El tiempo que se ha jugado.

Estos elementos se guardan, cargan y retroceden automáticamente cuando el usuario invoca los comandos apropiados. No necesitas hacer nada para que este estado se maneje, y no hay forma de evitar que estas cosas se guarden.

Estado del usuario

El otro tipo de estado que Ren'Py puede guardar y restaurar es el estado del usuario. El estado del usuario consiste en todas las variables que se han cambiado después del final de la fase `init`, y todos los objetos transitivamente accesibles desde tales variables.

Es importante aclarar lo que significa que una variable sea cambiada. En Python y Ren'Py, las variables hacen referencia a objetos. Una variable cambia cuando se actualiza para hacer referencia a un nuevo objeto. Una variable no cambia cuando el objeto al que hace referencia cambia. Por ejemplo, en el siguiente código:

```
init:
    $ a = 0
    $ state = object()

$ a = 1
$ b = [ ]
$ state.love_love_points = a + 1
```

En este ejemplo, las variables `a` y `b` son cambiadas por el código fuera del bloque `init` (asumiendo que ese código se ejecuta), mientras que `state` no es cambiada. En el código fuera del bloque `init`, `a` se le asigna un objeto entero diferente, mientras que `b` se le asigna una nueva lista vacía. Aunque un campo del objeto al que hace referencia `state` ha cambiado, `state` en sí mismo sigue haciendo referencia al mismo objeto y no se considera que haya cambiado (por lo tanto, `state` y su objeto asociado no se consideran parte del estado del usuario).

El estado del usuario se recopila encontrando primero todas las variables que han cambiado fuera de los bloques `init`. Luego, Ren'Py encuentra todos los objetos accesibles desde una de esas variables a través de alguna combinación de acceso a campos, iteración o iteración sobre elementos (como en un diccionario). Esta combinación de variables y valores de objetos constituye el estado del usuario.

Es importante asegurar que cada objeto en el estado del usuario se pueda serializar (pickle) mediante el módulo `pickle` de Python. La mayoría de las construcciones de Python se pueden serializar, incluyendo booleanos, enteros, números de punto flotante, cadenas, listas, tuplas, diccionarios y la mayoría de los objetos. También puedes hacer referencia a tus propias clases y funciones, siempre que estén definidas en un bloque Python (no un bloque `python hide`) dentro de un bloque `init` y siempre existan con el mismo nombre en versiones posteriores del guión. Hay algunos objetos de Python que no se pueden serializar, como archivos, iteradores y generadores. Estos objetos no deben usarse fuera de bloques `python hide`.

Aunque estas reglas pueden parecer complejas, se espera que en la práctica se puedan reducir a una heurística simple: cualquier variable cambiada fuera de un bloque `init`, y cualquier objeto accesible desde dicha variable, se guardará, cargará y retrocederá correctamente.

Estado que no se guarda

Hay algo de estado mantenido por Ren'Py que no es parte del estado del intérprete. Esto incluye:

- Asignaciones de nombres de imagen a elementos mostrables, creadas por la declaración `image`.
- Variables de configuración (`config.varname`).
- Estilos (`style.stylename`).

Como resultado, las variables de configuración y los estilos deben configurarse en bloques `init` y luego dejarse intactos durante el resto del juego. Como la declaración `image` solo se puede ejecutar desde dentro de un bloque `init`, es imposible configurar una asignación de nombre de imagen a elemento mostrable fuera de un bloque de imagen. Para asegurar la compatibilidad entre versiones del guión, una vez que un nombre de imagen o estilo está presente en una versión lanzada del juego, debe estar presente en todas las versiones futuras lanzadas del juego.

Detalles

A medida que se juega, Ren'Py registra todos los cambios en el estado del usuario y del intérprete. Cuando se guarda la partida, escribe este registro en disco, junto con el estado actual. Cuando se carga la partida, las variables se restablecen a lo que eran cuando el código `init` en la versión actual del guión terminó de ejecutarse. Luego, el estado del usuario guardado se fusiona con esto, sobrescribiendo cualquier variable que también se asignó en el código `init`. Finalmente, se activa un retroceso.

El retroceso que se activa en la carga termina cuando puede encontrar una declaración que tenga el mismo nombre que tenía cuando se encontró en el registro. Cuando el guión no ha cambiado, todas las declaraciones tienen el mismo nombre, por lo que el efecto

del retroceso es llevar el control de vuelta al inicio de la declaración que se estaba ejecutando cuando el usuario guardó. Sin embargo, cuando el guión ha cambiado, las únicas declaraciones que conservan sus nombres son aquellas que tienen un nombre explícito especificado (estas declaraciones son etiquetas, declaraciones `menu` con un nombre explícito y declaraciones `call ... from ...`). El juego retrocederá al inicio de la declaración más reciente que exista tanto en el juego antiguo como en el nuevo.

Cuando ocurre un retroceso, tanto el estado del usuario como el del intérprete se restauran a lo que eran cuando la declaración a la que se está retrocediendo comenzó a ejecutarse. Luego, la declaración se ejecuta de nuevo y el juego continúa normalmente.

Ten en cuenta que solo podemos retroceder la declaración actual, y no los sitios de retorno listados en la pila de retorno. Si el nombre de un sitio de retorno cambia, no podremos regresar de una llamada a procedimiento y el guión fallará. Sin embargo, si un sitio de retorno tiene un nombre explícito, se regresa a ese nombre incluso si el guión cambia. Por esta razón, es importante que cada sitio de llamada en un juego lanzado tenga una cláusula `from` asociada, que proporcione un nombre para el sitio de retorno que pueda aparecer en ambos guiones.

Finalmente, si está permitido, el usuario puede invocar explícitamente el retroceso. Cuando ocurre tal retroceso, primero buscamos una declaración anterior que sea un punto de control (los puntos de control son declaraciones `say` y `menu`, así como bloques Python que llamaron a `renpy.checkpoint`). Una vez que se encuentra un punto de control, buscamos una declaración que tenga un nombre que exista en el guión actual (normalmente es la misma declaración). Luego retrocedemos a esa declaración y comenzamos a ejecutar de nuevo. Esto significa que cuando ocurre un retroceso, el juego suele revertirse al inicio de la declaración `say` o `menu` que se ejecutó antes de la declaración actualmente en ejecución. Hay una variable que controla el comportamiento de carga y guardado:

Variable: `save_name = ...`

Este es un nombre que se asociará con los archivos de guardado. Se espera que el juego actualice esto regularmente con el nombre de la sección del guión que se está ejecutando actualmente. Cuando se muestra un archivo de guardado al usuario, este nombre se mostrará con él. Alternativamente, nunca cambies esto y no se mostrará ningún nombre de guardado.

Ejecutar código después de una carga

Si existe la etiqueta `after_load`, se llama después de que haya ocurrido una carga. Es responsable de actualizar las variables que pueden haber cambiado entre versiones del guión y de realizar cualquier otra acción que pueda ser necesaria en respuesta a la carga. Cuando se ejecuta una declaración `return`, la ejecución se transfiere al punto donde se guardó la partida.

Ten en cuenta que al llamar a `after_load`, la pila de retorno no es segura, por lo que no se debe permitir al usuario guardar antes de que el control regrese de `after_load`. Para estar seguros, esto significa que el código `after_load` no debe ser interactivo.

Funciones de carga/guardado

Las siguientes funciones son útiles para el código que quiere gestionar la carga y el guardado por sí mismo.

Function: `renpy.can_load (filename):`

Devuelve `true` si se puede cargar `filename` (mediante el mecanismo de carga/guardado), o `false` en caso contrario. Esta es una comprobación muy básica, que principalmente asegura que el archivo existe.

Function: `renpy.list_saved_games (regex=r'.')`:

Escanea los juegos guardados que conocemos y devuelve información sobre ellos. Devuelve una lista de tuplas, donde cada tupla representa un juego guardado y consiste en:

- El nombre de archivo del guardado.
- La `extra_info` que se pasó a `renpy.save`.
- Un elemento mostrable, la captura de pantalla utilizada para mostrar el juego.
- La hora en que se guardó el juego, en segundos desde 1/1/1970 UTC.

La expresión regular coincide al inicio del nombre de archivo y filtra la lista.

Function: `renpy.load (filename)`:

Hace que el mecanismo de carga/guardado cargue `filename`. Esta función nunca retorna.

Function: `renpy.rename_save (old, new)`:

Renombra el guardado `old` a `new`, sobrescribiendo cualquier guardado existente llamado `new`.

Function: `renpy.save (filename, extra_info=)`:

Guarda la partida actual en `filename`.

- `filename` - Una cadena que da el nombre de archivo en el que guardar. Ten en cuenta que esto no corresponde a ningún archivo en particular en el disco, sino que debe tratarse como un identificador de juego arbitrario. Las ranuras de guardado normales son representaciones en base 10 de enteros, mientras que otros nombres no aparecerán en el selector de archivos.
- `extra_info` - Información adicional guardada con la partida. Normalmente es el valor de `save_name`.

Se debe llamar a `renpy.take_screenshot` antes de `renpy.save`, excepto cuando se está dentro del menú del juego, donde se ha llamado automáticamente.

Function: `renpy.scan_saved_game (name)`:

Escanea un único juego guardado y devuelve información sobre él. Si no existe un juego guardado con el nombre de archivo `name`, esta función devuelve `None`. De lo contrario, devuelve un triple que consiste en:

- La `extra_info` que se pasó a `renpy.save`.
- Un elemento mostrable, la captura de pantalla utilizada para mostrar el juego.
- La hora en que se guardó el juego, en segundos desde 1/1/1970 UTC.

Function: `renpy.take_screenshot (size=None)`:

Esta función debe llamarse antes de `renpy.save` para tomar la captura de pantalla que se utilizará como parte de un juego guardado. Se llama automáticamente al entrar en el menú del juego, pero puede tener sentido llamarla manualmente si fueras un guardado.

- `size` - El tamaño de la captura de pantalla. Si es `None`, se usa el valor predeterminado de `(config.thumbnail_width, config.thumbnail_height)`.

Function: `renpy.unlink_save (filename):`

Elimina (borra) el archivo de guardado con el nombre dado. Lanza una excepción si el archivo no se puede eliminar.

Datos persistentes

Ren'Py también admite datos persistentes, que son datos guardados que no están asociados con un punto particular en un juego. Los datos persistentes son datos a los que se accede a través de los campos del objeto `persistent`, que está vinculado a la variable `persistent`.

Variable: `persistent = ...`

La variable `persistent` está vinculada al objeto persistente especial. Todos los datos accesibles a través de los campos de `persistent` se guardan cuando Ren'Py termina y se cargan cuando Ren'Py se reanuda. El objeto `persistent` es especial en que el acceso a un campo indefinido en él tendrá un valor `None`, en lugar de causar una excepción.

Un ejemplo de uso de `persistent` es la creación de una galería de imágenes desbloqueable. Esto se hace almacenando una bandera en `persistent` que determina si la galería se ha desbloqueado, como en:

```
label gallery:
    if not persistent.gallery_unlocked:
        show background
        centered "Aún no has desbloqueado esta galería."
        $ renpy.full_restart()
    # Mostrar la galería aquí.
```

Cuando el usuario obtiene un final que hace que la galería se desbloquee, la bandera debe establecerse en `True`:

```
$ persistent.gallery_unlocked = True
```

Persistencia multijuego

La persistencia multijuego es una característica que permite compartir información entre juegos de Ren'Py. Esto puede ser útil si planeas hacer una serie de juegos y quieres que compartan información.

Para usar datos multipersistentes, se debe crear un objeto `MultiPersistent` dentro de un bloque `init`. El usuario puede entonces actualizar este objeto y guardarlo en disco llamando a su método `save`. Los campos indefinidos toman por defecto el valor `None`. Para asegurar que el objeto se pueda cargar de nuevo, sugerimos no asignar instancias de objetos de tipos definidos por el usuario.

Function: `MultiPersistent (key):`

Crea un nuevo objeto `MultiPersistent`. Solo debe llamarse dentro de un bloque `init` y devuelve un nuevo objeto `MultiPersistent` correspondiente a `key`.

- **key** - Una clave que se utiliza para acceder a los datos multipersistentes. Para evitar conflictos, sugerimos que la clave tenga como sufijo un dominio que poseas... por lo que si posees `renpy.org`, podrías usar la clave `"demo.renpy.org"`.

Method: `MultiPersistent.save ():`

Guarda un objeto `MultiPersistent` en disco. Los cambios no se persisten hasta que se llama a este método.

Ejemplo. Parte 1 de un juego en varias partes:

```
init:
    $ mp = MultiPersistent("demo.renpy.org")

label start:
    # ...
    # Registrar el hecho de que el usuario completó la parte 1.
    $ mp.beat_part_1 = True
    $ mp.save()
    e "¡Completaste la parte 1. ¡Nos vemos en la parte 2!"
```

Y el código para la parte 2:

```
init:
    $ mp = MultiPersistent("demo.renpy.org")

label start:
    if mp.beat_part_1:
        e "Veo que has completado la parte 1, ¡así que bienvenido de nuevo!"
    else:
        e "Hmm, no has jugado la parte 1, ¿por qué no pruebas primero?"
```

Ubicación de los datos. El lugar donde se almacenan los datos varía según el sistema operativo que se utilice:

- Windows: %APPDATA%/RenPy/persistent
- Mac OS X: /Library/RenPy/persistent
- Linux/Otros: /.renpy/persistent

Ofuscación de tu juego

Por alguna razón, muchas personas parecen querer distribuir sus juegos en una forma ofuscada, dificultando la lectura del guión o la visualización de las imágenes asociadas con un juego. Ren'Py admite esto, hasta cierto punto, compilando guiones y permitiendo archivar imágenes. Aunque un programador con habilidades moderadas podría descifrar fácilmente esta ofuscación, debería ser suficiente para proteger un juego de una navegación casual.

La ofuscación del guión es prácticamente automática. Cada vez que el juego se ejecuta, cualquier archivo `.rpy` en el directorio del juego se escribe como un archivo `.rpyc`. Estos archivos `.rpyc` son suficientes para ejecutar el juego, por lo que simplemente ejecutar el juego una vez (para crear los archivos `.rpyc`) y luego eliminar (o, más apropiadamente, mover) los archivos `.rpy` te dejará con un juego ejecutable con un guión ofuscado. Como beneficio adicional, los archivos `.rpyc` ya están analizados, lo que mejora el tiempo de carga del juego (si un directorio contiene archivos `.rpy` y `.rpyc` con el mismo nombre base, se elige el más nuevo, lo que significa que todos los juegos obtienen esta mejora de rendimiento).

Las imágenes se pueden archivar usando el lanzador de Ren'Py. Solo elige *Archivar imágenes* en el menú *Herramientas*, y Ren'Py creará un archivo que contiene tus imágenes. También añadirá un archivo de guión que contiene código que le dice a Ren'Py dónde buscar las imágenes archivadas.

Al construir distribuciones, puedes usar la opción de excluir archivos para excluir los archivos `.rpy`.

Localización de Ren'Py

Aunque Ren'Py está configurado por defecto para funcionar en un entorno de habla inglesa, no se limita a tales configuraciones. Suponiendo que se cargue una fuente adecuada, los guiones de Ren'Py pueden contener cualquier idioma expresable en Unicode.

Hay dos cosas en la biblioteca de Ren'Py que pueden necesitar ser traducidas al idioma del usuario. La primera es el menú principal. No hay soporte explícito para hacer esto, pero como la variable `config.main_menu` admite cambiar el texto del menú principal, también admite traducir dicho texto.

La segunda cosa que necesita ser traducida es el menú del juego. El diccionario `config.translations` se utiliza para traducir el texto del menú del juego a tu idioma. Si una clave en este mapa corresponde al texto en inglés que se mostraría, se muestra el valor correspondiente a esa clave en su lugar. Por ejemplo:

```
init:
    $ config.translations = {
        "Yes" : u"HIja'",
        "No"  : u"ghobe'",
        # etc.
    }
```

Propiedades y estilos

Ren'Py contiene un sistema de estilos, que se utiliza para determinar la apariencia de los elementos mostrables. Este sistema permite personalizar en gran medida la apariencia de los juegos de Ren'Py.

Cualquier función que tome un parámetro ****properties** puede recibir las propiedades descritas a continuación, incluyendo propiedades con prefijo, pero no todas las propiedades son entendidas por todas las funciones. Estas funciones también pueden recibir un parámetro `style`. Se puede especificar un estilo proporcionando directamente el objeto de estilo, o proporcionando una cadena que dé el nombre del estilo.

```
$ ui.text("Hello, World", xalign=0.5, yalign=0.5, size=42, style='default')
$ ui.text('How are you doing?', style=style.centered_text)
$ ui.saybehavior()
$ ui.interact()
```

Estilos

Cada elemento mostrable tiene un estilo predeterminado que se usa cuando no se proporciona ningún estilo explícitamente. A los elementos mostrables utilizados por Ren'Py a

menudo se les asignan estilos únicos, lo que permite personalizar su apariencia utilizando el sistema de estilos.

Se puede acceder a un estilo como un campo en el objeto **style**. Las propiedades con prefijo son campos en los estilos, que permiten asignarles valores. Los estilos solo deben crearse y modificarse dentro de bloques **init**.

```
init:
    $ style.button_text.size = 22
    $ style.button_text.hover_size = 28
```

Se pueden crear nuevos estilos utilizando la función **Style**. Estos estilos deben asignarse a uno o más campos en el objeto **style** en el momento de la creación.

Function: Style (parent):

Construye un nuevo estilo, con el estilo dado como su padre. Se espera que el resultado de esta llamada se asigne a un campo del objeto **style**.

Por compatibilidad, los estilos también se pueden crear utilizando la función **style.create**.

Function: style.create (name, parent):

Crea un nuevo estilo.

- **name** - El nombre del nuevo estilo, como una cadena.
- **parent** - El padre del nuevo estilo, como una cadena. Es 'default' o algo más específico.

Function: style.rebuild ():

Hace que todos los estilos se reconstruyan, lo que permite cambiar estilos fuera del código **init**. Hay varios inconvenientes:

- Los cambios en los estilos no se guardan.
- Los cambios en los estilos persisten a través de un reinicio del juego.
- Los cambios en los estilos indexados se ignorarán a menos que el estilo indexado se haya definido primero en un bloque **init**.

Métodos de estilo

Los objetos de estilo tienen métodos que pueden ser útiles al personalizar estilos.

Method: Style.clear ():

Elimina todas las personalizaciones de propiedades del estilo en el que se llama.

Method: Style.take (other):

Espera que **other** sea un objeto de estilo. Toma personalizaciones de estilo de **other** y las aplica a este estilo.

Method: Style.set_parent (parent):

Cambia el padre de este estilo a **parent**.

Estilos indexados

Se accede a los estilos indexados indexando un estilo con un valor. Si un estilo indexado no existe, la indexación lo crea. Los estilos indexados son ligeros, lo que permite que haya un gran número de ellos. El padre de un estilo indexado es el estilo del que se indexó.

init python:

```
# Crea el nuevo estilo indexado button['Foo']
style.button['Foo'].background = "#f00"
style.button['Foo'].xalign = 1.0
```

label example:

python:

```
# Accede al estilo button['Foo']
ui.textbutton('Foo', style=style.button['Foo'], clicked=ui.returns('Foo'))
# Accede al estilo button['Bar'], creándolo si es necesario.
ui.textbutton('Bar', style=style.button['Bar'], clicked=ui.returns('Bar'))
ui.interact()
```

Prefijos de propiedades

El sistema de estilos nos permite especificar nuevos valores para las propiedades de un elemento mostrable dependiendo de la situación en la que se encuentre ese elemento mostrable. Hay dos componentes de la situación. El elemento mostrable puede estar seleccionado o no seleccionado, y puede estar activado, enfocado, sin enfocar o no enfocable. Un elemento mostrable está activado cuando se hace clic en el botón que lo contiene, y vuelve a un estado anterior si la función `clicked` devuelve `None`. Las propiedades con prefijo permiten establecer el valor de las propiedades para algún subconjunto de estas situaciones.

En la siguiente tabla, se proporciona una lista de prefijos y situaciones. Se indica con un * las situaciones a las que se aplica un prefijo.

prefijo	no seleccionado				seleccionado		
	activado	enfocado	sin enfocar	no enfocable	activado	enfocado	sin enfocar
(sin prefijo)	*	*	*	*	*	*	*
activate_	*				*		
hover_		*				*	
idle_			*				*
insensitive_				*			
selected_					*	*	*
selected_ activate_					*		
selected_ hover_						*	
selected_ idle_							*
selected_ insensitive_							

Resolución de conflictos

Hay dos tipos de conflictos que pueden ocurrir.

Si tenemos múltiples prefijos que asignarían a la misma propiedad en la misma situación al mismo tiempo, entonces tomamos el más específico, donde un más específico se define como más bajo en la tabla anterior. Este es el caso cuando se da más de un prefijo para una propiedad en la llamada a una función que toma propiedades como argumentos.

Cuando la asignación ocurre en momentos distintos en el tiempo, entonces el valor de la propiedad para una situación dada se determina por la última asignación realizada. Este es el caso cuando tenemos asignación a propiedades de estilo.

Para evitar conflictos, asigna valores a las propiedades con prefijo en orden de más específico a menos específico.

Estilos indexados y herencia

Los estilos se pueden indexar con una cadena o un número. Esto permite tener fácilmente estilos específicos para un botón, sin tener que definir todos los estilos con antelación. Los estilos indexados no aparecen en la jerarquía de estilos, pero sí aparecen cuando se usa el inspector de estilos.

Cuando los estilos heredan propiedades, la herencia se realiza primero a través de la forma indexada de un estilo, luego la forma no indexada de ese estilo, luego la forma indexada del padre de ese estilo, luego la forma no indexada del padre, y así sucesivamente. Específicamente, si `style.mm_button` hereda de `style.button` que hereda de `style.default`, entonces los componentes de `style.mm_button["Start Game"]` se toman de:

1. `style.mm_button["Start Game"]`
2. `style.mm_button`
3. `style.button["Start Game"]`
4. `style.button`
5. `style.default["Start Game"]`
6. `style.default`

Tomando la propiedad del estilo con el número más bajo que tenga esa propiedad definida.

Lista de estilos

Los estilos que están disponibles para personalizar varían según los diseños y temas que se estén utilizando. Para obtener un árbol completo de estilos, incluyendo una breve explicación de para qué se usa cada uno, presiona **Mayús+D** dentro de tu juego para acceder al menú de desarrollador y luego elige *Style Hierarchy* (ten en cuenta que `config.developer` debe establecerse en `True` para habilitar el acceso a la consola de desarrollador).

Lista de propiedades

Esta es una lista de propiedades, agrupadas de forma flexible según el uso de la propiedad.

Propiedades de texto

Las siguientes propiedades son utilizadas por los elementos mostrables de tipo **Text**.

- **antialias** — Si es **True**, el texto se renderizará con antialiasing. De lo contrario, se dejará con bordes dentados.
- **black_color** — Debe ser un triple RGBA. Cuando se usa un **SFont**, este es el color al que se mapeará el negro. Esta propiedad se ignora cuando se usa una fuente **TrueType**. El canal alfa en este color se ignora.
- **bold** — Si es **True**, este texto se renderizará en negrita.
- **color** — El color en el que se mostrará el texto en la pantalla, como una tupla RGBA. Cuando se usa un **SFont**, este es el color en el que se mostrará el blanco de ese **SFont**.
- **drop_shadow** — Se usa para controlar la generación de una sombra paralela en el texto. Es una tupla de 2 elementos, una lista de tuplas o **None**. Si es una tupla, los 2 elementos controlan los desplazamientos de la sombra en X e Y, respectivamente. Si es una lista, se espera que sea una lista de tuplas de 2 elementos, cada una de las cuales es la sombra. Usar una lista como [(-1, -1), (1, -1), (-1, 1), (1, 1)] crea texto contorneado en lugar de una sombra. Si es **None**, no se crea sombra para el texto.
- **drop_shadow_color** — El color de la sombra. Ejemplo: "#000".
- **first_indent** — Se usa para dar, en píxeles, la sangría de la primera línea de texto en el widget de texto. Se puede usar para sangrar la primera línea de un párrafo (no es una buena idea en un monitor de ordenador, es mejor dejar una línea en blanco entre párrafos).
- **font** — La fuente que se usará para renderizar el texto. Es el nombre de un archivo que contiene una fuente **TrueType** o el nombre de una fuente. En el primer caso, la fuente se carga desde el archivo. En el segundo, el nombre de archivo se usa para buscar **SFonts** registrados con **register_sfont** y fuentes **TrueType** que se encuentran en el directorio de fuentes del sistema.
- **size** — El tamaño de la fuente que se usa para mostrar el texto en la pantalla. Ten en cuenta que el significado de esto puede variar de una fuente a otra y solo tiene una relación débil con el número de píxeles de alto que tendrá la fuente en la pantalla.
- **italic** — Si es **True**, este texto se renderizará en cursiva.
- **justify** — Si es **True**, se inserta espacio en blanco adicional entre las palabras para que los márgenes izquierdo y derecho de cada línea sean uniformes. Esto no se realiza en la última línea de un párrafo.
- **language** — El idioma (en realidad, la familia de idiomas) que se usa para mostrar el texto. Actualmente, hay dos opciones: "**western**" para texto de estilo occidental que se divide en espacios, y **eastasian** para texto chino/japonés, que se puede dividir entre caracteres.

- **layout** — Controla cómo ocurre el salto de línea. Por defecto es "**greedy**", que intenta poner tanto como sea posible en cada línea. También puede ser "**subtitle**", que intenta que las líneas sean uniformes en longitud.
- **line_spacing** — Se usa para aumentar o disminuir el espacio entre líneas de texto en un número constante de píxeles. Un valor positivo aumenta, mientras que uno negativo disminuye el espacio.
- **minwidth** — El ancho mínimo en píxeles de este texto. Si el texto renderizado es más pequeño que esto, se rellena con espacios en blanco a la derecha hasta que tenga al menos esta longitud en píxeles.
- **outlines** — Una lista de tuplas de 2 o 4 componentes, que dan el tamaño, el color y, opcionalmente, los desplazamientos x e y de los contornos alrededor del texto. Un contorno es un bloque de texto que se ha expandido en tamaño por un número dado de píxeles. Los desplazamientos x e y se pueden omitir. La primera tupla en la lista está más lejos de la pantalla, mientras que la última está más cerca (nuevo en 6.8.0).
- **rest_indent** — Se usa para dar, en píxeles, la sangría de la segunda línea y posteriores de un widget de texto. Se puede usar para dar una sangría francesa a citas de diálogo.
- **slow_cps** — Si es un número, hace que el texto se muestre a una velocidad de esa cantidad de caracteres por segundo. Si es **True**, toma ese número de la preferencia *Velocidad de texto*.
- **slow_cps_multiplier** — Un multiplicador que se aplicará al número de caracteres mostrados por segundo. Por ejemplo, 2.0 hace que los caracteres se muestren al doble de velocidad.
- **slow_abortable** — Si es **True**, el texto lento se puede abortar haciendo clic con el ratón.
- **text_align** — Se usa para controlar la alineación horizontal de las líneas de texto en el área asignada al widget **Text** que contiene ese texto. Solo tiene efecto si el texto tiene más de una línea. Es un número entre 0 y 1, que da la fracción de espacio vacío que debe estar a la izquierda de cada línea de texto (para centrar el texto, debe ser 0.5).
- **text_y_fudge** — Ajusta la posición y de un bloque de texto añadiendo espacio en blanco encima de la primera línea.
- **underline** — Si es **true**, este texto se renderizará con subrayado.

Propiedades de ventana

Las propiedades controlan la apariencia de **Window** y **Button**.

- **background** — Un elemento mostrable que se usa como fondo de la ventana. Debe ser un elemento mostrable que siempre dibuje exactamente el tamaño que se le solicita, lo que generalmente significa un **Solid** o un **Frame**. También puede ser

`None`, lo que significa que no hay fondo en esta ventana (el resto de propiedades de ventana que se refieren a un fondo siguen funcionando; piensa en ellas como si su fondo fuera transparente).

- `foreground` — Se dibuja de la misma manera que el fondo, excepto que se dibuja encima del contenido de la ventana.
- `left_margin` — La cantidad de espacio transparente a la izquierda de esta ventana. Si es un número de punto flotante, se escala al ancho disponible.
- `right_margin` — La cantidad de espacio transparente a la derecha de esta ventana. Si es un número de punto flotante, se escala al ancho disponible.
- `top_margin` — La cantidad de espacio transparente en la parte superior de esta ventana. Si es un número de punto flotante, se escala a la altura disponible.
- `bottom_margin` — La cantidad de espacio transparente en la parte inferior de esta ventana. Si es un número de punto flotante, se escala a la altura disponible.
- `xmargin` — Una forma conveniente (y compatible con versiones anteriores) de establecer `left_margin` y `right_margin` al mismo valor.
- `ymargin` — Una forma conveniente (y compatible con versiones anteriores) de establecer `top_margin` y `bottom_margin` al mismo valor.
- `left_padding` — La cantidad de espacio entre el borde y el lado izquierdo del contenido de la ventana. Si es un flotante, se escala al ancho de la ventana.
- `right_padding` — La cantidad de espacio entre el borde y el lado derecho del contenido de la ventana. Si es un flotante, se escala al ancho de la ventana.
- `top_padding` — La cantidad de espacio entre el borde y la parte superior del contenido de la ventana. Si es un flotante, se escala a la altura de la ventana.
- `bottom_padding` — La cantidad de espacio entre el borde y la parte inferior del contenido de la ventana. Si es un flotante, se escala a la altura de la ventana.
- `xpadding` — Una forma conveniente (y compatible con versiones anteriores) de establecer `left_padding` y `right_padding` al mismo valor.
- `ypadding` — Una forma conveniente (y compatible con versiones anteriores) de establecer `top_padding` y `bottom_padding` al mismo valor.
- `xfill` — Si es `True`, la ventana se expandirá para ocupar todo el espacio disponible en la dirección x. Si es `False`, se encogerá para ajustarse a su contenido.
- `yfill` — Si es `True`, la ventana se expandirá para ocupar todo el espacio disponible en la dirección y. Si es `False`, se encogerá para ajustarse a su contenido.
- `xmaximum` — El tamaño máximo de esta ventana en la dirección x, incluyendo márgenes y relleno. Si es un número de punto flotante, se escala al ancho disponible.
- `ymaximum` — El tamaño máximo de esta ventana en la dirección y, incluyendo márgenes y relleno. Si es un número de punto flotante, se escala a la altura disponible.

- **xminimum** — El tamaño mínimo de esta ventana en la dirección x, incluyendo márgenes y relleno. Si la ventana es más pequeña que esto, se agranda hasta al menos este tamaño. Si es un número de punto flotante, se escala al ancho disponible.
- **yminimum** — El tamaño mínimo de esta ventana en la dirección y, incluyendo márgenes y relleno. Si la ventana es más pequeña que esto, se agranda hasta al menos este tamaño. Si es un número de punto flotante, se escala a la altura disponible.
- **size_group** — Si no es **None**, debe ser una cadena. Todas las ventanas o botones con una cadena dada como su **size_group** se renderizan con el mismo ancho (nuevo en 6.6.0).
- **clipping** — Si es **true** y el hijo es más grande que el tamaño máximo establecido para esta ventana, se recortará al tamaño de la ventana.

Propiedades de botón

Estas solo son aplicables a **Button**.

- **hover_sound** — El sonido que se reproduce cuando este widget se pone en estado **hover**.
- **activate_sound** — El sonido que se reproduce cuando el widget se activa, al hacer clic en él o seleccionarlo de otro modo.
- **focus_mask** — Se usa como una máscara que determina si un botón debe estar enfocado o no. Si es **False**, se usa un rectángulo predeterminado como máscara. Si es el valor especial **True**, se usa el elemento mostrable renderizado como máscara; de lo contrario, el valor se interpreta como un elemento mostrable, se renderiza al tamaño del botón y se usa. En cualquier caso, se usa el valor alfa de la máscara, y cualquier cantidad de alfa distinta de cero indica que un píxel es parte de la máscara (nuevo en 5.6.5).
- **focus_rect** — Si no es **None**, proporciona como una tupla (**x**, **y**, **width**, **height**) el rectángulo de enfoque de un botón, como un desplazamiento desde la esquina superior izquierda del botón. Para una navegación adecuada por teclado, no se permite que los rectángulos de enfoque de las imágenes se superpongan. Al establecer esto y **focus_mask** en un **imagebutton**, se pueden tener **imagebuttons** que se superpongan pero que aún admitan el enfoque por teclado. Casi con certeza nunca se debe establecer esto en una imagen individual, sino para cada botón individual (y es raro incluso hacer eso).
- **time_policy** — Una función responsable de modificar la base de tiempo del elemento mostrable de los elementos mostrables utilizados por los botones. Se espera que esta función reciba un tiempo, un objeto de estado y un objeto de estilo, y que devuelva un nuevo tiempo y un nuevo estado (el objeto de estado toma por defecto **None** la primera vez que se llama a **time_policy**). Esta función puede querer determinar el prefijo de estilo inspeccionando su campo **.prefix**. La **time_policy** predeterminada no cambia el tiempo.
- **child** — Si no es **None**, este elemento mostrable se muestra en lugar del hijo de este botón.

- **mouse** — Si no es **None**, una cadena que da el nombre del cursor del ratón que se usa cuando este botón está enfocado. El nombre debe estar listado en **config.mouse**. Esto también funciona con otros elementos mostrables enfocables.

Propiedades de barra

ui.bar tiene algunas propiedades específicas que controlan la apariencia de las barras. La barra tiene canales a la izquierda y a la derecha. El espacio restante es el espacio en el que la barra puede cambiar, siendo la división de la barra la fracción de este espacio que representa el valor de la barra como una fracción del rango. Cuando se dibuja la barra, primero se dibuja la sombra del pulgar, seguida de los lados izquierdo y derecho de la barra, seguidos del pulgar.

- **bar_vertical** — Si es **true**, la barra se dibuja verticalmente; si es **false**, horizontalmente.
- **bar_invert** — Si es **true**, el valor de la barra se representa en el lado derecho/superior de la barra, en lugar de en el lado izquierdo/inferior.
- **bar_resizing** — Si es **true**, redimensionamos las sub-barras en lugar de renderizarlas a tamaño completo y luego recortarlas.
- **left_gutter** — El tamaño del canal izquierdo de una barra horizontal, en píxeles (nombre interno: **fore_gutter**).
- **right_gutter** — El tamaño del canal derecho de una barra horizontal, en píxeles (nombre interno: **aft_gutter**).
- **top_gutter** — El tamaño del canal superior de una barra vertical, en píxeles (nombre interno: **fore_gutter**).
- **bottom_gutter** — El tamaño del canal inferior de una barra vertical, en píxeles (nombre interno: **aft_gutter**).
- **left_bar** — Un elemento mostrable que se usa para dibujar el lado izquierdo de una barra horizontal. Este elemento mostrable se renderiza primero al tamaño completo de la barra y luego se recorta para que solo el lado izquierdo sea visible (nombre interno: **fore_bar**).
- **right_bar** — Un elemento mostrable que se usa para dibujar el lado derecho de una barra horizontal. Este elemento mostrable se renderiza primero al tamaño completo de la barra y luego se recorta para que solo el lado derecho sea visible (nombre interno: **aft_bar**).
- **top_bar** — Un elemento mostrable que se usa para dibujar la parte superior de una barra vertical. Este elemento mostrable se renderiza primero al tamaño completo de la barra y luego se recorta para que solo la parte superior sea visible (nombre interno: **fore_bar**).
- **bottom_bar** — Un elemento mostrable que se usa para dibujar la parte inferior de una barra vertical. Este elemento mostrable se renderiza primero al tamaño completo de la barra y luego se recorta para que solo la parte inferior sea visible (nombre interno: **aft_bar**).

- **thumb** — Si no es **None**, es una imagen de pulgar que se dibuja sobre la división entre los lados de la barra.
- **thumb_shadow** — Si no es **None**, una sombra del pulgar que se dibuja debajo de la división entre los lados de la barra.
- **thumb_offset** — La cantidad en que el pulgar se superpone a las barras, en píxeles. Para que las barras izquierda y derecha continúen sin interrupción, establece esto a la mitad del ancho del pulgar. Por compatibilidad, tomamos el valor absoluto de esta propiedad.
- **unscrollable** — Controla qué sucede si la barra no se puede desplazar (si el rango se establece en 0, como es el caso de un **ui.viewport** que contiene un elemento mostrable más pequeño que él mismo). Hay tres valores posibles:
 - **None** - Renderiza la barra normalmente (predeterminado).
 - **"insensitive"** - Renderiza la barra en modo insensible. Se puede estilizar de manera diferente a una barra normal.
 - **"hide"** - Impide que la barra se renderice por completo. Se asignará espacio para la barra, pero no se dibujará nada en ese espacio (nuevo en 6.8.0).

Propiedades de caja

Los elementos mostrables **HBox**, **VBox** y **Fixed** son instancias de un elemento mostrable común **MultiBox**. Estas propiedades controlan cómo se distribuyen estos elementos mostrables.

- **box_layout** — Si es **"horizontal"**, la caja se distribuye horizontalmente. Si es **"vertical"**, la caja se distribuye verticalmente. Si es **"fixed"**, la distribución de la caja se controla mediante las propiedades de posición de los hijos de la caja, y el resto de estas propiedades se ignoran. Si es **None**, la distribución de la caja se controla mediante la función que creó la caja.
- **spacing** — El espacio entre elementos en la caja, en píxeles. También se usa para **ui.grid** y **ui.side**. Anteriormente se conocía como **box_spacing**, un alias que todavía funciona.
- **first_spacing** — Si no es **None**, el espacio entre los dos primeros elementos de la caja, en píxeles. Solo se usa para cajas. Anteriormente se conocía como **box_first_spacing**, un alias que todavía funciona.
- **xfill** — Si es **True**, la caja se expandirá para ocupar todo el espacio disponible en la dirección x. Si es **False**, se encogerá para ajustarse a su contenido.
- **yfill** — Si es **True**, la caja se expandirá para ocupar todo el espacio disponible en la dirección y. Si es **False**, se encogerá para ajustarse a su contenido.

Propiedades de posición

Las propiedades de posición se aplican a elementos mostrables que son más pequeños que el espacio que se les asigna. Se aplican a todos los elementos mostrables y controlan la colocación del widget en el espacio. Por ejemplo, las propiedades de posición se pueden usar para controlar la colocación de una ventana de diálogo en la pantalla.

Las propiedades de posición funcionan mejor cuando un elemento mostrable pequeño se coloca en un espacio vacío. Este es el caso de las ventanas de diálogo y los menús que se muestran como parte del menú principal. Las propiedades de posición funcionan en otros elementos mostrables, pero las vicisitudes de cómo se asigna el espacio pueden hacer que algunos resultados sean contradictorios.

El posicionamiento funciona colocando el elemento mostrable de modo que un ancla (especificada en relación con el elemento mostrable) esté en la misma ubicación que la posición (especificada en relación con el espacio disponible).

- **xanchor** — La ubicación en el eje x del ancla, en relación con la esquina superior izquierda del elemento mostrable. Puede ser un número absoluto de píxeles o un número de punto flotante que se interpreta como una fracción del tamaño del elemento mostrable.
- **xpos** — La ubicación en el eje x de la posición, en relación con la esquina superior izquierda del espacio disponible. Puede ser un número absoluto de píxeles o un número de punto flotante que se interpreta como una fracción del tamaño del espacio disponible.
- **xalign** — Un atajo para especificar **xanchor** y **xpos** al mismo tiempo. Casi siempre se debe asignar un número de punto flotante. Por ejemplo, **xanchor=0.5** centrará un elemento mostrable en el espacio disponible.
- **yanchor** — La ubicación en el eje y del ancla, en relación con la esquina superior izquierda del elemento mostrable. Puede ser un número absoluto de píxeles o un número de punto flotante que se interpreta como una fracción del tamaño del elemento mostrable.
- **ypos** — La ubicación en el eje y de la posición, en relación con la esquina superior izquierda del espacio disponible. Puede ser un número absoluto de píxeles o un número de punto flotante que se interpreta como una fracción del tamaño del espacio disponible.
- **yalign** — Un atajo para especificar **yanchor** y **ypos** al mismo tiempo. Casi siempre se debe asignar un número de punto flotante.
- **xoffset** — Un desplazamiento, en píxeles, que se añade a la posición calculada usando **xpos** y **xanchor**.
- **yoffset** — Un desplazamiento, en píxeles, que se añade a la posición calculada usando **ypos** y **yanchor**.

Suponiendo que **xpos**, **ypos**, **xanchor** y **yanchor** se han convertido a píxeles si es necesario, la posición de la esquina superior izquierda de un elemento mostrable en relación con la esquina superior izquierda del área en la que se distribuye se puede expresar como:

- `x = xpos - xanchor + xoffset`
- `y = ypos - yanchor + yoffset`

Lista de propiedades

Esta es una lista de propiedades, agrupadas de forma flexible según el uso de la propiedad.

Propiedades de texto

Las siguientes propiedades son utilizadas por los elementos mostrables de tipo **Text**.

- **antialias** — Si es **True**, el texto se renderizará con antialiasing. De lo contrario, se dejará con bordes dentados.
- **black_color** — Debe ser un triple RGBA. Cuando se usa un **SFont**, este es el color al que se mapeará el negro. Esta propiedad se ignora cuando se usa una fuente **TrueType**. El canal alfa en este color se ignora.
- **bold** — Si es **True**, este texto se renderizará en negrita.
- **color** — El color en el que se mostrará el texto en la pantalla, como una tupla RGBA. Cuando se usa un **SFont**, este es el color en el que se mostrará el blanco de ese **SFont**.
- **drop_shadow** — Se usa para controlar la generación de una sombra paralela en el texto. Es una tupla de 2 elementos, una lista de tuplas o **None**. Si es una tupla, los 2 elementos controlan los desplazamientos de la sombra en X e Y, respectivamente. Si es una lista, se espera que sea una lista de tuplas de 2 elementos, cada una de las cuales es la sombra. Usar una lista como `[ (-1, -1), (1, -1), (-1, 1), (1, 1) ]` crea texto contorneado en lugar de una sombra. Si es **None**, no se crea sombra para el texto.
- **drop_shadow_color** — El color de la sombra. Ejemplo: `"#000"`.
- **first_indent** — Se usa para dar, en píxeles, la sangría de la primera línea de texto en el widget de texto. Se puede usar para sangrar la primera línea de un párrafo (no es una buena idea en un monitor de ordenador, es mejor dejar una línea en blanco entre párrafos).
- **font** — La fuente que se usará para renderizar el texto. Es el nombre de un archivo que contiene una fuente **TrueType** o el nombre de una fuente. En el primer caso, la fuente se carga desde el archivo. En el segundo, el nombre de archivo se usa para buscar **SFonts** registrados con **register_sfont** y fuentes **TrueType** que se encuentran en el directorio de fuentes del sistema.
- **size** — El tamaño de la fuente que se usa para mostrar el texto en la pantalla. Ten en cuenta que el significado de esto puede variar de una fuente a otra y solo tiene una relación débil con el número de píxeles de alto que tendrá la fuente en la pantalla.

- **italic** — Si es **True**, este texto se renderizará en cursiva.
- **justify** — Si es **True**, se inserta espacio en blanco adicional entre las palabras para que los márgenes izquierdo y derecho de cada línea sean uniformes. Esto no se realiza en la última línea de un párrafo.
- **language** — El idioma (en realidad, la familia de idiomas) que se usa para mostrar el texto. Actualmente, hay dos opciones: **"western"** para texto de estilo occidental que se divide en espacios, y **."eastasian"** para texto chino/japonés, que se puede dividir entre caracteres.
- **layout** — Controla cómo ocurre el salto de línea. Por defecto es **"greedy"**, que intenta poner tanto como sea posible en cada línea. También puede ser **"subtitle"**, que intenta que las líneas sean uniformes en longitud.
- **line_spacing** — Se usa para aumentar o disminuir el espacio entre líneas de texto en un número constante de píxeles. Un valor positivo aumenta, mientras que uno negativo disminuye el espacio.
- **minwidth** — El ancho mínimo en píxeles de este texto. Si el texto renderizado es más pequeño que esto, se rellena con espacios en blanco a la derecha hasta que tenga al menos esta longitud en píxeles.
- **outlines** — Una lista de tuplas de 2 o 4 componentes, que dan el tamaño, el color y, opcionalmente, los desplazamientos x e y de los contornos alrededor del texto. Un contorno es un bloque de texto que se ha expandido en tamaño por un número dado de píxeles. Los desplazamientos x e y se pueden omitir. La primera tupla en la lista está más lejos de la pantalla, mientras que la última está más cerca (nuevo en 6.8.0).
- **rest_indent** — Se usa para dar, en píxeles, la sangría de la segunda línea y posteriores de un widget de texto. Se puede usar para dar una sangría francesa a citas de diálogo.
- **slow_cps** — Si es un número, hace que el texto se muestre a una velocidad de esa cantidad de caracteres por segundo. Si es **True**, toma ese número de la preferencia *Velocidad de texto*.
- **slow_cps_multiplier** — Un multiplicador que se aplicará al número de caracteres mostrados por segundo. Por ejemplo, 2.0 hace que los caracteres se muestren al doble de velocidad.
- **slow_abortable** — Si es **True**, el texto lento se puede abortar haciendo clic con el ratón.
- **text_align** — Se usa para controlar la alineación horizontal de las líneas de texto en el área asignada al widget **Text** que contiene ese texto. Solo tiene efecto si el texto tiene más de una línea. Es un número entre 0 y 1, que da la fracción de espacio vacío que debe estar a la izquierda de cada línea de texto (para centrar el texto, debe ser 0.5).
- **text_y_fudge** — Ajusta la posición y de un bloque de texto añadiendo espacio en blanco encima de la primera línea.

- **underline** — Si es **true**, este texto se renderizará con subrayado.

Propiedades de ventana

Las propiedades controlan la apariencia de **Window** y **Button**.

- **background** — Un elemento mostrable que se usa como fondo de la ventana. Debe ser un elemento mostrable que siempre dibuje exactamente el tamaño que se le solicita, lo que generalmente significa un **Solid** o un **Frame**. También puede ser **None**, lo que significa que no hay fondo en esta ventana (el resto de propiedades de ventana que se refieren a un fondo siguen funcionando; piensa en ellas como si su fondo fuera transparente).
- **foreground** — Se dibuja de la misma manera que el fondo, excepto que se dibuja encima del contenido de la ventana.
- **left_margin** — La cantidad de espacio transparente a la izquierda de esta ventana. Si es un número de punto flotante, se escala al ancho disponible.
- **right_margin** — La cantidad de espacio transparente a la derecha de esta ventana. Si es un número de punto flotante, se escala al ancho disponible.
- **top_margin** — La cantidad de espacio transparente en la parte superior de esta ventana. Si es un número de punto flotante, se escala a la altura disponible.
- **bottom_margin** — La cantidad de espacio transparente en la parte inferior de esta ventana. Si es un número de punto flotante, se escala a la altura disponible.
- **xmargin** — Una forma conveniente (y compatible con versiones anteriores) de establecer **left_margin** y **right_margin** al mismo valor.
- **ymargin** — Una forma conveniente (y compatible con versiones anteriores) de establecer **top_margin** y **bottom_margin** al mismo valor.
- **left_padding** — La cantidad de espacio entre el borde y el lado izquierdo del contenido de la ventana. Si es un flotante, se escala al ancho de la ventana.
- **right_padding** — La cantidad de espacio entre el borde y el lado derecho del contenido de la ventana. Si es un flotante, se escala al ancho de la ventana.
- **top_padding** — La cantidad de espacio entre el borde y la parte superior del contenido de la ventana. Si es un flotante, se escala a la altura de la ventana.
- **bottom_padding** — La cantidad de espacio entre el borde y la parte inferior del contenido de la ventana. Si es un flotante, se escala a la altura de la ventana.
- **xpadding** — Una forma conveniente (y compatible con versiones anteriores) de establecer **left_padding** y **right_padding** al mismo valor.
- **ypadding** — Una forma conveniente (y compatible con versiones anteriores) de establecer **top_padding** y **bottom_padding** al mismo valor.
- **xfill** — Si es **True**, la ventana se expandirá para ocupar todo el espacio disponible en la dirección x. Si es **False**, se encogerá para ajustarse a su contenido.

- **yfill** — Si es **True**, la ventana se expandirá para ocupar todo el espacio disponible en la dirección y. Si es **False**, se encogerá para ajustarse a su contenido.
- **xmaximum** — El tamaño máximo de esta ventana en la dirección x, incluyendo márgenes y relleno. Si es un número de punto flotante, se escala al ancho disponible.
- **ymaximum** — El tamaño máximo de esta ventana en la dirección y, incluyendo márgenes y relleno. Si es un número de punto flotante, se escala a la altura disponible.
- **xminimum** — El tamaño mínimo de esta ventana en la dirección x, incluyendo márgenes y relleno. Si la ventana es más pequeña que esto, se agranda hasta al menos este tamaño. Si es un número de punto flotante, se escala al ancho disponible.
- **yminimum** — El tamaño mínimo de esta ventana en la dirección y, incluyendo márgenes y relleno. Si la ventana es más pequeña que esto, se agranda hasta al menos este tamaño. Si es un número de punto flotante, se escala a la altura disponible.
- **size_group** — Si no es **None**, debe ser una cadena. Todas las ventanas o botones con una cadena dada como su **size_group** se renderizan con el mismo ancho (nuevo en 6.6.0).
- **clipping** — Si es **true** y el hijo es más grande que el tamaño máximo establecido para esta ventana, se recortará al tamaño de la ventana.

Propiedades de botón

Estas solo son aplicables a **Button**.

- **hover_sound** — El sonido que se reproduce cuando este widget se pone en estado **hover**.
- **activate_sound** — El sonido que se reproduce cuando el widget se activa, al hacer clic en él o seleccionarlo de otro modo.
- **focus_mask** — Se usa como una máscara que determina si un botón debe estar enfocado o no. Si es **False**, se usa un rectángulo predeterminado como máscara. Si es el valor especial **True**, se usa el elemento mostrable renderizado como máscara; de lo contrario, el valor se interpreta como un elemento mostrable, se renderiza al tamaño del botón y se usa. En cualquier caso, se usa el valor alfa de la máscara, y cualquier cantidad de alfa distinta de cero indica que un píxel es parte de la máscara (nuevo en 5.6.5).
- **focus_rect** — Si no es **None**, proporciona como una tupla (**x**, **y**, **width**, **height**) el rectángulo de enfoque de un botón, como un desplazamiento desde la esquina superior izquierda del botón. Para una navegación adecuada por teclado, no se permite que los rectángulos de enfoque de las imágenes se superpongan. Al establecer esto y **focus_mask** en un **imagebutton**, se pueden tener **imagebuttons** que se superpongan pero que aún admitan el enfoque por teclado. Casi con certeza nunca se debe establecer esto en una imagen individual, sino para cada botón individual (y es raro incluso hacer eso).

- **time_policy** — Una función responsable de modificar la base de tiempo del elemento mostrable de los elementos mostrables utilizados por los botones. Se espera que esta función reciba un tiempo, un objeto de estado y un objeto de estilo, y que devuelva un nuevo tiempo y un nuevo estado (el objeto de estado toma por defecto `None` la primera vez que se llama a `time_policy`). Esta función puede querer determinar el prefijo de estilo inspeccionando su campo `.prefix`. La `time_policy` predeterminada no cambia el tiempo.
- **child** — Si no es `None`, este elemento mostrable se muestra en lugar del hijo de este botón.
- **mouse** — Si no es `None`, una cadena que da el nombre del cursor del ratón que se usa cuando este botón está enfocado. El nombre debe estar listado en `config.mouse`. Esto también funciona con otros elementos mostrables enfocables.

Propiedades de barra

`ui.bar` tiene algunas propiedades específicas que controlan la apariencia de las barras. La barra tiene canales a la izquierda y a la derecha. El espacio restante es el espacio en el que la barra puede cambiar, siendo la división de la barra la fracción de este espacio que representa el valor de la barra como una fracción del rango. Cuando se dibuja la barra, primero se dibuja la sombra del pulgar, seguida de los lados izquierdo y derecho de la barra, seguidos del pulgar.

- **bar_vertical** — Si es `true`, la barra se dibuja verticalmente; si es `false`, horizontalmente.
- **bar_invert** — Si es `true`, el valor de la barra se representa en el lado derecho/superior de la barra, en lugar de en el lado izquierdo/inferior.
- **bar_resizing** — Si es `true`, redimensionamos las sub-barras en lugar de renderizarlas a tamaño completo y luego recortarlas.
- **left_gutter** — El tamaño del canal izquierdo de una barra horizontal, en píxeles (nombre interno: `fore_gutter`).
- **right_gutter** — El tamaño del canal derecho de una barra horizontal, en píxeles (nombre interno: `aft_gutter`).
- **top_gutter** — El tamaño del canal superior de una barra vertical, en píxeles (nombre interno: `fore_gutter`).
- **bottom_gutter** — El tamaño del canal inferior de una barra vertical, en píxeles (nombre interno: `aft_gutter`).
- **left_bar** — Un elemento mostrable que se usa para dibujar el lado izquierdo de una barra horizontal. Este elemento mostrable se renderiza primero al tamaño completo de la barra y luego se recorta para que solo el lado izquierdo sea visible (nombre interno: `fore_bar`).

- **right_bar** — Un elemento mostrable que se usa para dibujar el lado derecho de una barra horizontal. Este elemento mostrable se renderiza primero al tamaño completo de la barra y luego se recorta para que solo el lado derecho sea visible (nombre interno: **aft_bar**).
- **top_bar** — Un elemento mostrable que se usa para dibujar la parte superior de una barra vertical. Este elemento mostrable se renderiza primero al tamaño completo de la barra y luego se recorta para que solo la parte superior sea visible (nombre interno: **fore_bar**).
- **bottom_bar** — Un elemento mostrable que se usa para dibujar la parte inferior de una barra vertical. Este elemento mostrable se renderiza primero al tamaño completo de la barra y luego se recorta para que solo la parte inferior sea visible (nombre interno: **aft_bar**).
- **thumb** — Si no es **None**, es una imagen de pulgar que se dibuja sobre la división entre los lados de la barra.
- **thumb_shadow** — Si no es **None**, una sombra del pulgar que se dibuja debajo de la división entre los lados de la barra.
- **thumb_offset** — La cantidad en que el pulgar se superpone a las barras, en píxeles. Para que las barras izquierda y derecha continúen sin interrupción, establece esto a la mitad del ancho del pulgar. Por compatibilidad, tomamos el valor absoluto de esta propiedad.
- **unscrollable** — Controla qué sucede si la barra no se puede desplazar (si el rango se establece en 0, como es el caso de un **ui.viewport** que contiene un elemento mostrable más pequeño que él mismo). Hay tres valores posibles:
 - **None** - Renderiza la barra normalmente (predeterminado).
 - **"insensitive"** - Renderiza la barra en modo insensible. Se puede estilizar de manera diferente a una barra normal.
 - **"hide"** - Impide que la barra se renderice por completo. Se asignará espacio para la barra, pero no se dibujará nada en ese espacio (nuevo en 6.8.0).

Propiedades de caja

Los elementos mostrables **HBox**, **VBox** y **Fixed** son instancias de un elemento mostrable común **MultiBox**. Estas propiedades controlan cómo se distribuyen estos elementos mostrables.

- **box_layout** — Si es **"horizontal"**, la caja se distribuye horizontalmente. Si es **"vertical"**, la caja se distribuye verticalmente. Si es **"fixed"**, la distribución de la caja se controla mediante las propiedades de posición de los hijos de la caja, y el resto de estas propiedades se ignoran. Si es **None**, la distribución de la caja se controla mediante la función que creó la caja.
- **spacing** — El espacio entre elementos en la caja, en píxeles. También se usa para **ui.grid** y **ui.side**. Anteriormente se conocía como **box_spacing**, un alias que todavía funciona.

- **first_spacing** — Si no es `None`, el espacio entre los dos primeros elementos de la caja, en píxeles. Solo se usa para cajas. Anteriormente se conocía como `box_first_spacing`, un alias que todavía funciona.
- **xfill** — Si es `True`, la caja se expandirá para ocupar todo el espacio disponible en la dirección x. Si es `False`, se encogerá para ajustarse a su contenido.
- **yfill** — Si es `True`, la caja se expandirá para ocupar todo el espacio disponible en la dirección y. Si es `False`, se encogerá para ajustarse a su contenido.

Propiedades de posición

Las propiedades de posición se aplican a elementos mostrables que son más pequeños que el espacio que se les asigna. Se aplican a todos los elementos mostrables y controlan la colocación del widget en el espacio. Por ejemplo, las propiedades de posición se pueden usar para controlar la colocación de una ventana de diálogo en la pantalla.

Las propiedades de posición funcionan mejor cuando un elemento mostrable pequeño se coloca en un espacio vacío. Este es el caso de las ventanas de diálogo y los menús que se muestran como parte del menú principal. Las propiedades de posición funcionan en otros elementos mostrables, pero las vicisitudes de cómo se asigna el espacio pueden hacer que algunos resultados sean contradictorios.

El posicionamiento funciona colocando el elemento mostrable de modo que un ancla (especificada en relación con el elemento mostrable) esté en la misma ubicación que la posición (especificada en relación con el espacio disponible).

- **xanchor** — La ubicación en el eje x del ancla, en relación con la esquina superior izquierda del elemento mostrable. Puede ser un número absoluto de píxeles o un número de punto flotante que se interpreta como una fracción del tamaño del elemento mostrable.
- **xpos** — La ubicación en el eje x de la posición, en relación con la esquina superior izquierda del espacio disponible. Puede ser un número absoluto de píxeles o un número de punto flotante que se interpreta como una fracción del tamaño del espacio disponible.
- **xalign** — Un atajo para especificar **xanchor** y **xpos** al mismo tiempo. Casi siempre se debe asignar un número de punto flotante. Por ejemplo, **xanchor**=0.5 centrará un elemento mostrable en el espacio disponible.
- **yanchor** — La ubicación en el eje y del ancla, en relación con la esquina superior izquierda del elemento mostrable. Puede ser un número absoluto de píxeles o un número de punto flotante que se interpreta como una fracción del tamaño del elemento mostrable.
- **ypos** — La ubicación en el eje y de la posición, en relación con la esquina superior izquierda del espacio disponible. Puede ser un número absoluto de píxeles o un número de punto flotante que se interpreta como una fracción del tamaño del espacio disponible.
- **yalign** — Un atajo para especificar **yanchor** y **ypos** al mismo tiempo. Casi siempre se debe asignar un número de punto flotante.

- **xoffset** — Un desplazamiento, en píxeles, que se añade a la posición calculada usando **xpos** y **xanchor**.
- **yoffset** — Un desplazamiento, en píxeles, que se añade a la posición calculada usando **ypos** y **yanchor**.

Suponiendo que **xpos**, **ypos**, **xanchor** y **yanchor** se han convertido a píxeles si es necesario, la posición de la esquina superior izquierda de un elemento mostrable en relación con la esquina superior izquierda del área en la que se distribuye se puede expresar como:

- $x = xpos - xanchor + xoffset$
- $y = ypos - yanchor + yoffset$

Personalización de la interfaz

En Ren'Py, hay cuatro formas de personalizar los menús fuera del juego:

1. Los **diseños (layouts)** controlan la apariencia de las diversas pantallas que componen el menú del juego. Controlan la colocación de los componentes en esas pantallas y lo que hacen esos componentes si interactúan.
2. Los **temas (themes)** controlan la apariencia de botones, barras, marcos, etiquetas y mensajes.
3. Se puede personalizar los menús principal y del juego para añadir opciones adicionales a esos menús, más allá de las proporcionadas por los diseños.
4. Finalmente, se puede usar el sistema de estilos para personalizar aún más la apariencia y distribución de las diversas pantallas de menús fuera del juego.

De estas, la única personalización obligatoria es la selección de un tema.

Orden de personalizaciones

Esperamos que la personalización del juego ocurra en el siguiente orden:

1. Si es necesario, establecer la versión del guión (`config.script_version`), el ancho de la pantalla (`config.screen_width`) y la altura de la pantalla (`config.screen_height`).
2. Si es necesario, seleccionar uno o más diseños.
3. Seleccionar un tema maestro.
4. Si es necesario, seleccionar uno o más componentes de tema que anulen partes del tema maestro.
5. Si es necesario, personalizar los menús principal y del juego, y usar el sistema de estilos para personalizar aún más la interfaz.

Diseños (Layouts)

Los diseños controlan la apariencia de los menús fuera del juego. Hay cinco tipos principales de diseños:

- Los diseños de **menú principal** (`main_menu`) controlan la apariencia del menú principal.
- Los diseños de **navegación** (`navigation`) controlan la apariencia de la navegación del menú del juego.
- Los diseños de **carga/guardado** (`load_save`) controlan la apariencia de las pantallas de carga y guardado.
- Los diseños de **confirmación sí/no** (`yesno_prompt`) controlan la apariencia de las preguntas de sí/no al usuario.
- Los diseños de **preferencias** (`preferences`) controlan la apariencia de las pantallas de preferencias.
- Los diseños de **preferencias de joystick** (`joystick_preferences`) controlan la apariencia de las pantallas de preferencias de joystick.

También hay diseños independientes que no entran en ninguna de estas categorías. Aunque un juego necesita exactamente uno de cada uno de los cinco tipos principales de diseños para funcionar correctamente, puede tener tantos diseños independientes como necesite. Cada tipo de diseño proporciona cierta funcionalidad a la interfaz. La funcionalidad proporcionada por cada tipo de diseño se describe a continuación. Se espera que la selección de diseños ocurra antes de la selección de un tema. Si no se selecciona ningún diseño de un tipo dado, se utiliza la variante clásica. Por ejemplo, si no se ha seleccionado ningún diseño `yesno_prompt`, se utiliza `layout.classic_yesno_prompt`.

Ten en cuenta que hay combinaciones de diseños y temas que no se verán bien juntos. Esto no es un error... es imposible que cada diseño funcione con todos los demás diseños y todos los temas, y aún tener un rango ilimitado de diseños y temas. *Caveat factor*.

Diseños de menú principal

Los diseños de menú principal son responsables de definir la apariencia del menú principal. Esto incluye el fondo del menú principal y los botones que se usan para ir al menú del juego o iniciar un juego.

Function: `layout.classic_main_menu ()`:

Muestra el menú principal clásico, que apila los botones del menú principal verticalmente.

Function: `layout.grouped_main_menu ()`:

Define un diseño de menú principal que agrupa botones en filas horizontales y luego apila esas filas verticalmente.

Variable: `config.main_menu_per_group = 2`

El número de botones colocados en cada fila horizontal.

(imagen 2)

Function: `layout.imagemap_main_menu (ground, selected, hotspots, idle=None, variant=None)`:

- **ground** - La imagen de fondo. Se usa para las partes de la pantalla que no están definidas.
- **selected** - La imagen seleccionada. Se usa para las zonas interactivas que están en estado **hover**.
- **hotspots** - Una lista de tuplas que definen las zonas interactivas. Cada tupla consiste en:
 1. La coordenada x del lado izquierdo.
 2. La coordenada y del lado superior.
 3. La coordenada x del lado derecho.
 4. La coordenada y del lado inferior.
 5. El nombre (sin traducir) del botón del menú principal al que equivale esta zona interactiva, o una etiqueta en el programa a la que saltar cuando se hace clic en esta zona.
- **idle** - Si no es **None**, una imagen que se usa para imágenes que no están en estado **hover**. Si es **None**, toma el valor de **ground** (nuevo en 6.10.1).
- **variant** - La variante del menú principal a definir. La variable `_main_menu_variant` se usa para seleccionar una variante para mostrar al usuario (nuevo en 6.10.1).

A pesar del nombre, esta función puede tomar elementos mostrables arbitrarios (como **Animation**) además de imágenes.

Diseños de navegación

Los diseños de navegación son responsables de definir la navegación a través del menú del juego. Esto incluye el fondo de los menús del juego y los botones que se usan para moverse entre las pantallas del menú del juego.

Function: `layout.classic_navigation ()`:

Muestra los botones de navegación en una caja vertical.

Function: `layout.grouped_navigation ()`:

Muestra los botones de navegación en grupos horizontales, que luego se apilan verticalmente.

Variable: `config.navigation_per_group = 2`

El número de botones de navegación por grupo horizontal.

(imagen 3)

Function: `layout.imagemap_navigation (ground, idle, hover, selected_idle, selected_hover, hotspots)`:

Este diseño utiliza un mapa de imagen para manejar la navegación del menú del juego.

- **ground** - El elemento mostrable que se usa para los botones deshabilitados y las áreas que no están en una zona interactiva.
- **idle** - El elemento mostrable que se usa para zonas interactivas no seleccionadas y sin foco.

- **hover** - El elemento mostrable que se usa para zonas interactivas no seleccionadas y con foco.
- **selected_idle** - El elemento mostrable que se usa para zonas interactivas seleccionadas y sin foco.
- **selected_hover** - El elemento mostrable que se usa para zonas interactivas seleccionadas y con foco.
- **hotspots** - Una lista de tuplas que definen las zonas interactivas. Cada tupla consiste en:
 1. La coordenada x del lado izquierdo.
 2. La coordenada y del lado superior.
 3. La coordenada x del lado derecho.
 4. La coordenada y del lado inferior.
 5. El nombre (sin traducir) del botón del menú del juego al que equivale esta zona interactiva.

A pesar del nombre, esta función puede tomar elementos mostrables arbitrarios además de imágenes.

Diseños de carga/guardado

Los diseños de carga/guardado son responsables de definir las pantallas de carga y guardado.

(imagen 4) (imagen 5)

Function: `layout.classic_load_save ()`:

Este diseño muestra las pantallas de carga y guardado con un cierto número de filas y columnas de ranuras, con una fila de botones de navegación en la parte superior.

Variable: `config.file_page_rows = 5`

El número de filas de ranuras de archivo a mostrar.

Variable: `config.file_page_cols = 2`

El número de columnas de ranuras de archivo a mostrar.

Variable: `config.file_quick_access_pages = 8`

El número de páginas de archivos para las que proporcionar botones de acceso rápido.

Variable: `config.disable_file_pager = False`

Si es `True`, el paginador está deshabilitado. Esto impide el acceso a páginas distintas de la primera página de archivos.

Variable: `config.disable_thumbnails = False`

Si es `True`, no se muestran miniaturas.

Variable: `config.load_save_empty_thumbnail = None`

Si no es `None`, debe ser un elemento mostrable que se mostrará con las ranuras de carga/guardado vacías.

(imagen 6)

Variable: `config.time_format = "%b%d, %H: %M"`

El formato utilizado para las horas de los archivos en las ranuras de entrada de archivos.

Variable: `config.file_entry_format = "%(time)s\n%(save_name)s"`

El formato de las entradas de archivo en las ranuras de entrada de archivos.

Function: `layout.scrolling_load_save ()`:

Utiliza un área de desplazamiento que contiene entradas del selector de archivos. El usuario elige una de estas entradas para cargar o guardar un archivo. Hay una miniatura grande a la derecha de la pantalla, que corresponde a la entrada que está actualmente en estado `hover` (`config.thumbnail_width` y `config.thumbnail_height` controlan el tamaño de esta miniatura).

Variable: `config.load_save_slots = 50`

El número de ranuras normales a mostrar.

Variable: `config.load_save_auto_slots = 5`

El número de ranuras de autoguardado a mostrar.

Variable: `config.load_save_quick_slots = 5`

El número de ranuras de guardado rápido a mostrar.

Variable: `config.load_save_empty_thumbnail = None`

Cuando no es `None`, debe ser un elemento mostrable que se mostrará en el marco de la miniatura cuando no se haya puesto el cursor sobre ninguna ranura de guardado.

(imagen 7)

Variable: `config.time_format = "%b%d, %H: %M"`

El formato utilizado para las horas de los archivos en las ranuras de entrada de archivos.

Variable: `config.file_entry_format = "%(time)s\n%(save_name)s"`

El formato de las entradas de archivo en las ranuras de entrada de archivos.

Function: `layout.imagemap_load_save (ground, idle, hover, selected_idle, selected_hover, hotspots, variant=None)`:

Este diseño utiliza un mapa de imagen para manejar la carga y el guardado.

- **ground** - El elemento mostrable que se usa para los botones deshabilitados y las áreas que no están en una zona interactiva.
- **idle** - El elemento mostrable que se usa para zonas interactivas no seleccionadas y sin foco.
- **hover** - El elemento mostrable que se usa para zonas interactivas no seleccionadas y con foco.
- **selected_idle** - El elemento mostrable que se usa para zonas interactivas seleccionadas y sin foco.
- **selected_hover** - El elemento mostrable que se usa para zonas interactivas seleccionadas y con foco.
- **hotspots** - Una lista de tuplas que definen las zonas interactivas. Cada tupla consiste en:
 1. La coordenada x del lado izquierdo.
 2. La coordenada y del lado superior.
 3. La coordenada x del lado derecho.
 4. La coordenada y del lado inferior.
 5. La función de esta zona interactiva.

La función de la zona interactiva debe ser una de las siguientes:

- "previous" - ir a la página anterior.
- "next" - ir a la página siguiente.
- "page_auto" - ir a la página de autoguardado.
- "page_quick" - ir a la página de guardado rápido.
- "page_1", "page_2", "page_3", ... - ir a la página especificada. Los números de página deben comenzar con 1 y ser densos (sin omitir números).
- "slot_0", "slot_1", "slot_2", ... - una ranura de guardado/carga. Los números de ranura deben comenzar con 0 y ser densos.
- **variant** - Permite definir solo las pantallas de guardado o carga. Puede ser "save", "load" o None para definir las pantallas de guardado y carga a la vez. Si se define una pantalla de guardado, se debe definir una pantalla de carga (quizás con parámetros diferentes), y viceversa.

Las capturas de pantalla y el texto de las ranuras se colocan dentro de ventanas que se distribuyen en relación con la ranura. Ajustar `style.file_picker_ss_window` controla la colocación de la captura de pantalla, y ajustar `style.file_picker_text_window` controla la colocación del texto por ranura. Normalmente será necesario ajustar estos estilos, ya que el valor predeterminado coloca ambos en la parte superior izquierda.

Las funciones de las zonas interactivas también pueden incluir los nombres no traducidos de los botones del menú del juego. Si se define al menos uno de estos botones, la navegación no se muestra y se espera que el mapa de imagen defina todos los botones relevantes del menú del juego.

A pesar del nombre, esta función puede tomar elementos mostrables arbitrarios además de imágenes. Las imágenes o elementos mostrables utilizados deben ser transparentes para permitir que la navegación se vea a través de ellos, a menos que los botones del menú del juego se definan aquí.

Variable: `config.disable_thumbnails = False`

Si es True, no se muestran miniaturas.

Variable: `config.load_save_empty_thumbnail = None`

Si no es None, debe ser un elemento mostrable que se mostrará con las ranuras de carga/guardado vacías.

Variable: `config.time_format = "%b%d, %H: %M"`

El formato utilizado para las horas de los archivos en las ranuras de entrada de archivos.

Variable: `config.file_entry_format = "%(time)s\n%(save_name)s"`

(imagen 8) El formato de las entradas de archivo en las ranuras de entrada de archivos.

Diseños de confirmación sí/no

Los diseños de confirmación sí/no son responsables de definir las pantallas de preguntas de sí/no.

Function: `layout.classic_yesno_prompt ()`:

Muestra la pregunta de sí/no clásica, que es solo una pregunta sobre dos botones.

Function: `layout.imagemap_yesno_prompt (ground, idle, hover, hotspots, prompt_images=`

Este diseño utiliza un mapa de imagen para manejar las preguntas de sí/no al usuario.

- **ground** - El elemento mostrable que se usa para los botones deshabilitados y las áreas que no están en una zona interactiva.
- **idle** - El elemento mostrable que se usa para zonas interactivas sin foco.
- **hover** - El elemento mostrable que se usa para zonas interactivas con foco.
- **hotspots** - Una lista de tuplas que definen las zonas interactivas. Cada tupla consiste en:
 1. La coordenada x del lado izquierdo.
 2. La coordenada y del lado superior.
 3. La coordenada x del lado derecho.
 4. La coordenada y del lado inferior.
 5. El botón reemplazado por esta zona interactiva (uno de "Yes" o "No").
- **prompt_images** - Un mapa que enumera las imágenes de pregunta que se usarán en lugar del texto de la pregunta.

Cuando se va a mostrar una pregunta, primero se busca en `prompt_images`. Si se encuentra una imagen de mensaje, se muestra al usuario. De lo contrario, se busca la pregunta predeterminada (`layout.ARE_YOU_SURE`) y, si se encuentra, se muestra esa imagen al usuario. De lo contrario, la pregunta se renderiza como texto.

La colocación de una imagen de pregunta se controla mediante propiedades de estilo asociadas con la imagen. La colocación del texto de la pregunta se puede controlar mediante `style.yesno_prompt`, y el estilo del texto en sí se controla mediante `style.yesno_prompt_text`. Las preguntas utilizadas actualmente son:

- `layout.ARE_YOU_SURE` - `u"¿Estás seguro?"`
- `layout.DELETE_SAVE` - `u"¿Estás seguro de que quieres eliminar este guardado?"`
- `layout.OVERWRITE_SAVE` - `u"¿Estás seguro de que quieres sobrescribir tu guardado?"`
- `layout.LOADING` - `uÇargar hará que se pierda el progreso no guardado.\n¿Estás seguro de que quieres hacer esto?"`
- `layout.QUIT` - `u"¿Estás seguro de que quieres salir?"`
- `layout.MAIN_MENU` - `u"¿Estás seguro de que quieres volver al menú principal?\nEsto perderá el progreso no guardado."`

(imagen 9)

Las funciones de las zonas interactivas también pueden incluir los nombres no traducidos de los botones del menú del juego. Si se define al menos uno de estos botones, la navegación no se muestra y se espera que el mapa de imagen defina todos los botones relevantes del menú del juego.

A pesar del nombre, esta función puede tomar elementos mostrables arbitrarios además de imágenes. Las imágenes o elementos mostrables utilizados deben ser transparentes para permitir que la navegación se vea a través de ellos, a menos que los botones del menú del juego se definan aquí.

Diseños de preferencias

Los diseños de preferencias se utilizan para definir la pantalla de preferencias.

Function: `layout.classic_preferences ()`:

Utiliza un diseño de preferencias de tres columnas.

Variable: `config.sample_sound = None`

Un archivo de sonido utilizado para probar el volumen del sonido.

Variable: `config.sample_voice = None`

Un archivo de sonido utilizado para probar el volumen de la voz.

Variable: `config.has_transitions = True`

Si es `True`, se muestra la opción de habilitar o deshabilitar transiciones.

Variable: `config.has_cps = True`

Si es `True`, se muestra la opción de controlar la velocidad del texto. (imagen 10)

Variable: `config.has_afm = True`

Si es `True`, se muestra la opción de controlar el modo de avance automático.

Variable: `config.has_skipping = True`

Si es `True`, se muestra la opción de controlar si se saltan los mensajes ya leídos o no.

Variable: `config.has_skip_after_choice = True`

Si es `True`, se muestra la opción de controlar el salto después de las opciones.

Variable: `config.always_has_joystick = False`

Si es `True`, el enlace a la página de joystick está siempre activo.

Variable: `config.has_joystick = True`

Si es `True`, se muestra el enlace a la página de joystick.

Function: `layout.two_column_preferences ()`:

Utiliza un diseño de preferencias de dos columnas.

(imagen 11)

Function: `layout.one_column_preferences ()`:

Utiliza un diseño de preferencias de una columna. Las variables de configuración son las mismas que para `layout.classic_preferences`.

Function: `layout.imagemap_preferences (ground, idle, hover, selected_idle, selected_hover, hotspots)`:

Este diseño utiliza un mapa de imagen para manejar las preferencias.

- **ground** - El elemento mostrable que se usa para los botones deshabilitados y las áreas que no están en una zona interactiva.
- **idle** - El elemento mostrable que se usa para zonas interactivas no seleccionadas y sin foco.
- **hover** - El elemento mostrable que se usa para zonas interactivas no seleccionadas y con foco.
- **selected_idle** - El elemento mostrable que se usa para zonas interactivas seleccionadas y sin foco.
- **selected_hover** - El elemento mostrable que se usa para zonas interactivas seleccionadas y con foco.
- **hotspots** - Una lista de tuplas que definen las zonas interactivas. Cada tupla consiste en:

1. La coordenada x del lado izquierdo.
2. La coordenada y del lado superior.
3. La coordenada x del lado derecho.
4. La coordenada y del lado inferior.
5. La función de esta zona interactiva.

Hay dos tipos de zonas interactivas: botones y barras. Los botones son:

- "Window", "Fullscreen" - La preferencia de visualización.
- ".All", "Some", "None" - Las preferencias de transiciones.
- "Joystick" - El botón que se usa para saltar a las preferencias de joystick.
- "Seen Messages", ".All Messages" - Controlan qué mensajes se saltan.
- "Begin Skipping" - Fuerza el inicio del salto.
- "Stop Skipping", "Keep Skipping" - Controlan el salto en los menús.
- "Sound Test", "Voice Test" - Prueban los canales de audio correspondientes.

El otro tipo de zona interactiva es una barra horizontal. Las imágenes seleccionadas se usan para la parte llena de la barra, mientras que las imágenes no seleccionadas se usan para la parte vacía de la barra. Las barras conocidas son:

- "Music Volume" - Volumen de música.
- "Sound Volume" - Volumen de sonido.
- "Voice Volume" - Volumen de voz.
- ".Auto-Forward Time" - Tiempo de avance automático.
- "Text Speed" - Velocidad del texto.

Las funciones de las zonas interactivas también pueden incluir los nombres no traducidos de los botones del menú del juego. Si se define al menos uno de estos botones, la navegación no se muestra y se espera que el mapa de imagen defina todos los botones relevantes del menú del juego.

A pesar del nombre, esta función puede tomar elementos mostrables arbitrarios además de imágenes. Las imágenes o elementos mostrables utilizados deben ser transparentes para permitir que la navegación se vea a través de ellos, a menos que el menú del juego se defina aquí.

(imagen 12)

Diseños de preferencias de joystick

Los diseños de preferencias de joystick se utilizan para definir la pantalla de preferencias de joystick.

Function: `layout.classic_joystick_preferences ()`:

El diseño estándar de preferencias de joystick.

Diseños independientes

Estos proporcionan funcionalidad interesante a los juegos de Ren'Py, y son independientes, por lo que es posible llamar a tantos como se desee.

Function: `layout.button_menu ()`:

Cambia los menús dentro del juego para que usen botones definidos en el tema actual.

Definición de nuevos diseños

Esta sección contiene algunas notas sobre cómo definir tus propios diseños. Si no estás interesado en definir tus propias etiquetas, puedes ignorar esta sección de forma segura. En lo que respecta a la definición de diseños, hay dos principios importantes que debes seguir:

1. Se debe llamar a `layout.provides` con el tipo de diseño que se está definiendo, y debe llamarse antes de que se llame a cualquier función de tema. Por ejemplo, si se está definiendo un diseño de navegación, se debe llamar a `layout.provides("navigation")`.
2. El diseño debe implementar el contrato de su tipo de diseño. Estos contratos se describen a continuación.

Aunque los diseños predeterminados se habilitan mediante el uso de funciones en el objeto `layout`, esta no es de ninguna manera la única forma de proporcionar un diseño. Para un diseño definido por el usuario, debería ser suficiente colocar la llamada a `layout.provides` en un bloque `init -2 python`, con el resto del código residiendo en bloques `init` o etiquetas según corresponda.

Si tu diseño define nuevas variables de configuración, debes establecer `config.locked` en `False` antes de crearlas, y luego volver a `True` cuando hayas terminado de crearlas. Suggerimos que las variables de configuración tengan como prefijo el tipo de diseño para el que se utilizan. Las variables de carga/guardado deben tener el prefijo `config.load_save_`, las variables de navegación con `config.navigation_`, y así sucesivamente (por razones de compatibilidad, los diseños predeterminados no se ajustan a este estándar).

Diseños de menú principal

Los diseños de menú principal deben llamar a:

```
layout.provides('main_menu')
```

Se espera que los diseños de menú principal definan una etiqueta `main_menu_screen`. Se espera que esta etiqueta:

- Muestre una ventana de pantalla completa de tipo `mm_root`. El código para hacer esto es:

```
python:
    ui.window(style='mm_root')
    ui.null()
```

- Cree un mapa de teclas que deshabilite el enlace `game_menu` (esto es opcional, pero puede evitar transiciones del menú principal a sí mismo).

```
python:
    ui.keymap(game_menu=ui.returns(None))
```

- Muestre el menú principal en la pantalla. Cómo se hace esto generalmente depende del diseño en sí, pero recomendamos encarecidamente usar `config.main_menu` para controlar la generación del menú. Ten en cuenta que la documentación de `config.main_menu` dice que las cadenas simples causan un salto fuera del contexto; usa `ui.jumpsoutofcontext` para implementar esto correctamente. Usar `config.main_menu` asegura que tu diseño funcione incluso con extensiones (como galerías de imágenes) que añaden opciones al menú principal.

Diseños de navegación

Los diseños de navegación deben llamar a:

```
layout.provides('navigation')
```

Se espera que los diseños de navegación redefinan la función `layout.navigation`. La forma de hacerlo es primero definir una función en un bloque `init python` y luego asignar esa función a `layout.navigation`.

Function: `layout.navigation (screen):`

Esta función está destinada a ser establecida por un diseño de navegación. Es legítimo reemplazar esta función por una de tu elección, siempre que la firma siga siendo la misma. Esta función muestra la navegación en el menú del juego. Se espera que establezca el fondo del menú del juego. Si `screen` no es `None`, también se espera que muestre los botones de navegación definidos en `config.game_menu`, resaltando el llamado `screen`.

Se sugiere que tu función de navegación use `config.game_menu` para determinar los botones del menú del juego que se mostrarán al usuario (pero ten en cuenta que el menú del juego se extiende con menos frecuencia que el menú principal, por lo que esto es correspondientemente menos importante).

Diseños de carga/guardado

Los diseños de carga/guardado deben llamar a:

```
layout.provides('load_save')
```

Se espera que los diseños de carga/guardado proporcionen dos etiquetas: `load_screen` y `save_screen`, que se utilizan para cargar y guardar, respectivamente. Estas pantallas deben llamar a `layout.navigation` con `"load"` o `"save"` como argumento, y por lo demás no están restringidas en cómo proporcionan la funcionalidad de carga y guardado. Consulta la sección sobre funciones de carga/guardado para las funciones que se usarían para implementar realmente estas pantallas.

Diseños de confirmación sí/no

Los diseños de confirmación sí/no deben llamar a:

```
layout.provides('yesno_prompt')
```

Se espera que los diseños de navegación redefinan la función `layout.yesno_prompt`. La forma de hacerlo es primero definir una función en un bloque `init python` y luego asignar esa función a `layout.yesno_prompt`.

Function: `layout.yesno_prompt (screen, message):`

Esta función está destinada a ser personalizada por diseños `yesno_prompt`. Se puede reemplazar por otra función, siempre que la firma siga siendo la misma.

- **screen** - El botón de pantalla que debe resaltarse cuando se muestra esta pregunta. Si es `None`, no se muestra ninguna navegación del menú del juego.
- **message** - El mensaje que se muestra al usuario para pedirle que responda sí o no.

Esta función devuelve `True` si el usuario hace clic en **Yes**, o `False` si el usuario hace clic en **No**.

Diseños de preferencias

Los diseños de preferencias deben llamar a:

```
layout.provides('preferences')
```

El diseño de preferencias debe definir una etiqueta `preferences_screen`, que contiene código para establecer las preferencias. Esta pantalla debe llamar a `layout.navigation("preferences")`. Luego debe permitir al usuario modificar las preferencias, utilizando las variables y funciones definidas en la sección de preferencias. Finalmente, debe:

- Permitir al usuario comenzar a saltar, saltando a la etiqueta `_begin_skipping`.
- Permitir al usuario ir a las preferencias de joystick, saltando a la etiqueta `joystick_preferences_s`.

Diseños de preferencias de joystick

Los diseños de preferencias de joystick deben llamar a:

```
layout.provides('joystick_preferences')
```

El diseño de preferencias debe definir una etiqueta `joystick_preferences_screen`, que contiene código para establecer las preferencias de joystick. Esta pantalla debe llamar a `layout.navigation("joystick_preferences")`. Desafortunadamente, las preferencias de joystick no están documentadas en este momento.

Ejemplo

Un ejemplo de un nuevo diseño de menú principal:

```
init -2 python:
    layout.provides('main_menu')
    style.mm_button = Style(style.button, help="main menu button")
    style.mm_button_text = Style(style.button_text, help="main menu button (text)")
    style.mm_button.size_group = "mm"

label main_menu_screen:
    python:
        layout.button(u"Start Game", "mm", clicked=ui.jumpsoutofcontext('start'), xpos=
        layout.button(u"Continue Game", "mm", clicked=_intra_jumps("load_screen", "ma
        layout.button(u"Preferences", "mm", clicked=_intra_jumps("preferences_screen"
        layout.button(u"Quit", "mm", clicked=ui.jumps("_quit"), xpos=550, ypos=490)
        ui.interact()
```

Temas

Los temas proporcionan una forma sencilla de cambiar la apariencia de los menús principal y del juego. Una sola llamada a una función aplica estilos a muchos de los elementos de los menús principal y del juego, dando una apariencia coherente a la interfaz.

Las funciones de tema deben llamarse después de que se hayan establecido las variables `config.screen_width`, `config.screen_height` y `library.script_version`, y después de que se haya llamado a cualquier función de diseño. Deben llamarse antes de que se modifique cualquier estilo manualmente.

Funciones de tema

Estas funciones de tema son:

(imagen 13)

Function: `theme.roundrect (widget="#003c78", widget_hover="#0050a0", widget_text="#c8c8c8", widget_selected="#ffffffc8", disabled="#404040", disabled_text="#c8c8c8", label="#ffffff", frame="#6496c8", window="#000000c0", text_size=None, small_text_size=None, mm_root=..., gm_root=..., less_rounded=False):`

Activa el uso del tema `roundrect`. Por defecto, este tema estiliza el juego con un esquema de colores azul. Sin embargo, proporcionando uno o más de los parámetros que se indican a continuación, se puede cambiar el esquema de colores.

- `widget` - El color de fondo de botones y controles deslizantes sin foco.
- `widget_hover` - El color de fondo de botones y controles deslizantes con foco.
- `widget_text` - El color de texto de botones no seleccionados.
- `widget_selected` - El color de texto de botones seleccionados.
- `disabled` - El color de fondo de botones deshabilitados.

- `disabled_text` - El color de texto de botones deshabilitados.
- `label` - El color de texto de etiquetas no seleccionadas.
- `frame` - El color de fondo de los marcos.
- `mm_root` - Un elemento mostrable (como una `Image` o `Solid`) que se usará como fondo del menú principal.
- `gm_root` - Un elemento mostrable (como una `Image` o `Solid`) que se usará como fondo del menú del juego.
- `less_rounded` - Si es `True`, hace que los botones parezcan menos redondeados en el modo 800x600 (no tiene efecto en el modo 640x480).
- `text_size` - El tamaño del texto, como títulos de botones, etiquetas y mensajes. El valor predeterminado es 18 si la pantalla tiene 640 píxeles de ancho o menos, y 22 en caso contrario.
- `small_text_size` - El tamaño del texto en botones grandes. El valor predeterminado es 12 si la pantalla tiene 640 píxeles de ancho o menos, y 16 en caso contrario.

`widget`, `widget_hover`, `disabled` y `frame` pueden ser colores simples o tuplas que contienen dos colores. En este último caso, se utiliza un degradado vertical.

Funciones de componente

Las siguientes funciones existen para permitirte añadir elementos del tema `roundrect` a otro tema. El otro tema debe haberse configurado antes de que estas funciones puedan usarse. Los argumentos son como para `roundrect`, excepto que todos deben especificarse (no hay valores predeterminados).

- `theme.roundrect_labels(text_size, label)`
- `theme.roundrect_frames(less_rounded, frame)`
- `theme.roundrect_buttons(text_size, less_rounded, widget, widget_hover, widget_text, widget_selected, disabled, disabled_text)`
- `theme.roundrect_large_buttons(text_size, less_rounded, widget, widget_hover, widget_text, widget_selected, disabled, disabled_text)`
- `theme.roundrect_prompts(text_size, label)`
- `theme.roundrectBars(widget, widget_hover)`

(imagen 14)

Function: `theme.outline (inside="#fff", idle="#e66", hover="#48f", selected="#84f", insensitive="#ccc", label="#484", prompt="#484", background="#fee", large_button="#fff", text_size=22, small_text_size=16):`

Esta función selecciona un tema que se basa en contornear el texto con diferentes colores.

- `inside` - El color del texto dentro de los diversos contornos.

- `idle` - El color del contorno del texto de un botón o barra inactivo.
- `hover` - El color del contorno de un botón o barra en estado `hover`.
- `selected` - El color del contorno de un botón seleccionado.
- `insensitive` - El color del contorno de un botón insensible.
- `label` - El color del contorno de una etiqueta.
- `prompt` - El color del contorno de un mensaje.
- `background` - Un elemento mostrable utilizado para los fondos del menú del juego y del menú principal.
- `large_button` - El color de fondo de los fondos grandes.
- `text_size` - El tamaño del texto grande (utilizado para botones, etiquetas y mensajes).
- `small_text_size` - El tamaño del texto pequeño (utilizado en botones grandes).

Funciones de componente

Las siguientes funciones existen para permitirte añadir elementos del tema `outline` a otro tema. El otro tema debe haberse configurado antes de que estas funciones puedan usarse. Los argumentos son como para `theme.outline`, excepto que todos deben especificarse (no hay valores predeterminados).
(imagen 15)

- `theme.outline_frames()`
- `theme.outline_buttons(inside, idle, hover, selected, insensitive, text_size)`
- `theme.outline_large_buttons(inside, idle, hover, selected, insensitive, text_size, large_button)`
- `theme.outline_prompts(inside, prompt, text_size)`
- `theme.outline_labels(inside, label, text_size)`
- `theme.outlineBars(inside, idle, hover)`

Function: `theme.ancient ()`:

Este es un tema que intenta emular el tema utilizado por Ren'Py 6.5.0 cuando no se especificaba explícitamente ningún tema.

Modificadores de tema

Estas son funciones que se pueden llamar después de una función de tema, lo que permite cambiar una parte de un tema.

Function: `theme.image_buttons (d):`

Se utiliza para definir botones en términos de tuplas de 5 nombres de archivo de imagen. Espera que su único parámetro, `d`, sea un diccionario que asigne nombres de botones no traducidos a tuplas de 5 elementos. Cada tupla de 5 debe contener 5 nombres de archivo, que dan la imagen utilizada para el botón cuando:

1. El botón está inactivo.
2. El botón está en estado `hover`.
3. El botón está seleccionado e inactivo.
4. El botón está seleccionado y en estado `hover`.
5. El botón está insensible.

en ese orden.

Function: `theme.image_labels (d):`

Reemplaza etiquetas con imágenes. Toma un único parámetro, `d`, que se espera que sea un mapa del texto de una etiqueta a un archivo de imagen.

Tema personalizado

También es posible definir tu propio tema de Ren'Py. Un tema personalizado consiste en código Ren'Py que hace lo siguiente.

- Llama a `layout.defaults()` para inicializar los diseños. Llamar a esta función es responsabilidad del tema, y los temas personalizados también deben hacerlo.
- Configure los estilos base para reflejar los requisitos del tema.

A menudo, los estilos base vienen en pares `nombre/nombre_text`. En estos casos, `nombre` representa un `Button` o `Window` con el nombre de estilo, en el que vive un `Text` con el estilo `nombre_text`. Los estilos base son:

- `style.frame` (hereda de `style.default`) - Utilizado para marcos sobre los cuales el resto de la interfaz puede colocarse cómodamente.
- `style.button` (hereda de `style.default`) y `style.button_text` (hereda de `style.default`) - Utilizados para botones, especialmente botones cuyo propósito principal es navegar por la interfaz (como los botones de navegación del menú principal y del menú del juego).
- `style.small_button` (hereda de `style.button`) y `style.small_button_text` (hereda de `style.button_text`) - Utilizados para botones de navegación más pequeños. Puede tener sentido establecer un ancho mínimo en los botones, pero `small_button` debe poder reducirse lo máximo posible.

- `style.radio_button` (hereda de `style.button`) y `style.radio_button_text` (hereda de `style.button_text`) - Utilizados para botones que están organizados en un grupo, de modo que solo uno de los botones del grupo puede estar seleccionado a la vez.
- `style.check_button` (hereda de `style.button`) y `style.check_button_text` (hereda de `style.button_text`) - Utilizados para botones que alternan su estado de selección para indicar si una opción está activada o no (no se utilizan en ninguno de los diseños predefinidos).
- `style.large_button` (hereda de `style.default`) y `style.large_button_text` (hereda de `style.default`) - Utilizados para botones grandes, como las entradas del selector de archivos, que pueden contener una gran cantidad de texto y otra información.
- `style.label` (hereda de `style.default`) y `style.label_text` (hereda de `style.default`) - Utilizados para etiquetas, que son pequeños mensajes de texto que nunca cambian.
- `style.prompt` (hereda de `style.default`) y `style.prompt_text` (hereda de `style.default`) - Utilizados para mensajes, textos más largos que pueden cambiar en tiempo de ejecución.
- `style.bar` (hereda de `style.default`) y `style.vbar` (hereda de `style.default`) - Utilizados para barras horizontales y verticales, respectivamente. Las barras generalmente están destinadas a indicar una cantidad o un progreso, pero no se espera que sean ajustadas por el usuario.
- `style.slider` (hereda de `style.default`) y `style.vslider` (hereda de `style.default`) - Utilizados para controles deslizantes horizontales y verticales, respectivamente. Los controles deslizantes son barras que se utilizan para ajustar un valor.
- `style.scrollbar` (hereda de `style.default`) y `style.vscrollbar` (hereda de `style.default`) - Utilizados para barras de desplazamiento horizontales y verticales, respectivamente.
- `style.mm_root` (hereda de `style.default`) y `style.gm_root` (hereda de `style.default`) - Utilizados para los fondos de los menús principal y del juego, respectivamente.

Generalmente, los temas no deben ajustar los márgenes, las propiedades de posicionamiento o los tamaños máximos de estos estilos. Una excepción es que se espera que las barras establezcan un tamaño máximo en una dirección perpendicular a la orientación de la barra (`ymaximum` para `bar` y `scrollbar`; `xmaximum` para `vbar` y `vscrollbar`). No se aplican limitaciones a los estilos `_text`.

Menús principal y del juego

Esta sección describe cómo se puede personalizar el contenido de los diversos menús. Si solo quieres cambiar la apariencia de los menús (en la medida en que lo permite el sistema de estilos), usa temas o estilos. En particular, si quieres cambiar la imagen de fondo del menú principal y del menú del juego, usa las propiedades `mm_root` y `gm_root` en un tema.

Para cambiar solo el texto de los elementos del menú, considera usar `config.translations`. Para cambiar las pantallas de los menús, deberás cambiar el contenido de las variables `config.main_menu` y `config.game_menu`. De lo contrario, querrás usar un diseño que cambie la apariencia de los menús, o escribir tu propio diseño.

Menú principal

El menú principal se puede personalizar estableciendo la variable `config.main_menu`. Esta variable debe ser una lista de tuplas. El primer componente de cada tupla es el nombre del botón en el menú principal. El segundo componente puede ser una de tres cosas:

- una cadena, en cuyo caso es una etiqueta en la que comienza la ejecución del juego (después de un salto fuera del contexto del menú).
- una función, en cuyo caso se llama a la función cuando se hace clic en el botón.
- Finalmente, puede ser `None`, lo que hace que el botón sea insensible.

El tercer componente debe ser una cadena que contenga una expresión Python, que determina cuándo el botón está habilitado. El cuarto componente opcional es una cadena que contiene una expresión Python, que determina si el botón se muestra o no (cambiado en 6.6.2).

Para saltar del menú principal a una de las pantallas del menú del juego, se debe usar la función `_intra_jumps` para especificar la transición apropiada.

Function: `_intra_jumps (label, transition):`

Salta a `label` mientras permanece en el mismo contexto.

`transition` debe ser una cadena que dé un campo en el objeto `config`. Cuando se ejecuta esta función, se accede a ese campo y se utiliza para especificar la transición que se usa.

El valor predeterminado de `config.main_menu` es:

```
config.main_menu = [
    (u"Start Game", "start", "True"),
    (u"Continue Game", _intra_jumps("load_screen", "main_game_transition"), "True"),
    (u"Preferences", _intra_jumps("preferences_screen", "main_game_transition"), "True"),
    (u"Quit", ui.jumps("_quit"), "True")
]
```

Menú del juego

Lo primero que se puede desear hacer al modificar el menú del juego es añadir una pantalla. Esto se hace en dos pasos. El primero es crear la pantalla, y el segundo es añadirla a la lista `config.game_menu` para que se pueda acceder a ella desde el menú del juego.

Cada pantalla se representa en el código como una etiqueta a la que se salta para mostrar la pantalla. Hay cinco pasos que esta etiqueta debe realizar para mostrar la pantalla. Primero, debe llamar a `layout.navigation(screen)`, donde `screen` es el nombre de la pantalla (el primer componente de las tuplas en `config.game_menu`, descrito a continuación). Segundo, debe llamar a las funciones `ui` para añadir componentes a la pantalla. Tercero, debe llamar a `ui.interact` para interactuar con el usuario. Cuarto, debe examinar los resultados de `_game_interact` y reaccionar adecuadamente. Finalmente, debe

saltar de vuelta a la etiqueta de la pantalla, mostrando la pantalla de nuevo después de cada interacción.

Para que el usuario pueda verla, la pantalla debe añadirse a `config.game_menu`. Esta es una lista de tuplas de cuatro o cinco componentes. El primero es el nombre de la pantalla. Se utiliza para determinar si el botón utilizado para llegar a esa pantalla debe indicarse como seleccionado. El segundo componente es el texto utilizado para ese botón. El tercer componente es una función que se ejecuta cuando se hace clic en el botón. Normalmente, una función apropiada es `_intra_jumps(label, transition)`, donde `label` es el nombre de la etiqueta de tu pantalla. El cuarto componente es una cadena que contiene una expresión Python. Si la expresión no es verdadera, el botón es insensible. El quinto componente opcional también es una cadena que contiene una expresión Python. Si la expresión no es verdadera, el botón no se muestra (cambiado en 6.6.2). **Method:**

`renpy.Displayable.is_focused (self):`

Este método determina si el elemento mostrable actual tiene el foco. Probablemente no tenga sentido sobrescribirlo.

Method: `renpy.Displayable.set_style_prefix (self, prefix):`

Establece el prefijo de estilo de este elemento mostrable y sus hijos a uno de: `insensitive_`, `idle_`, `hover_`, `activate_`, `selected_insensitive_`, `selected_idle_`, `selected_hover_` o `selected_activate_`.

Method: `renpy.Displayable.render (self, width, height, st, at):`

Es responsable de dibujar el elemento mostrable. Esto se hace creando un objeto `renpy.Render`, copiando otros objetos en ese objeto en los lugares apropiados, y luego devolviendo ese `Render`.

- **width, height** - El área asignada a este elemento mostrable. El elemento mostrable creado puede ser más grande o más pequeño que esto, sin consecuencias.
- **st** - La base de tiempo del elemento mostrable, el número de segundos que este elemento mostrable se ha mostrado.
- **at** - La base de tiempo de animación, el número de segundos que una imagen con la misma etiqueta que este elemento mostrable se ha mostrado.

Este método no debe llamarse directamente sobre elementos mostrables hijos (consulta `renpy.render` en su lugar).

Method: `renpy.Displayable.event (self, ev, x, y, st):`

Se llama cuando ocurre un evento.

- **ev** - El objeto de evento de pygame. Se puede leer para obtener el tipo de evento y otros parámetros.
- **x, y** - La posición del ratón en el momento del evento. Es relativa a la esquina superior izquierda de este elemento mostrable, y debe usarse con preferencia a datos similares contenidos en `ev`.
- **st** - La base de tiempo del elemento mostrable.

Si este método devuelve un valor distinto de `None`, ese valor se devuelve desde `ui.interact`. Si devuelve `None`, el procesamiento del evento continúa con los otros elementos mostrables. Si se lanza `renpy.IgnoreEvent`, ningún otro elemento mostrable verá el evento.

Puede tener sentido llamar a este método sobre elementos mostrables hijos.

Method: `renpy.Displayable.get_placement (self):`

Devuelve una tupla de (`xpos`, `ypos`, `xalign`, `yalign`, `xoffset`, `yoffset`) que se usa para colocar este elemento mostrable en la pantalla.

El valor predeterminado obtiene los valores apropiados del estilo.

Method: `renpy.Displayable.visit (self):`

Debe sobrescribirse para devolver una lista de todos los elementos mostrables hijos de este elemento mostrable.

Method: `renpy.Displayable.per_interact (self):`

Se llama una vez por interacción, para informar al elemento mostrable que ha comenzado una nueva interacción.

A menudo, esto debe llamar a `renpy.redraw(self, 0)` para asegurarse de que el elemento mostrable se vuelva a dibujar para cada interacción. Esto es especialmente útil si el elemento mostrable participa en el retroceso.

renpy.Render

Dado que no se va a crear una subclase de `renpy.Render`, omitimos el parámetro `self` en las descripciones de los métodos.

Function: `renpy.Render (width, height):`

Construye un nuevo objeto de renderizado con el ancho y alto dados.

Method: `renpy.Render.blit (source, pos):`

Copia el contenido del renderizado fuente a este. `Pos` es una tupla de 2 elementos que da la posición de la esquina superior izquierda del renderizado fuente, relativa a la esquina superior izquierda de este renderizado, en píxeles.

Method: `renpy.Render.fill (color):`

Rellena el renderizado con el color dado, que debe ser una tupla RGBA.

Method: `renpy.Render.get_size ():`

Devuelve el ancho y alto de este renderizado como un par.

Method: `renpy.Render.kill ():`

Desasigna la memoria utilizada por este `Render`. Debe llamarse en un `Render` devuelto por `renpy.render` o `renpy.Render.subsurface` si el renderizado no se copia a otro `Render`. Una vez que se llama a esto, el renderizado no debe copiarse a una superficie ni se debe llamar a ningún otro método. Es seguro llamar a esto en un `Render` que se ha copiado en algún lugar, ya que estas llamadas se ignoran.

Method: `renpy.Render.subsurface (pos):`

Devuelve un renderizado que es una sub-superficie de este renderizado. `Pos` debe ser una tupla (`x`, `y`, `w`, `h`), donde `x` e `y` son las coordenadas de la esquina superior izquierda de la sub-superficie, `w` es el ancho y `h` es el alto de la sub-superficie.

Method: `renpy.Render.add_focus (displayable, arg, x, y, w, h, mx=None, my=None, mask=None):`

Se usa para indicar que una subregión de este `Render` es elegible para recibir el foco.

- `displayable` - El elemento mostrable que recibirá el foco.
- `arg` - Ignorado, debe ser `None`.
- `x`, `y`, `w`, `h` - Si no son `None`, una subregión rectangular de este `Render` que recibirá el foco. Si todos son `None`, este elemento mostrable tiene foco en toda la pantalla.

- **mx, my** - Si no son **None**, desplazamientos de la máscara de enfoque desde la esquina superior izquierda de este **Render**.
- **mask** - Un **Render** que se usa como máscara de enfoque cuando se necesita el enfoque del ratón. El ratón solo enfocará el elemento mostrable cuando esté sobre un píxel no transparente en la máscara.

Method: `renpy.Render.canvas ()`:

Esta función devuelve un objeto lienzo que tiene métodos correspondientes a las funciones de `pygame.draw`, excepto que el primer argumento (la superficie) se omite. Por ejemplo, en lugar de llamar a `pygame.draw.line(surface, (255, 255, 255, 255), (100, 100), (200, 200))`, se llamaría a `c.line((255, 255, 255, 255), (100, 100), (200, 200))`. El código que usa un lienzo puede no ser portable a plataformas que no sean `pygame`, en caso de que decidamos admitirlas en el futuro.

renpy.Container

La clase `renpy.Container` implementa un elemento mostrable que posiciona a sus hijos utilizando propiedades de estilo, de manera similar a `Fixed` y `ui.fixed`. A diferencia del elemento mostrable devuelto por esas funciones, `renpy.Container` admite añadir y eliminar hijos, y se puede crear una subclase para cambiar su comportamiento (incluyendo el comportamiento de los métodos `render` y `event`).

Un `renpy.Container` implementa todos los métodos de un `renpy.Displayable`. Estos métodos se pueden sobrescribir en una subclase, y esto se hace a menudo con los métodos `render` y `event`. Un contenedor tiene tres campos adicionales:

Field: `children = []`

Una lista de los hijos de este contenedor.

Field: `children = None`

El último hijo añadido a este contenedor y no eliminado, o `None` si no hay hijos en este contenedor.

Field: `offsets = []`

Una lista de tuplas `(x, y)` que dan las coordenadas de las esquinas superiores izquierdas de los hijos de este elemento mostrable.

También tiene tres métodos (los presentamos sin el parámetro `self`, ya que esperamos que se llamen y solo raramente se sobrescriban):

Method: `renpy.Container.add (d)`:

Añade un `d` a este contenedor.

Method: `renpy.Container.remove (d)`:

Elimina la primera instancia de `d` de este contenedor.

Method: `renpy.Container.update ()`:

Debe llamarse después de `add` o `remove` para indicar que el contenedor ha cambiado y debe volverse a dibujar.

Funciones de utilidad

Function: `renpy.render (displayable, width, height, st, at)`:

Llama a esta función para obtener un **Render** de un elemento mostrable. El objeto devuelto por esta función debe tratarse como inmutable, ya que puede almacenarse en una caché. Si

ya se ha renderizado un objeto con el mismo ancho y alto (y no se ha invalidado mediante `renpy.redraw`), se devolverá el renderizado en caché.

Function: `renpy.redraw (displayable, time):`

Hace que el elemento mostrable proporcionado se vuelva a dibujar después de que hayan transcurrido `time` segundos. `Time` puede ser cero para asegurar un redibujado inmediato.

Function: `renpy.timeout (time):`

Hace que ocurra un evento en `time` segundos. El tipo de evento no está definido.

Function: `renpy.easy_displayable (d, none=False):`

Toma un elemento mostrable fácilz devuelve un `Displayable`. Los elementos mostrables fáciles pueden ser `Displayable` o cadenas. Si es una cadena:

- Si la cadena comienza con `#`, se devuelve un `Solid` del color dado.
- De lo contrario, se devuelve una `Image`.

Si `none` es `True` y `d` es `None`, se devuelve `None`. De lo contrario, si `d` es `None`, se lanza una excepción.

Uso de elementos mostrables definidos por el usuario

Una vez que se ha definido una clase de elemento mostrable definida por el usuario, se pueden crear objetos de esa clase. Tal objeto se puede mostrar con una declaración `show`, o añadir a la capa transitoria con `ui.add`.

Definición de funciones `At` y transiciones

Una función `At` es una función adecuada para usar en una lista `at`, como parte de una declaración `show`. Son funciones que toman un elemento mostrable como argumento y devuelven un segundo elemento mostrable (definido por el usuario o no). Las transiciones toman dos argumentos de palabra clave, `new_widget` y `old_widget`, que representan las escenas nueva y antigua, respectivamente. Devuelven un elemento mostrable que realiza la transición, que debe tener su campo `delay` establecido en la duración de la transición.

Notas

Aunque haremos nuestro mejor esfuerzo para evitar romper código innecesariamente, nos reservamos el derecho de cambiar la API de elementos mostrables según sea necesario para apoyar el desarrollo futuro de Ren'Py. Como resultado, el código que usa la API de elementos mostrables puede no ser compatible con versiones futuras de Ren'Py.

Módulos Python

Es posible escribir módulos Python e importarlos en Ren'Py. Estos módulos Python definen su propio espacio de nombres, que es diferente del espacio de nombres en el que se ejecuta Python en Ren'Py. Los objetos definidos en módulos Python no participan en el retroceso, aunque pueden guardarse si son accesibles desde el store principal de Ren'Py. Los módulos Python se pueden colocar en el directorio `game`, mientras que los paquetes Python son subdirectorios del directorio `game` que contienen archivos `__init__.py`. La única codificación que admitimos es utf-8.

Sugerimos comenzar cada módulo con:

```
import renpy.store as store
import renpy.exports as renpy
```

Esto hace que el `store`, el espacio de nombres en el que se ejecuta Python en Ren'Py, esté disponible como `store`. También hace que las diversas funciones documentadas en este manual estén disponibles bajo `renpy` (un simple `import renpy` no hará esto, accederá a los internos de Ren'Py en su lugar. Disculpas por la complejidad adicional).

En general, los objetos creados dentro de módulos Python no participan en el retroceso. Esto significa que no debes cambiar tales objetos después de su creación inicial. Si quieres producir un objeto que participe en el retroceso, puedes usar los equivalentes definidos en el módulo `store`: `store.list`, `store.dict`, `store.set`, `store.object`. También puedes crear una clase que herede de `store.object`, y los objetos de esa clase participarán en el retroceso.

Probablemente sea más fácil no cambiar los objetos que se crean en módulos Python.

Ejemplo

Como ejemplo trivial, podríamos tener el siguiente módulo, que se coloca en `magic.py` en el directorio `game`:

```
import renpy.store as store
import renpy.exports as renpy
```

```
def add(a, b):
    return a + b
```

En un archivo de guión de Ren'Py, podemos importar el módulo usando una declaración `import`:

```
init:
    $ import magic
```

Luego puedes llamar a los métodos del módulo:

```
$ result = magic.add(1, 2)
"1 + 2 = %(result)d"
```

Herramientas de desarrollador

Ren'Py incluye varias características para facilitar la vida del desarrollador. Muchas de ellas necesitan que la variable `config.developer` esté establecida en `True` para funcionar.

Soporte de editor con Shift+E

La variable `config.editor` permite a un desarrollador especificar un comando de editor que se ejecuta cuando se produce la pulsación de tecla `launch_editor` (por defecto, `Mayús+E`). Consulta *Integrar SciTE con Ren'Py* para obtener información sobre cómo usar Ren'Py con el editor de texto SciTE personalizado. Consulta *Integrar Crimson Editor con Ren'Py* para obtener información sobre cómo usar Ren'Py con Crimson Editor.

Menú de desarrollador con Shift+D

Cuando `config.developer` es `true`, al pulsar Mayús+D se muestra un menú de desarrollador que proporciona un fácil acceso a algunas de las características que se indican a continuación.

Recarga con Shift+R

Cuando `config.developer` es `true`, al pulsar Mayús+R se guarda la partida actual, se recarga el guión del juego y se recarga la partida. Esto a menudo te colocará en la última declaración sin cambios encontrada antes de que se pulsara Mayús+R.

Esto permite al desarrollador hacer cambios en el guión con un editor externo y no tener que salir y reiniciar Ren'Py para ver el efecto de los cambios.

Ten en cuenta que el estado del juego, que incluye los valores de las variables y las listas de escenas, se conserva durante la recarga. Esto significa que si una de esas declaraciones se cambia, es necesario retroceder y volver a ejecutar la declaración para ver su nuevo efecto.

Inspección de estilos con Shift+I

Cuando `config.developer` es `true`, al pulsar Mayús+I se produce una inspección de estilos. Esto mostrará una lista de elementos mostrables debajo del ratón. Para cada elemento mostrable, se mostrará el tipo, el estilo utilizado y el tamaño en el que se está renderizando.

Volcado de estilos con Shift+Y

Cuando `config.developer` es `True`, al pulsar la tecla `dump_styles` (por defecto, Mayús+Y), se escribe una descripción de cada estilo que Ren'Py conoce en el archivo `styles.txt`. Esta descripción incluye cada propiedad que es parte del estilo, el valor de esa propiedad y el estilo del que se hereda la propiedad.

Salto rápido

Cuando `config.developer` es `True`, al pulsar la tecla `fast_skip` (por defecto, '>') se hace que el juego salte inmediatamente a la siguiente interacción importante. A estos efectos, una interacción importante es aquella que no es causada por una declaración `say`, una transición o un comando de pausa. Normalmente, esto significa saltar al siguiente menú, pero también se detendrá cuando ocurran formas de interacción definidas por el usuario.

Teletransporte a una línea

Ren'Py admite el teletransporte a una línea en el guión, sin que el desarrollador tenga que jugar todo el juego para llegar allí. Aunque esta técnica de teletransporte tiene varias advertencias asociadas, puede ser útil para proporcionar una vista previa en vivo.

Para invocar el teletransporte, ejecuta Ren'Py con el argumento de línea de comandos `-warp` seguido de una combinación `nombre de archivo:línea`, para especificar dónde te gustaría teletransportarte. Por ejemplo:

```
run_game --warp script.rpy:458
```

Cuando se invoca el teletransporte, Ren'Py hace varias cosas. Primero encuentra todas las declaraciones `scene` en el programa. Luego intenta encontrar un camino desde las declaraciones `scene` hasta cada declaración alcanzable en el juego. Luego elige la declaración alcanzable más cercana a, pero antes o en, la línea dada. Trabaja hacia atrás desde esa declaración hasta una declaración `scene`, registrando el camino que tomó. Ren'Py entonces ejecuta la declaración `scene` y cualquier declaración `show` o `hide` encontrada a lo largo de ese camino. Finalmente, transfiere el control a la declaración encontrada.

Hay varias advertencias bastante importantes sobre la función de teletransporte. La primera es que solo examina un único camino, lo que significa que aunque el camino puede ser representativo de alguna ruta de ejecución, es posible que haya un error a lo largo de otra ruta. En general, el camino no considera la lógica del juego, por lo que también es posible tener un camino que en realidad no sea alcanzable (esto solo es realmente un problema en juegos con mucho control, especialmente aquellos que usan mucho código Python). El mayor problema, sin embargo, es que no se ejecuta ningún código Python antes de la declaración a la que se teletransporta. Esto significa que todas las variables no estarán inicializadas, lo que puede provocar fallos cuando se usen. Para superar esto, se puede definir una etiqueta `after_warp`, que se llama después de un teletransporte pero antes de que se ejecute la declaración teletransportada. El código al que llega esta etiqueta puede configurar variables en el programa y luego regresar a la vista previa.

La función de teletransporte requiere que `config.developer` sea `True` para funcionar.

Cuando se usa `renpy.sh` o `renpy.py` desde el SDK para ejecutar Ren'Py, también deberás dar la opción `-game`. Por ejemplo:

```
./renpy.sh --game ./the_question/game/ --warp script.rpy:98
```

Modo NVL

El modo NVL es un modo en el que Ren'Py muestra más de una línea de diálogo en la pantalla a la vez. Esto se usa para emular mejor el formato de novela visual japonés, mientras que la configuración predeterminada de Ren'Py emula el formato de aventura japonés.

Para usar el modo NVL, se deben declarar personajes con un `kind` de `nvl`, o con un `kind` que tenga un `kind` de `nvl`. Por ejemplo:

```
init:
    $ e = Character("Eileen", color="#c8ffc8", kind=nvl)
    $ narrator = NVLCharacter(None, kind=nvl)
```

Cuando se habla diálogo con un personaje en modo NVL, se añade al final de un búfer, y luego se muestra al usuario todo el contenido de ese búfer. El búfer se puede limpiar usando la declaración `nvl clear`. Debe limpiarse antes del primer uso.

```
nvl clear
e "Esta es la primera línea de la primera pantalla."
e "Esta es la segunda línea de la primera pantalla."
nvl clear
e "Esta es la primera línea de la segunda pantalla."
e "Esta es la segunda línea de la segunda pantalla."
```


La declaración `nvl show transition` hace que ocurra una transición desde la pantalla anterior a una pantalla con el búfer mostrado. La declaración `nvl hide transition` hace que ocurra una transición desde una pantalla con el búfer mostrado a una pantalla sin el búfer mostrado.

Menús

La función `nvl_menu` muestra un menú en modo NVL. Para usarla, debe asignarse a `menu` en un bloque `init`, usando un código como:

```
init:
    $ menu = nvl_menu
```

Como los menús en modo NVL se muestran sin añadirse al búfer, sugerimos limpiar el búfer (usando `nvl clear`) inmediatamente después de cada menú.

Funciones

Para cada una de las declaraciones de modo NVL, hay una función Python equivalente. También hay una función sin equivalente de declaración.

Function: `nvl_clear ()`:

Equivalente a `nvl clear`.

Function: `nvl_hide (transition)`:

Equivalente a `nvl hide transition`.

Function: `nvl_show (transition)`:

Equivalente a `nvl show transition`.

Function: `nvl_erase ()`:

Elimina la línea inferior de la pantalla de modo NVL. Si no hay líneas en la pantalla de modo NVL, no hace nada. Esto se puede usar para reemplazar la línea inferior con otra cosa, o para volver a mostrarla con texto diferente.

Function: `nvl_window ()`:

Muestra la página NVL actual sin causar una interacción.

Variables

Variable: `config.nvl_page_ctc = None`

Si no es `None`, se espera que sea un elemento mostrable que proporcione un indicador de clic para continuar que se usará cuando la línea actual de diálogo en modo NVL vaya seguida de una declaración `nvl clear`.

Variable: `config.nvl_page_ctc_position = "nestled"`

Da la posición del indicador de clic para continuar cuando se usa `config.nvl_page_ctc`. Consulta el argumento `ctc_position` de `Character` para conocer los valores legales.

Variable: `config.nvl_paged_rollback = False`

Cuando es `true`, cambia el mecanismo de retroceso para mostrar páginas enteras de retroceso a la vez. Para que esto funcione, los finales de página deben ser predecibles, lo que generalmente significa que las declaraciones `nvl clear` deben seguir inmediatamente al diálogo en modo NVL (en lugar de ocurrir solo después de que una página haya terminado). Esto no funciona bien cuando se mezclan los modos ADV y NVL, ya que la idea de una página no está bien definida en ese caso.

Variable: `nvl_variant = None`

Si no es `None`, es un valor por el que se indexan todos los estilos utilizados en el modo NVL. Por ejemplo, si `nvl_variant` se establece en `"foo"`, se usará `style.nvl_window["foo"]` en lugar de `style.nvl_window`.

Soporte de voz

Ren'Py incluye soporte para añadir actuación de voz a un juego. Esto se hace mediante dos declaraciones adicionales.

La declaración `voice "filename"` reproduce el archivo de voz dado en `filename`.

La declaración `voice sustain` continúa reproduciendo el archivo de voz anterior.

Un archivo de voz se reproduce al inicio de la primera interacción que sigue a una declaración `voice`. Se termina al inicio de la siguiente interacción, a menos que se dé una declaración `voice sustain` que continúe reproduciéndolo.

```
voice "eileen1.ogg"
e "Ahora estoy diciendo algo."
"Pero ella dejó de decirlo, porque esta declaración comenzó."
voice "eileen2.ogg"
e "Ahora estoy diciendo algo más..."
voice sustain
e "¡Y seguiré diciéndolo!"
```

Equivalentes de función

Hay dos funciones correspondientes a las dos declaraciones de voz.

Function: `voice (filename):`

Reproduce la voz en `filename`. Equivalente a `voice`.

Function: `voice_sustain ():`

Continúa reproduciendo el archivo de voz actual. Equivalente a `voice sustain`.

Modelo de renderizado

Hay tres entidades principales en el modelo de renderizado de Ren'Py: la lista de escenas actual, la lista de escenas antigua y la pantalla. Las dos primeras necesitan un poco de explicación, pero la pantalla es simplemente lo que se renderiza en el monitor del usuario.

Interacciones

A lo largo de este documento, nos referiremos a las interacciones con el usuario. Una interacción es cualquier período de tiempo contiguo, incluyendo de longitud cero, en el que mostramos algo al usuario, esperamos y obtenemos la entrada del usuario, o ambas cosas. Así, `renpy.pause(0)` causa una interacción de longitud cero en la que solo mostramos algo al usuario, mientras que una declaración `say` (una línea de diálogo) comienza una interacción que comienza dibujando el diálogo en la pantalla y continúa hasta que el usuario hace clic con su ratón, lo que potencialmente es para siempre.

Listas de escenas

Las listas de escenas son estructuras de datos con toda la información necesaria para dibujar una escena en la pantalla. En particular, tienen capas, cada una de las cuales tiene una lista ordenada de imágenes para mostrar. Puede haber cualquier número de capas, pero por defecto hay dos capas: la capa maestra y la capa transitoria.

Capa maestra

La capa maestra contiene todas las imágenes que típicamente abarcan múltiples interacciones. La imagen de fondo y los gráficos de los personajes irán en la capa maestra, por ejemplo.

Capa transitoria

La capa transitoria contiene imágenes que abarcan solo una única interacción con el usuario. Lo más común en la capa transitoria es el diálogo y la narración renderizados.

Orden

Para cada capa, las imágenes en ella tienen un orden *z*. El usuario puede especificar explícitamente el orden *z* de las imágenes, pero más comúnmente simplemente obtienen el orden *z* implícito de incrementar el orden *z* para cada nueva imagen colocada en la capa.

Dibujo

El dibujo en la pantalla se realiza durante una interacción con el usuario, y solo entonces. Al comenzar la interacción, si no se especifica ninguna transición, Ren'Py simplemente dibuja la lista de escenas actual en la pantalla. Si se especifica una transición, Ren'Py comienza la transición al inicio de la interacción y continúa hasta que la transición termina. Las animaciones, por el contrario, comienzan al inicio de la interacción y continúan hasta el final de la interacción.

Una vez que una interacción termina, la lista de escenas actual se copia a la lista de escenas antigua.

Declaraciones y sus comportamientos

Scene y Show

La declaración **scene** (si se usa para especificar una imagen) y la declaración **show** ambas (por defecto) colocan imágenes en la capa maestra de la lista de escenas actual, pero no activan una interacción. Por lo tanto, puedes pensar en ellas como encolando imágenes para mostrar.

with

La declaración **with** especifica una transición y causa una interacción. Técnicamente, el usuario puede hacer clic para terminar prematuramente la transición, pero por lo demás la interacción causada por una declaración **with** no toma entrada del usuario.

with None

La declaración **with** cuando se le da el argumento **None** es un caso especial. No causa una interacción, tampoco especifica una transición. Simplemente copia la lista de escenas actual en la lista de escenas antigua y retorna (esto se puede usar para hacer la transición desde cosas que nunca se dibujaron en la pantalla).

Cláusula with

Una declaración que tiene una cláusula **with**:

```
statement args with transition
```

es equivalente a:

```
with None
statement args
with transition
```

y se comporta exactamente así.

say

La declaración **say** causa una interacción que dura hasta que el usuario la termina (con el teclado o el ratón), excepto si la auto-lectura está habilitada, en cuyo caso la interacción dura la duración de la auto-lectura. En particular, cualquier cosa que se mostró antes de una declaración **say** se mostrará al inicio de la declaración **say**.

Declaraciones definidas por el usuario

Las declaraciones definidas por el usuario permiten añadir tus propias declaraciones a Ren'Py. Esto hace posible añadir cosas que no están soportadas por la sintaxis actual de Ren'Py. Las declaraciones definidas por el usuario están actualmente limitadas a una sola línea y no pueden contener bloques.

Las declaraciones definidas por el usuario deben definirse en un bloque **python early**. Además, el nombre de archivo que contiene la declaración definida por el usuario debe cargarse antes que cualquier archivo que la use. Dado que Ren'Py carga archivos en orden de clasificación Unicode, generalmente tiene sentido prefijar el nombre de cualquier archivo que contenga una declaración definida por el usuario con 00. Una declaración definida por el usuario no se puede usar en el archivo en el que se define.

Function: `renpy.statements.register (name, parse=..., execute=..., predict=..., lint=..., next=...):`

Registra una declaración definida por el usuario.

- **name** es una lista separada por espacios de nombres que comienzan la declaración, o la cadena vacía para definir una nueva declaración predeterminada (la declaración predeterminada reemplazará a la declaración **say**).
- **parse** es una función que toma un objeto **lexer**. Esta función debe analizar la declaración y devolver un objeto. Este objeto se pasa como argumento a todas las demás funciones. El argumento **lexer** tiene los siguientes métodos:

- `l.eol()` - True si estamos al final de la línea.
 - `l.match(re)` - Coincide con una cadena de expresión regular arbitraria.
 - `l.keyword(s)` - Coincide con `s`.
 - `l.name()` - Coincide con cualquier nombre que no sea palabra clave. Ten en cuenta que esto solo cuenta las palabras clave integradas.
 - `l.word()` - Coincide con cualquier palabra.
 - `l.string()` - Coincide con una cadena de Ren'Py.
 - `l.integer()` - Coincide con un entero, devuelve una cadena que contiene el entero.
 - `l.float()` - Coincide con un número de punto flotante.
 - `l.simple_expression()` - Coincide con una expresión Python simple, la devuelve como una cadena.
 - `l.rest()` - Salta los espacios en blanco y luego devuelve el resto de la línea.
 - `l.checkpoint()` - Devuelve un objeto opaco que representa el punto actual en el análisis.
 - `l.revert(o)` - Dado el objeto devuelto por `l.checkpoint()`, regresa allí.
- `execute` es una función que se llama cuando se ejecuta la declaración. Se le pasa un solo objeto, el argumento devuelto por `parse`.
 - `predict` es una función que se llama para predecir las imágenes utilizadas por la declaración. Se le pasa un solo objeto, el argumento devuelto por `parse`. Debe devolver una lista de elementos mostrables utilizados por la declaración.
 - `lint` se llama para comprobar la declaración. Se le pasa un solo objeto, el argumento devuelto por `parse`. Debe llamar a `renpy.error` para informar de errores.
 - `next` se llama para determinar la siguiente declaración. Se le pasa un solo objeto, el argumento devuelto por `parse`. Debe devolver una etiqueta o devolver `None` si la ejecución debe continuar con la siguiente declaración.

Funciones de utilidad de Lint

Estas funciones son útiles para escribir funciones de `lint`.

Function: `renpy.lint.check_text_tags (s):`

Comprueba las etiquetas de texto en `s` para verificar su corrección. Devuelve una cadena de error si hay un error, o `None` si no hay error.

Ejemplo

Esto crea una nueva declaración "line" que permite especificar líneas de texto sin comillas.

```
python early:
    def parse_smartline(lex):
        who = lex.simple_expression()
        what = lex.rest()
```

```

    return (who, what)

def execute_smartline(o):
    who, what = o
    renpy.say(who, what)

def lint_smartline(o):
    who, what = o
    try:
        eval(who)
    except:
        renpy.error("Character not defined: %s" % who)
    tte = renpy.lint.check_text_tags(what)
    if tte:
        renpy.error(tte)

renpy.statements.register("l", parse=parse_smartline,
                          execute=execute_smartline, lint=lint_smartline)

```

Esto se usa así:

```
line e "Estas comillas aparecerán," dijo Eileen, "y no necesitan ser escapadas con bar"
```

Variables de entorno

Las siguientes variables de entorno controlan el comportamiento de Ren'Py:

- **RENPY_SCALE_FACTOR** - Si se establece, se analiza como un número de punto flotante y la pantalla se escala por esa cantidad. Por ejemplo, si **RENPY_SCALE_FACTOR** se establece en "0.5", todo es la mitad del tamaño normal.
- **RENPY_SCALE_FAST** - Si se establece y Ren'Py comienza a escalar la pantalla, esta usará un filtrado de vecino más cercano en lugar de un filtrado bilineal más lento pero de mayor calidad. Generalmente no debería ser necesario establecer esto.
- **RENPY_DISABLE_JOYSTICK** - Si se establece, la detección de joystick está deshabilitada. Usa esto si un joystick defectuoso está haciendo que Ren'Py avance cuando no se desea.
- **RENPY_DISABLE_FULLSCREEN** - Si se establece, Ren'Py se negará a entrar en modo de pantalla completa.
- **RENPY_DISABLE_SOUND** - Evita que ocurra la reproducción de sonido. Si esta variable contiene "pss", la reproducción de sonido se deshabilita. Si contiene "mixer", el control de volumen se deshabilita. Un valor de "pss,mixer" deshabilita ambos.
- **RENPY_SOUND_BUFSIZE** - Controla el tamaño del búfer de sonido. Valores mayores que el predeterminado (2048) pueden evitar que el sonido se salte, a costa de un mayor retraso desde que se invoca un sonido hasta que se reproduce.

- `RENPY_NOMIXER` - Si se establece, evita que Ren'Py intente controlar el mezclador de audio del sistema.
- `RENPY_EDITOR` - El valor predeterminado de `config.editor`.
- `RENPY_EDITOR_FILE_SEPARATOR` - El valor predeterminado de `config.editor_file_separator`.
- `RENPY_EDITOR_TRANSIENT` - El valor predeterminado de `config.editor_transient`.
- `RENPY_SCREENSHOT_PATTERN` - Un patrón utilizado para crear nombres de archivo de capturas de pantalla. Debe contener una única sustitución `%d`. Por ejemplo, establecer esto en `"screenshot %04d.jpg"` hará que Ren'Py escriba capturas de pantalla en JPEG en lugar de los PNG habituales.
- `RENPY_LESS_MEMORY` - Hace que Ren'Py reduzca su uso de memoria, a cambio de reducciones en velocidad.
- `RENPY_LESS_UPDATES` - Hace que Ren'Py reduzca el número de actualizaciones de pantalla que ocurren.
- `RENPY_LESS_MOUSE` - Hace que Ren'Py deshabilite el ratón en todo momento.
- `RENPY_BASE` - Esta variable de entorno es exportada por Ren'Py a los procesos ejecutados por él. Contiene la ruta completa al directorio que contiene `renpy.exe`, `renpy.sh` o `renpy.app`.

Como Ren'Py usa SDL, su comportamiento también puede ser controlado por las variables de entorno de SDL.

Al inicio, Ren'Py buscará en el directorio de Ren'Py (el que contiene `renpy.exe` o `renpy.py`) el archivo `.environment.txt`. Si existe, se evaluará como un archivo Python, y los valores definidos en ese archivo se usarán como los valores predeterminados de las variables de entorno (nuevo en 6.9.0).

Índice de funciones

Ten en cuenta que debido a una limitación de MediaWiki, los guiones bajos se reemplazan por espacios en esta página.

- Alpha
- `anim.Blink`
- `anim.Edge`
- `anim.Filmstrip`
- `anim.SMAnimation`
- `anim.State`
- `anim.TransitionAnimation`
- Animation

- At
- Bar
- Button
- Character
- Character.copy
- color
- ComposeTransition
- ConditionSwitch
- CropMove
- define.move transitions
- Dissolve
- DynamicCharacter
- DynamicDisplayable
- FactorZoom
- Fade
- Fixed
- Frame
- HBox
- im.Alpha
- im.AlphaMask
- im.Composite
- im.Crop
- im.FactorScale
- im.Flip
- im.Grayscale
- im.Image
- im.Map
- im.matrix
- im.matrix.brightness

- `im.matrix.contrast`
- `im.matrix.desaturate`
- `im.matrix.hue`
- `im.matrix.identity`
- `im.matrix.invert`
- `im.matrix.opacity`
- `im.matrix.saturation`
- `im.matrix.tint`
- `im.MatrixColor`
- `im.ramp`
- `im.Recolor`
- `im.Rotozoom`
- `im.Scale`
- `im.Sepia`
- `im.Tile`
- `im.Twocolor`
- `Image`
- `ImageDissolve`
- `ImageReference`
- `layout.button menu`
- `layout.classic joystick preferences`
- `layout.classic load save`
- `layout.classic main menu`
- `layout.classic navigation`
- `layout.classic preferences`
- `layout.classic yesno prompt`
- `layout.compat`
- `layout.defaults`
- `layout.grouped main menu`

- `layout.grouped` navigation
- `layout.imagemap` load save
- `layout.imagemap` main menu
- `layout.imagemap` navigation
- `layout.imagemap` preferences
- `layout.imagemap` yesno prompt
- `layout.joystick` preferences
- `layout.navigation`
- `layout.one column` preferences
- `layout.provides`
- `layout.scrolling` load save
- `layout.two column` preferences
- `layout.yesno` prompt
- `LiveComposite`
- `Motion`
- `Move`
- `MoveTransition`
- `Movie`
- `MultipleTransition`
- `Null`
- `nvl` erase
- `Pan`
- `ParameterizedText`
- `Particles`
- `Pause`
- `Pixellate`
- `Position`
- `renpy.block` rollback
- `renpy.cache` pin

- `renpy.cache unpin`
- `renpy.call` in new context
- `renpy.can load`
- `renpy.checkpoint`
- `renpy.choice` for skipping
- `renpy.clear game runtime`
- `renpy.context`
- `renpy.context` nesting level
- `renpy.copy images`
- `renpy.count displayables in layer`
- `renpy.current interact type`
- `renpy.curried call` in new context
- `renpy.curry`
- `renpy.display menu`
- `renpy.display say`
- `renpy.dynamic`
- `renpy.easy displayable`
- `renpy.exists`
- `renpy.file`
- `renpy.free memory`
- `renpy.full restart`
- `renpy.game menu`
- `renpy.get all labels`
- `renpy.get at list`
- `renpy.get filename line`
- `renpy.get game runtime`
- `renpy.get placement`
- `renpy.get reshow say`
- `renpy.get roll forward`

- `renpy.get transition`
- `renpy.has label`
- `renpy.hide`
- `renpy.image`
- `renpy.imagemap`
- `renpy.input`
- `renpy.invoke` in new context
- `renpy.in rollback`
- `renpy.jump`
- `renpy.jump` out of context
- `renpy.last interact type`
- `renpy.layer at list`
- `renpy.list saved games`
- `renpy.load`
- `renpy.loadable`
- `renpy.load module`
- `renpy.log`
- `renpy.movie cutscene`
- `renpy.movie start displayable`
- `renpy.movie stop`
- `renpy.music.get playing`
- `renpy.music.play`
- `renpy.music.queue`
- `renpy.music.register channel`
- `renpy.music.set mixer`
- `renpy.music.set music`
- `renpy.music.set pan`
- `renpy.music.set queue empty callback`
- `renpy.music.set volume`

- `renpy.music.stop`
- `renpy.partial`
- `renpy.pause`
- `renpy.play`
- `renpy.pop` return
- `renpy.predict`
- `renpy.predict display say`
- `renpy.quit`
- `renpy.random.choice`
- `renpy.random.randint`
- `renpy.random.random`
- `renpy.redraw`
- `renpy.register bmfont`
- `renpy.register mudgefont`
- `renpy.register sfont`
- `renpy.rename save`
- `renpy.render`
- `renpy.Render` (constructor)
- `renpy.reshow say`
- `renpy.restart interaction`
- `renpy.rollback`
- `renpy.roll forward info`
- `renpy.save`
- `renpy.scan saved game`
- `renpy.scene`
- `renpy.seen audio`
- `renpy.seen image`
- `renpy.seen label`
- `renpy.show`

- `renpy.showing`
- `renpy.shown` window
- `renpy.show` display say
- `renpy.sound.is` playing
- `renpy.sound.play`
- `renpy.sound.queue`
- `renpy.sound.set` mixer
- `renpy.sound.set` pan
- `renpy.sound.set` volume
- `renpy.sound.stop`
- `renpy.take` screenshot
- `renpy.timeout`
- `renpy.transition`
- `renpy.unlink` save
- `renpy.watch`
- `renpy.with` statement
- `Revolve`
- `RotoZoom`
- `ShowingSwitch`
- `SizeZoom`
- `SnowBlossom`
- `Solid`
- `SplineMotion`
- `Style`
- `Style.clear`
- `style.create`
- `style.rebuild`
- `Style.set` parent
- `Style.take`

- Text
- theme.ancient
- theme.image buttons
- theme.image labels
- theme.outline
- theme.roundrect
- theme.roundrect red
- Transform
- ui.add
- ui.adjustment
- ui.at
- ui.autobar
- ui.bar
- ui.button
- ui.callsinnewcontext
- ui.clear
- ui.close
- ui.detached
- ui.fixed
- ui.frame
- ui.gamemenus
- ui.grid
- ui.hbox
- ui.image
- ui.imagebutton
- ui.imagemap
- ui.input
- ui.interact
- ui.jumps

- `ui.jumpsoutofcontext`
- `ui.keymap`
- `ui.layer`
- `ui.menu`
- `ui.null`
- `ui.pausebehavior`
- `ui.remove`
- `ui.returns`
- `ui.saybehavior`
- `ui.side`
- `ui.sizer`
- `ui.soundstopbehavior`
- `ui.tag`
- `ui.text`
- `ui.textbutton`
- `ui.timer`
- `ui.transform`
- `ui.vbox`
- `ui.viewport`
- `ui.window`
- `VBox`
- `Viewport`
- `Window`
- `Zoom`

